EGRE 246 – Advanced Engineering Programming

Dr. Dan Resler
Dept. of Electrical & Computer Engineering
Virginia Commonwealth University

January 2024



Introduction

- **Software testing** is defined as an activity to check whether the actual results match the expected results and to ensure that the software system is free of defects
- Although testing can determine the correctness of software under the assumption of some specific hypotheses, testing cannot identify all the defects within the software.
- A study conducted by NIST in 2002 reports that software bugs cost the U.S. economy $59.5 billion annually
- Software testing is an exercise in risk and cost management
- Could easily present an entire semester course on the subject of software testing; we therefore are only going to cover an introduction



Introduction

- **Software testing** is defined as an activity to check whether the actual results match the expected results and to ensure that the software system is free of defects
- Although testing can determine the correctness of software under the assumption of some specific hypotheses, testing cannot identify all the defects within the software.
- A study conducted by NIST in 2002 reports that software bugs cost the U.S. economy $59.5 billion annually
- Software testing is an exercise in risk and cost management
- Could easily present an entire semester course on the subject of software testing; we therefore are only going to cover an introduction



Introduction

- **Software testing** is defined as an activity to check whether the actual results match the expected results and to ensure that the software system is free of defects
- Although testing can determine the correctness of software under the assumption of some specific hypotheses, testing cannot identify all the defects within the software. Why not?
- A study conducted by NIST in 2002 reports that software bugs cost the U.S. economy $59.5 billion annually
- Software testing is an exercise in risk and cost management
- Could easily present an entire semester course on the subject of software testing; we therefore are only going to cover an introduction



Introduction

- **Software testing** is defined as an activity to check whether the actual results match the expected results and to ensure that the software system is free of defects
- Although testing can determine the correctness of software under the assumption of some specific hypotheses, testing cannot identify all the defects within the software. Why not?

> *Testing shows the presence, not absence of bugs.* Edsger Dijkstra

- A study conducted by NIST in 2002 reports that software bugs cost the U.S. economy $59.5 billion annually
- Software testing is an exercise in risk and cost management
- Could easily present an entire semester course on the subject of software testing; we therefore are only going to cover an introduction



Introduction

- **Software testing** is defined as an activity to check whether the actual results match the expected results and to ensure that the software system is free of defects
- Although testing can determine the correctness of software under the assumption of some specific hypotheses, testing cannot identify all the defects within the software. Why not?

> *Testing shows the presence, not absence of bugs.* Edsger Dijkstra

- A study conducted by NIST in 2002 reports that software bugs cost the U.S. economy $59.5 billion annually
- Software testing is an exercise in risk and cost management
- Could easily present an entire semester course on the subject of software testing; we therefore are only going to cover an introduction



Introduction

- **Software testing** is defined as an activity to check whether the actual results match the expected results and to ensure that the software system is free of defects
- Although testing can determine the correctness of software under the assumption of some specific hypotheses, testing cannot identify all the defects within the software. Why not?

> *Testing shows the presence, not absence of bugs.* Edsger Dijkstra

- A study conducted by NIST in 2002 reports that software bugs cost the U.S. economy $59.5 billion annually
- Software testing is an exercise in risk and cost management
- Could easily present an entire semester course on the subject of software testing; we therefore are only going to cover an introduction



Introduction

- **Software testing** is defined as an activity to check whether the actual results match the expected results and to ensure that the software system is free of defects
- Although testing can determine the correctness of software under the assumption of some specific hypotheses, testing cannot identify all the defects within the software. Why not?

> *Testing shows the presence, not absence of bugs.* Edsger Dijkstra

- A study conducted by NIST in 2002 reports that software bugs cost the U.S. economy $59.5 billion annually
- Software testing is an exercise in risk and cost management
- Could easily present an entire semester course on the subject of software testing; we therefore are only going to cover an introduction



Ad hoc Testing

- **Ad hoc testing** is informal, direct, often random testing to try to uncover software bugs
- Usually takes the form of trying various inputs to see if the results are accurate
- It is performed by improvisation: the tester seeks to find bugs by any means that seem appropriate
- Usually is not documented or repeatable



Ad hoc Testing

- **Ad hoc testing** is informal, direct, often random testing to try to uncover software bugs
- Usually takes the form of trying various inputs to see if the results are accurate
- It is performed by improvisation: the tester seeks to find bugs by any means that seem appropriate
- Usually is not documented or repeatable



Ad hoc Testing

- **Ad hoc testing** is informal, direct, often random testing to try to uncover software bugs
- Usually takes the form of trying various inputs to see if the results are accurate
- It is performed by improvisation: the tester seeks to find bugs by any means that seem appropriate
- Usually is not documented or repeatable



Ad hoc Testing

- **Ad hoc testing** is informal, direct, often random testing to try to uncover software bugs
- Usually takes the form of trying various inputs to see if the results are accurate
- It is performed by improvisation: the tester seeks to find bugs by any means that seem appropriate
- Usually is not documented or repeatable



Automated Testing

- Software test automation is the use of software separate from the software being tested to control the execution of tests
- Usually compares actual outcomes with predicted or hoped for outcomes
- Test suites can be created, maintained and updated, repeatedly applied, analyzed, and can also provide reports
- Some testing software can also generate the tests themselves
- Software testing frameworks are available for most modern languages; they are usually provided in separate libraries
- We will be using **Unity** `http://www.throwtheswitch.org/unity` for **unit testing**
- Unit testing is a type of software testing where specific parts, modules, or subroutines (i.e. functions in C) are tested to see if they perform correctly
- Though Unity is not as feature-rich as some other testing platforms, it has the advantage that it is very lightweight and easy to install and use on most machines



Automated Testing

- Software test automation is the use of software separate from the software being tested to control the execution of tests
- Usually compares actual outcomes with predicted or hoped for outcomes
- Test suites can be created, maintained and updated, repeatedly applied, analyzed, and can also provide reports
- Some testing software can also generate the tests themselves
- Software testing frameworks are available for most modern languages; they are usually provided in separate libraries
- We will be using **Unity** `http://www.throwtheswitch.org/unity` for **unit testing**
- Unit testing is a type of software testing where specific parts, modules, or subroutines (i.e. functions in C) are tested to see if they perform correctly
- Though Unity is not as feature-rich as some other testing platforms, it has the advantage that it is very lightweight and easy to install and use on most machines



Automated Testing

- Software test automation is the use of software separate from the software being tested to control the execution of tests
- Usually compares actual outcomes with predicted or hoped for outcomes
- Test suites can be created, maintained and updated, repeatedly applied, analyzed, and can also provide reports
- Some testing software can also generate the tests themselves
- Software testing frameworks are available for most modern languages; they are usually provided in separate libraries
- We will be using **Unity** `http://www.throwtheswitch.org/unity` for **unit testing**
- Unit testing is a type of software testing where specific parts, modules, or subroutines (i.e. functions in C) are tested to see if they perform correctly
- Though Unity is not as feature-rich as some other testing platforms, it has the advantage that it is very lightweight and easy to install and use on most machines



Automated Testing

- Software test automation is the use of software separate from the software being tested to control the execution of tests
- Usually compares actual outcomes with predicted or hoped for outcomes
- Test suites can be created, maintained and updated, repeatedly applied, analyzed, and can also provide reports
- Some testing software can also generate the tests themselves
- Software testing frameworks are available for most modern languages; they are usually provided in separate libraries
- We will be using **Unity** `http://www.throwtheswitch.org/unity` for **unit testing**
- Unit testing is a type of software testing where specific parts, modules, or subroutines (i.e. functions in C) are tested to see if they perform correctly
- Though Unity is not as feature-rich as some other testing platforms, it has the advantage that it is very lightweight and easy to install and use on most machines



Automated Testing

- Software test automation is the use of software separate from the software being tested to control the execution of tests
- Usually compares actual outcomes with predicted or hoped for outcomes
- Test suites can be created, maintained and updated, repeatedly applied, analyzed, and can also provide reports
- Some testing software can also generate the tests themselves
- Software testing frameworks are available for most modern languages; they are usually provided in separate libraries
- We will be using **Unity** `http://www.throwtheswitch.org/unity` for **unit testing**
- Unit testing is a type of software testing where specific parts, modules, or subroutines (i.e. functions in C) are tested to see if they perform correctly
- Though Unity is not as feature-rich as some other testing platforms, it has the advantage that it is very lightweight and easy to install and use on most machines



Automated Testing

- Software test automation is the use of software separate from the software being tested to control the execution of tests
- Usually compares actual outcomes with predicted or hoped for outcomes
- Test suites can be created, maintained and updated, repeatedly applied, analyzed, and can also provide reports
- Some testing software can also generate the tests themselves
- Software testing frameworks are available for most modern languages; they are usually provided in separate libraries
- We will be using **Unity** `http://www.throwtheswitch.org/unity` for **unit testing**
- Unit testing is a type of software testing where specific parts, modules, or subroutines (i.e. functions in C) are tested to see if they perform correctly
- Though Unity is not as feature-rich as some other testing platforms, it has the advantage that it is very lightweight and easy to install and use on most machines



Automated Testing

- Software test automation is the use of software separate from the software being tested to control the execution of tests
- Usually compares actual outcomes with predicted or hoped for outcomes
- Test suites can be created, maintained and updated, repeatedly applied, analyzed, and can also provide reports
- Some testing software can also generate the tests themselves
- Software testing frameworks are available for most modern languages; they are usually provided in separate libraries
- We will be using **Unity** `http://www.throwtheswitch.org/unity` for **unit testing**
- Unit testing is a type of software testing where specific parts, modules, or subroutines (i.e. functions in C) are tested to see if they perform correctly
- Though Unity is not as feature-rich as some other testing platforms, it has the advantage that it is very lightweight and easy to install and use on most machines



Automated Testing

- Software test automation is the use of software separate from the software being tested to control the execution of tests
- Usually compares actual outcomes with predicted or hoped for outcomes
- Test suites can be created, maintained and updated, repeatedly applied, analyzed, and can also provide reports
- Some testing software can also generate the tests themselves
- Software testing frameworks are available for most modern languages; they are usually provided in separate libraries
- We will be using **Unity** `http://www.throwtheswitch.org/unity` for **unit testing**
- Unit testing is a type of software testing where specific parts, modules, or subroutines (i.e. functions in C) are tested to see if they perform correctly
- Though Unity is not as feature-rich as some other testing platforms, it has the advantage that it is very lightweight and easy to install and use on most machines



Unit Testing with Unity

- Let's create a simple example program for a unit testing demo (file `maxMain.c`):

```
#include <stdio.h>
#include "max.h"

int main(void){
  int x,y;
  printf("enter two ints: ");
  scanf("%d%d",&x,&y);
  printf("max(%d,%d) = %d\n",x,y,max(x,y));
}
```

- File `max.h`:

```
int max (int, int);
```

- File `max.c`:

```
int max(int a, int b) { return (a>=b?a:b); }
```



Unit Testing with Unity

- Let's create a simple example program for a unit testing demo (file `maxMain.c`):

```
#include <stdio.h>
#include "max.h"

int main(void){
  int x,y;
  printf("enter two ints: ");
  scanf("%d%d",&x,&y);
  printf("max(%d,%d) = %d\n",x,y,max(x,y));
}
```

- File `max.h`:

```
int max (int, int);
```

- File `max.c`:

```
int max(int a, int b) { return (a>=b?a:b); }
```



Unit Testing with Unity

- Let's create a simple example program for a unit testing demo (file `maxMain.c`):

```
#include <stdio.h>
#include "max.h"

int main(void){
  int x,y;
  printf("enter two ints: ");
  scanf("%d%d",&x,&y);
  printf("max(%d,%d) = %d\n",x,y,max(x,y));
}
```

- File `max.h`:

```
int max (int, int);
```

- File `max.c`:

```
int max(int a, int b) { return (a>=b?a:b); }
```

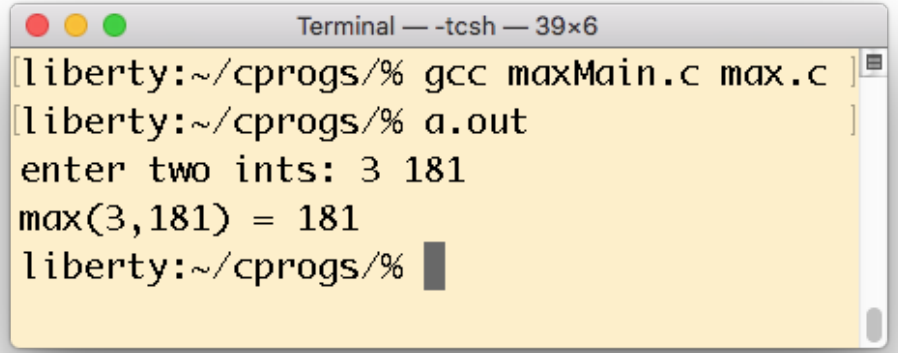


Unit Testing with Unity

- Let's create a simple example program for a unit testing demo (file maxMain.c):

```
#include <stdio.h>
#include "max.h"

int main(void){
  int x,y;
  printf("enter two ints: ");
  scanf("%d%d",&x,&y);
  printf("max(%d,%d) = %d\n",x,y,max(x,y));
}
```



- File max.h:

```
int max (int, int);
```

- File max.c:

```
int max(int a, int b) { return (a>=b?a:b); }
```

Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
  2. Initialize the Unity test framework
  3. Add *test suites* (functions) to test your code
  4. Add tests to the suites
  5. Run tests using the appropriate Unity calls
  6. Report the results
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
  2. Initialize the Unity test framework
  3. Add *test suites* (functions) to test your code
  4. Add tests to the suites
  5. Run tests using the appropriate Unity calls
  6. Report the results
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
  2. Initialize the Unity test framework
  3. Add *test suites* (functions) to test your code
  4. Add tests to the suites
  5. Run tests using the appropriate Unity calls
  6. Report the results
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
  2. Initialize the Unity test framework
  3. Add *test suites* (functions) to test your code
  4. Add tests to the suites
  5. Run tests using the appropriate Unity calls
  6. Report the results
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
  2. Initialize the Unity test framework
  3. Add *test suites* (functions) to test your code
  4. Add tests to the suites
  5. Run tests using the appropriate Unity calls
  6. Report the results
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
  2. Initialize the Unity test framework
  3. Add *test suites* (functions) to test your code
  4. Add tests to the suites
  5. Run tests using the appropriate Unity calls
  6. Report the results
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
  2. Initialize the Unity test framework
  3. Add *test suites* (functions) to test your code
  4. Add tests to the suites
  5. Run tests using the appropriate Unity calls
  6. Report the results
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

- We are going to write unit tests using Unity for the function `max` in our example
- A typical sequence of steps for using the Unity framework:
  1. Create a unit testing version of your code with the required Unity functions
  2. Initialize the Unity test framework
  3. Add *test suites* (functions) to test your code
  4. Add tests to the suites
  5. Run tests using the appropriate Unity calls
  6. Report the results
- Let's build up our test program step by step . . .



Unit Testing with Unity *continued*

File `maxUnity.c`

```
#include <stdio.h>
#include "unity.h"
#include "max.h"

void setUp(void){ /* set stuff up here */ }
void tearDown(void){ /* clean stuff up here */ }

int main (void){

}
```

1) Create a unit testing version of your code



Unit Testing with Unity *continued*

File `maxUnity.c`

```
#include <stdio.h>
#include "unity.h"
#include "max.h"

void setUp(void){ /* set stuff up here */ }
void tearDown(void){ /* clean stuff up here */ }
```

1) Create a unit testing version of your code
2) Initialize the Unity test framework

```
int main (void){
  UNITY_BEGIN();


  return UNITY_END();
}
```



Unit Testing with Unity *continued*

File `maxUnity.c`

```
#include <stdio.h>
#include "unity.h"
#include "max.h"

void setUp(void){ /* set stuff up here */ }
void tearDown(void){ /* clean stuff up here */ }

void maxTest1(void){


}
void maxTest2(void){



}
void maxTest3(void){


}

int main (void){
  UNITY_BEGIN();



  return UNITY_END();
}
```

1) Create a unit testing version of your code
2) Initialize the Unity test framework
3) Add *test suites* to test your code



Unit Testing with Unity *continued*

File `maxUnity.c`

```
#include <stdio.h>
#include "unity.h"
#include "max.h"

void setUp(void){ /* set stuff up here */ }
void tearDown(void){ /* clean stuff up here */ }

void maxTest1(void){
  TEST_ASSERT_EQUAL_INT(2,max(1,2));
  TEST_ASSERT_EQUAL_INT(2,max(2,1));
}
void maxTest2(void){
  TEST_ASSERT_EQUAL_INT(2,max(2,2));
  TEST_ASSERT_EQUAL_INT(0,max(0,0));
  TEST_ASSERT_EQUAL_INT(-1,max(-1,-1));
}
void maxTest3(void){
  TEST_ASSERT_EQUAL_INT(-1,max(-1,-2));
  TEST_ASSERT_EQUAL_INT(-1,max(-2,-1));
}

int main (void){
  UNITY_BEGIN();



  return UNITY_END();
}
```

1) Create a unit testing version of your code
2) Initialize the Unity test framework
3) Add *test suites* to test your code
4) Add tests to the suites



Unit Testing with Unity *continued*

File `maxUnity.c`

```
#include <stdio.h>
#include "unity.h"
#include "max.h"

void setUp(void){ /* set stuff up here */ }
void tearDown(void){ /* clean stuff up here */ }

void maxTest1(void){
  TEST_ASSERT_EQUAL_INT(2,max(1,2));
  TEST_ASSERT_EQUAL_INT(2,max(2,1));
}
void maxTest2(void){
  TEST_ASSERT_EQUAL_INT(2,max(2,2));
  TEST_ASSERT_EQUAL_INT(0,max(0,0));
  TEST_ASSERT_EQUAL_INT(-1,max(-1,-1));
}
void maxTest3(void){
  TEST_ASSERT_EQUAL_INT(-1,max(-1,-2));
  TEST_ASSERT_EQUAL_INT(-1,max(-2,-1));
}

int main (void){
  UNITY_BEGIN();
  RUN_TEST(maxTest1);
  RUN_TEST(maxTest2);
  RUN_TEST(maxTest3);
  return UNITY_END();
}
```

1) Create a unit testing version of your code
2) Initialize the Unity test framework
3) Add *test suites* to test your code
4) Add tests to the suites
5) Run the tests using the appropriate Unity calls
6) Report the results



Unit Testing with Unity *continued*

File `maxUnity.c`

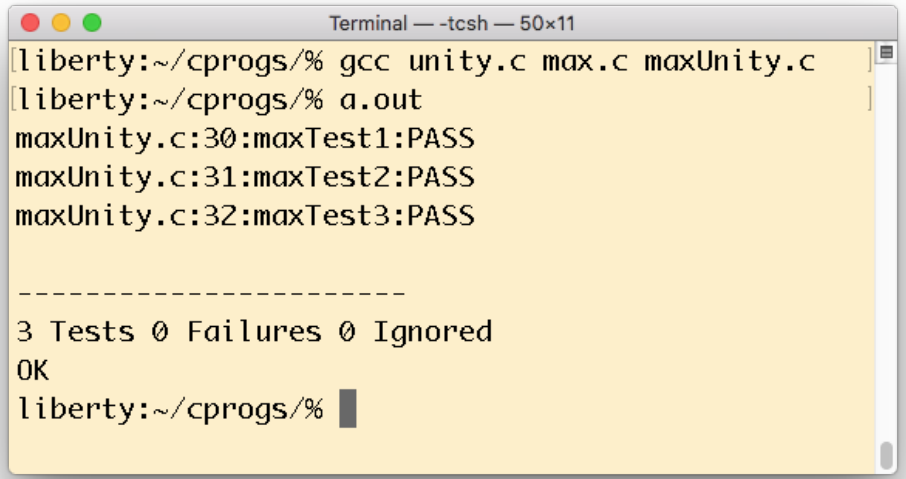
```
#include <stdio.h>
#include "unity.h"
#include "max.h"

void setUp(void){ /* set stuff up here */ }
void tearDown(void){ /* clean stuff up here */ }

void maxTest1(void){
  TEST_ASSERT_EQUAL_INT(2,max(1,2));
  TEST_ASSERT_EQUAL_INT(2,max(2,1));
}
void maxTest2(void){
  TEST_ASSERT_EQUAL_INT(2,max(2,2));
  TEST_ASSERT_EQUAL_INT(0,max(0,0));
  TEST_ASSERT_EQUAL_INT(-1,max(-1,-1));
}
void maxTest3(void){
  TEST_ASSERT_EQUAL_INT(-1,max(-1,-2));
  TEST_ASSERT_EQUAL_INT(-1,max(-2,-1));
}

int main (void){
  UNITY_BEGIN();
  RUN_TEST(maxTest1);
  RUN_TEST(maxTest2);
  RUN_TEST(maxTest3);
  return UNITY_END();
}
```

1) Create a unit testing version of your code
2) Initialize the Unity test framework
3) Add *test suites* to test your code
4) Add tests to the suites
5) Run the tests using the appropriate Unity calls
6) Report the results



Unit Testing with Unity *continued*

- File `maxError.c`:

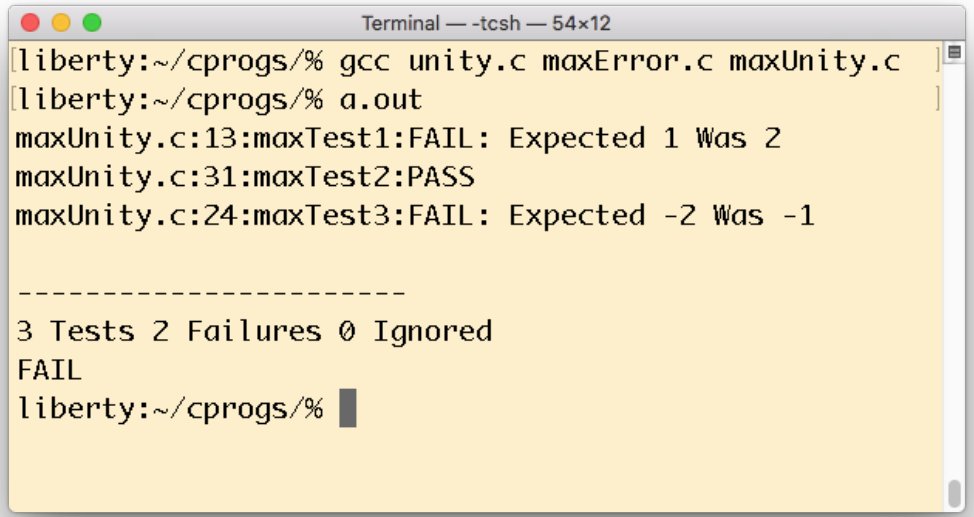
```
int max(int a, int b) { return (a<=b?a:b); }
```



Unit Testing with Unity *continued*

- File `maxError.c`:

```
int max(int a, int b) { return (a<=b?a:b); }
```



Unity Assertions

Basic Fail and Ignore

```
TEST_FAIL()
TEST_IGNORE()
```

Boolean

```
TEST_ASSERT (condition)
TEST_ASSERT_TRUE (condition)
TEST_ASSERT_UNLESS (condition)
TEST_ASSERT_FALSE (condition)
TEST_ASSERT_NULL (pointer)
TEST_ASSERT_NOT_NULL (pointer)
```

Signed and Unsigned Integers (of all sizes)

```
TEST_ASSERT_EQUAL_INT (exp, act)
TEST_ASSERT_EQUAL_INT8 (exp, act)
TEST_ASSERT_EQUAL_INT16 (exp, act)
TEST_ASSERT_EQUAL_INT32 (exp, act)
TEST_ASSERT_EQUAL_INT64 (exp, act)
TEST_ASSERT_EQUAL (exp, act)
TEST_ASSERT_NOT_EQUAL (exp, act)
TEST_ASSERT_EQUAL_UINT (exp, act)
TEST_ASSERT_EQUAL_UINT8 (exp, act)
TEST_ASSERT_EQUAL_UINT16 (exp, act)
TEST_ASSERT_EQUAL_UINT32 (exp, act)
TEST_ASSERT_EQUAL_UINT64 (exp, act)
```

Unsigned Integers (of all sizes) in Hexadecimal

```
TEST_ASSERT_EQUAL_HEX (exp, act)
TEST_ASSERT_EQUAL_HEX8 (exp, act)
TEST_ASSERT_EQUAL_HEX16 (exp, act)
TEST_ASSERT_EQUAL_HEX32 (exp, act)
TEST_ASSERT_EQUAL_HEX64 (exp, act)
```

Masked and Bit-level Comparisons

```
TEST_ASSERT_BITS (mask, exp, act)
TEST_ASSERT_BITS_HIGH (mask, act)
TEST_ASSERT_BITS_LOW (mask, act)
TEST_ASSERT_BIT_HIGH (bit, act)
```

Structs and Strings

```
TEST_ASSERT_EQUAL_PTR (exp, act)
TEST_ASSERT_EQUAL_STRING (exp, act)
TEST_ASSERT_EQUAL_MEMORY (exp, act, len)
```

Arrays

```
TEST_ASSERT_EQUAL_INT_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_INT8_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_INT16_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_INT32_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_INT64_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_UINT_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_UINT8_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_UINT16_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_UINT32_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_UINT64_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_HEX_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_HEX8_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_HEX16_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_HEX32_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_HEX64_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_PTR_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_STRING_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_MEMORY_ARRAY (exp, act, len,
elem)
```

Each Equal (Comparing Arrays to a Single Val)

(these follow the pattern of the arrays, but are named like this)

```
TEST_ASSERT_EACH_EQUAL_INT8 (exp, act, elem)
```

Floating Point (If Enabled)

```
TEST_ASSERT_FLOAT_WITHIN (delta, exp, act)
TEST_ASSERT_EQUAL_FLOAT (exp, act)
TEST_ASSERT_EQUAL_FLOAT_ARRAY (exp, act, elem)
TEST_ASSERT_FLOAT_IS_INF (act)
TEST_ASSERT_FLOAT_IS_NEG_INF (act)
TEST_ASSERT_FLOAT_IS_NAN (act)
```



Unity Assertions *continued*

```
TEST_ASSERT_EQUAL (exp, act)
TEST_ASSERT_NOT_EQUAL (exp, act)
TEST_ASSERT_EQUAL_UINT (exp, act)
TEST_ASSERT_EQUAL_UINT8 (exp, act)
TEST_ASSERT_EQUAL_UINT16 (exp, act)
TEST_ASSERT_EQUAL_UINT32 (exp, act)
TEST_ASSERT_EQUAL_UINT64 (exp, act)
```

Unsigned Integers (of all sizes) in Hexadecimal

```
TEST_ASSERT_EQUAL_HEX (exp, act)
TEST_ASSERT_EQUAL_HEX8 (exp, act)
TEST_ASSERT_EQUAL_HEX16 (exp, act)
TEST_ASSERT_EQUAL_HEX32 (exp, act)
TEST_ASSERT_EQUAL_HEX64 (exp, act)
```

Masked and Bit-level Comparisons

```
TEST_ASSERT_BITS (mask, exp, act)
TEST_ASSERT_BITS_HIGH (mask, act)
TEST_ASSERT_BITS_LOW (mask, act)
TEST_ASSERT_BIT_HIGH (bit, act)
TEST_ASSERT_BIT_LOW (bit, act)
```

Integer Ranges (of all sizes)

```
TEST_ASSERT_INT_WITHIN (delta, exp, act)
TEST_ASSERT_INT8_WITHIN (delta, exp, act)
TEST_ASSERT_INT16_WITHIN (delta, exp, act)
TEST_ASSERT_INT32_WITHIN (delta, exp, act)
TEST_ASSERT_INT64_WITHIN (delta, exp, act)
TEST_ASSERT_UINT_WITHIN (delta, exp, act)
TEST_ASSERT_UINT8_WITHIN (delta, exp, act)
TEST_ASSERT_UINT16_WITHIN (delta, exp, act)
TEST_ASSERT_UINT32_WITHIN (delta, exp, act)
TEST_ASSERT_UINT64_WITHIN (delta, exp, act)
TEST_ASSERT_HEX_WITHIN (delta, exp, act)
TEST_ASSERT_HEX8_WITHIN (delta, exp, act)
TEST_ASSERT_HEX16_WITHIN (delta, exp, act)
TEST_ASSERT_HEX32_WITHIN (delta, exp, act)
TEST_ASSERT_HEX64_WITHIN (delta, exp, act)
```

```
TEST_ASSERT_EQUAL_HEX8_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_HEX16_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_HEX32_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_HEX64_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_PTR_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_STRING_ARRAY (exp, act, elem)
TEST_ASSERT_EQUAL_MEMORY_ARRAY (exp, act, len,
elem)
```

Each Equal (Comparing Arrays to a Single Val)

(these follow the pattern of the arrays, but are named like this)

```
TEST_ASSERT_EACH_EQUAL_INT8 (exp, act, elem)
```

Floating Point (If Enabled)

```
TEST_ASSERT_FLOAT_WITHIN (delta, exp, act)
TEST_ASSERT_EQUAL_FLOAT (exp, act)
TEST_ASSERT_EQUAL_FLOAT_ARRAY (exp, act, elem)
TEST_ASSERT_FLOAT_IS_INF (act)
TEST_ASSERT_FLOAT_IS_NEG_INF (act)
TEST_ASSERT_FLOAT_IS_NAN (act)
TEST_ASSERT_FLOAT_IS_DETERMINATE (act)
TEST_ASSERT_FLOAT_IS_NOT_INF (act)
TEST_ASSERT_FLOAT_IS_NOT_NEG_INF (act)
TEST_ASSERT_FLOAT_IS_NOT_NAN (act)
TEST_ASSERT_FLOAT_IS_NOT_DETERMINATE (act)
```

Double (If Enabled)

```
TEST_ASSERT_DOUBLE_WITHIN (delta, exp, act)
TEST_ASSERT_EQUAL_DOUBLE (exp, act)
TEST_ASSERT_EQUAL_DOUBLE_ARRAY (exp, act, elem)
TEST_ASSERT_DOUBLE_IS_INF (act)
TEST_ASSERT_DOUBLE_IS_NEG_INF (act)
TEST_ASSERT_DOUBLE_IS_NAN (act)
TEST_ASSERT_DOUBLE_IS_DETERMINATE (act)
TEST_ASSERT_DOUBLE_IS_NOT_INF (act)
TEST_ASSERT_DOUBLE_IS_NOT_NEG_INF (act)
TEST_ASSERT_DOUBLE_IS_NOT_NAN (act)
TEST_ASSERT_DOUBLE_IS_NOT_DETERMINATE (act)
```



Test-driven Software Development

- **Test-driven development (TDD)** is an evolutionary approach to software development where you write the software tests *before* you write the code to fulfill those tests.
- Basic idea of TDD using unit testing of functions:
  - write syntactically correct stubs for all specified functions
  - write test cases based on the specifications
  - run test cases
  - fix bugs
  - repeat



Test-driven Software Development

- **Test-driven development (TDD)** is an evolutionary approach to software development where you write the software tests *before* you write the code to fulfill those tests.
- Basic idea of TDD using unit testing of functions:
  - write syntactically correct stubs for all specified functions
  - write complete unit test suites for your functions
  - run test suites – all tests should fail!
  - repeat {
    - write code to pass tests
    - run test suites
    - refactor code
  - } until all tests pass



Test-driven Software Development

- **Test-driven development (TDD)** is an evolutionary approach to software development where you write the software tests *before* you write the code to fulfill those tests.
- Basic idea of TDD using unit testing of functions:
  - write syntactically correct stubs for all specified functions
  - write complete unit test suites for your functions
  - run test suites – all tests should fail!
  - repeat {
    - } until all tests pass



Test-driven Software Development

- **Test-driven development (TDD)** is an evolutionary approach to software development where you write the software tests *before* you write the code to fulfill those tests.
- Basic idea of TDD using unit testing of functions:
  - write syntactically correct stubs for all specified functions
  - write complete unit test suites for your functions
  - run test suites – all tests should fail!
  - repeat {

    } until all tests pass



Test-driven Software Development

- **Test-driven development (TDD)** is an evolutionary approach to software development where you write the software tests *before* you write the code to fulfill those tests.
- Basic idea of TDD using unit testing of functions:
  - write syntactically correct stubs for all specified functions
  - write complete unit test suites for your functions
  - run test suites – all tests should fail!
  - repeat {
    - write or modify your functions to meet specs
    - run test suites
    - } until all tests pass



Test-driven Software Development

- **Test-driven development (TDD)** is an evolutionary approach to software development where you write the software tests *before* you write the code to fulfill those tests.
- Basic idea of TDD using unit testing of functions:
  - write syntactically correct stubs for all specified functions
  - write complete unit test suites for your functions
  - run test suites – all tests should fail!
  - repeat {
    - write or modify your functions to meet specs
    - run test suites

    } until all tests pass



Test-driven Software Development

- **Test-driven development (TDD)** is an evolutionary approach to software development where you write the software tests *before* you write the code to fulfill those tests.
- Basic idea of TDD using unit testing of functions:
  - write syntactically correct stubs for all specified functions
  - write complete unit test suites for your functions
  - run test suites – all tests should fail!
  - repeat {
    - ★ write or modify your functions to meet specs
    - ★ run test suites

    } until all tests pass



Test-driven Software Development

- **Test-driven development (TDD)** is an evolutionary approach to software development where you write the software tests *before* you write the code to fulfill those tests.
- Basic idea of TDD using unit testing of functions:
  - write syntactically correct stubs for all specified functions
  - write complete unit test suites for your functions
  - run test suites – all tests should fail!
  - repeat {
    - ★ write or modify your functions to meet specs
    - ★ run test suites

    } until all tests pass



TDD Example

- Let's write the function `squareRoot` defined below that returns the square root of its argument to a precision of 0.0001 using Newton's algorithm

```
double squareRoot(long);
```

$$squareRoot(n) = \begin{cases} \sqrt{n} & \text{if } n >= 1 \\ NaN & \text{otherwise} \end{cases}$$

- We will use the Unity test framework to do our test-driven development



TDD Example

- Let's write the function `squareRoot` defined below that returns the square root of its argument to a precision of 0.0001 using Newton's algorithm

```
double squareRoot(long);
```

$$squareRoot(n) = \begin{cases} \sqrt{n} & \text{if } n >= 1 \\ NaN & \text{otherwise} \end{cases}$$

- We will use the Unity test framework to do our test-driven development



TDD Example *continued*

- First create `squareRoot.c` with just a syntactically correct function stub for our function:

```
double squareRoot(long n) {
  return -9999.999; // value is irrelevant
}
```

- Now let's write our tests. How you write these, and in what order, is arbitrary though most programmers develop a style or custom and practice in how they do it:
  - test routine to test square roots with no fractional part:

```
double t = 0.0001;

void squareRootTest1(void) {
  TEST_ASSERT_DOUBLE_WITHIN(t,1.0,squareRoot(1));
  TEST_ASSERT_DOUBLE_WITHIN(t,10.0,squareRoot(100));
  TEST_ASSERT_DOUBLE_WITHIN(t,520.0,squareRoot(270784));
  TEST_ASSERT_DOUBLE_WITHIN(t,1211.0,squareRoot(1466521));
}
```



TDD Example *continued*

- First create `squareRoot.c` with just a syntactically correct function stub for our function:

```
double squareRoot(long n) {
  return -9999.999; // value is irrelevant
}
```

- Now let's write our tests. How you write these, and in what order, is arbitrary though most programmers develop a style or custom and practice in how they do it:
  - test routine to test square roots with no fractional part:

```
double t = 0.0001;

void squareRootTest1(void) {
  TEST_ASSERT_DOUBLE_WITHIN(t,1.0,squareRoot(1));
  TEST_ASSERT_DOUBLE_WITHIN(t,10.0,squareRoot(100));
  TEST_ASSERT_DOUBLE_WITHIN(t,528.0,squareRoot(278784));
  TEST_ASSERT_DOUBLE_WITHIN(t,1211.0,squareRoot(1466521));
}
```



TDD Example *continued*

- First create `squareRoot.c` with just a syntactically correct function stub for our function:

```
double squareRoot(long n) {
  return -9999.999; // value is irrelevant
}
```

- Now let's write our tests. How you write these, and in what order, is arbitrary though most programmers develop a style or custom and practice in how they do it:
  - test routine to test square roots with no fractional part:

```
double t = 0.0001;

void squareRootTest1(void) {
  TEST_ASSERT_DOUBLE_WITHIN(t,1.0,squareRoot(1));
  TEST_ASSERT_DOUBLE_WITHIN(t,10.0,squareRoot(100));
  TEST_ASSERT_DOUBLE_WITHIN(t,528.0,squareRoot(278784));
  TEST_ASSERT_DOUBLE_WITHIN(t,1211.0,squareRoot(1466521));
}
```



TDD Example *continued*

- More tests:
  - test routine to test square roots with a fractional part:

```
void squareRootTest2(void) {
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,2.236068,squareRoot(5),"test #2-1");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,20.469489,squareRoot(419),"test #2-2");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,26480.164746,squareRoot(701199123),"test #2-3");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,sqrt(532),squareRoot(532),"test #2-4");
}
```

  - test routine to test illegal inputs:

```
void squareRootTest3(void) {
  TEST_ASSERT_TRUE(isnan(squareRoot(0)));
  TEST_ASSERT_TRUE(isnan(squareRoot(-1)));
  TEST_ASSERT_TRUE(isnan(squareRoot(-144)));
}
```



TDD Example *continued*

- More tests:
  - test routine to test square roots with a fractional part:

```
void squareRootTest2(void) {
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,2.236068,squareRoot(5),"test #2-1");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,20.469489,squareRoot(419),"test #2-2");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,26480.164746,squareRoot(701199123),"test #2-3");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,sqrt(532),squareRoot(532),"test #2-4");
}
```

  - test routine to test illegal inputs:

```
void squareRootTest3(void) {
  TEST_ASSERT_TRUE(isnan(squareRoot(0)));
  TEST_ASSERT_TRUE(isnan(squareRoot(-1)));
  TEST_ASSERT_TRUE(isnan(squareRoot(-144)));
}
```



TDD Example *continued*

- More tests:
  - test routine to test square roots with a fractional part:

```
void squareRootTest2(void) {
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,2.236068,squareRoot(5),"test #2-1");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,20.469489,squareRoot(419),"test #2-2");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,26480.164746,squareRoot(701199123),"test #2-3");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,sqrt(532),squareRoot(532),"test #2-4");
}
```

  - test routine to test illegal inputs:

```
void squareRootTest3(void) {
  TEST_ASSERT_TRUE(isnan(squareRoot(0)));
  TEST_ASSERT_TRUE(isnan(squareRoot(-1)));
  TEST_ASSERT_TRUE(isnan(squareRoot(-144)));
}
```



TDD Example *continued*

- Now wrap our tests in a Unity program (file `squareRootUnity.c`):

```
#include <stdio.h>
#include "unity.h"
#include "squareRoot.h"
#include <math.h>

void setUp(void) {}
void tearDown(void) {}

double t = 0.0000001;

void squareRootTest1(void) {
  TEST_ASSERT_DOUBLE_WITHIN(t,1.0,squareRoot(1));
  TEST_ASSERT_DOUBLE_WITHIN(t,10.0,squareRoot(100));
  TEST_ASSERT_DOUBLE_WITHIN(t,528.0,squareRoot(278784));
  TEST_ASSERT_DOUBLE_WITHIN(t,1211.0,squareRoot(1466521));
}

void squareRootTest2(void) {
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,2.236068,squareRoot(5),"test #2-1");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,20.469489,squareRoot(419),"test #2-2");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,26480.164746,squareRoot(701199123),"test #2-3");
  TEST_ASSERT_DOUBLE_WITHIN_MESSAGE(t,sqrt(532),squareRoot(532),"test #2-4");
}
```



TDD Example *continued*

- Rest of file `squareRootUnity.c`:

```
void squareRootTest3(void) {
  TEST_ASSERT_TRUE_MESSAGE(isnan(squareRoot(0)),"test #3-1");
  TEST_ASSERT_TRUE_MESSAGE(isnan(squareRoot(-1)),"test #3-2");
  TEST_ASSERT_TRUE_MESSAGE(isnan(squareRoot(-144)),"test #3-3");
}

int main ( void ){
  UNITY_BEGIN();
  RUN_TEST(squareRootTest1);
  RUN_TEST(squareRootTest2);
  RUN_TEST(squareRootTest3);
  return UNITY_END();
}
```

- Next let's run our test suites (see next page).



TDD Example *continued*

- Rest of file `squareRootUnity.c`:

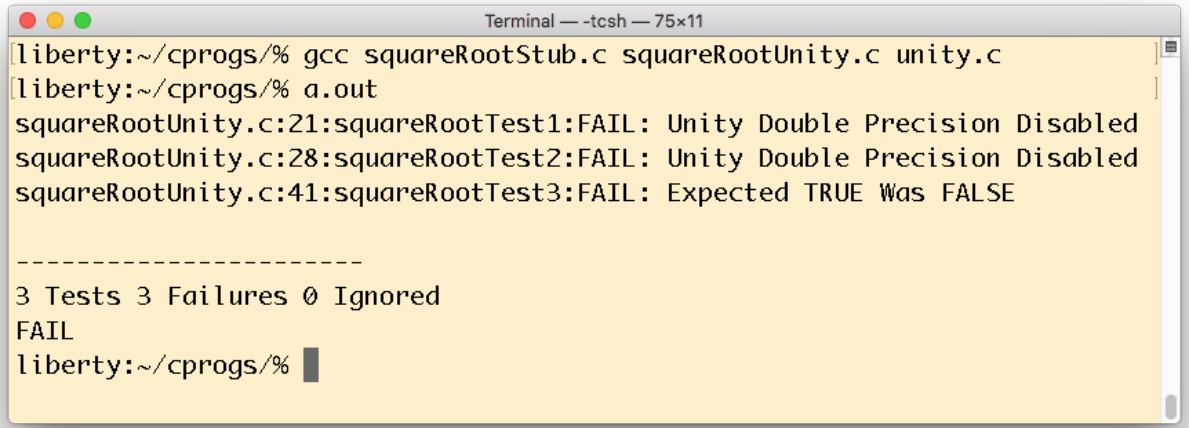
```
void squareRootTest3(void) {
  TEST_ASSERT_TRUE_MESSAGE(isnan(squareRoot(0)),"test #3-1");
  TEST_ASSERT_TRUE_MESSAGE(isnan(squareRoot(-1)),"test #3-2");
  TEST_ASSERT_TRUE_MESSAGE(isnan(squareRoot(-144)),"test #3-3");
}

int main ( void ){
  UNITY_BEGIN();
  RUN_TEST(squareRootTest1);
  RUN_TEST(squareRootTest2);
  RUN_TEST(squareRootTest3);
  return UNITY_END();
}
```

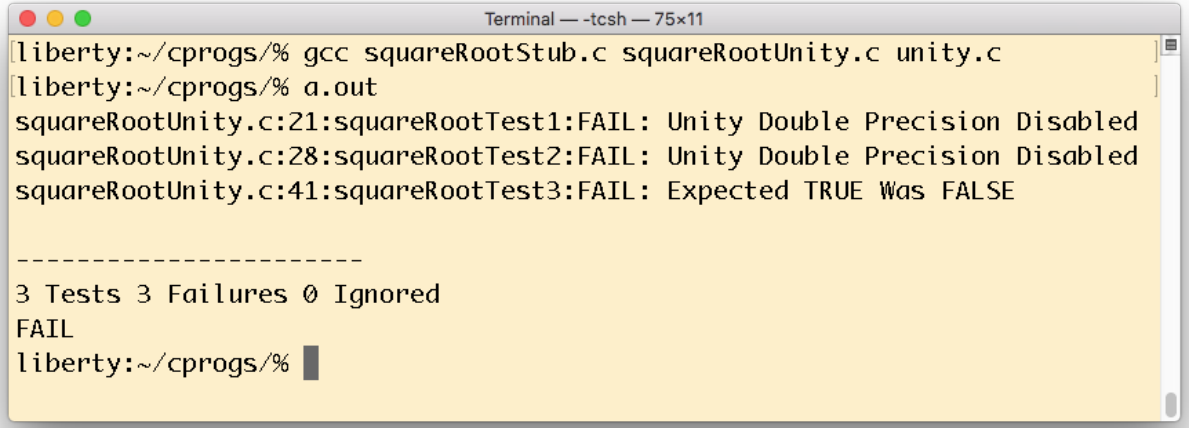
- Next let's run our test suites (see next page).



TDD Example *continued*



TDD Example *continued*



Use flag '-D UNITY_INCLUDE_DOUBLE' for double tests

TDD Example *continued*

Terminal — -tcsh — 75×11

```
liberty:~/cprogs/% gcc squareRootStub.c squareRootUnity.c unity.c
liberty:~/cprogs/% a.out
squareRootUnity.c:21:squareRootTest1:FAIL: Unity Double Precision Disabled
squareRootUnity.c:28:squareRootTest2:FAIL: Unity Double Precision Disabled
squareRootUnity.c:41:squareRootTest3:FAIL: Expected TRUE Was FALSE

-----------------------
3 Tests 3 Failures 0 Ignored
FAIL
liberty:~/cprogs/%
```

Use flag '-D UNITY_INCLUDE_DOUBLE' for double tests

Terminal — -tcsh — 91×10

```
liberty:~/cprogs/% gcc squareRootStub.c squareRootUnity.c unity.c -D UNITY_INCLUDE_DOUBLE
liberty:~/cprogs/% a.out
squareRootUnity.c:20:squareRootTest1:FAIL: Expected 1 Was -9999.999
squareRootUnity.c:27:squareRootTest2:FAIL: Expected 2.236068 Was -9999.999. test #2-1
squareRootUnity.c:34:squareRootTest3:FAIL: test #3-1

-----------------------
3 Tests 3 Failures 0 Ignored
FAIL
liberty:~/cprogs/%
```



TDD Example *continued*

- Let's begin to develop our function by adding code to check for illegal input:

```
#include <math.h>

double squareRoot(long x) {
  if(x==0) return NAN; // <-- error! should be x<=0
  return -9999.999;
}
```

- Now run the test suites . . .



TDD Example *continued*

- Let's begin to develop our function by adding code to check for illegal input:

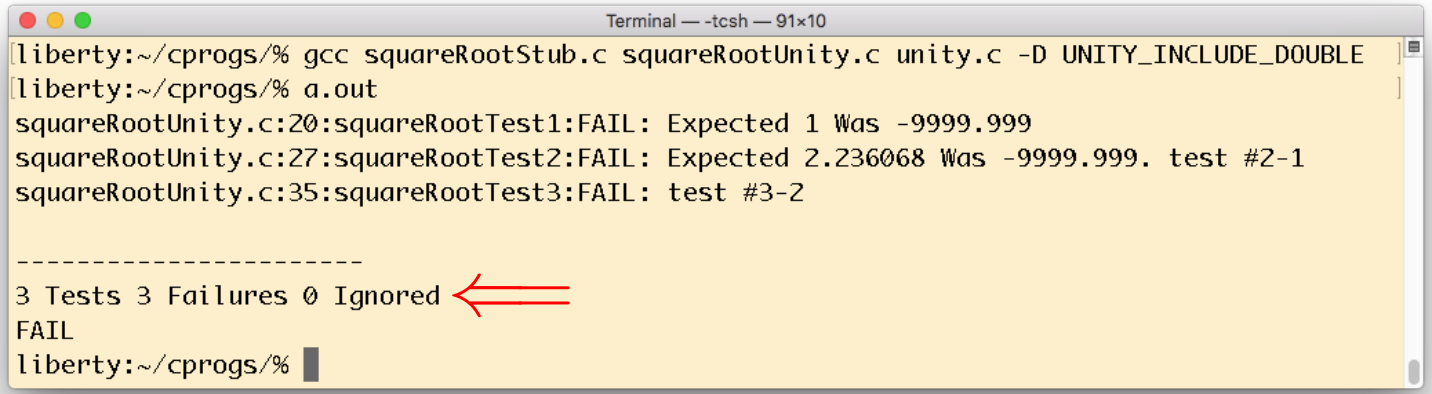
```
#include <math.h>

double squareRoot(long x) {
  if(x==0) return NAN; // <-- error! should be x<=0
  return -9999.999;
}
```

- Now run the test suites . . .



TDD Example *continued*



TDD Example *continued*

- Now continue to edit your function and running your test suites until all tests pass
- Final version of squareRoot:

```
#include <math.h>
double squareRoot(long x) {
  if(x<=0) return NAN;
  int prev,k = 0;
  int kmax = 1000;
  double s = 1;
  for(k=0;k<kmax;k++) {
    prev = s;
    s = (s + x/s)/2;
    if(prev == s) break;
  }
  return s;
}
```



TDD Example *continued*

- Now continue to edit your function and running your test suites until all tests pass
- Final version of squareRoot:

```
#include <math.h>
double squareRoot(long x) {
  if(x<=0) return NAN;
  int prev,k = 0;
  int kmax = 1000;
  double s = 1;
  for(k=0;k<kmax;k++) {
    prev = s;
    s = (s + x/s)/2;
    if(prev == s) break;
  }
  return s;
}
```



TDD Example *continued*

Terminal — -tcsh — 91×10

```
liberty:~/cprogs/% gcc squareRootStub.c squareRootUnity.c unity.c -D UNITY_INCLUDE_DOUBLE
liberty:~/cprogs/% a.out
squareRootUnity.c:40:squareRootTest1:PASS
squareRootUnity.c:41:squareRootTest2:PASS
squareRootUnity.c:42:squareRootTest3:PASS

-----------------------
3 Tests 0 Failures 0 Ignored
OK
liberty:~/cprogs/%
```



Regression Testing

- It is a common occurrence that modifying code or developing new functionality in a program inadvertently “breaks” the code that had previously worked!
- Therefore it is important that whenever you change or add code to your program you re-run all of the tests that had previously passed to see if everything still works
- **Regression testing** is re-running all tests after a change to ensure that previously developed and tested software still performs correctly (if not, that would be called a **regression**)
- Automatic software testing frameworks like Unity make it easy to perform regression testing after every code modification



Regression Testing

- It is a common occurrence that modifying code or developing new functionality in a program inadvertently “breaks” the code that had previously worked!
- Therefore it is important that whenever you change or add code to your program you re-run all of the tests that had previously passed to see if everything still works
- **Regression testing** is re-running all tests after a change to ensure that previously developed and tested software still performs correctly (if not, that would be called a **regression**)
- Automatic software testing frameworks like Unity make it easy to perform regression testing after every code modification



Regression Testing

- It is a common occurrence that modifying code or developing new functionality in a program inadvertently “breaks” the code that had previously worked!
- Therefore it is important that whenever you change or add code to your program you re-run all of the tests that had previously passed to see if everything still works
- **Regression testing** is re-running all tests after a change to ensure that previously developed and tested software still performs correctly (if not, that would be called a **regression**)
- Automatic software testing frameworks like Unity make it easy to perform regression testing after every code modification



Regression Testing

- It is a common occurrence that modifying code or developing new functionality in a program inadvertently “breaks” the code that had previously worked!
- Therefore it is important that whenever you change or add code to your program you re-run all of the tests that had previously passed to see if everything still works
- **Regression testing** is re-running all tests after a change to ensure that previously developed and tested software still performs correctly (if not, that would be called a **regression**)
- Automatic software testing frameworks like Unity make it easy to perform regression testing after every code modification



Structures

- Arrays are data structures that provide a means to store homogeneous data; structures allow us to create heterogeneous data structures
- The structure car:

```
#define SIZE 50

struct car {
  int year;
  int doors;
  float horsepower;
  char model[SIZE+1];
};
```

- Use:

```
struct car mycar;
mycar.year = 2014;
mycar.doors = 4;
mycar.horsepower = 192.0;
strncpy(mycar.model,"Honda Accord", SIZE);
```



Structures

- Arrays are data structures that provide a means to store homogeneous data; structures allow us to create heterogeneous data structures
- The structure car:

```
#define SIZE 50

struct car {
  int year;
  int doors;
  float horsepower;
  char model[SIZE+1];
};
```

- Use:

```
struct car mycar;
mycar.year = 2014;
mycar.doors = 4;
mycar.horsepower = 192.0;
strncpy(mycar.model,"Honda Accord", SIZE);
```



Structures

- Arrays are data structures that provide a means to store homogeneous data; structures allow us to create heterogeneous data structures
- The structure car:

```
#define SIZE 50

struct car {
  int year;
  int doors;
  float horsepower;
  char model[SIZE+1];
};
```

called a "structure tag"

- Use:

```
struct car mycar;
mycar.year = 2014;
mycar.doors = 4;
mycar.horsepower = 192.0;
strncpy(mycar.model,"Honda Accord", SIZE);
```



Structures

- Arrays are data structures that provide a means to store homogeneous data; structures allow us to create heterogeneous data structures
- The structure car:

```
#define SIZE 50

struct car {
  int year;
  int doors;
  float horsepower;
  char model[SIZE+1];
};
```

called a "structure tag"

- Use:

```
struct car mycar;
mycar.year = 2014;
mycar.doors = 4;
mycar.horsepower = 192.0;
strncpy(mycar.model,"Honda Accord", SIZE);
```



Structures

- Arrays are data structures that provide a means to store homogeneous data; structures allow us to create heterogeneous data structures
- The structure car:

```
#define SIZE 50

struct car {
  int year;
  int doors;
  float horsepower;
  char model[SIZE+1];
};
```

called a "structure tag"

- Use:

```
struct car mycar;
mycar.year = 2014;
mycar.doors = 4;
mycar.horsepower = 192.0;
strncpy(mycar.model,"Honda Accord", SIZE);
```

required with structure tags



Structures *continued*

- Components of structures often called **fields** or **members**
- Fields can be of any type including other structures:

```
struct date {
  int day, month, year;
};

struct funnyStruct {
  struct date timeStamp;
  float info[100];
};

struct funnyStruct fs;
fs.timeStamp.day = 14;
fs.info[24] = 3.198;
```

- Allows for hierarchical data structures



Structures *continued*

- Components of structures often called **fields** or **members**
- Fields can be of any type including other structures:

```
struct date {
  int day, month, year;
};

struct funnyStruct {
  struct date timeStamp;
  float info[100];
};

struct funnyStruct fs;
fs.timeStamp.day = 14;
fs.info[24] = 3.198;
```

- Allows for hierarchical data structures



Structures *continued*

- Components of structures often called **fields** or **members**
- Fields can be of any type including other structures:

```
struct date {
  int day, month, year;
};

struct funnyStruct {
  struct date timeStamp;
  float info[100];
};

struct funnyStruct fs;
fs.timeStamp.day = 14;
fs.info[24] = 3.198;
```

- Allows for hierarchical data structures



Structures *continued*

- Often have arrays of structures:

```
struct date { int day, month, year; };

struct widget {
  float model;
  float cost;
  int inventory;
  struct date added;
};

struct widget stock[1000];

stock[212].cost = 0.21;
stock[212].added.year = 2003;
```



Structures *continued*

```
#include <stdio.h>

struct date { int day, month, year; };

void f1(struct date *d1, struct date *d2) {
  (*d1).day = 20; (*d1).month = 6; (*d1).year = 1974;
  (*d2).day = 20; (*d2).month = 6; (*d2).year = 1974;
}

struct date f2(int d, int m, int y) {
  struct date temp;
  temp.day = d; temp.month = m; temp.year = y;
  return temp;
}

int main() {
  struct date day1, day2, day3;
  day1.day = 5; day1.month = 9; day1.year = 2012;
  day2.day = 5; day2.month = 9; day2.year = 2012;
  f1(&day1,&day2);
  printf("%d-%d-%d\n",day1.month,day1.day,day1.year);
  printf("%d-%d-%d\n",day2.month,day2.day,day2.year);
  day3 = f2(12,4,2018);
  printf("%d-%d-%d\n",day3.month,day3.day,day3.
}
```



Structures *continued*

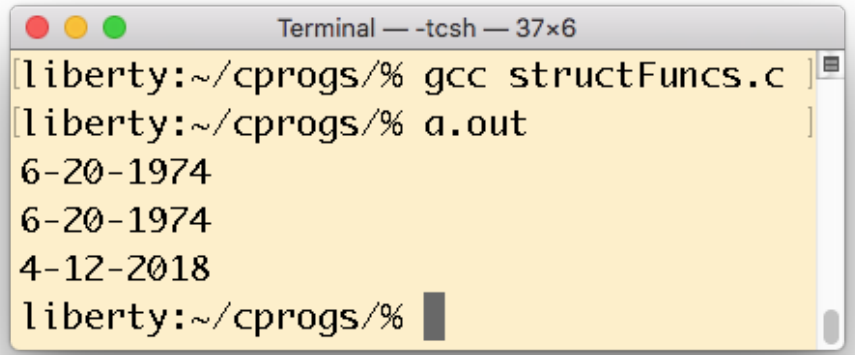
```
#include <stdio.h>

struct date { int day, month, year; };

void f1(struct date *d1, struct date *d2) {
  (*d1).day = 20; (*d1).month = 6; (*d1).year = 1974;
  (*d2).day = 20; (*d2).month = 6; (*d2).year = 1974;
}

struct date f2(int d, int m, int y) {
  struct date temp;
  temp.day = d; temp.month = m; temp.year = y;
  return temp;
}

int main() {
  struct date day1, day2, day3;
  day1.day = 5; day1.month = 9; day1.year = 2012;
  day2.day = 5; day2.month = 9; day2.year = 2012;
  f1(&day1,&day2);
  printf("%d-%d-%d\n",day1.month,day1.day,day1.year);
  printf("%d-%d-%d\n",day2.month,day2.day,day2.year);
  day3 = f2(12,4,2018);
  printf("%d-%d-%d\n",day3.month,day3.day,day3.
}
```



Structures *continued*

- As an alternative to declaring a structure tag, we can use `typedef` to define a genuine type name
- A definition of a type named `Part`:

```
typedef struct {
  int number;
  char name[NAME_LEN+1];
  int on_hand;
} Part;
```

- `Part` can be used in the same way as the built-in types:

```
Part part1, part2;
```



Structures *continued*

- As an alternative to declaring a structure tag, we can use `typedef` to define a genuine type name
- A definition of a type named `Part`:

```
typedef struct {
  int number;
  char name[NAME_LEN+1];
  int on_hand;
} Part;
```

- `Part` can be used in the same way as the built-in types:

```
Part part1, part2;
```



Structures *continued*

- As an alternative to declaring a structure tag, we can use `typedef` to define a genuine type name
- A definition of a type named `Part`:

```
typedef struct {
  int number;
  char name[NAME_LEN+1];
  int on_hand;
} Part;
```

- `Part` can be used in the same way as the built-in types:

```
Part part1, part2;
```



Structures *continued*

- When it comes time to name a structure, we can usually choose either to declare a structure tag or to use typedef
- However, declaring a structure tag is mandatory when the structure is to be used in a linked list



Structures *continued*

- When it comes time to name a structure, we can usually choose either to declare a structure tag or to use `typedef`
- However, declaring a structure tag is mandatory when the structure is to be used in a linked list



Dynamic Variables

- When speaking about programming languages the word **dynamic** usually refers to that which happens at run-time (as contrasted with compile-time)
- Therefore a **dynamic variable** is a variable that is created at run-time
- Most variables we've seen up to this point are defined at compile-time and cannot change at run-time
- For example:

```
int x;
```

At run-time how would we associate a new memory location with variable x?



Dynamic Variables

- When speaking about programming languages the word **dynamic** usually refers to that which happens at run-time (as contrasted with compile-time)
- Therefore a **dynamic variable** is a variable that is created at run-time
- Most variables we've seen up to this point are defined at compile-time and cannot change at run-time
- For example:

```
int x;
```

At run-time how would we associate a new memory location with variable x?



Dynamic Variables

- When speaking about programming languages the word **dynamic** usually refers to that which happens at run-time (as contrasted with compile-time)
- Therefore a **dynamic variable** is a variable that is created at run-time
- Most variables we've seen up to this point are defined at compile-time and cannot change at run-time
- For example:

```
int x;
```

At run-time how would we associate a new memory location with variable x?



Dynamic Variables

- When speaking about programming languages the word **dynamic** usually refers to that which happens at run-time (as contrasted with compile-time)
- Therefore a **dynamic variable** is a variable that is created at run-time
- Most variables we've seen up to this point are defined at compile-time and cannot change at run-time
- For example:

```
int x;
```

  At run-time how would we associate a new memory location with variable x?



Dynamic Variables

- When speaking about programming languages the word **dynamic** usually refers to that which happens at run-time (as contrasted with compile-time)
- Therefore a **dynamic variable** is a variable that is created at run-time
- Most variables we've seen up to this point are defined at compile-time and cannot change at run-time
- For example:

```
int x;
```

  At run-time how would we associate a new memory location with variable x? We can't!



Dynamic Variables *continued*

- But wait – I have created and used dynamic arrays in gcc!

```
#include <stdio.h>
int main(void){
  int size,i;
  printf("size of array? ");
  scanf("%d",&size);
  int list[size];
  for(i=0; i<size; i++)
    list[i] = i*2;
  for(i=0; i<size; i++)
    printf("%d ",list[i]);
  printf("\n");
}
```

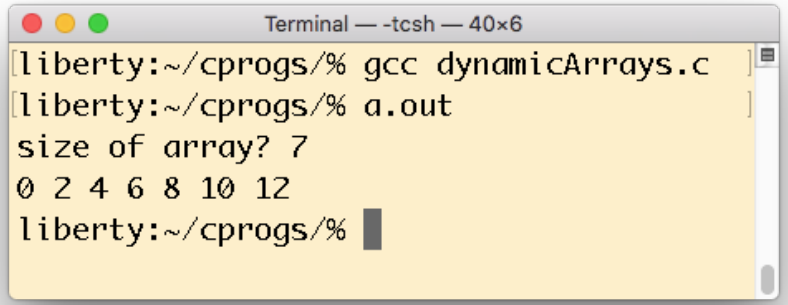
- Not part of K&R C
- However, the GCC compilers support dynamic arrays



Dynamic Variables *continued*

- But wait – I have created and used dynamic arrays in gcc!

```
#include <stdio.h>
int main(void){
  int size,i;
  printf("size of array? ");
  scanf("%d",&size);
  int list[size];
  for(i=0; i<size; i++)
    list[i] = i*2;
  for(i=0; i<size; i++)
    printf("%d ",list[i]);
  printf("\n");
}
```

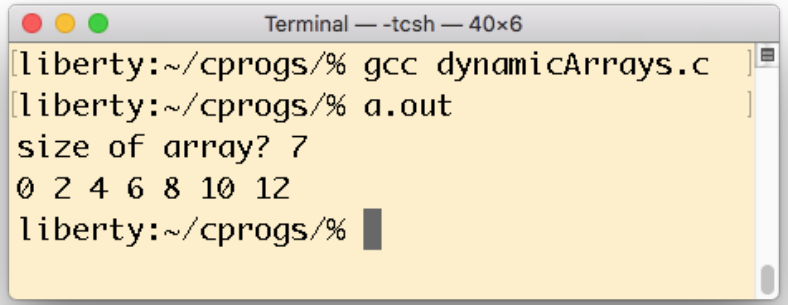


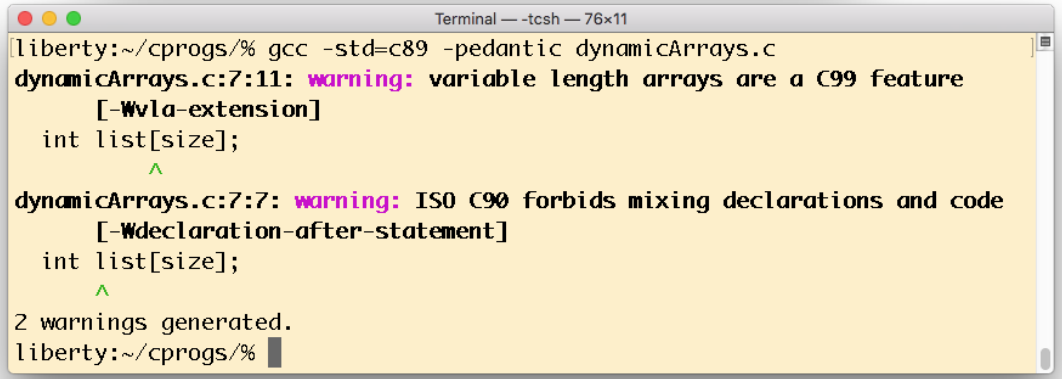
- Not part of K&R C
- However, the GCC compilers support dynamic arrays

Dynamic Variables *continued*

- But wait – I have created and used dynamic arrays in gcc!

```
#include <stdio.h>
int main(void){
  int size,i;
  printf("size of array? ");
  scanf("%d",&size);
  int list[size];
  for(i=0; i<size; i++)
    list[i] = i*2;
  for(i=0; i<size; i++)
    printf("%d ",list[i]);
  printf("\n");
}
```



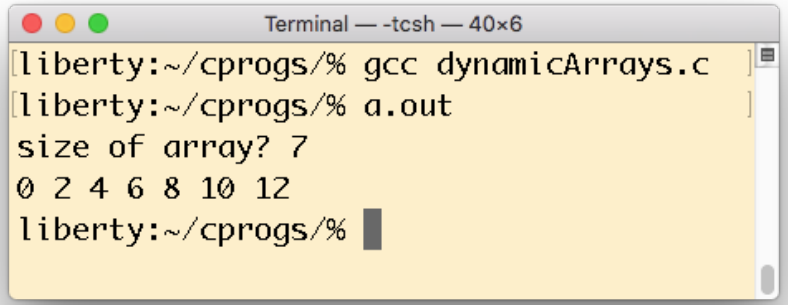


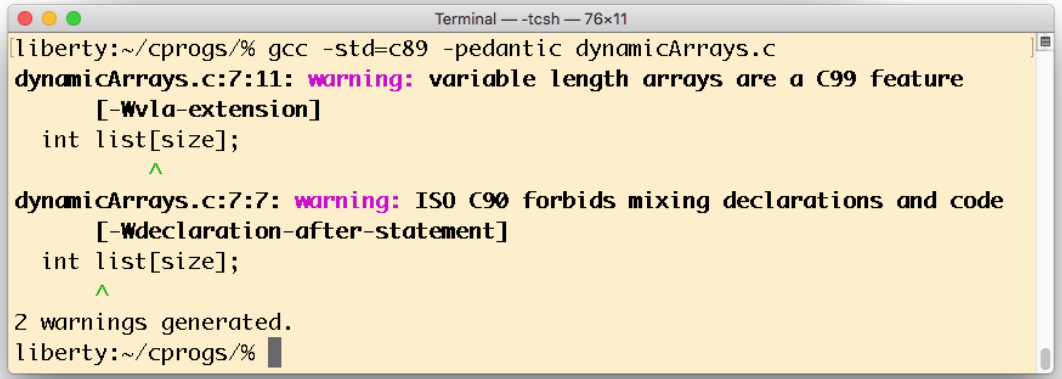
- Not part of K&R C
- However, the GCC compilers support dynamic arrays

Dynamic Variables *continued*

- But wait – I have created and used dynamic arrays in gcc!

```
#include <stdio.h>
int main(void){
  int size,i;
  printf("size of array? ");
  scanf("%d",&size);
  int list[size];
  for(i=0; i<size; i++)
    list[i] = i*2;
  for(i=0; i<size; i++)
    printf("%d ",list[i]);
  printf("\n");
}
```





- Not part of K&R C
- However, the GCC compilers support dynamic arrays

Dynamic Variables *continued*

- We are going to explore a different kind of dynamic varible
- Dynamic variables are not declared at compile-time – they are created at run-time and do not have a name
- You access dynamic variables using a pointer



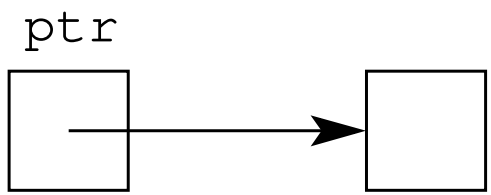
Dynamic Variables *continued*

- We are going to explore a different kind of dynamic varible
- Dynamic variables are not declared at compile-time – they are created at run-time and do not have a name
- You access dynamic variables using a pointer



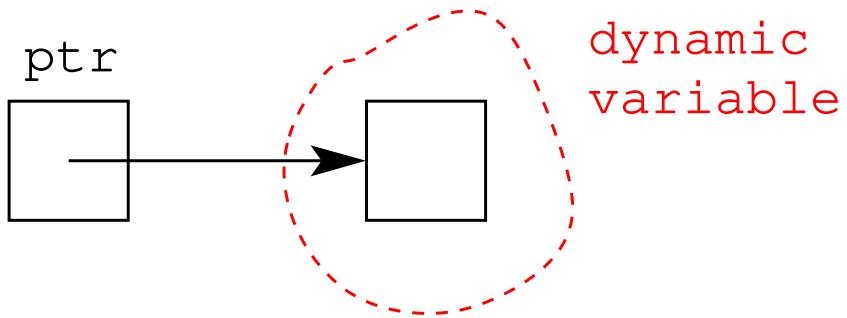
Dynamic Variables *continued*

- We are going to explore a different kind of dynamic varible
- Dynamic variables are not declared at compile-time – they are created at run-time and do not have a name
- You access dynamic variables using a pointer



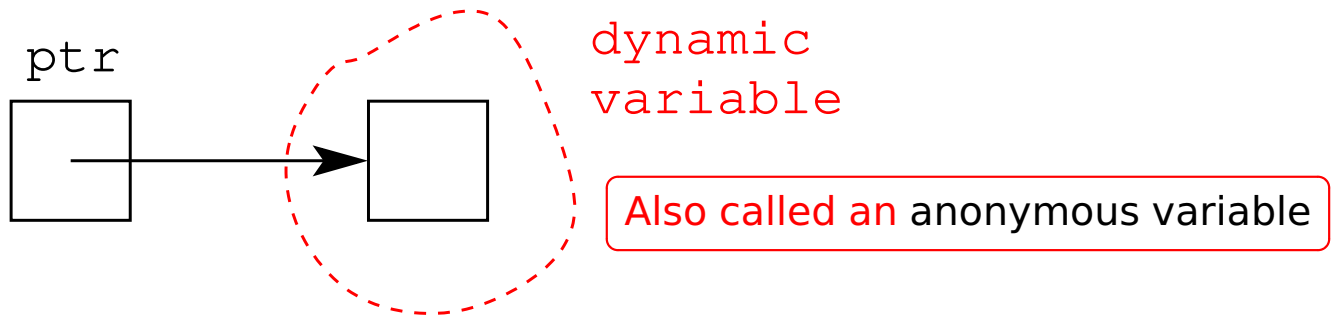
Dynamic Variables *continued*

- We are going to explore a different kind of dynamic varible
- Dynamic variables are not declared at compile-time – they are created at run-time and do not have a name
- You access dynamic variables using a pointer



Dynamic Variables *continued*

- We are going to explore a different kind of dynamic varible
- Dynamic variables are not declared at compile-time – they are created at run-time and do not have a name
- You access dynamic variables using a pointer



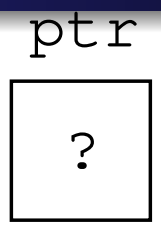
Using malloc

```
int *ptr;
```



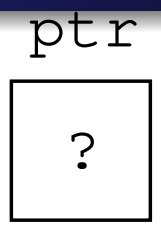
Using malloc

```
int *ptr;
```



Using malloc

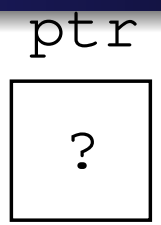
```
int *ptr;
```



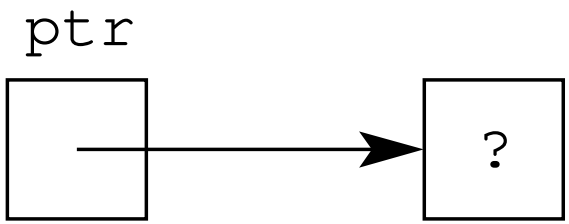

```
ptr = (int*) malloc(sizeof(int));
```

Using malloc

```
int *ptr;
```

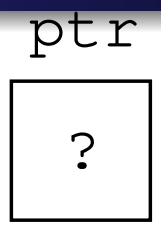



```
ptr = (int*) malloc(sizeof(int));
```

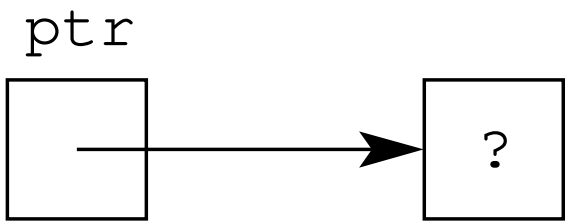


Using malloc

```
int *ptr;
```



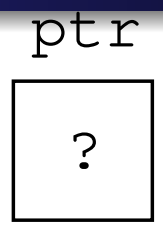

```
ptr = (int*) malloc(sizeof(int));
```



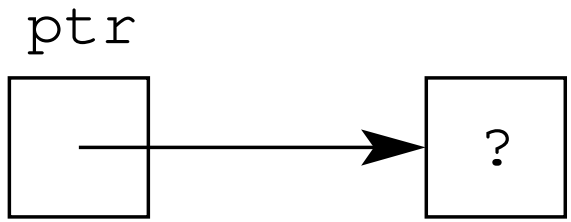

```
*ptr = 2;
```

Using malloc

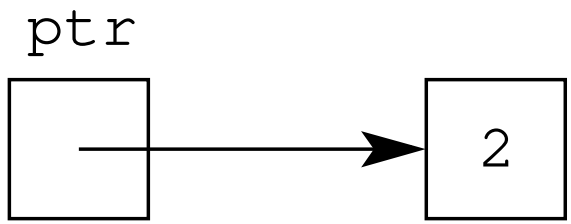
```
int *ptr;
```




```
ptr = (int*) malloc(sizeof(int));
```

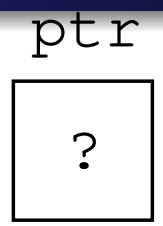



```
*ptr = 2;
```

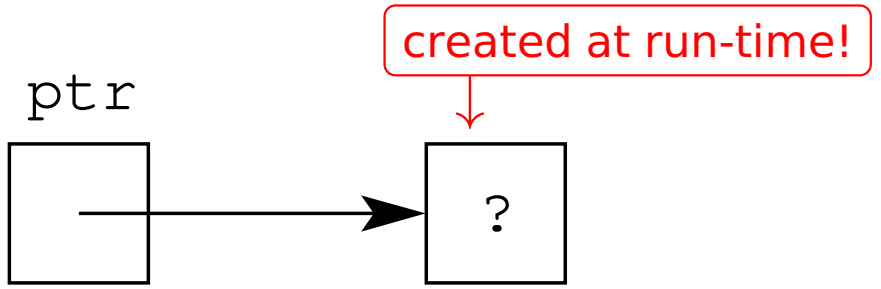


Using malloc

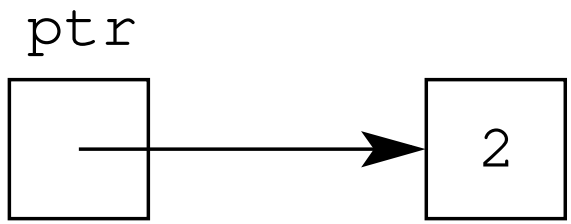
```
int *ptr;
```




```
ptr = (int*) malloc(sizeof(int));
```




```
*ptr = 2;
```



Using `malloc` *continued*

- malloc returns a void pointer

```
void* malloc( size_t size );
```

- Therefore you should cast the value returned by `malloc` to the appropriate pointer type

```
float *fptr;
char *cptr;
fptr = (float*) malloc(sizeof(float));
cptr = (char*) malloc(sizeof(char));
```



Using `malloc` *continued*

- malloc returns a void pointer

```
void* malloc( size_t size );
```

- Therefore you should cast the value returned by `malloc` to the appropriate pointer type

```
float *fptr;
char *cptr;
fptr = (float*) malloc(sizeof(float));
cptr = (char*) malloc(sizeof(char));
```



NULL Pointers

- The NULL pointer in C is a pointer value compatible with all pointer types that indicates the pointer is pointing to nothing
- Used primarily to:
  - Initialize a pointer variable to point to nothing
  - Indicate the end of a pointer data structure (such as a linked list)
  - Indicate a function failed
- Use:

```
int *ptr = NULL;
// ...
if (ptr != NULL)
  // do something
```

- Dereferencing a NULL pointer is undefined in C and will usually cause your program to crash



NULL Pointers

- The NULL pointer in C is a pointer value compatible with all pointer types that indicates the pointer is pointing to nothing
- Used primarily to:
  - initialize a pointer variable to point to nothing
  - to check if a pointer variable is pointing to a non-NULL (therefore possibly valid) address
- Use:

```
int *ptr = NULL;
// ...
if (ptr != NULL)
  // do something
```

- Dereferencing a NULL pointer is undefined in C and will usually cause your program to crash



NULL Pointers

- The NULL pointer in C is a pointer value compatible with all pointer types that indicates the pointer is pointing to nothing
- Used primarily to:
  - initialize a pointer variable to point to nothing
  - to check if a pointer variable is pointing to a non-NULL (therefore possibly valid) address
- Use:

```
int *ptr = NULL;
// ...
if (ptr != NULL)
  // do something
```

- Dereferencing a NULL pointer is undefined in C and will usually cause your program to crash



NULL Pointers

- The NULL pointer in C is a pointer value compatible with all pointer types that indicates the pointer is pointing to nothing
- Used primarily to:
  - initialize a pointer variable to point to nothing
  - to check if a pointer variable is pointing to a non-NULL (therefore possibly valid) address
- Use:

```
int *ptr = NULL;
// ...
if (ptr != NULL)
  // do something
```

- Dereferencing a NULL pointer is undefined in C and will usually cause your program to crash



NULL Pointers

- The NULL pointer in C is a pointer value compatible with all pointer types that indicates the pointer is pointing to nothing
- Used primarily to:
  - initialize a pointer variable to point to nothing
  - to check if a pointer variable is pointing to a non-NULL (therefore possibly valid) address
- Use:

```
int *ptr = NULL;
// ...
if (ptr != NULL)
  // do something
```

- Dereferencing a NULL pointer is undefined in C and will usually cause your program to crash



NULL Pointers

- The NULL pointer in C is a pointer value compatible with all pointer types that indicates the pointer is pointing to nothing
- Used primarily to:
  - initialize a pointer variable to point to nothing
  - to check if a pointer variable is pointing to a non-NULL (therefore possibly valid) address
- Use:

```
int *ptr = NULL;
// ...
if (ptr != NULL)
  // do something
```

- Dereferencing a NULL pointer is undefined in C and will usually cause your program to crash



Using malloc for Dynamic Arrays

- Works as one would expect:

```
#define SIZE 100
int *list = (int *) malloc(sizeof(int)*SIZE);
```

- malloc allocates a *continuous* block of memory
- Example of use:

```
#include <stdio.h>
#include <stdlib.h>

#define LIST_SIZE 10

int main() {
  int *list = (int *) malloc(sizeof(int)*LIST_SIZE),i;
  for (i=0; i < LIST_SIZE; i++) scanf("%d ",&list[i]);
  for (i=0; i < LIST_SIZE; i++) printf("%d ",list[i]);
  printf("\n");
}
```



Using malloc for Dynamic Arrays

- Works as one would expect:

```
#define SIZE 100
int *list = (int *) malloc(sizeof(int)*SIZE);
```

- `malloc` allocates a *continuous* block of memory
- Example of use:

```
#include <stdio.h>
#include <stdlib.h>

#define LIST_SIZE 10

int main() {
  int *list = (int *) malloc(sizeof(int)*LIST_SIZE),i;
  for (i=0; i < LIST_SIZE; i++) scanf("%d ",&list[i]);
  for (i=0; i < LIST_SIZE; i++) printf("%d ",list[i]);
  printf("\n");
}
```



Using malloc for Dynamic Arrays

- Works as one would expect:

```
#define SIZE 100
int *list = (int *) malloc(sizeof(int)*SIZE);
```

- `malloc` allocates a *continuous* block of memory
- Example of use:

```
#include <stdio.h>
#include <stdlib.h>

#define LIST_SIZE 10

int main() {
  int *list = (int *) malloc(sizeof(int)*LIST_SIZE),i;
  for (i=0; i < LIST_SIZE; i++) scanf("%d ",&list[i]);
  for (i=0; i < LIST_SIZE; i++) printf("%d ",list[i]);
  printf("\n");
}
```



Using malloc for Dynamic Arrays *continued*

- Using `malloc` for storing a string:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_STRING_SIZE 100

int main() {
  char *str = (char *) malloc(sizeof(char)*MAX_STRING_SIZE+1);
  printf("string? ");
  scanf("%s",str);
  printf("string entered = %s\n",str);
}
```



Using malloc for Structures

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define NAME_SIZE 100

typedef struct {
  int number;
  char name[NAME_SIZE+1];
  int on_hand;
} Part;

int main() {
  Part *p = (Part *) malloc(sizeof(Part));
  (*p).number = 10221; strcpy((*p).name,"widget 5"); (*p).on_hand = 1200;
  printf("%d: %s (# in stock: %d)\n",(*p).number,(*p).name,(*p).on_hand);
}
```



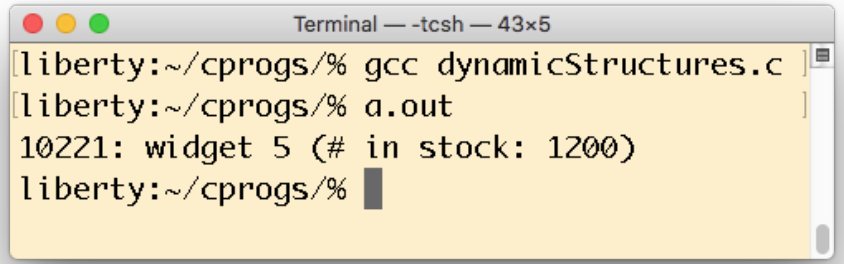
Using malloc for Structures

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define NAME_SIZE 100

typedef struct {
  int number;
  char name[NAME_SIZE+1];
  int on_hand;
} Part;

int main() {
  Part *p = (Part *) malloc(sizeof(Part));
  (*p).number = 10221; strcpy((*p).name,"widget 5"); (*p).on_hand = 1200;
  printf("%d: %s (# in stock: %d)\n",(*p).number,(*p).name,(*p).on_hand);
}
```



The Arrow Operator

- Note that because the `.` operator has higher precedence than the `*` dereference operator you must always use parenthesis when accessing a structure field with a pointer, e.g. from the previous example:

```
(*p).number
```

- Because having to always use these parentheses is error prone, the arrow opertaor `->` was introduced:

```
p->number  ≡  (*p).number
```



The Arrow Operator

- Note that because the `.` operator has higher precedence than the `*` dereference operator you must always use parenthesis when accessing a structure field with a pointer, e.g. from the previous example:

```
(*p).number
```

What would `*p.number` mean?

- Because having to always use these parentheses is error prone, the arrow opertaor `->` was introduced:

```
p->number  ≡  (*p).number
```



The Arrow Operator

- Note that because the `.` operator has higher precedence than the `*` dereference operator you must always use parenthesis when accessing a structure field with a pointer, e.g. from the previous example:

  `(*p).number`

  What would `*p.number` mean?

- Because having to always use these parentheses is error prone, the arrow opertaor `->` was introduced:

  `p->number` $\equiv$ `(*p).number`



The Arrow Operator *continued*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define NAME_SIZE 100

typedef struct {
  int number;
  char name[NAME_SIZE+1];
  int on_hand;
} Part;

int main() {
  Part *p = (Part *) malloc(sizeof(Part));
  p->number = 10221; strcpy(p->name,"widget 5"); p->on_hand = 1200;
  printf("%d: %s (# in stock: %d)\n",p->number,p->name,p->on_hand);
}
```



Deallocating Memory

- Most modern languages automatically manage their memory with a system called a **garbage collector**
- A garbage collector program runs in the background while your code executes locating memory no longer being used by your program and then reclaims it making it available once again
- C does not provide automatic garbage collection; therefore the programmer must manage their own memory
- The language provides a way to deallocate all dynamically allocated memory (i.e. memory allocated in your code with `malloc`) when it is no longer needed



Deallocating Memory

- Most modern languages automatically manage their memory with a system called a **garbage collector**
- A garbage collector program runs in the background while your code executes locating memory no longer being used by your program and then reclaims it making it available once again
- C does not provide automatic garbage collection; therefore the programmer must manage their own memory
- The language provides a way to deallocate all dynamically allocated memory (i.e. memory allocated in your code with `malloc`) when it is no longer needed



Deallocating Memory

- Most modern languages automatically manage their memory with a system called a **garbage collector**
- A garbage collector program runs in the background while your code executes locating memory no longer being used by your program and then reclaims it making it available once again
- C does not provide automatic garbage collection; therefore the programmer must manage their own memory
- The language provides a way to deallocate all dynamically allocated memory (i.e. memory allocated in your code with `malloc`) when it is no longer needed



Deallocating Memory

- Most modern languages automatically manage their memory with a system called a **garbage collector**
- A garbage collector program runs in the background while your code executes locating memory no longer being used by your program and then reclaims it making it available once again
- C does not provide automatic garbage collection; therefore the programmer must manage their own memory
- The language provides a way to deallocate all dynamically allocated memory (i.e. memory allocated in your code with `malloc`) when it is no longer needed



Deallocating Memory *continued*

- C provides the `free` operator to deallocate memory that has been previously allocated by `malloc`
- Usage:

```
int *p1 = (int*) malloc(sizeof(int));
// ...
// assume the var pointed to by p1 is no longer needed, so:
free(p1);
```

  - If `p1` does not point to a block of memory allocated with `malloc` (or equivalent), it causes undefined behavior
- If memory management is important for your code, **it's the programmer's responsibilty** in C to free up all memory they no longer need



Deallocating Memory *continued*

- C provides the `free` operator to deallocate memory that has been previously allocated by `malloc`
- Usage:

```
int *p1 = (int*) malloc(sizeof(int));
// ...
// assume the var pointed to by p1 is no longer needed, so:
free(p1);
```

  - if `p1` does not point to a block of memory allocated with `malloc` (or equivalent), it causes undefined behavior
  - if `p1` is a null pointer, `free` does nothing
  - `free` does not change the value of `p1` itself – it still points to the same (now invalid) location

- If memory management is important for your code, **it's the programmer's responsibilty** in C to free up all memory they no longer need



Deallocating Memory *continued*

- C provides the `free` operator to deallocate memory that has been previously allocated by `malloc`
- Usage:

```
int *p1 = (int*) malloc(sizeof(int));
// ...
// assume the var pointed to by p1 is no longer needed, so:
free(p1);
```

  - if `p1` does not point to a block of memory allocated with `malloc` (or equivalent), it causes undefined behavior
  - if `p1` is a null pointer, `free` does nothing
  - `free` does not change the value of `p1` itself – it still points to the same (now invalid) location

- If memory management is important for your code, **it's the programmer's responsibilty** in C to free up all memory they no longer need



Deallocating Memory *continued*

- C provides the `free` operator to deallocate memory that has been previously allocated by `malloc`
- Usage:

```
int *p1 = (int*) malloc(sizeof(int));
// ...
// assume the var pointed to by p1 is no longer needed, so:
free(p1);
```

  - if `p1` does not point to a block of memory allocated with `malloc` (or equivalent), it causes undefined behavior
  - if `p1` is a null pointer, `free` does nothing
  - `free` does not change the value of `p1` itself – it still points to the same (now invalid) location

- If memory management is important for your code, **it's the programmer's responsibilty** in C to free up all memory they no longer need



Deallocating Memory *continued*

- C provides the `free` operator to deallocate memory that has been previously allocated by `malloc`
- Usage:

```
int *p1 = (int*) malloc(sizeof(int));
// ...
// assume the var pointed to by p1 is no longer needed, so:
free(p1);
```

  - if `p1` does not point to a block of memory allocated with `malloc` (or equivalent), it causes undefined behavior
  - if `p1` is a null pointer, `free` does nothing
  - `free` does not change the value of `p1` itself – it still points to the same (now invalid) location

- If memory management is important for your code, **it's the programmer's responsibilty** in C to free up all memory they no longer need



Deallocating Memory *continued*

- C provides the `free` operator to deallocate memory that has been previously allocated by `malloc`
- Usage:

```
int *p1 = (int*) malloc(sizeof(int));
// ...
// assume the var pointed to by p1 is no longer needed, so:
free(p1);
```

  - if `p1` does not point to a block of memory allocated with `malloc` (or equivalent), it causes undefined behavior
  - if `p1` is a null pointer, `free` does nothing
  - `free` does not change the value of `p1` itself – it still points to the same (now invalid) location
- If memory management is important for your code, **it's the programmer's responsibilty** in C to free up all memory they no longer need



How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . .



How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . .



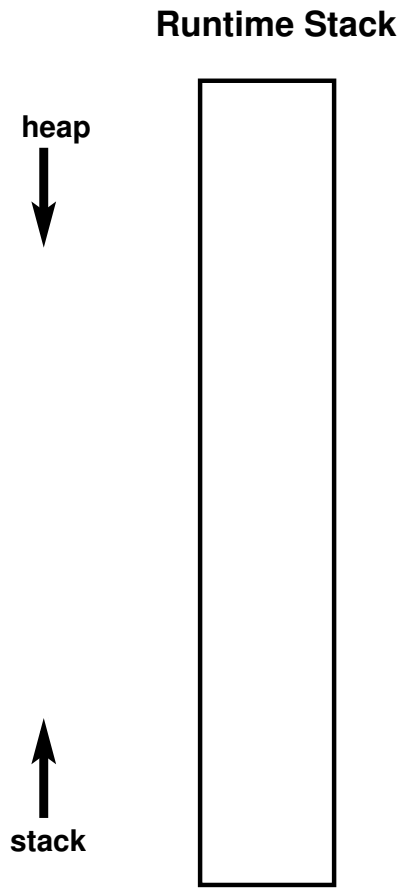
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . .



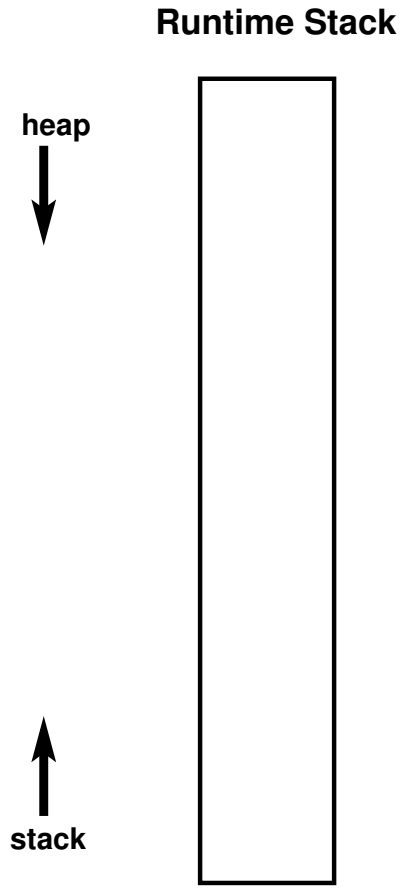
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . .



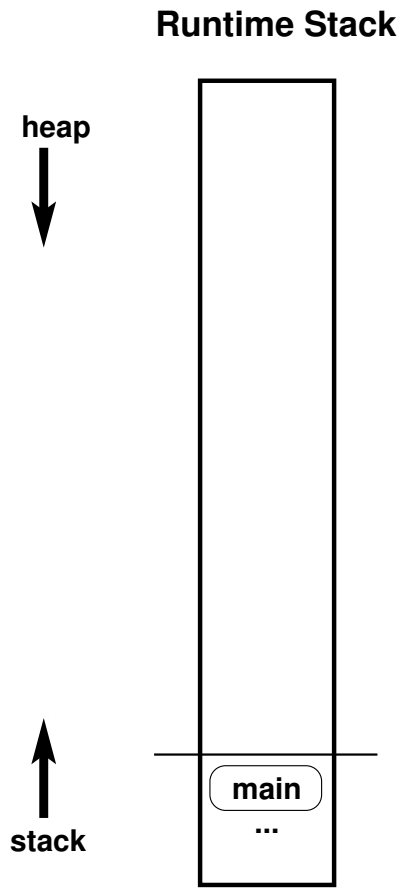
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . .



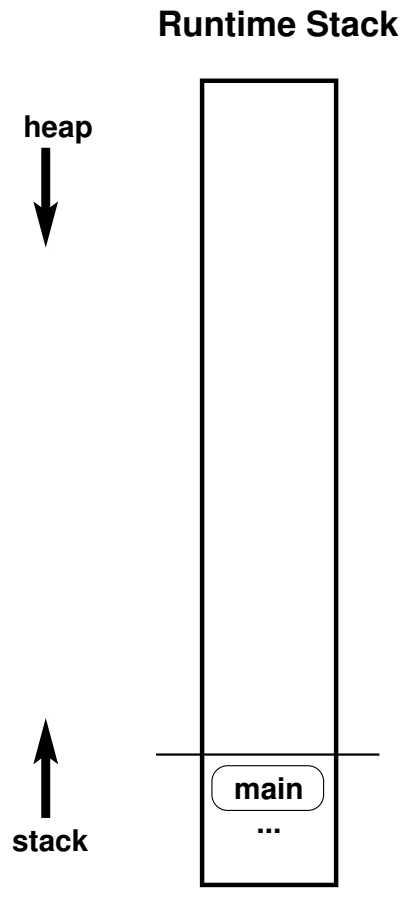
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . .



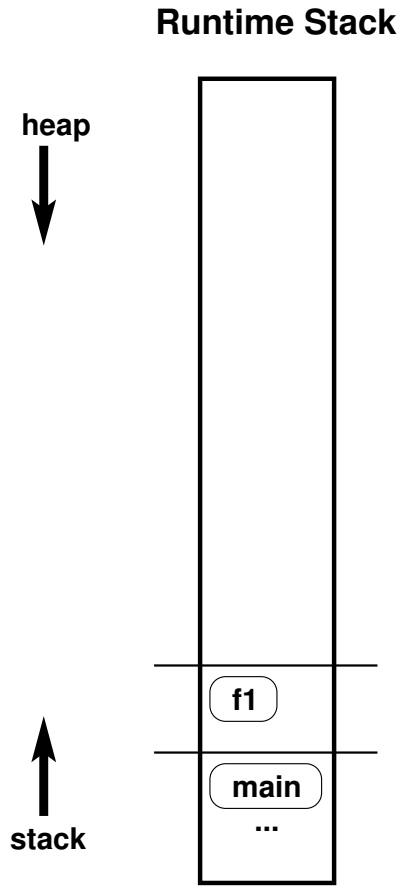
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . .



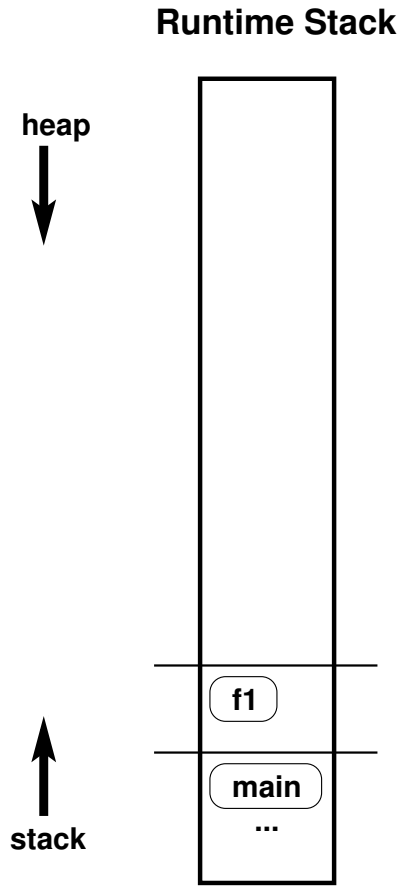
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . .



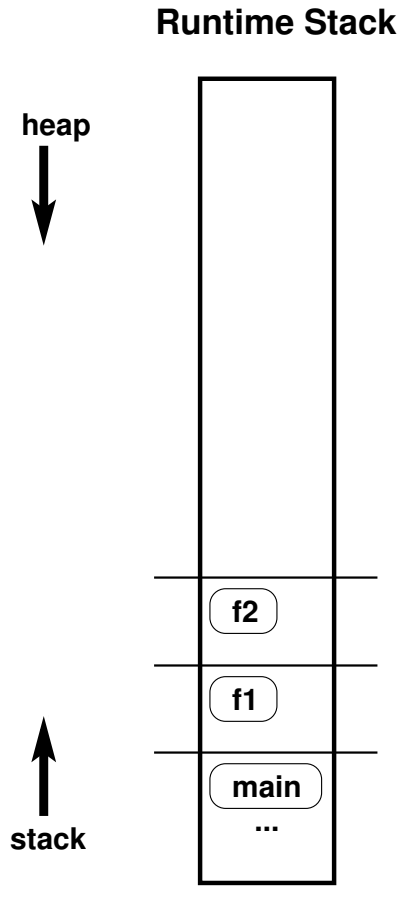
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . . in `f1` call `f2`. . .



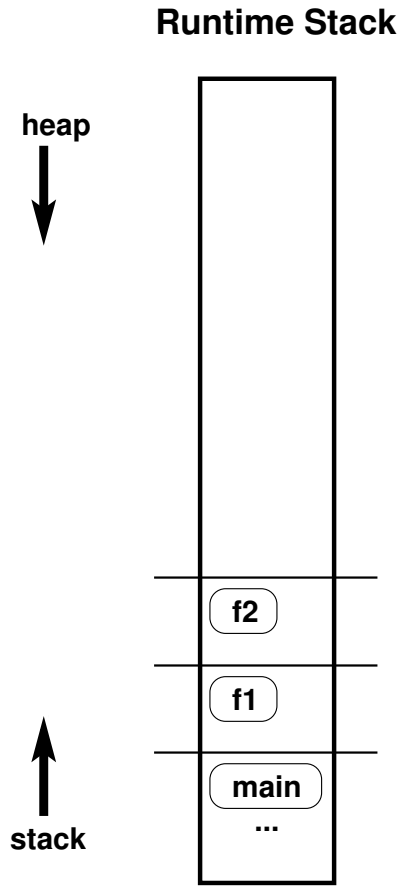
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . . in `f1` call `f2`. . .



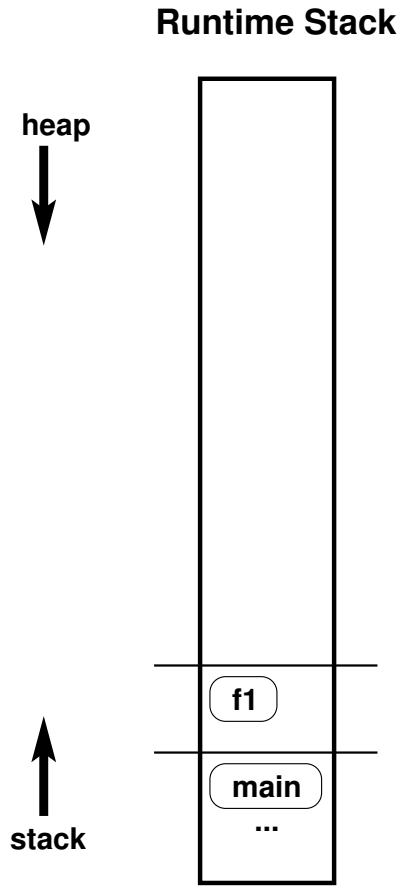
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . . in `f1` call `f2`. . . now exit `f2`. . .



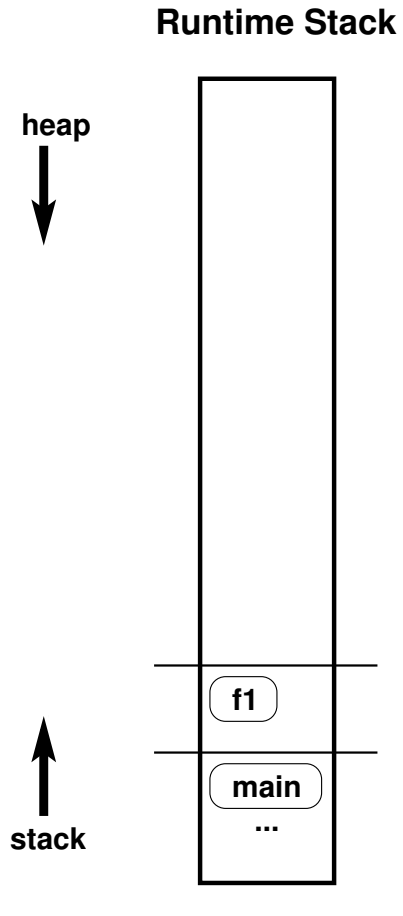
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . . in `f1` call `f2`. . . now exit `f2`. . .



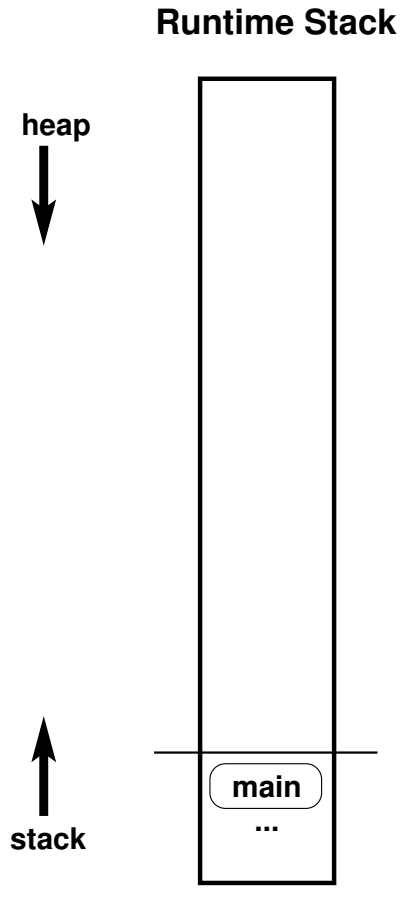
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . . in `f1` call `f2`. . . now exit `f2`. . . exit `f1`. . .



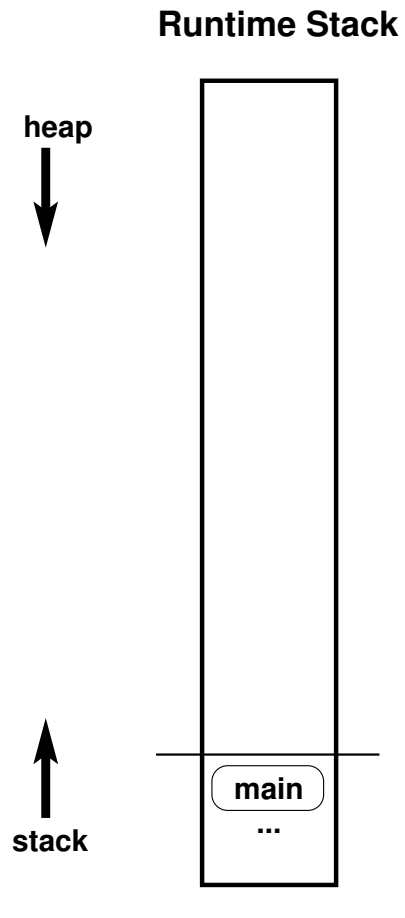
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . . in `f1` call `f2`. . . now exit `f2`. . . exit `f1`. . .



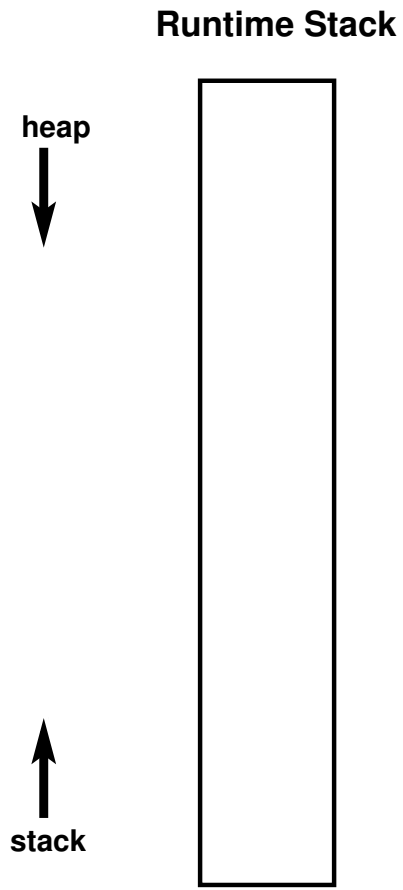
How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . . in `f1` call `f2`. . . now exit `f2`. . . exit `f1`. . . exit program (`main`)



How Dynamic Memory Works

- When your programs runs the runtime system creates a **runtime stack** of **activation records** (also called **frames**)
- Everytime a function is called (this includes `main`) a frame is pushed upon this runtime stack
- The frame contains local variables and other control information; dynamic variables (those created by `malloc`) will live on the **heap**
- So when your program begins the runtime system looks like this . . .
- Now your program begins and `main` is called. . . now in `main` call `f1` . . . in `f1` call `f2`. . . now exit `f2`. . . exit `f1`. . . exit program (`main`)



How Dynamic Memory Works *continued*

- Let's execute the following code:

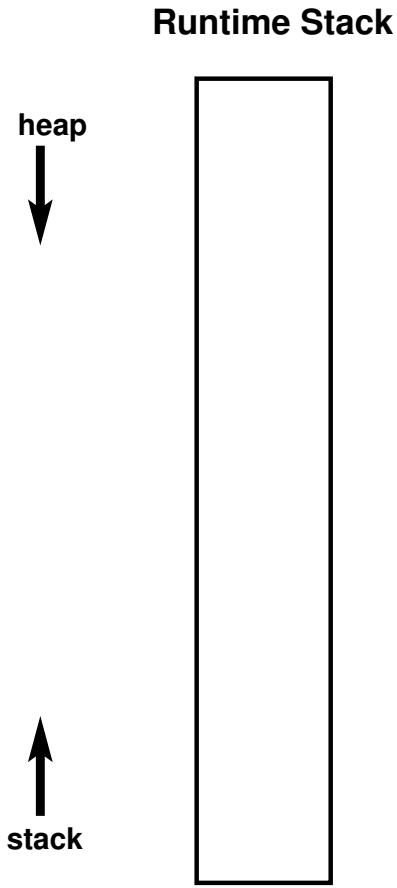
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

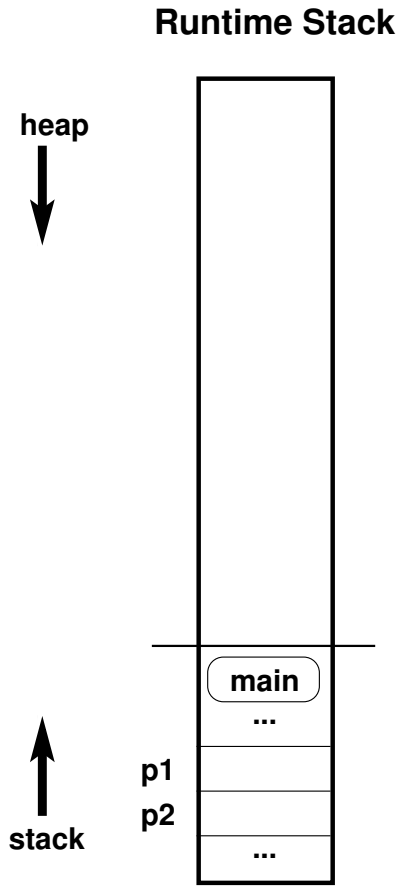
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

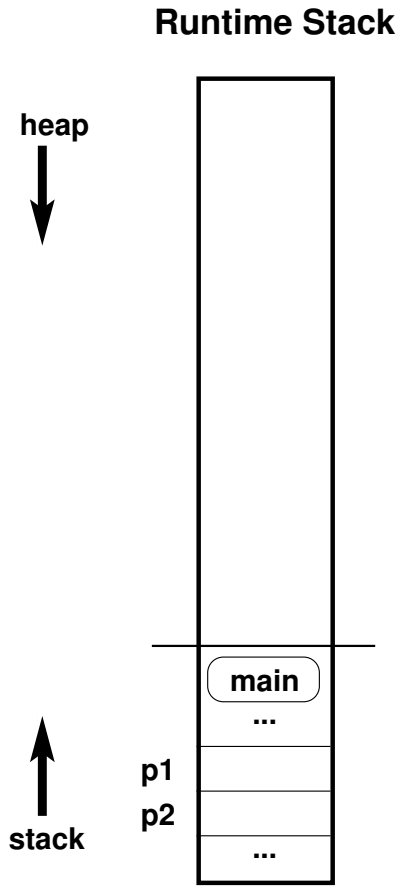
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

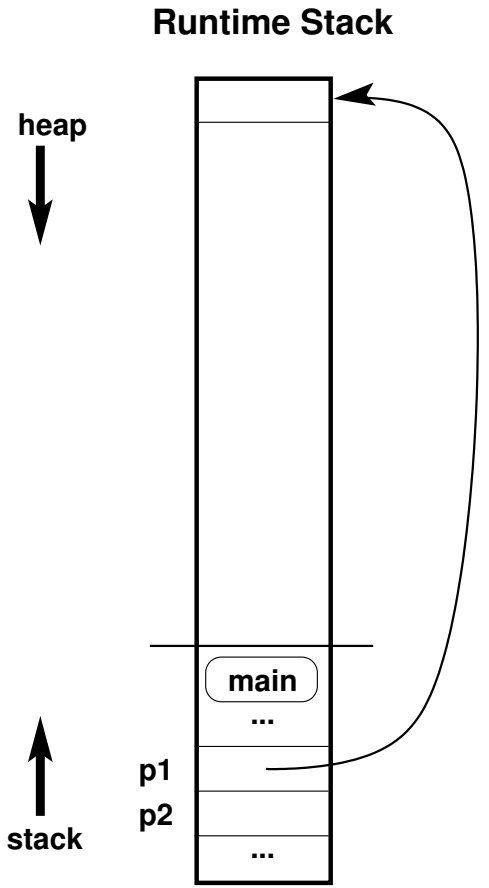
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

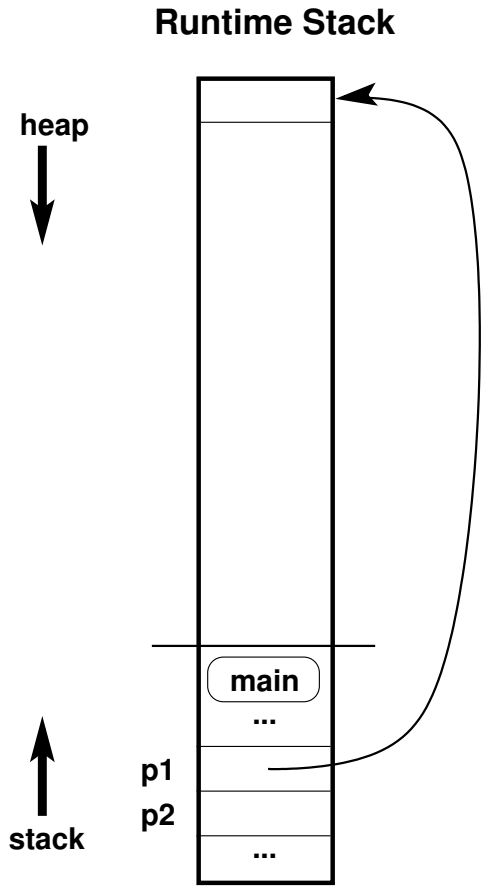
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

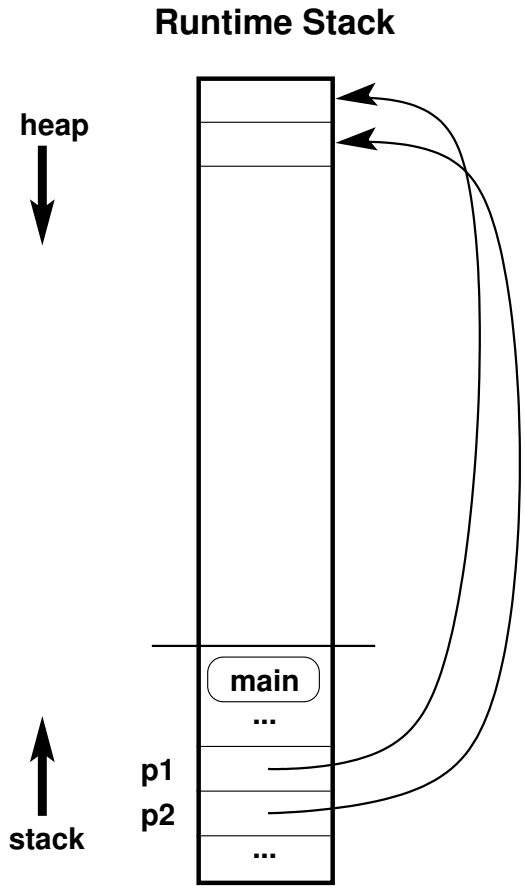
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

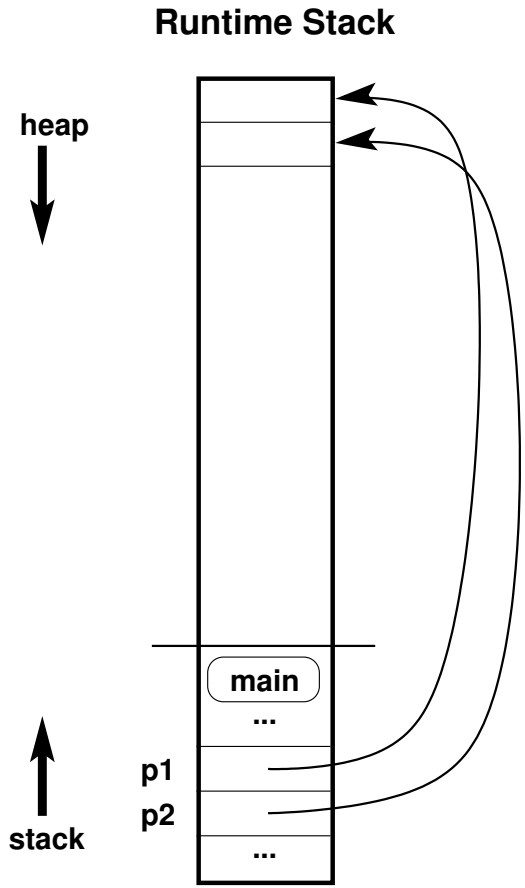
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

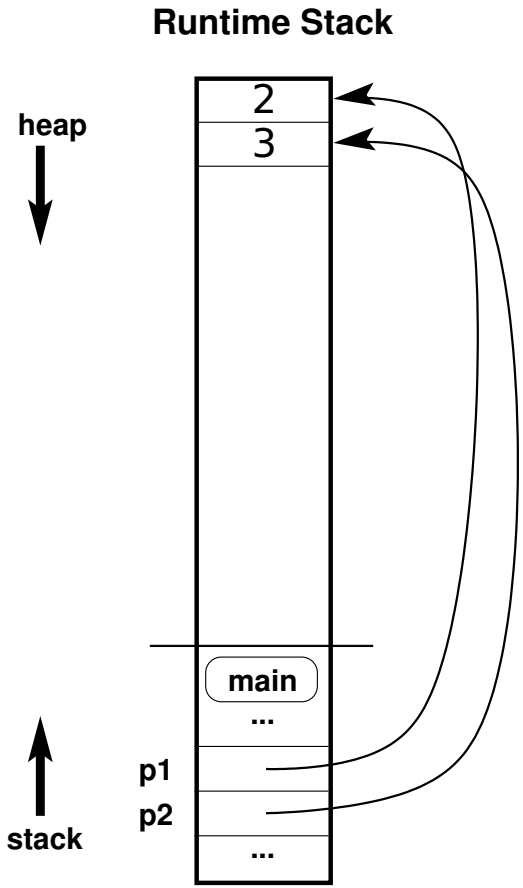
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

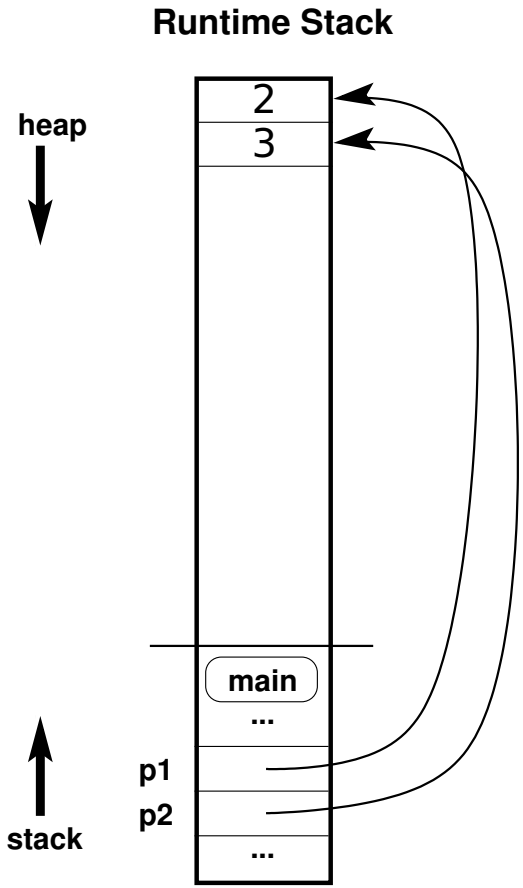
```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```



How Dynamic Memory Works *continued*

- Let's execute the following code:

```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```

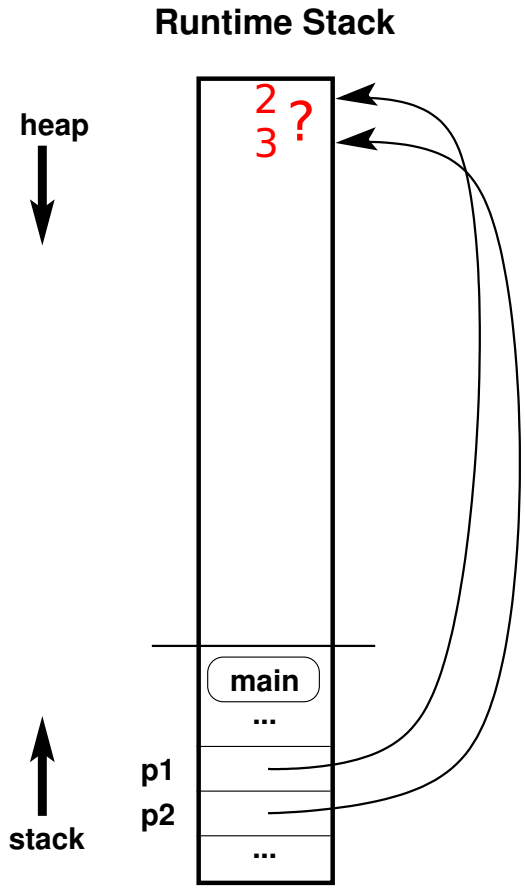


How Dynamic Memory Works *continued*

- Let's execute the following code:

```
int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  p2 = (int *) malloc(sizeof(int));
  *p1=2; *p2=3;
  free(p1); free(p2);
}
```

- **Note `p1` & `p2` are not changed by the `free` operations!**
- **After the free operations, the anonymous variables pointed to by `p1` & `p2` are no longer considered "alive"**
- **Yet they can still be accessed via the pointers!**
- **This creates something called a dangling pointer – it should be avoided as it creates undefined behavior**



How Dynamic Memory Works *continued*

- Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  *p1=2;
  printf("*p1 before free: %d\n",*p1);
  free(p1);
  printf("*p1 after free: %d\n",*p1); // *p1 value undefined!
  p2 = (int *) malloc(sizeof(int));
  *p2=1000;
  printf("*p1 after p2 allocated: %d\n",*p1);
}
```

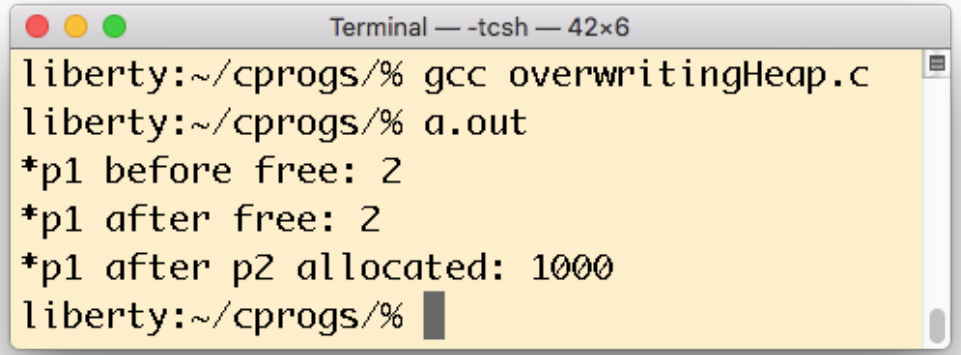


How Dynamic Memory Works *continued*

- Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
  int *p1,*p2;
  p1 = (int *) malloc(sizeof(int));
  *p1=2;
  printf("*p1 before free: %d\n",*p1);
  free(p1);
  printf("*p1 after free: %d\n",*p1); // *p1 value undefined!
  p2 = (int *) malloc(sizeof(int));
  *p2=1000;
  printf("*p1 after p2 allocated: %d\n",*p1);
}
```



How Dynamic Memory Works *continued*

- If a program's memory is not managed properly it is possible to create allocated memory on the heap that is no longer accessible by your program
- Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
  int *p1;
  p1 = (int *) malloc(sizeof(int));
  *p1=100;
  p1 = (int *) malloc(sizeof(int));
  *p1=1000;
  // ...
}
```

- Note that the anonymous variable containing the value 100 is no longer accessible!
- Called a **memory leak**
- Excessive memory leaks can so reduce available memory that your program might not be able to continue to run



How Dynamic Memory Works *continued*

- If a program's memory is not managed properly it is possible to create allocated memory on the heap that is no longer accessible by your program
- Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
  int *p1;
  p1 = (int *) malloc(sizeof(int));
  *p1=100;
  p1 = (int *) malloc(sizeof(int));
  *p1=1000;
  // ...
}
```

- Note that the anonymous variable containing the value 100 is no longer accessible!
- Called a **memory leak**
- Excessive memory leaks can so reduce available memory that your program might not be able to continue to run



How Dynamic Memory Works *continued*

- If a program's memory is not managed properly it is possible to create allocated memory on the heap that is no longer accessible by your program
- Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
  int *p1;
  p1 = (int *) malloc(sizeof(int));
  *p1=100;
  p1 = (int *) malloc(sizeof(int));
  *p1=1000;
  // ...
}
```

- Note that the anonymous variable containing the value 100 is no longer accessible!
- Called a **memory leak**
- Excessive memory leaks can so reduce available memory that your program might not be able to continue to run



How Dynamic Memory Works *continued*

- If a program's memory is not managed properly it is possible to create allocated memory on the heap that is no longer accessible by your program
- Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
  int *p1;
  p1 = (int *) malloc(sizeof(int));
  *p1=100;
  p1 = (int *) malloc(sizeof(int));
  *p1=1000;
  // ...
}
```

- Note that the anonymous variable containing the value 100 is no longer accessible!
- Called a **memory leak**
- Excessive memory leaks can so reduce available memory that your program might not be able to continue to run



How Dynamic Memory Works *continued*

- If a program's memory is not managed properly it is possible to create allocated memory on the heap that is no longer accessible by your program
- Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
  int *p1;
  p1 = (int *) malloc(sizeof(int));
  *p1=100;
  p1 = (int *) malloc(sizeof(int));
  *p1=1000;
  // ...
}
```

- Note that the anonymous variable containing the value 100 is no longer accessible!
- Called a **memory leak**
- Excessive memory leaks can so reduce available memory that your program might not be able to continue to run



How Dynamic Memory Works *continued*

- If a program's memory is not managed properly it is possible to create allocated memory on the heap that is no longer accessible by your program
- Consider the following code:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
  int *p1;
  p1 = (int *) malloc(sizeof(int));
  *p1=100;
  p1 = (int *) malloc(sizeof(int));
  *p1=1000;
  // ...
}
```

- Note that the anonymous variable containing the value 100 is no longer accessible!
- Called a **memory leak**
- Excessive memory leaks can so reduce available memory that your program might not be able to continue to run the solution?



Dynamic Array Example

```
#include <stdio.h>
#include <stdlib.h>

#define EXPANSION_SIZE 10
#define DEBUG 1

int arraySize = 0, listSize = 0;

int *insert(int *list, int item) {
  int *l = list;
  if (listSize >= arraySize) {
    if (DEBUG) printf("DEBUG: expanding array by %d to %d\n",EXPANSION_SIZE,arraySize+EXPANSION_SIZE);
    l = (int *) malloc((arraySize=arraySize+EXPANSION_SIZE)*sizeof(int));
    if (list != NULL) {
      for(int i=0; i < listSize; i++) l[i] = list[i];
      free(list);
    }
  }
  l[listSize++] = item;
  return l;
}

int main(void){
  int *list=NULL,i=0;
  for (i=1; i <= 21; i++) list = insert(list,i*2);
  for(i = 0; i < listSize; i++) printf("%d ",list[i]);
  printf("\n");
}
```



Dynamic Array Example

```
#include <stdio.h>
#include <stdlib.h>

#define EXPANSION_SIZE 10
#define DEBUG 1

int arraySize = 0, listSize = 0;

int *insert(int *list, int item) {
  int *l = list;
  if (listSize >= arraySize) {
    if (DEBUG) printf("DEBUG: expanding array by %d to %d\n",EXPANSION_SIZE,arraySize+EXPANSION_SIZE);
    l = (int *) malloc((arraySize=arraySize+EXPANSION_SIZE)*sizeof(int));
    if (list != NULL) {
      for(int i=0; i < listSize; i++) l[i] = list[i];
      free(list);
    }
  }
  l[listSize++] = item;
  return l;
}

int main(void){
  int *list=NULL,i=0;
  for (i=1; i <= 21; i++) list = insert(list,i*2);
  for(i = 0; i < listSize; i++) printf("%d ",list[i]);
  printf("\n");
}
```

```
Terminal — -tcsh — 59×7
liberty:~/cprogs/% gcc dynamicExpandingArrays.c
liberty:~/cprogs/% a.out
DEBUG: expanding array by 10 to 10
DEBUG: expanding array by 10 to 20
DEBUG: expanding array by 10 to 30
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42
liberty:~/cprogs/%
```

Introduction

- Most beginning programmers struggle more with their problem solving techniques than with the actual syntax of the language
- They understand the building-block constructs of the language (variables, if statements, loops, functions, etc) but they have trouble using them to actually write a program to solve a problem
- In this section we are going to introduce a problem solving technique called *stepwise refinement*
- **Stepwise refinement** (also called **top-down design**) is a way of developing a computer program by first describing general functions, then breaking each function down into details which are refined in successive steps until the whole program is written
- We will illustrate how this technique works through developing a tic-tac-toe playing program



Introduction

- Most beginning programmers struggle more with their problem solving techniques than with the actual syntax of the language
- They understand the building-block constructs of the language (variables, if statements, loops, functions, etc) but they have trouble using them to actually write a program to solve a problem
- In this section we are going to introduce a problem solving technique called *stepwise refinement*
- **Stepwise refinement** (also called **top-down design**) is a way of developing a computer program by first describing general functions, then breaking each function down into details which are refined in successive steps until the whole program is written
- We will illustrate how this technique works through developing a tic-tac-toe playing program



Introduction

- Most beginning programmers struggle more with their problem solving techniques than with the actual syntax of the language
- They understand the building-block constructs of the language (variables, if statements, loops, functions, etc) but they have trouble using them to actually write a program to solve a problem
- In this section we are going to introduce a problem solving technique called *stepwise refinement*
- **Stepwise refinement** (also called **top-down design**) is a way of developing a computer program by first describing general functions, then breaking each function down into details which are refined in successive steps until the whole program is written
- We will illustrate how this technique works through developing a tic-tac-toe playing program



Introduction

- Most beginning programmers struggle more with their problem solving techniques than with the actual syntax of the language
- They understand the building-block constructs of the language (variables, if statements, loops, functions, etc) but they have trouble using them to actually write a program to solve a problem
- In this section we are going to introduce a problem solving technique called *stepwise refinement*
- **Stepwise refinement** (also called **top-down design**) is a way of developing a computer program by first describing general functions, then breaking each function down into details which are refined in successive steps until the whole program is written
- We will illustrate how this technique works through developing a tic-tac-toe playing program



Introduction

- Most beginning programmers struggle more with their problem solving techniques than with the actual syntax of the language
- They understand the building-block constructs of the language (variables, if statements, loops, functions, etc) but they have trouble using them to actually write a program to solve a problem
- In this section we are going to introduce a problem solving technique called *stepwise refinement*
- **Stepwise refinement** (also called **top-down design**) is a way of developing a computer program by first describing general functions, then breaking each function down into details which are refined in successive steps until the whole program is written
- We will illustrate how this technique works through developing a tic-tac-toe playing program



Program Example: Tic-Tac-Toe

- We've determined to write a program that plays the game of tic-tac-toe
- The specifications state that a human player plays against the computer
- The human player gets to choose whether they go first (and play Xs) or second (and play Os)
- Each player will in turn choose a legal location on the board for their X or O until either there is a winner or the game is a draw
- We will develop the program systematically using stepwise refinement assuming we are a somewhat naive programmer



Program Example: Tic-Tac-Toe

- We've determined to write a program that plays the game of tic-tac-toe
- The specifications state that a human player plays against the computer
- The human player gets to choose whether they go first (and play Xs) or second (and play Os)
- Each player will in turn choose a legal location on the board for their X or O until either there is a winner or the game is a draw
- We will develop the program systematically using stepwise refinement assuming we are a somewhat naive programmer



Program Example: Tic-Tac-Toe

- We've determined to write a program that plays the game of tic-tac-toe
- The specifications state that a human player plays against the computer
- The human player gets to choose whether they go first (and play Xs) or second (and play Os)
- Each player will in turn choose a legal location on the board for their X or O until either there is a winner or the game is a draw
- We will develop the program systematically using stepwise refinement assuming we are a somewhat naive programmer



Program Example: Tic-Tac-Toe

- We've determined to write a program that plays the game of tic-tac-toe
- The specifications state that a human player plays against the computer
- The human player gets to choose whether they go first (and play Xs) or second (and play Os)
- Each player will in turn choose a legal location on the board for their X or O until either there is a winner or the game is a draw
- We will develop the program systematically using stepwise refinement assuming we are a somewhat naive programmer



Program Example: Tic-Tac-Toe

- We've determined to write a program that plays the game of tic-tac-toe
- The specifications state that a human player plays against the computer
- The human player gets to choose whether they go first (and play Xs) or second (and play Os)
- Each player will in turn choose a legal location on the board for their X or O until either there is a winner or the game is a draw
- We will develop the program systematically using stepwise refinement assuming we are a somewhat naive programmer



Program Example: Tic-Tac-Toe *continued*

- Suppose we have no idea where to begin! How can we start?
- Many years ago Niklaus Wirth, the developer of the computer language Pascal, wrote a book called *Algorithms + Data Structures = Programs*
- So let's start there and break our problem into considerations about two things – the data structure(s) needed followed then by the algorithms
- Though we will work through this process step-by-step in these notes, realize that software development is often a cyclic process – one develops a first solution for the data structures, then begins writing algorithms, only to realize their data structures need modifying, which means algorithms need changing, etc.



Program Example: Tic-Tac-Toe *continued*

- Suppose we have no idea where to begin! How can we start?
- Many years ago Niklaus Wirth, the developer of the computer language Pascal, wrote a book called *Algorithms + Data Structures = Programs*
- So let's start there and break our problem into considerations about two things – the data structure(s) needed followed then by the algorithms
- Though we will work through this process step-by-step in these notes, realize that software development is often a cyclic process – one develops a first solution for the data structures, then begins writing algorithms, only to realize their data structures need modifying, which means algorithms need changing, etc.



Program Example: Tic-Tac-Toe *continued*

- Suppose we have no idea where to begin! How can we start?
- Many years ago Niklaus Wirth, the developer of the computer language Pascal, wrote a book called *Algorithms + Data Structures = Programs*
- So let's start there and break our problem into considerations about two things – the data structure(s) needed followed then by the algorithms
- Though we will work through this process step-by-step in these notes, realize that software development is often a cyclic process – one develops a first solution for the data structures, then begins writing algorithms, only to realize their data structures need modifying, which means algorithms need changing, etc.



Program Example: Tic-Tac-Toe *continued*

- Suppose we have no idea where to begin! How can we start?
- Many years ago Niklaus Wirth, the developer of the computer language Pascal, wrote a book called *Algorithms + Data Structures = Programs*
- So let's start there and break our problem into considerations about two things – the data structure(s) needed followed then by the algorithms
- Though we will work through this process step-by-step in these notes, realize that software development is often a cyclic process – one develops a first solution for the data structures, then begins writing algorithms, only to realize their data structures need modifying, which means algorithms need changing, etc.



Tic-Tac-Toe Program Development – Data Structures

- We will need to represent the tic-tac-toe board in our program
- How should we do that?

- Though not necessary, we're going to develop our code step-by-step always producing a program that will compile
- Therefore, our very first draft, file `ttt0.c`:

```
#include <stdio.h>
#define SIZE 3

int main(void){
  char board[SIZE][SIZE];
  return 0;
}
```

- Note that we defined our board local to main; it is a good programming practice to avoid declaring global variables
- By defining it in `main` we will need to pass it to any routines (functions) we develop that need to access the board



Tic-Tac-Toe Program Development – Data Structures

- We will need to represent the tic-tac-toe board in our program
- How should we do that?

- Though not necessary, we're going to develop our code step-by-step always producing a program that will compile
- Therefore, our very first draft, file `ttt0.c`:

```
#include <stdio.h>
#define SIZE 3

int main(void){
  char board[SIZE][SIZE];
  return 0;
}
```

- Note that we defined our board local to main; it is a good programming practice to avoid declaring global variables
- By defining it in `main` we will need to pass it to any routines (functions) we develop that need to access the board



Tic-Tac-Toe Program Development – Data Structures

- We will need to represent the tic-tac-toe board in our program
- How should we do that?

```
char board[3][3];
```

- Though not necessary, we're going to develop our code step-by-step always producing a program that will compile
- Therefore, our very first draft, file `ttt0.c`:

```
#include <stdio.h>
#define SIZE 3

int main(void){
  char board[SIZE][SIZE];
  return 0;
}
```

- Note that we defined our board local to main; it is a good programming practice to avoid declaring global variables
- By defining it in `main` we will need to pass it to any routines (functions) we develop that need to access the board



Tic-Tac-Toe Program Development – Data Structures

- We will need to represent the tic-tac-toe board in our program
- How should we do that?

```
char board[3][3];
```

- Though not necessary, we're going to develop our code step-by-step always producing a program that will compile
- Therefore, our very first draft, file `ttt0.c`:

```
#include <stdio.h>
#define SIZE 3

int main(void){
  char board[SIZE][SIZE];
  return 0;
}
```

- Note that we defined our board local to main; it is a good programming practice to avoid declaring global variables
- By defining it in `main` we will need to pass it to any routines (functions) we develop that need to access the board



Tic-Tac-Toe Program Development – Data Structures

- We will need to represent the tic-tac-toe board in our program
- How should we do that?

```
char board[3][3];
```

- Though not necessary, we're going to develop our code step-by-step always producing a program that will compile
- Therefore, our very first draft, file `ttt0.c`:

```
#include <stdio.h>
#define SIZE 3

int main(void){
  char board[SIZE][SIZE];
  return 0;
}
```

- Note that we defined our board local to main; it is a good programming practice to avoid declaring global variables
- By defining it in `main` we will need to pass it to any routines (functions) we develop that need to access the board



Tic-Tac-Toe Program Development – Data Structures

- We will need to represent the tic-tac-toe board in our program
- How should we do that?

```
char board[3][3];
```

- Though not necessary, we're going to develop our code step-by-step always producing a program that will compile
- Therefore, our very first draft, file `ttt0.c`:

```
#include <stdio.h>
#define SIZE 3

int main(void){
  char board[SIZE][SIZE];
  return 0;
}
```

- Note that we defined our board local to main; it is a good programming practice to avoid declaring global variables
- By defining it in `main` we will need to pass it to any routines (functions) we develop that need to access the board



Tic-Tac-Toe Program Development – Data Structures

- We will need to represent the tic-tac-toe board in our program
- How should we do that?

```
char board[3][3];
```

- Though not necessary, we're going to develop our code step-by-step always producing a program that will compile
- Therefore, our very first draft, file `ttt0.c`:

```
#include <stdio.h>
#define SIZE 3

int main(void){
  char board[SIZE][SIZE];
  return 0;
}
```

- Note that we defined our board local to main; it is a good programming practice to avoid declaring global variables
- By defining it in `main` we will need to pass it to any routines (functions) we develop that need to access the board



Tic-Tac-Toe Program Development – Algorithms

- We wish to develop versions of our program using stepwise refinement

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3



int main(void){
  char board[SIZE][SIZE];


  return 0;
}
```



Tic-Tac-Toe Program Development – Algorithms

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3



int main(void){
  char board[SIZE][SIZE];


  return 0;
}
```



Tic-Tac-Toe Program Development – Algorithms

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3



int main(void){
  char board[SIZE][SIZE];


  return 0;
}
```



Tic-Tac-Toe Program Development – Algorithms

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?
- We wish it to play the game!

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3


int main(void){
  char board[SIZE][SIZE];


  return 0;
}
```



Tic-Tac-Toe Program Development – Algorithms

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?
- We wish it to play the game!
- So we add a function call that does just that . . .

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3


int main(void){
  char board[SIZE][SIZE];


  return 0;
}
```



Tic-Tac-Toe Program Development – Algorithms

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3



int main(void){
  char board[SIZE][SIZE];

  playGame(board);
  return 0;
}
```

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?
- We wish it to play the game!
- So we add a function call that does just that . . .



Tic-Tac-Toe Program Development – Algorithms

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3


int main(void){
  char board[SIZE][SIZE];

  playGame(board);
  return 0;
}
```

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?
- We wish it to play the game!
- So we add a function call that does just that . . .

- If we had the confidence that `playGame()` was written correctly, would we be confident that our program was correct?



Tic-Tac-Toe Program Development – Algorithms

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3


int main(void){
  char board[SIZE][SIZE];

  playGame(board);
  return 0;
}
```

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?
- We wish it to play the game!
- So we add a function call that does just that . . .

- If we had the confidence that `playGame()` was written correctly, would we be confident that our program was correct? **Yes!**



Tic-Tac-Toe Program Development – Algorithms

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3


int main(void){
  char board[SIZE][SIZE];

  playGame(board);
  return 0;
}
```

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?
- We wish it to play the game!
- So we add a function call that does just that . . .

- If we had the confidence that `playGame()` was written correctly, would we be confident that our program was correct? **Yes!**
- Now remember we also want this version of our program to compile . . .



Tic-Tac-Toe Program Development – Algorithms

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?
- We wish it to play the game!
- So we add a function call that does just that . . .

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3

void playGame(char [SIZE][SIZE]);

int main(void){
  char board[SIZE][SIZE];

  playGame(board);
  return 0;
}

void playGame(char brd[SIZE][SIZE]) {
}
```

- If we had the confidence that `playGame()` was written correctly, would we be confident that our program was correct? **Yes!**
- Now remember we also want this version of our program to compile . . .



Tic-Tac-Toe Program Development – Algorithms

- We wish to develop versions of our program using stepwise refinement
- And each step should compile without errors
- At the most general level, what do we want our program to do at this point?
- We wish it to play the game!
- So we add a function call that does just that . . .

- File `ttt1.c`:

```
#include <stdio.h>
#define SIZE 3

void playGame(char [SIZE][SIZE]);

int main(void){
  char board[SIZE][SIZE];

  playGame(board);
  return 0;
}

void playGame(char brd[SIZE][SIZE]) {
}
```

- If we had the confidence that `playGame()` was written correctly, would we be confident that our program was correct? **Yes!**
- Now remember we also want this version of our program to compile . . .
- We now have version #2 of our program!



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - [illegible]
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - draw the board
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - draw the board
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - draw the board
    - player takes turn
    - repeat until game is over
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - draw the board, then
    - player takes turn
    - repeat until game is over
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - draw the board, then
    - ★ player takes turn
    - ★ repeat until game is over
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - draw the board, then
    - ★ player takes turn
    - ★ repeat until game is over
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - draw the board, then
    - ★ player takes turn
    - ★ repeat until game is over
- If we correctly implement these steps do we have confidence we have solved the problem?
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- So how do we write the `playGame()` function?
- Again, suppose we have no idea; what do we do?
- Often a good place to start: consider generally how you would really play the game with someone else and jot down the steps
  - choose who goes first
  - draw the board, then
    - ★ player takes turn
    - ★ repeat until game is over
- If we correctly implement these steps do we have confidence we have solved the problem? **Yes!**
- We now have an algorithm for our program



Tic-Tac-Toe Program Development – Algorithms *continued*

- Begin by writing down your algorithm



Tic-Tac-Toe Program Development – Algorithms *continued*

```
choose who goes first
draw the board
  player takes turn
  repeat until game is over
```

- Begin by writing down your algorithm



Tic-Tac-Toe Program Development – Algorithms *continued*

```
choose who goes first
draw the board
  player takes turn
  repeat until game is over
```

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  draw_the_board(brd);
    player_takes_turn(brd);
    repeat_until_game_is_over(brd);

}
```

we need to pass our board into some routines

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  draw_the_board(brd);
    player_takes_turn(brd);
    repeat_until_game_is_over(brd);

}
```

we need to pass our board into some routines

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call
- But wait — that repeat_until_game_is_over() step is a loop that belongs in playGame() (because `playGame()` has the steps you wish to repeat), not in a function by itself



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  draw_the_board(brd);
    player_takes_turn(brd);
    repeat_until_game_is_over(brd);

}
```

we need to pass our board into some routines

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call
- But wait — that repeat_until_game_is_over() step is a loop that belongs in playGame() (because `playGame()` has the steps you wish to repeat), not in a function by itself
- Therefore rewrite the repeat step as a loop in playGame()



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  draw_the_board(brd);
  do {
    player_takes_turn(brd);
  while (!game_is_over(brd));
}
```

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call
- But wait — that repeat_until_game_is_over() step is a loop that belongs in playGame() (because `playGame()` has the steps you wish to repeat), not in a function by itself
- Therefore rewrite the repeat step as a loop in playGame()



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  draw_the_board(brd);
  do {
    player_takes_turn(brd);
  while (!game_is_over(brd));
}
```

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call
- But wait — that repeat_until_game_is_over() step is a loop that belongs in playGame() (because `playGame()` has the steps you wish to repeat), not in a function by itself
- Therefore rewrite the repeat step as a loop in playGame()
- Do we have confidence that this will work?



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  draw_the_board(brd);
  do {
    player_takes_turn(brd);
  while (!game_is_over(brd));
}
```

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call
- But wait — that repeat_until_game_is_over() step is a loop that belongs in playGame() (because `playGame()` has the steps you wish to repeat), not in a function by itself
- Therefore rewrite the repeat step as a loop in playGame()
- Do we have confidence that this will work? Well - kind of!



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  draw_the_board(brd);
  do {
    player_takes_turn(brd);
  while (!game_is_over(brd));
}
```

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call
- But wait — that repeat_until_game_is_over() step is a loop that belongs in playGame() (because `playGame()` has the steps you wish to repeat), not in a function by itself
- Therefore rewrite the repeat step as a loop in playGame()
- Do we have confidence that this will work? Well - kind of!
- We notice that on a computer we will want to redraw the board after every move



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  draw_the_board(brd);
  do {
    player_takes_turn(brd);
  while (!game_is_over(brd));
}
```

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call
- But wait — that repeat_until_game_is_over() step is a loop that belongs in playGame() (because `playGame()` has the steps you wish to repeat), not in a function by itself
- Therefore rewrite the repeat step as a loop in playGame()
- Do we have confidence that this will work? Well - kind of!
- We notice that on a computer we will want to redraw the board after every move
- So we move the draw_the_board() call into the loop



Tic-Tac-Toe Program Development – Algorithms *continued*

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  do {
    draw_the_board(brd);
    player_takes_turn(brd);
  while (!game_is_over(brd));
}
```

- Begin by writing down your algorithm
- Then make this part of your playGame() function, and make every action item a functon call
- But wait — that repeat_until_game_is_over() step is a loop that belongs in playGame() (because `playGame()` has the steps you wish to repeat), not in a function by itself
- Therefore rewrite the repeat step as a loop in playGame()
- Do we have confidence that this will work? Well - kind of!
- We notice that on a computer we will want to redraw the board after every move
- So we move the draw_the_board() call into the loop



Tic-Tac-Toe Program Development – Algorithms *continued*

- Remember we will want this new version to compile
- Part of file `ttt2.c`:

```
...
#include <stdbool.h>
...
void choose_who_goes_first();
void draw_the_board(char [SIZE][SIZE]);
void player_takes_turn(char [SIZE][SIZE]);
bool game_is_over(char [SIZE][SIZE]);
...
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  do {
    draw_the_board(brd);
    player_takes_turn(brd);
  } while (!game_is_over(brd));
}

void choose_who_goes_first() { }
void draw_the_board(char brd[SIZE][SIZE]) { }
void player_takes_turn(char brd[SIZE][SIZE]) { }
bool game_is_over(char brd[SIZE][SIZE]){ return true; }
```



Tic-Tac-Toe Program Development – Algorithms *continued*

- Remember we will want this new version to compile
- Part of file `ttt2.c`:

```
...
#include <stdbool.h>
...
void choose_who_goes_first();
void draw_the_board(char [SIZE][SIZE]);
void player_takes_turn(char [SIZE][SIZE]);
bool game_is_over(char [SIZE][SIZE]);
...
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  do {
    draw_the_board(brd);
    player_takes_turn(brd);
  } while (!game_is_over(brd));
}

void choose_who_goes_first() { }
void draw_the_board(char brd[SIZE][SIZE]) { }
void player_takes_turn(char brd[SIZE][SIZE]) { }
bool game_is_over(char brd[SIZE][SIZE]){ return true; }
```



Tic-Tac-Toe Program Development – Algorithms *continued*

- We now continue the stepwise refinement of our program by writing all of the new functions
- Note that each function is defined such as to do a single logical task (i.e. we do not specify a function called `draw_board_and_player_takes_turn()`)
- Also note this method lends itself well to TDD – you would write the tests first, then each function
- How do we know when to stop subdividing our functions into calls to other functions?
- There is no hard and fast rule, but generally you stop when you are able to correctly write a small routine (<20 lines) to complete the task at hand



Tic-Tac-Toe Program Development – Algorithms *continued*

- We now continue the stepwise refinement of our program by writing all of the new functions
- Note that each function is defined such as to do a single logical task (i.e. we do not specify a function called `draw_board_and_player_takes_turn()`)
- Also note this method lends itself well to TDD – you would write the tests first, then each function
- How do we know when to stop subdividing our functions into calls to other functions?
- There is no hard and fast rule, but generally you stop when you are able to correctly write a small routine (<20 lines) to complete the task at hand



Tic-Tac-Toe Program Development – Algorithms *continued*

- We now continue the stepwise refinement of our program by writing all of the new functions
- Note that each function is defined such as to do a single logical task (i.e. we do not specify a function called `draw_board_and_player_takes_turn()`)
- Also note this method lends itself well to TDD – you would write the tests first, then each function
- How do we know when to stop subdividing our functions into calls to other functions?
- There is no hard and fast rule, but generally you stop when you are able to correctly write a small routine (<20 lines) to complete the task at hand



Tic-Tac-Toe Program Development – Algorithms *continued*

- We now continue the stepwise refinement of our program by writing all of the new functions
- Note that each function is defined such as to do a single logical task (i.e. we do not specify a function called `draw_board_and_player_takes_turn()`)
- Also note this method lends itself well to TDD – you would write the tests first, then each function
- How do we know when to stop subdividing our functions into calls to other functions?
- There is no hard and fast rule, but generally you stop when you are able to correctly write a small routine (<20 lines) to complete the task at hand



Tic-Tac-Toe Program Development – Algorithms *continued*

- We now continue the stepwise refinement of our program by writing all of the new functions
- Note that each function is defined such as to do a single logical task (i.e. we do not specify a function called `draw_board_and_player_takes_turn()`)
- Also note this method lends itself well to TDD – you would write the tests first, then each function
- How do we know when to stop subdividing our functions into calls to other functions?
- There is no hard and fast rule, but generally you stop when you are able to correctly write a small routine ($<$20 lines) to complete the task at hand



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {



}
```



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {



}
```

- Now let's describe in comments what we want our routine to do



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {
   // if it's computer's turn then computer makes move


}
```

- Now let's describe in comments what we want our routine to do



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {
   // if it's computer's turn then computer makes move
   // if it's human's turn then computer makes move


}
```

- Now let's describe in comments what we want our routine to do



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {
   // if it's computer's turn then computer makes move
   // if it's human's turn then computer makes move
   // place a mark on the board
}
```

- Now let's describe in comments what we want our routine to do



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {
   // if it's computer's turn then computer makes move
   // if it's human's turn then computer makes move
   // place a mark on the board
}
```

- Now let's describe in comments what we want our routine to do
- Notice we need to know whose turn it is in this routine



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {
  // if it's computer's turn then computer makes move
  // if it's human's turn then computer makes move
  // place a mark on the board
}
```

- Now let's describe in comments what we want our routine to do
- Notice we need to know whose turn it is in this routine
- Remember that our `playGame()` routine loops once per move

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  do {
    draw_the_board(brd);
    player_takes_turn(brd);
  } while (!game_is_over(brd));
}
```



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {
  // if it's computer's turn then computer makes move
  // if it's human's turn then computer makes move
  // place a mark on the board
}
```

- Now let's describe in comments what we want our routine to do
- Notice we need to know whose turn it is in this routine
- Remember that our `playGame()` routine loops once per move
- Therefore `playGame()` has to know whose turn it is when it calls `player_takes_turn()`!

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  do {
    draw_the_board(brd);
    player_takes_turn(brd);
  } while (!game_is_over(brd));
}
```



Tic-Tac-Toe Program Development – Algorithms *continued*

- Let's do one more routine – `player_takes_turn()`

```
void player_takes_turn(char brd[SIZE][SIZE]) {
  // if it's computer's turn then computer makes move
  // if it's human's turn then computer makes move
  // place a mark on the board
}
```

- Now let's describe in comments what we want our routine to do
- Notice we need to know whose turn it is in this routine
- Remember that our `playGame()` routine loops once per move
- Therefore `playGame()` has to know whose turn it is when it calls `player_takes_turn()`!
- So let's pass whose turn it is into this routine
- Now we will need to modify `playerGame()` to keep track of whose turn it is and then pass this into `player_takes_turn()`

```
void playGame(char brd[SIZE][SIZE]) {
  choose_who_goes_first();
  do {
    draw_the_board(brd);
    player_takes_turn(brd);
  } while (!game_is_over(brd));
}
```



Tic-Tac-Toe Program Development – Algorithms *continued*

- If we add separate routines to make a move for the computer and the human, plus ones to mark the board and to get the appropriate symbol, `player_takes_turn()` is straightforward to write:

- We will assume that `humanMoves()` and `computerMoves()` select legal moves (i.e. we will eventually write them so they only return legal moves)



Tic-Tac-Toe Program Development – Algorithms *continued*

- If we add separate routines to make a move for the computer and the human, plus ones to mark the board and to get the appropriate symbol, `player_takes_turn()` is straightforward to write:

```
void player_takes_turn(char brd[SIZE][SIZE],int player) {
  int move = (player==HUMAN?humanMoves(brd):computerMoves(brd));
  placeMarkOnBoard(brd,move,getSymbol(player));
}
```

- We will assume that `humanMoves()` and `computerMoves()` select legal moves (i.e. we will eventually write them so they only return legal moves)



Tic-Tac-Toe Program Development – Algorithms *continued*

- If we add separate routines to make a move for the computer and the human, plus ones to mark the board and to get the appropriate symbol, `player_takes_turn()` is straightforward to write:

```
void player_takes_turn(char brd[SIZE][SIZE],int player) {
  int move = (player==HUMAN?humanMoves(brd):computerMoves(brd));
  placeMarkOnBoard(brd,move,getSymbol(player));
}
```

- We will assume that `humanMoves()` and `computerMoves()` select legal moves (i.e. we will eventually write them so they only return legal moves)



Tic-Tac-Toe Program Development – Algorithms *continued*

- To complete the program you would continue to refine and test each function until all code had been written correctly
- Complete program will either be presented in class or given as a project or lab assignment



Tic-Tac-Toe Program Development – Algorithms *continued*

- To complete the program you would continue to refine and test each function until all code had been written correctly
- Complete program will either be presented in class or given as a project or lab assignment



Debugging

- One of the most important skills a programmer can have is the ability to debug code
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - Trace flow through the program
  - Watch variables change
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debugging

- One of the most important skills a programmer can have is the ability to debug code Note it is a skill one can develop!
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - Trace flow through the program
  - Watch variables change as the program executes
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debugging

- One of the most important skills a programmer can have is the ability to debug code Note it is a skill one can develop!
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - trace the execution path through your program
  - view the values of variables
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debugging

- One of the most important skills a programmer can have is the ability to debug code Note it is a skill one can develop!
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - trace the execution path through your program
  - 'observe' the updating of variables
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debugging

- One of the most important skills a programmer can have is the ability to debug code Note it is a skill one can develop!
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - trace the execution path through your program
  - 'observe' the updating of variables
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debugging

- One of the most important skills a programmer can have is the ability to debug code Note it is a skill one can develop!
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - trace the execution path through your program
  - 'observe' the updating of variables
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debugging

- One of the most important skills a programmer can have is the ability to debug code Note it is a skill one can develop!
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - trace the execution path through your program
  - 'observe' the updating of variables
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debugging

- One of the most important skills a programmer can have is the ability to debug code Note it is a skill one can develop!
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - trace the execution path through your program
  - 'observe' the updating of variables
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debugging

- One of the most important skills a programmer can have is the ability to debug code Note it is a skill one can develop!
- Assuming that your approach and algorithm(s) are correct, debugging consists of isolating and fixing problems
- To isolate problems one must be able to:
  - trace the execution path through your program
  - 'observe' the updating of variables
- Use a source-level debugger (best option) or diagnostic prints
- Always develop & debug your programs in small, incremental steps
- Ask yourself: Will the program work if the part I am working on now is not correct? If the answer is no, **do not continue on other portions of the code until you are sure the current section is correct!**



Debuggers

- A **debugger** is a program (or part of an IDE) that enables you to step through and examine the execution of another program
- It is useful to:
  - trace the flow of control
  - examine the state of the program
- Traditionally it was used to discover bugs in code
- Debuggers are also very useful in learning how programs work
- We will not be teaching and using a specific debugger this semester



Debuggers

- A **debugger** is a program (or part of an IDE) that enables you to step through and examine the execution of another program
- It is useful to:
  - trace the flow of control
  - examine the state (variables)
- Traditionally it was used to discover bugs in code
- Debuggers are also very useful in learning how programs work
- We will not be teaching and using a specific debugger this semester



Debuggers

- A **debugger** is a program (or part of an IDE) that enables you to step through and examine the execution of another program
- It is useful to:
  - trace the flow of control
  - examine the state (variables)
- Traditionally it was used to discover bugs in code
- Debuggers are also very useful in learning how programs work
- We will not be teaching and using a specific debugger this semester



Debuggers

- A **debugger** is a program (or part of an IDE) that enables you to step through and examine the execution of another program
- It is useful to:
  - trace the flow of control
  - examine the state (variables)
- Traditionally it was used to discover bugs in code
- Debuggers are also very useful in learning how programs work
- We will not be teaching and using a specific debugger this semester



Debuggers

- A **debugger** is a program (or part of an IDE) that enables you to step through and examine the execution of another program
- It is useful to:
  - trace the flow of control
  - examine the state (variables)
- Traditionally it was used to discover bugs in code
- Debuggers are also very useful in learning how programs work
- We will not be teaching and using a specific debugger this semester



Debuggers

- A **debugger** is a program (or part of an IDE) that enables you to step through and examine the execution of another program
- It is useful to:
  - trace the flow of control
  - examine the state (variables)
- Traditionally it was used to discover bugs in code
- Debuggers are also very useful in learning how programs work
- We will not be teaching and using a specific debugger this semester



Debuggers

- A **debugger** is a program (or part of an IDE) that enables you to step through and examine the execution of another program
- It is useful to:
  - trace the flow of control
  - examine the state (variables)
- Traditionally it was used to discover bugs in code
- Debuggers are also very useful in learning how programs work
- We will not be teaching and using a specific debugger this semester



Debugging with Scaffolding

```
//...



int main(void) {
  // ...
  inputWidgets(widgets);

  // ...




  // ...
  return 0;
}
```



Debugging with Scaffolding

```
//...
#define DEBUG true
#define DEBUG1 false

int main(void) {
  // ...
  inputWidgets(widgets);
  if (DEBUG) printWidgets(widgets);
  // ...
  if (DEBUG1) {
    printf("DEBUG1:  x = %d\n",x);
    // ...
  }
  // ...
  return 0;
}
```



DOs and DON'Ts

1. Subdivide your program into functions (and modules, which we will cover later) – remember, **divide & conquer**
2. Try to restrict each method to a maximum of 20 lines of code (less is better!)
3. Treat each method as a 'black box' with inputs/outputs only allowed via the parameter list and/or return statement (exception: accessing encapsulated data in modules; more on this later)



DOs and DON'Ts

1. Subdivide your program into functions (and modules, which we will cover later) – remember, **divide & conquer**
2. Try to restrict each method to a maximum of 20 lines of code (less is better!)
3. Treat each method as a 'black box' with inputs/outputs only allowed via the parameter list and/or return statement (exception: accessing encapsulated data in modules; more on this later)

DO:

DOs and DON'Ts

1. Subdivide your program into functions (and modules, which we will cover later) – remember, **divide & conquer**
2. Try to restrict each method to a maximum of 20 lines of code (less is better!)
3. Treat each method as a 'black box' with inputs/outputs only allowed via the parameter list and/or return statement (exception: accessing encapsulated data in modules; more on this later)

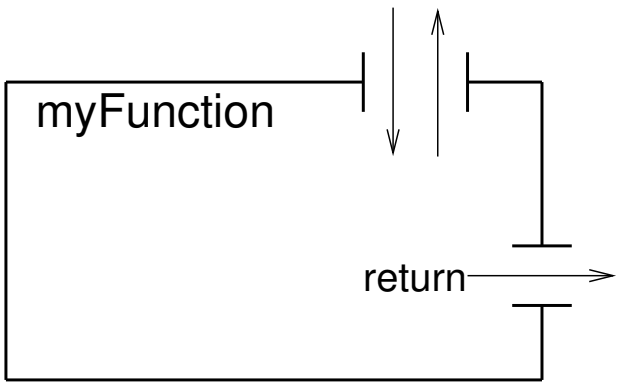
DO: **DON'T:**

DOs and DON'Ts

1. Subdivide your program into functions (and modules, which we will cover later) – remember, **divide & conquer**
2. Try to restrict each method to a maximum of 20 lines of code (less is better!)
3. Treat each method as a 'black box' with inputs/outputs only allowed via the parameter list and/or return statement (exception: accessing encapsulated data in modules; more on this later)

DO:

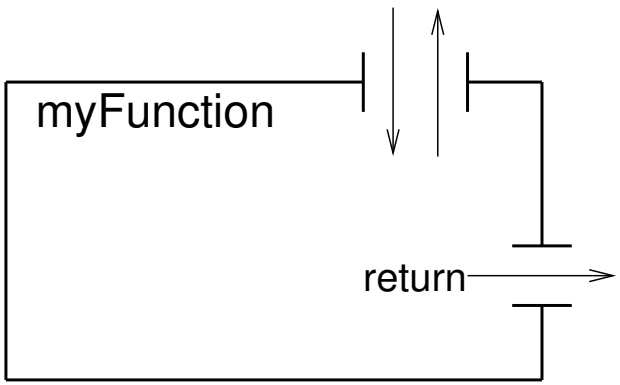
DON'T:



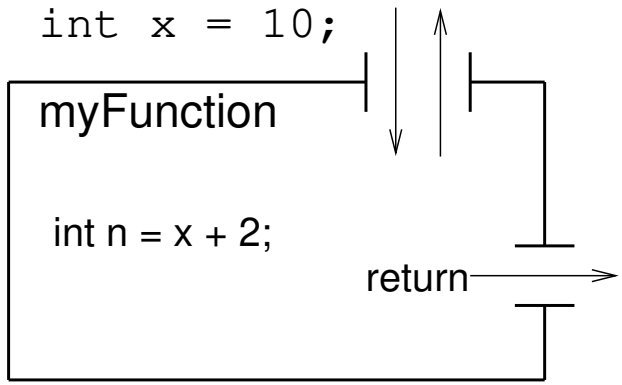
DOs and DON'Ts

1. Subdivide your program into functions (and modules, which we will cover later) – remember, **divide & conquer**
2. Try to restrict each method to a maximum of 20 lines of code (less is better!)
3. Treat each method as a 'black box' with inputs/outputs only allowed via the parameter list and/or return statement (exception: accessing encapsulated data in modules; more on this later)

DO:



DON'T:



DOs and DON'Ts

4. Do one logical task per function



DOs and DON'Ts

4. Do one logical task per function

DON'T:

```
int main(void) {
  inputDataAndBringAboutWorldPeace();
  printResultsAndCelebrate();
}
```

DOs and DON'Ts

4. Do one logical task per function

DON'T:

```
int main(void) {
  inputDataAndBringAboutWorldPeace();
  printResultsAndCelebrate();
}
```

DO:

```
int main(void) {
  inputData();
  bringAboutWorldPeace();
  printResults();
  celebrate();
}
```



DOs and DON'Ts

5. Use top-down design to build a system by refining subtasks; assume both programs below accomplish exactly the same thing:

DOs and DON'Ts

5. Use top-down design to build a system by refining subtasks; assume both programs below accomplish exactly the same thing:

DO:

```
int main(void) {
  inputData();
  bringAboutWorldPeace();
  printResults();
  celebrate();
}
```



DOs and DON'Ts

5. Use top-down design to build a system by refining subtasks; assume both programs below accomplish exactly the same thing:

DO:

```
int main(void) {
  inputData();
  bringAboutWorldPeace();
  printResults();
  celebrate();
}
```

DON'T:

```
int main(void) {
  inputData2();
}
```

DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

DO:	NOT:
sumOfScores	s
numStudents	ns
averageScore	as

   - For methods, use verb-object pairs . . .

DO:	NOT:
getAverage()	average()
displayMenu()	menu()
calculateSum()	sum()



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO: **NOT:**

`sumOfScores`	`s`
`annualSalary`	`asal`
`endOfFile`	`eof`

   - For methods, use verb-object pairs . . .

DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

DO:	**NOT:**
`sumOfScores`	`s`
`annualSalary`	`asal`
`endOfFile`	`eof`

   - For methods, use verb-object pairs . . .

DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

DO:	**NOT:**
`sumOfScores`	`s`
`annualSalary`	`asal`
`endOfFile`	`eof`

   - For methods, use verb-object pairs . . .



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

DO:	**NOT:**
`sumOfScores`	`s`
`annualSalary`	`asal`
`endOfFile`	`eof`

   - For methods, use verb-object pairs . . .



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO:

     ```
     sumOfScores
     annualSalary
     endOfFile
     ```

     NOT:

     ```
     s
     asal
     eof
     ```

   - For methods, use verb-object pairs . . .

     DO:

     ```
     inputSalaryData
     ```

DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO:

     ```
     sumOfScores
     annualSalary
     endOfFile
     ```

     NOT:

     ```
     s
     asal
     eof
     ```

   - For methods, use verb-object pairs . . .

     DO:

     ```
     inputSalaryData
     ```

     NOT:

     ```
     input
     ```



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO:

     ```
     sumOfScores
     annualSalary
     endOfFile
     ```

     NOT:

     ```
     s
     asal
     eof
     ```

   - For methods, use verb-object pairs . . .

     DO:

     ```
     inputSalaryData
     computeMonthlySales
     isCorrectCode
     ```

     NOT:

     ```
     input
     cms
     iscc
     ```



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO:

     ```
     sumOfScores
     annualSalary
     endOfFile
     ```

     NOT:

     ```
     s
     asal
     eof
     ```

   - For methods, use verb-object pairs . . .

     DO:

     ```
     inputSalaryData
     computeMonthlySales
     isCorrectCode
     ```

     NOT:

     ```
     input
     cms
     iscc
     ```



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO:
     ```
     sumOfScores
     annualSalary
     endOfFile
     ```

     NOT:
     ```
     s
     asal
     eof
     ```

   - For methods, use verb-object pairs . . .

     DO:
     ```
     inputSalaryData
     computeMonthlySales
     isCorrectCode
     ```

     NOT:
     ```
     input
     cms
     iscc
     ```



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO:

     ```
     sumOfScores
     annualSalary
     endOfFile
     ```

     NOT:

     ```
     s
     asal
     eof
     ```

   - For methods, use verb-object pairs . . .

     DO:

     ```
     inputSalaryData
     computeMonthlySales
     isCorrectCode
     ```

     NOT:

     ```
     input
     cms
     iscc
     ```



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO:
     ```
     sumOfScores
     annualSalary
     endOfFile
     ```
     NOT:
     ```
     s
     asal
     eof
     ```

   - For methods, use verb-object pairs . . .

     DO:
     ```
     inputSalaryData
     computeMonthlySales
     isCorrectCode
     ```
     NOT:
     ```
     input
     cms
     iscc
     ```



DOs and DON'Ts

6. Use identifiers that connote meaning
   - For variables . . .

     DO:
     ```
     sumOfScores
     annualSalary
     endOfFile
     ```
     NOT:
     ```
     s
     asal
     eof
     ```

   - For methods, use verb-object pairs . . .

     DO:
     ```
     inputSalaryData
     computeMonthlySales
     isCorrectCode
     ```
     NOT:
     ```
     input
     cms
     iscc
     ```



DOs and DON'Ts

7. Formatting of your code is important!

DON'T:

```
#define DEBUG false

     int main(void
        ) {

                       inputWidgets   (widgets);
if (DEBUG  )printWidgets(widgets)      ;
               Results r = doAnalysis(widgets);
reportResults(r)
   ;                              }
  }
```



DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

   DON'T:

   ```
   if(flag == false) ...
   ```

9. Do not use a conditional to return a boolean value

10. Avoid overuse of variables unless it aids in readability



DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

10. Avoid overuse of variables unless it aids in readability



DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

10. Avoid overuse of variables unless it aids in readability



DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

10. Avoid overuse of variables unless it aids in readability

DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

10. Avoid overuse of variables unless it aids in readability



DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

DON'T:

```
if(x > y)
  return true;
else
  return false;
```

10. Avoid overuse of variables unless it aids in readability



DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

DON'T:

```
if(x > y)
  return true;
else
  return false;
```

DO:

```
return (x > y);
```

10. Avoid overuse of variables unless it aids in readability

DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

DON'T:

```
if(x > y)
  return true;
else
  return false;
```

DO:

```
return (x > y);
```

10. Avoid overuse of variables unless it aids in readability

DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

DON'T:

```
if(x > y)
  return true;
else
  return false;
```

DO:

```
return (x > y);
```

10. Avoid overuse of variables unless it aids in readability

DON'T:

```
int x = f(x);
doSomething(x);
```

DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

DON'T:

```
if(x > y)
  return true;
else
  return false;
```

DO:

```
return (x > y);
```

10. Avoid overuse of variables unless it aids in readability

DON'T:

```
int x = f(x);
doSomething(x);
```

DO:

```
doSomething(f(x));
```



DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

DON'T:

```
if(x > y)
  return true;
else
  return false;
```

DO:

```
return (x > y);
```

10. Avoid overuse of variables unless it aids in readability

DON'T:

```
int x = f(x);
doSomething(x);
```

DO:

```
doSomething(f(x));
```

DOs and DON'Ts

And a few more minor points ...

8. Do not use '==' to test a boolean variable

DON'T:

```
if(flag == false) ...
while(notDone == true) ...
```

DO:

```
if(!flag) ...
while(notDone) ...
```

9. Do not use a conditional to return a boolean value

DON'T:

```
if(x > y)
  return true;
else
  return false;
```

DO:

```
return (x > y);
```

10. Avoid overuse of variables unless it aids in readability

DON'T:

```
int x = f(x);
doSomething(x);
```

DO:

```
doSomething(f(x));
```



Abstraction

- **Abstraction** is the principle of representing only the essential details of data or actions while hiding the inessential ones
- It is a key concept in programming (and life!) that enables us to reduce complexity
- Example: `printf("answer = %d\n",n);`
- Abstractions in computer science generally allow us to separate the concept of ***what*** something does from ***how*** it does it
- As we write more and more complex code it is important to design our programs using layers of user-defined abstractions



Abstraction

- **Abstraction** is the principle of representing only the essential details of data or actions while hiding the inessential ones
- It is a key concept in programming (and life!) that enables us to reduce complexity
- Example: `printf("answer = %d\n",n);`
- Abstractions in computer science generally allow us to separate the concept of ***what*** something does from ***how*** it does it
- As we write more and more complex code it is important to design our programs using layers of user-defined abstractions



Abstraction

- **Abstraction** is the principle of representing only the essential details of data or actions while hiding the inessential ones
- It is a key concept in programming (and life!) that enables us to reduce complexity
- Example: `printf("answer = %d\n",n);`
- Abstractions in computer science generally allow us to separate the concept of ***what*** something does from ***how*** it does it
- As we write more and more complex code it is important to design our programs using layers of user-defined abstractions



Abstraction

- **Abstraction** is the principle of representing only the essential details of data or actions while hiding the inessential ones
- It is a key concept in programming (and life!) that enables us to reduce complexity
- Example: `printf("answer = %d\n",n);`
- Abstractions in computer science generally allow us to separate the concept of ***what*** something does from ***how*** it does it
- As we write more and more complex code it is important to design our programs using layers of user-defined abstractions



Abstraction

- **Abstraction** is the principle of representing only the essential details of data or actions while hiding the inessential ones
- It is a key concept in programming (and life!) that enables us to reduce complexity
- Example: `printf("answer = %d\n",n);`
- Abstractions in computer science generally allow us to separate the concept of ***what*** something does from ***how*** it does it
- As we write more and more complex code it is important to design our programs using layers of user-defined abstractions



Data Abstractions

```
typedef struct {
  int day, month, year;
} Date;

Date today, yesterday;
```

- We now can use `Date` as a data type abstraction representing a calendar date
- Even defining a simple `float` variable involves data abstraction – to use it you do not need to concern yourself with the inessential details of how the number is actually stored in memory



Data Abstractions

```
typedef struct {
  int day, month, year;
} Date;

Date today, yesterday;
```

- We now can use `Date` as a data type abstraction representing a calendar date
- Even defining a simple `float` variable involves data abstraction – to use it you do not need to concern yourself with the inessential details of how the number is actually stored in memory



Operation Abstractions

- To create abstractions for actions or operations we use *functions*
- Examples:

```
do_report();
x = getAnswer(a,b);
findLargest(list);
sort(l,size);
```

- C provides us with many built-in function abstractions:

```
scanf("%d",&x);
float a = sin(n);
n = y + atof(str1);
```



Operation Abstractions

- To create abstractions for actions or operations we use *functions*
- Examples:

```
do_report();
x = getAnswer(a,b);
findLargest(list);
sort(l,size);
```

- C provides us with many built-in function abstractions:

```
scanf("%d",&x);
float a = sin(n);
n = y + atof(str1);
```



Operation Abstractions

- To create abstractions for actions or operations we use *functions*
- Examples:

```
do_report();
x = getAnswer(a,b);
findLargest(list);
sort(l,size);
```

- C provides us with many built-in function abstractions:

```
scanf("%d",&x);
float a = sin(n);
n = y + atof(str1);
```



Abstraction and Program Development

- Most real-world modern programs are so complex it is impossible for one to keep track of *all* the details in the code at any given moment
- Therefore it is essential to build layers and layers of abstractions
- Using functions and stepwise refinement along with TDD allows us to handle much of this complexity
- In this section we will look at another way to help reduce complexity by adding more layers of abstractions: **modules**



Abstraction and Program Development

- Most real-world modern programs are so complex it is impossible for one to keep track of *all* the details in the code at any given moment
- Therefore it is essential to build layers and layers of abstractions
- Using functions and stepwise refinement along with TDD allows us to handle much of this complexity
- In this section we will look at another way to help reduce complexity by adding more layers of abstractions: **modules**



Abstraction and Program Development

- Most real-world modern programs are so complex it is impossible for one to keep track of *all* the details in the code at any given moment
- Therefore it is essential to build layers and layers of abstractions
- Using functions and stepwise refinement along with TDD allows us to handle much of this complexity
- In this section we will look at another way to help reduce complexity by adding more layers of abstractions: **modules**



Abstraction and Program Development

- Most real-world modern programs are so complex it is impossible for one to keep track of *all* the details in the code at any given moment
- Therefore it is essential to build layers and layers of abstractions
- Using functions and stepwise refinement along with TDD allows us to handle much of this complexity
- In this section we will look at another way to help reduce complexity by adding more layers of abstractions: **modules**



Introduction to Modules

- A **module** is a potentially separately-compiled, reusable software construct consisting of code and data that is usually used to represent an object or set of operations in a program
- Remember that functions are used to define operation abstractions by encapsulating algorithms and data
- Modules are often used to define object abstractions by encapsulating functions and data



Introduction to Modules

- A **module** is a potentially separately-compiled, reusable software construct consisting of code and data that is usually used to represent an object or set of operations in a program
- Remember that functions are used to define operation abstractions by encapsulating algorithms and data
- Modules are often used to define object abstractions by encapsulating functions and data



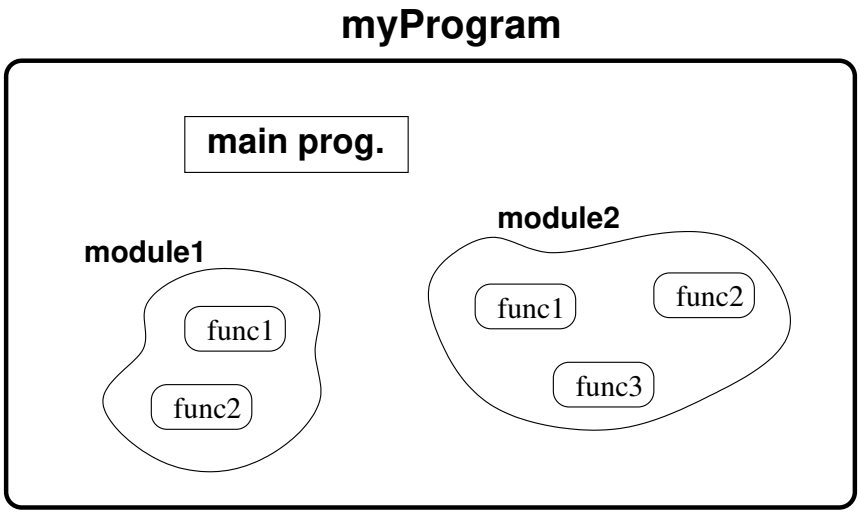
Introduction to Modules

- A **module** is a potentially separately-compiled, reusable software construct consisting of code and data that is usually used to represent an object or set of operations in a program
- Remember that functions are used to define operation abstractions by encapsulating algorithms and data
- Modules are often used to define object abstractions by encapsulating functions and data



Introduction to Modules

- A **module** is a potentially separately-compiled, reusable software construct consisting of code and data that is usually used to represent an object or set of operations in a program
- Remember that functions are used to define operation abstractions by encapsulating algorithms and data
- Modules are often used to define object abstractions by encapsulating functions and data



Introduction to Modules *continued*

- Abstractly modules can be thought of indepentant mechanisms (code) that provide services to your program
- Each module has an **interface** that describes the available services
- The details of the module, referred to as the **implementation**, is often hidden from the program that uses it
- The C library contains many modules for you to use
- An example: the C math library accessed via `math.h`

  https://en.wikipedia.org/wiki/C_mathematical_functions#Overview_of_functions
- It is possible to define and use your own modules



Introduction to Modules *continued*

- Abstractly modules can be thought of indepentant mechanisms (code) that provide services to your program
- Each module has an **interface** that describes the available services
- The details of the module, referred to as the **implementation**, is often hidden from the program that uses it
- The C library contains many modules for you to use
- An example: the C math library accessed via `math.h`

  https://en.wikipedia.org/wiki/C_mathematical_functions#Overview_of_functions
- It is possible to define and use your own modules



Introduction to Modules *continued*

- Abstractly modules can be thought of indepentant mechanisms (code) that provide services to your program
- Each module has an **interface** that describes the available services
- The details of the module, referred to as the **implementation**, is often hidden from the program that uses it
- The C library contains many modules for you to use
- An example: the C math library accessed via `math.h`

  `https://en.wikipedia.org/wiki/C_mathematical_functions#Overview_of_functions`
- It is possible to define and use your own modules



Introduction to Modules *continued*

- Abstractly modules can be thought of indepentant mechanisms (code) that provide services to your program
- Each module has an **interface** that describes the available services
- The details of the module, referred to as the **implementation**, is often hidden from the program that uses it
- The C library contains many modules for you to use
- An example: the C math library accessed via `math.h`

  https://en.wikipedia.org/wiki/C_mathematical_functions#Overview_of_functions
- It is possible to define and use your own modules



Introduction to Modules *continued*

- Abstractly modules can be thought of indepentant mechanisms (code) that provide services to your program
- Each module has an **interface** that describes the available services
- The details of the module, referred to as the **implementation**, is often hidden from the program that uses it
- The C library contains many modules for you to use
- An example: the C math library accessed via `math.h`

  `https://en.wikipedia.org/wiki/C_mathematical_functions#Overview_of_functions`
- It is possible to define and use your own modules



Introduction to Modules *continued*

- Abstractly modules can be thought of indepentant mechanisms (code) that provide services to your program
- Each module has an **interface** that describes the available services
- The details of the module, referred to as the **implementation**, is often hidden from the program that uses it
- The C library contains many modules for you to use
- An example: the C math library accessed via `math.h`

  `https://en.wikipedia.org/wiki/C_mathematical_functions#Overview_of_functions`
- It is possible to define and use your own modules



Introduction to Modules *continued*

- Dividing a program into modules has several advantages:

  abstraction allows us to cleanly separate the **what** from the **how**, i.e. to use code without having to concern ourselves about how it is implemented

  reusability can reuse useful modules across many programs

  maintainability updating code or fixing a bug in a single module will affect all programs that use that module (all you will need to do is recompile the programs that use the module)



Introduction to Modules *continued*

- Dividing a program into modules has several advantages:

  abstraction allows us to cleanly separate the **what** from the **how**, i.e. to use code without having to concern ourselves about how it is implemented

  reusability can reuse useful modules across many programs

  maintainability updating code or fixing a bug in a single module will affect all programs that use that module (all you will need to do is recompile the programs that use the module)



Introduction to Modules *continued*

- Dividing a program into modules has several advantages:

abstraction	allows us to cleanly separate the **what** from the **how**, i.e. to use code without having to concern ourselves about how it is implemented
reusability	can reuse useful modules across many programs
maintainability	updating code or fixing a bug in a single module will affect all programs that use that module (all you will need to do is recompile the programs that use the module)



Introduction to Modules *continued*

- Dividing a program into modules has several advantages:

abstraction	allows us to cleanly separate the **what** from the **how**, i.e. to use code without having to concern ourselves about how it is implemented
reusability	can reuse useful modules across many programs
maintainability	updating code or fixing a bug in a single module will affect all programs that use that module (all you will need to do is recompile the programs that use the module)



Creating Separately Compiled Modules

- Suppose we wish to create a separately compiled module called `mymodule` that contains the `int` variable `x` and two functions – `getX()` and `setX()`
- We want to use it like this – file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000;
  printf("value from getX(): %d\n",getX());
  return 0;
}
```



Creating Separately Compiled Modules

- Suppose we wish to create a separately compiled module called `mymodule` that contains the `int` variable `x` and two functions – `getX()` and `setX()`
- We want to use it like this – file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000;
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

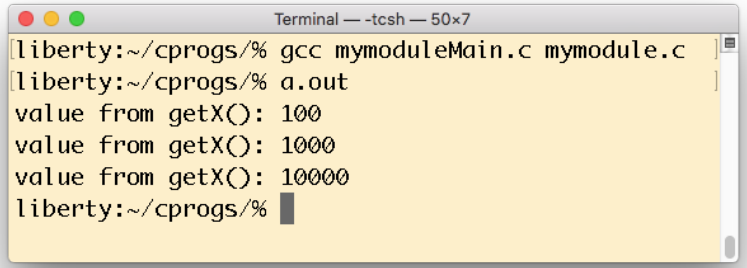


Creating Separately Compiled Modules

- Suppose we wish to create a separately compiled module called `mymodule` that contains the `int` variable `x` and two functions – `getX()` and `setX()`
- We want to use it like this – file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000;
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

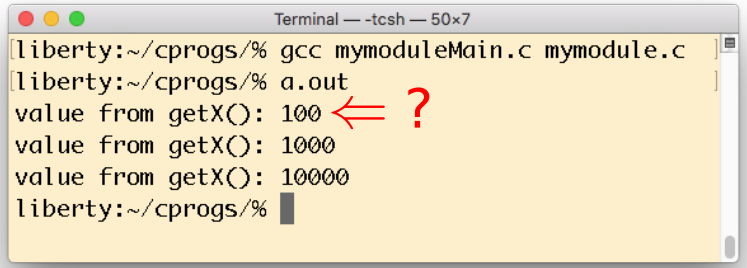


Creating Separately Compiled Modules

- Suppose we wish to create a separately compiled module called `mymodule` that contains the `int` variable `x` and two functions – `getX()` and `setX()`
- We want to use it like this – file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000;
  printf("value from getX(): %d\n",getX());
  return 0;
}
```



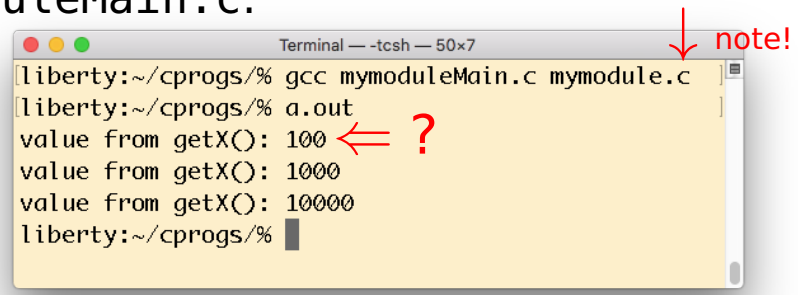
Creating Separately Compiled Modules

- Suppose we wish to create a separately compiled module called `mymodule` that contains the `int` variable `x` and two functions – `getX()` and `setX()`
- We want to use it like this – file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"
```

note!

```
int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000;
  printf("value from getX(): %d\n",getX());
  return 0;
}
```



Creating Separately Compiled Modules *continued*

- Though not required by the language, we are going to separate our modules into two files – a **specification** file (the "what") and an **implementation** file (the "how")
- Specification files will be header files (i.e. '`.h`' files) and the implementations will be C code files (i.e. '`.c`' files) without a `main` function
- Specification files will usually include only function prototypes and/or variable and type definitions



Creating Separately Compiled Modules *continued*

- Though not required by the language, we are going to separate our modules into two files – a **specification** file (the "what") and an **implementation** file (the "how")
- Specification files will be header files (i.e. '`.h`' files) and the implementations will be C code files (i.e. '`.c`' files) without a `main` function
- Specification files will usually include only function prototypes and/or variable and type definitions



Creating Separately Compiled Modules *continued*

- Though not required by the language, we are going to separate our modules into two files – a **specification** file (the "what") and an **implementation** file (the "how")
- Specification files will be header files (i.e. '`.h`' files) and the implementations will be C code files (i.e. '`.c`' files) without a `main` function
- Specification files will usually include only function prototypes and/or variable and type definitions



Creating Separately Compiled Modules *continued*

- Specification file `mymodule.h`:

- The file `mymodule.h` will be included into your main program file *and* `mymodule.c` (your module implementation file; see next page)
- A header guard is necessary so the definitions found in your header file are not included more than once in your code (which would produce a compiler error)



Creating Separately Compiled Modules *continued*

- Specification file `mymodule.h`:

```
int getX();
void setX(int);
```

function prototype(s)

- The file `mymodule.h` will be included into your main program file *and* `mymodule.c` (your module implementation file; see next page)
- A header guard is necessary so the definitions found in your header file are not included more than once in your code (which would produce a compiler error)



Creating Separately Compiled Modules *continued*

- Specification file `mymodule.h`:

```
extern int x;

int getX();
void setX(int);
```

variable(s) (and types)

function prototype(s)

- The file `mymodule.h` will be included into your main program file *and* `mymodule.c` (your module implementation file; see next page)
- A header guard is necessary so the definitions found in your header file are not included more than once in your code (which would produce a compiler error)



Creating Separately Compiled Modules *continued*

- Specification file `mymodule.h`:

```
#ifndef MYMODULE_H
#define MYMODULE_H

extern int x;

int getX();
void setX(int);

#endif
```

header guard

variable(s) (and types)

function prototype(s)

note!

- The file `mymodule.h` will be included into your main program file *and* `mymodule.c` (your module implementation file; see next page)
- A header guard is necessary so the definitions found in your header file are not included more than once in your code (which would produce a compiler error)



Creating Separately Compiled Modules *continued*

- Specification file `mymodule.h`:

```
#ifndef MYMODULE_H
#define MYMODULE_H

extern int x;

int getX();
void setX(int);

#endif
```

header guard

variable(s) (and types)

function prototype(s)

note!

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX():  %\n",getX());
  setX(1000);
  printf("value from getX():  %d\n",getX());
  x = 10000;
  printf("value from getX():  %d\n",getX());
  return 0;
}
```

- The file `mymodule.h` will be included into your main program file *and* `mymodule.c` (your module implementation file; see next page)
- A header guard is necessary so the definitions found in your header file are not included more than once in your code (which would produce a compiler error)



Creating Separately Compiled Modules *continued*

- Specification file `mymodule.h`:

```
#ifndef MYMODULE_H
#define MYMODULE_H

extern int x;

int getX();
void setX(int);

#endif
```

header guard

variable(s) (and types)

function prototype(s)

note!

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX():  %\n",getX());
  setX(1000);
  printf("value from getX():  %d\n",getX());
  x = 10000;
  printf("value from getX():  %d\n",getX());
  return 0;
}
```

- The file `mymodule.h` will be included into your main program file *and* `mymodule.c` (your module implementation file; see next page)
- A header guard is necessary so the definitions found in your header file are not included more than once in your code (which would produce a compiler error)

Creating Separately Compiled Modules *continued*

- Implementation file `mymodule.c`:

```
#include "mymodule.h"

int x = 100;

int getX() { return x; }
void setX(int n) { x = n; }
```

- Can compile separately: `gcc -c mymodule.c`
- ... or together with the main program:
  `gcc mymoduleMain.c mymodule.c`



Creating Separately Compiled Modules *continued*

- Implementation file `mymodule.c`:

```
#include "mymodule.h"

int x = 100;

int getX() { return x; }
void setX(int n) { x = n; }
```

- Can compile separately: `gcc -c mymodule.c`
- ... or together with the main program:
  `gcc mymoduleMain.c mymodule.c`



Creating Separately Compiled Modules *continued*

- Implementation file `mymodule.c`:

```
#include "mymodule.h"

int x = 100;

int getX() { return x; }
void setX(int n) { x = n; }
```

- Can compile separately: `gcc -c mymodule.c`
- ... or together with the main program:
  `gcc mymoduleMain.c mymodule.c`



Creating Separately Compiled Modules *continued*

- Let's take another look at file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000;
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

- Notice how we directly assign a value to module variable x
- Allowing a client program to directly manipulate data that is encapsulated in a module violates a programming practice called *information hiding*
- **Information hiding** is the process of not allowing clients to *directly* modify any data encapsulated in a module
- Providing a means to protect data from arbitrary outside modification assures **data integrity** of encapsulated data
- Most modern languages provide a means to protect encapsulated data in a module so that clients cannot directly modify it



Creating Separately Compiled Modules *continued*

- Let's take another look at file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000; <=
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

- Notice how we directly assign a value to module variable x
- Allowing a client program to directly manipulate data that is encapsulated in a module violates a programming practice called *information hiding*
- **Information hiding** is the process of not allowing clients to *directly* modify any data encapsulated in a module
- Providing a means to protect data from arbitrary outside modification assures **data integrity** of encapsulated data
- Most modern languages provide a means to protect encapsulated data in a module so that clients cannot directly modify it



Creating Separately Compiled Modules *continued*

- Let's take another look at file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000; <=
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

- Notice how we directly assign a value to module variable x
- Allowing a client program to directly manipulate data that is encapsulated in a module violates a programming practice called *information hiding*
- **Information hiding** is the process of not allowing clients to *directly* modify any data encapsulated in a module
- Providing a means to protect data from arbitrary outside modification assures **data integrity** of encapsulated data
- Most modern languages provide a means to protect encapsulated data in a module so that clients cannot directly modify it



Creating Separately Compiled Modules *continued*

- Let's take another look at file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000; <=
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

- Notice how we directly assign a value to module variable x
- Allowing a client program to directly manipulate data that is encapsulated in a module violates a programming practice called *information hiding*
- **Information hiding** is the process of not allowing clients to *directly* modify any data encapsulated in a module
- Providing a means to protect data from arbitrary outside modification assures **data integrity** of encapsulated data
- Most modern languages provide a means to protect encapsulated data in a module so that clients cannot directly modify it



Creating Separately Compiled Modules *continued*

- Let's take another look at file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000; <=
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

- Notice how we directly assign a value to module variable x
- Allowing a client program to directly manipulate data that is encapsulated in a module violates a programming practice called *information hiding*
- **Information hiding** is the process of not allowing clients to *directly* modify any data encapsulated in a module
- Providing a means to protect data from arbitrary outside modification assures **data integrity** of encapsulated data
- Most modern languages provide a means to protect encapsulated data in a module so that clients cannot directly modify it



Creating Separately Compiled Modules *continued*

- Let's take another look at file `mymoduleMain.c`:

```
#include <stdio.h>
#include "mymodule.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000; <=
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

- Notice how we directly assign a value to module variable x
- Allowing a client program to directly manipulate data that is encapsulated in a module violates a programming practice called *information hiding*
- **Information hiding** is the process of not allowing clients to *directly* modify any data encapsulated in a module
- Providing a means to protect data from arbitrary outside modification assures **data integrity** of encapsulated data
- Most modern languages provide a means to protect encapsulated data in a module so that clients cannot directly modify it



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add 'x = -10;' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`!
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add 'x = -10;' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`!
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add '`x = -10;`' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`!
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add '`x = -10;`' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`!
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add 'x = -10;' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`!
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add '`x = -10;`' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`!
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add '`x = -10;`' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`! `void setX(int n) { x = n; }`
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add '`x = -10;`' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`! `void setX(int n) { x = n; }`
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add '`x = -10;`' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`! `void setX(int n) { x = n; }`
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add '`x = -10;`' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`! `void setX(int n) { x = n; }`
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```

- forcing the programmer to access/modify data only through getters and setters ensures **data integrity**



Creating Separately Compiled Modules *continued*

- Let's assume the variable x in our example module should never be negative
- Notice that the way we have defined it currently a programmer could add '`x = -10;`' to their code!
- We would like to provide a mechanism so the programmer could not do this
- One possible way would be to first somehow prevent the coder from directly accessing x altogether (more on how to do this later)
- However – if we did that, how might the client actually change x?
- By using our module routine `setX(int)`! `void setX(int n) { x = n; }`
- In this manner we control access to the hidden variable x
- Then we would modify `setX(int)`:

```
void setX(int n) {
  if(x >= 0) x = n;
}
```

- forcing the programmer to access/modify data only through getters and setters ensures **data integrity**
- best practices: declare all data to be private (how? see next page) and force all access through getters/setters



Creating Separately Compiled Modules *continued*

- New modifed file `mymodule1.h` (note '`extern int x;`' has been removed):

```
#ifndef MYMODULE_H
#define MYMODULE_H

int getX();
void setX(int);

#endif
```

- New modified file `mymodule1.c`:

```
#include "mymodule1.h"

static int x = 100;

int getX() { return x; }
void setX(int n) { if (x >= 0) x = n; }
```



Creating Separately Compiled Modules *continued*

- New modifed file `mymodule1.h` (note '`extern int x;`' has been removed):

```
#ifndef MYMODULE_H
#define MYMODULE_H

int getX();
void setX(int);

#endif
```

- New modified file `mymodule1.c`:

```
#include "mymodule1.h"

static int x = 100;

int getX() { return x; }
void setX(int n) { if (x >= 0) x = n; }
```



Creating Separately Compiled Modules *continued*

- New modifed file `mymodule1.h` (note '`extern int x;`' has been removed):

```
#ifndef MYMODULE_H
#define MYMODULE_H

int getX();
void setX(int);

#endif
```

- New modified file `mymodule1.c`:

```
#include "mymodule1.h"

static int x = 100;

int getX() { return x; }
void setX(int n) { if (x >= 0) x = n; }
```

- static variables are only accessible within the file where they are declared



Creating Separately Compiled Modules *continued*

- New modifed file `mymodule1.h` (note '`extern int x;`' has been removed):

```
#ifndef MYMODULE_H
#define MYMODULE_H

int getX();
void setX(int);

#endif
```

- New modified file `mymodule1.c`:

```
#include "mymodule1.h"

static int x = 100;

int getX() { return x; }
void setX(int n) { if (x >= 0) x = n; }
```

- static variables are only accessible within the file where they are declared
- Therefore declaring a variable static is how you "hide" data in modules



Creating Separately Compiled Modules *continued*

- New modifed file `mymodule1.h` (note '`extern int x;`' has been removed):

```
#ifndef MYMODULE_H
#define MYMODULE_H

int getX();
void setX(int);

#endif
```

- New modified file `mymodule1.c`:

```
#include "mymodule1.h"

static int x = 100;

int getX() { return x; }
void setX(int n) { if (x >= 0) x = n; }
```

- static variables are only accessible within the file where they are declared
- Therefore declaring a variable static is how you "hide" data in modules
- Functions may also be declared static



Creating Separately Compiled Modules *continued*

- File `mymoduleMain1.c`:

```
#include <stdio.h>
#include "mymodule1.h" // <-- note

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000;
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

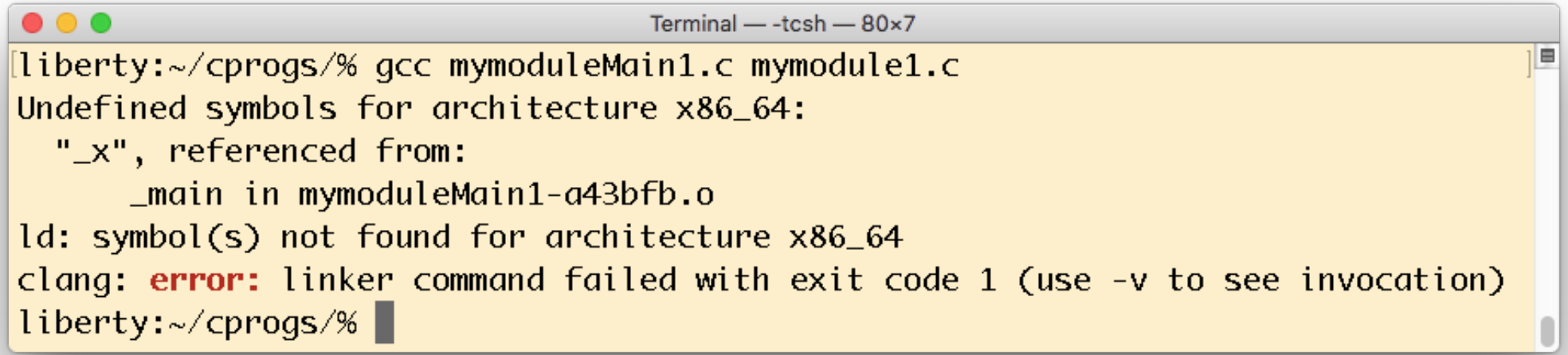


Creating Separately Compiled Modules *continued*

- File `mymoduleMain1.c`:

```
#include <stdio.h>
#include "mymodule1.h" // <-- note

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(1000);
  printf("value from getX(): %d\n",getX());
  x = 10000;
  printf("value from getX(): %d\n",getX());
  return 0;
}
```



Creating Separately Compiled Modules *continued*

- File `mymoduleMain2.c`:

```
#include <stdio.h>
#include "mymodule1.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(999);
  printf("value from getX(): %d\n",getX());
  return 0;
}
```

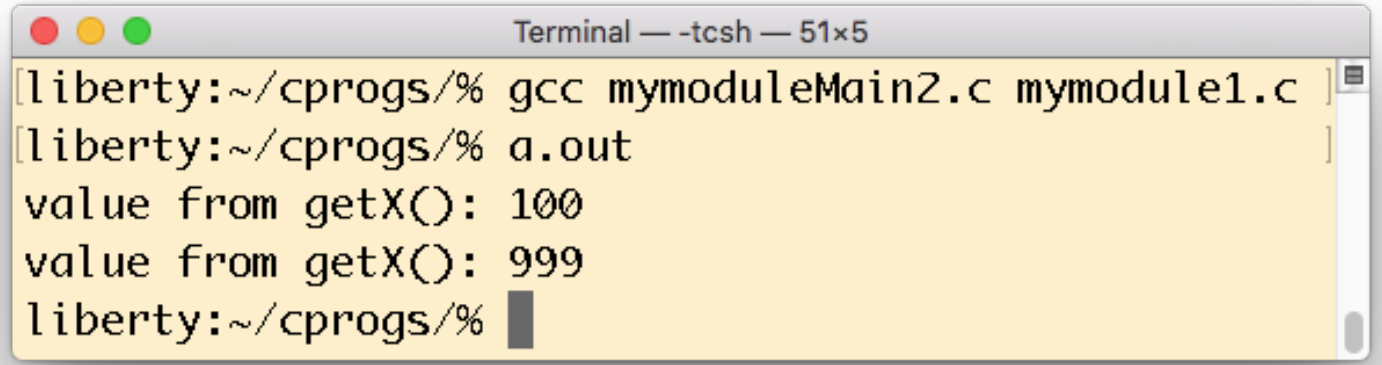


Creating Separately Compiled Modules *continued*

- File `mymoduleMain2.c`:

```
#include <stdio.h>
#include "mymodule1.h"

int main(void){
  printf("value from getX(): %d\n",getX());
  setX(999);
  printf("value from getX(): %d\n",getX());
  return 0;
}
```



Definitions

- A **data type** consists of a collection of values together with a set of basic operations defined on these values
- For example, the `int` data type defines a collection of legal values (`INT_MIN...INT_MAX`) and a set of operations on int values (e.g. $+, -, >=, ==$, etc.)
- Traditionally data types are *atomic* entities meaning they cannot be subdivided into component parts
- A data type that does not give a programmer access to the details of how the values and operations are implemented is called an **abstract data type** (**ADT**)
- We will be interested in defining useful ADTs this semester



Definitions

- A **data type** consists of a collection of values together with a set of basic operations defined on these values
- For example, the `int` data type defines a collection of legal values (`INT_MIN...INT_MAX`) and a set of operations on int values (e.g. $+, -, >=, =$, etc.)
- Traditionally data types are *atomic* entities meaning they cannot be subdivided into component parts
- A data type that does not give a programmer access to the details of how the values and operations are implemented is called an **abstract data type** (**ADT**)
- We will be interested in defining useful ADTs this semester



Definitions

- A **data type** consists of a collection of values together with a set of basic operations defined on these values
- For example, the `int` data type defines a collection of legal values (`INT_MIN...INT_MAX`) and a set of operations on int values (e.g. $+,-,>=,=$, etc.)
- Traditionally data types are *atomic* entities meaning they cannot be subdivided into component parts
- A data type that does not give a programmer access to the details of how the values and operations are implemented is called an **abstract data type** (**ADT**)
- We will be interested in defining useful ADTs this semester



Definitions

- A **data type** consists of a collection of values together with a set of basic operations defined on these values
- For example, the `int` data type defines a collection of legal values (`INT_MIN...INT_MAX`) and a set of operations on int values (e.g. $+,-,>=,=$, etc.)
- Traditionally data types are *atomic* entities meaning they cannot be subdivided into component parts
- A data type that does not give a programmer access to the details of how the values and operations are implemented is called an **abstract data type** (**ADT**)
- We will be interested in defining useful ADTs this semester



Definitions

- A **data type** consists of a collection of values together with a set of basic operations defined on these values
- For example, the `int` data type defines a collection of legal values (`INT_MIN...INT_MAX`) and a set of operations on int values (e.g. $+,-,>=,=$, etc.)
- Traditionally data types are *atomic* entities meaning they cannot be subdivided into component parts
- A data type that does not give a programmer access to the details of how the values and operations are implemented is called an **abstract data type** (**ADT**)
- We will be interested in defining useful ADTs this semester



Definitions *continued*

- A **data structure** is an abstract description of a way of organizing data to allow certain operations on it to be performed efficiently
- Therefore like a data type it too consists of values and operations on those values
- However traditionally data structures are builtup from component parts and therefore can be decomposed into other values
- For example: a list of strings
- A data structure is organized in that it consists of a set of associations or relationships (structure) involving its component elements
- For example arrays are data structures – they consist of a set of values with both relationships between those values and a set of operations on arrays
- We will define several useful data structures this semester



Definitions *continued*

- A **data structure** is an abstract description of a way of organizing data to allow certain operations on it to be performed efficiently
- Therefore like a data type it too consists of values and operations on those values
- However traditionally data structures are builtup from component parts and therefore can be decomposed into other values
- For example: a list of strings
- A data structure is organized in that it consists of a set of associations or relationships (structure) involving its component elements
- For example arrays are data structures – they consist of a set of values with both relationships between those values and a set of operations on arrays
- We will define several useful data structures this semester



Definitions *continued*

- A **data structure** is an abstract description of a way of organizing data to allow certain operations on it to be performed efficiently
- Therefore like a data type it too consists of values and operations on those values
- However traditionally data structures are builtup from component parts and therefore can be decomposed into other values
- For example: a list of strings
- A data structure is organized in that it consists of a set of associations or relationships (structure) involving its component elements
- For example arrays are data structures – they consist of a set of values with both relationships between those values and a set of operations on arrays
- We will define several useful data structures this semester



Definitions *continued*

- A **data structure** is an abstract description of a way of organizing data to allow certain operations on it to be performed efficiently
- Therefore like a data type it too consists of values and operations on those values
- However traditionally data structures are builtup from component parts and therefore can be decomposed into other values
- For example: a list of strings
- A data structure is organized in that it consists of a set of associations or relationships (structure) involving its component elements
- For example arrays are data structures – they consist of a set of values with both relationships between those values and a set of operations on arrays
- We will define several useful data structures this semester



Definitions *continued*

- A **data structure** is an abstract description of a way of organizing data to allow certain operations on it to be performed efficiently
- Therefore like a data type it too consists of values and operations on those values
- However traditionally data structures are builtup from component parts and therefore can be decomposed into other values
- For example: a list of strings
- A data structure is organized in that it consists of a set of associations or relationships (structure) involving its component elements
- For example arrays are data structures – they consist of a set of values with both relationships between those values and a set of operations on arrays
- We will define several useful data structures this semester



Definitions *continued*

- A **data structure** is an abstract description of a way of organizing data to allow certain operations on it to be performed efficiently
- Therefore like a data type it too consists of values and operations on those values
- However traditionally data structures are builtup from component parts and therefore can be decomposed into other values
- For example: a list of strings
- A data structure is organized in that it consists of a set of associations or relationships (structure) involving its component elements
- For example arrays are data structures – they consist of a set of values with both relationships between those values and a set of operations on arrays
- We will define several useful data structures this semester



Definitions *continued*

- A **data structure** is an abstract description of a way of organizing data to allow certain operations on it to be performed efficiently
- Therefore like a data type it too consists of values and operations on those values
- However traditionally data structures are builtup from component parts and therefore can be decomposed into other values
- For example: a list of strings
- A data structure is organized in that it consists of a set of associations or relationships (structure) involving its component elements
- For example arrays are data structures – they consist of a set of values with both relationships between those values and a set of operations on arrays
- We will define several useful data structures this semester



Example Data Structure: Lists

- A **list** is a data type that represents a countable number of ordered values where the same value may occur more than once.
- Generally lists will be empty or non-empty with a first item, a sequence of ordered items, a last item, and the ability to update the list and fetch items in the list
- Note that the concept and operations on a list are independent of the actual type of values stored in a list
- Our list data structure is a **container** – a type-independant data structure used to store, organize, and access data
- To define a **data structure** for a list we will need:



Example Data Structure: Lists

- A **list** is a data type that represents a countable number of ordered values where the same value may occur more than once.
- Generally lists will be empty or non-empty with a first item, a sequence of ordered items, a last item, and the ability to update the list and fetch items in the list
- Note that the concept and operations on a list are independent of the actual type of values stored in a list
- Our list data structure is a **container** – a type-independant data structure used to store, organize, and access data
- To define a **data structure** for a list we will need:



Example Data Structure: Lists

- A **list** is a data type that represents a countable number of ordered values where the same value may occur more than once.
- Generally lists will be empty or non-empty with a first item, a sequence of ordered items, a last item, and the ability to update the list and fetch items in the list
- Note that the concept and operations on a list are independent of the actual type of values stored in a list
- Our list data structure is a **container** – a type-independant data structure used to store, organize, and access data
- To define a **data structure** for a list we will need:



Example Data Structure: Lists

- A **list** is a data type that represents a countable number of ordered values where the same value may occur more than once.
- Generally lists will be empty or non-empty with a first item, a sequence of ordered items, a last item, and the ability to update the list and fetch items in the list
- Note that the concept and operations on a list are independent of the actual type of values stored in a list
- Our list data structure is a **container** – a type-independant data structure used to store, organize, and access data
- To define a **data structure** for a list we will need:
  - type definitions for the set of values



Example Data Structure: Lists

- A **list** is a data type that represents a countable number of ordered values where the same value may occur more than once.
- Generally lists will be empty or non-empty with a first item, a sequence of ordered items, a last item, and the ability to update the list and fetch items in the list
- Note that the concept and operations on a list are independent of the actual type of values stored in a list
- Our list data structure is a **container** – a type-independant data structure used to store, organize, and access data
- To define a **data structure** for a list we will need:
  - type definitions for the **set of values**
  - functions defintions for the **operations**



Example Data Structure: Lists

- A **list** is a data type that represents a countable number of ordered values where the same value may occur more than once.
- Generally lists will be empty or non-empty with a first item, a sequence of ordered items, a last item, and the ability to update the list and fetch items in the list
- Note that the concept and operations on a list are independent of the actual type of values stored in a list
- Our list data structure is a **container** – a type-independant data structure used to store, organize, and access data
- To define a **data structure** for a list we will need:
  - type definitions for the **set of values**
  - functions defintions for the **operations**



Example Data Structure: Lists

- A **list** is a data type that represents a countable number of ordered values where the same value may occur more than once.
- Generally lists will be empty or non-empty with a first item, a sequence of ordered items, a last item, and the ability to update the list and fetch items in the list
- Note that the concept and operations on a list are independent of the actual type of values stored in a list
- Our list data structure is a **container** – a type-independant data structure used to store, organize, and access data
- To define a **data structure** for a list we will need:
  - type definitions for the **set of values**
  - functions defintions for the **operations**



Example Data Structure: Lists - Defining the List Values

- We wish to define the *values* in our list in such a way that it is easy to change
- We will create an abstraction called `listItem` that will be used by all our operations as the type of items in the list
- Then to change our list item type we only need to change a single definition and then recompile
- It is often useful (but not always!) to define a "non list item" as well so when we wish an operation to return a list item that is not valid data



Example Data Structure: Lists - Defining the List Values

- We wish to define the *values* in our list in such a way that it is easy to change
- We will create an abstraction called `listItem` that will be used by all our operations as the type of items in the list
- Then to change our list item type we only need to change a single definition and then recompile
- It is often useful (but not always!) to define a “non list item” as well so when we wish an operation to return a list item that is not valid data



Example Data Structure: Lists - Defining the List Values

- We wish to define the *values* in our list in such a way that it is easy to change
- We will create an abstraction called `listItem` that will be used by all our operations as the type of items in the list
- Then to change our list item type we only need to change a single definition and then recompile
- It is often useful (but not always!) to define a “non list item” as well so when we wish an operation to return a list item that is not valid data



Example Data Structure: Lists - Defining the List Values

- We wish to define the *values* in our list in such a way that it is easy to change
- We will create an abstraction called `listItem` that will be used by all our operations as the type of items in the list
- Then to change our list item type we only need to change a single definition and then recompile
- It is often useful (but not always!) to define a “non list item” as well so when we wish an operation to return a list item that is not valid data



Example Data Structure: Lists - Defining the List Values

- We wish to define the *values* in our list in such a way that it is easy to change
- We will create an abstraction called `listItem` that will be used by all our operations as the type of items in the list
- Then to change our list item type we only need to change a single definition and then recompile
- It is often useful (but not always!) to define a “non list item” as well so when we wish an operation to return a list item that is not valid data

```
typedef int listItem;
#define NULL_ITEM INT_MIN
```



Example Data Structure: Lists - Defining the List Values

- We wish to define the *values* in our list in such a way that it is easy to change
- We will create an abstraction called `listItem` that will be used by all our operations as the type of items in the list
- Then to change our list item type we only need to change a single definition and then recompile
- It is often useful (but not always!) to define a “non list item” as well so when we wish an operation to return a list item that is not valid data

```
typedef int listItem;
#define NULL_ITEM INT_MIN
```

What is the drawback to defining this as a non-item?



Example Data Structure: Lists - Defining the List Structure

- How do we wish to store these values in our list?
- Perhaps the easiest way is to store values in an array of fixed size
- We now will be able to declare list variables: `list l1, l2, l3;`
- But since these are pointer variables pointing to nothing, we will need to write an operation that "creates" a valid list before we can use it



Example Data Structure: Lists - Defining the List Structure

- How do we wish to store these values in our list?
- Perhaps the easiest way is to store values in an array of fixed size

- We now will be able to declare list variables: `list l1, l2, l3;`
- But since these are pointer variables pointing to nothing, we will need to write an operation that "creates" a valid list before we can use it



Example Data Structure: Lists - Defining the List Structure

- How do we wish to store these values in our list?
- Perhaps the easiest way is to store values in an array of fixed size

```
typedef struct listType *list;

#define LIST_SIZE 100 // arbitrary
struct listType {
 listItem data[LIST_SIZE];
 int count;
 int maxSize;
};
```

- We now will be able to declare list variables: `list l1, l2, l3;`
- But since these are pointer variables pointing to nothing, we will need to write an operation that "creates" a valid list before we can use it



Example Data Structure: Lists - Defining the List Structure

- How do we wish to store these values in our list?
- Perhaps the easiest way is to store values in an array of fixed size

```
typedef struct listType *list;
```

Why define it as a pointer? (it's somewhat arbitrary)

```
#define LIST_SIZE 100 // arbitrary
struct listType {
 listItem data[LIST_SIZE];
 int count;
 int maxSize;
};
```

- We now will be able to declare list variables: `list l1, l2, l3;`
- But since these are pointer variables pointing to nothing, we will need to write an operation that "creates" a valid list before we can use it



Example Data Structure: Lists - Defining the List Structure

- How do we wish to store these values in our list?
- Perhaps the easiest way is to store values in an array of fixed size

```
typedef struct listType *list;
```

Why define it as a pointer? (it's somewhat arbitrary)

```
#define LIST_SIZE 100 // arbitrary
struct listType {
 listItem data[LIST_SIZE];
 int count;
 int maxSize;
};
```

- We now will be able to declare list variables: `list l1, l2, l3;`
- But since these are pointer variables pointing to nothing, we will need to write an operation that "creates" a valid list before we can use it



Example Data Structure: Lists - Defining the List Structure

- How do we wish to store these values in our list?
- Perhaps the easiest way is to store values in an array of fixed size

```
typedef struct listType *list;
```

Why define it as a pointer? (it's somewhat arbitrary)

```
#define LIST_SIZE 100 // arbitrary
struct listType {
 listItem data[LIST_SIZE];
 int count;
 int maxSize;
};
```

- We now will be able to declare list variables: `list l1, l2, l3;`
- But since these are pointer variables pointing to nothing, we will need to write an operation that "creates" a valid list before we can use it



Container Operations

- Container data structure operations generally fall into one of several possible categories:

constructors	creates a data container (i.e. allocates memory for container)
destructors	destroys a data container (i.e. deallocates memory for container)
getters	used to fetch data from a container
setters	used to change the **state** of a container (also called *mutators*)
predicates	boolean operations that test attributes of a container
reporters	used to report useful information about container



Container Operations

- Container data structure operations generally fall into one of several possible categories:

constructors	creates a data container (i.e. allocates memory for container)
destructors	destroys a data container (i.e. deallocates memory for container)
getters	used to fetch data from a container
setters	used to change the **state** of a container (also called *mutators*)
predicates	boolean operations that test attributes of a container
reporters	used to report useful information about container



Container Operations

- Container data structure operations generally fall into one of several possible categories:

constructors	creates a data container (i.e. allocates memory for container)
destructors	destroys a data container (i.e. deallocates memory for container)
getters	used to fetch data from a container
setters	used to change the **state** of a container (also called *mutators*)
predicates	boolean operations that test attributes of a container
reporters	used to report useful information about container



Container Operations

- Container data structure operations generally fall into one of several possible categories:

constructors	creates a data container (i.e. allocates memory for container)
destructors	destroys a data container (i.e. deallocates memory for container)
getters	used to fetch data from a container
setters	used to change the **state** of a container (also called *mutators*)
predicates	boolean operations that test attributes of a container
reporters	used to report useful information about container



Container Operations

- Container data structure operations generally fall into one of several possible categories:

constructors	creates a data container (i.e. allocates memory for container)
destructors	destroys a data container (i.e. deallocates memory for container)
getters	used to fetch data from a container
setters	used to change the **state** of a container (also called *mutators*)
predicates	boolean operations that test attributes of a container
reporters	used to report useful information about container



Container Operations

- Container data structure operations generally fall into one of several possible categories:

constructors	creates a data container (i.e. allocates memory for container)
destructors	destroys a data container (i.e. deallocates memory for container)
getters	used to fetch data from a container
setters	used to change the **state** of a container (also called *mutators*)
predicates	boolean operations that test attributes of a container
reporters	used to report useful information about container



Container Operations

- Container data structure operations generally fall into one of several possible categories:

constructors	creates a data container (i.e. allocates memory for container)
destructors	destroys a data container (i.e. deallocates memory for container)
getters	used to fetch data from a container
setters	used to change the **state** of a container (also called *mutators*)
predicates	boolean operations that test attributes of a container
reporters	used to report useful information about container



Example Data Structure: Lists - Defining the List Operations

- **constructor**

```
list createList(void);
```

Usage: `list l = createList();`

- **destructor**

```
void destroyList(list);
```

Usage: `destroyList(l);`

- **getter**

```
listItem getItem(list,int);
```

Usage: `listItem i = getItem(l,0); // 0 index is 1st item`



Example Data Structure: Lists - Defining the List Operations

- **constructor**

```
list createList(void);
```

Usage: `list l = createList();`

- **destructor**

```
void destroyList(list);
```

Usage: `destroyList(l);`

- **getter**

```
listItem getItem(list,int);
```

Usage: `listItem i = getItem(l,0); // 0 index is 1st item`



Example Data Structure: Lists - Defining the List Operations

- **constructor**

```
list createList(void);
```

Usage: `list l = createList();`

- **destructor**

```
void destroyList(list);
```

Usage: `destroyList(l);`

- **getter**

```
listItem getItem(list,int);
```

Usage: `listItem i = getItem(l,0); // 0 index is 1st item`



Example Data Structure: Lists - Defining the List Operations *continued*

- **setters (mutators)**

```
int addItem(list,listItem);
```

Usage:

```
// l ≡ [1,2,3]
int i = addItem(l,99);
// l ≡ [1,2,3,99], i ≡ 3
```



Example Data Structure: Lists - Defining the List Operations *continued*

- **setters (mutators)**

```
int addItem(list,listItem);
```

Usage:

```
// l ≡ [1,2,3]
int i = addItem(l,99);
// l ≡ [1,2,3,99], i ≡ 3
```

```
int insertItem(list,int,listItem);
```

Usage:

```
// l ≡ [1,2,3]
int i = insertItem(l,1,99);
// l ≡ [1,99,2,3], i ≡ 1
```



Example Data Structure: Lists - Defining the List Operations *continued*

- **setters (mutators)** *continued*

```
bool removeItem(list,int);
```

Usage:

```
// l ≡ [1,2,3]
bool success = removeItem(l,0);
// l ≡ [2,3], success ≡ true
```



Example Data Structure: Lists - Defining the List Operations *continued*

- **setters (mutators)** *continued*

```
bool removeItem(list,int);
```

Usage:

```
// l ≡ [1,2,3]
bool success = removeItem(l,0);
// l ≡ [2,3], success ≡ true
```

```
void clearList(l);
```

Usage:

```
// l ≡ [1,2,3]
clearList(l);
// l ≡ []
```



Example Data Structure: Lists - Defining the List Operations *continued*

- **setters (mutators)** *continued*

```
bool setItem(list,int,listItem);
```

Usage:

```
// l ≡ [1,2,3]
bool success = setItem(l,2,99);
// l ≡ [1,2,99], success ≡ true
```



Example Data Structure: Lists - Defining the List Operations *continued*

- **predicates**

```
bool listEmpty(l);
```

Usage:

```
// l ≡ [1,2,3]
bool isEmpty = listEmpty(l);
// isEmpty ≡ false
```



Example Data Structure: Lists - Defining the List Operations *continued*

- **predicates**

```
bool listEmpty(l);
```

Usage:

```
// l ≡ [1,2,3]
bool isEmpty = listEmpty(l);
// isEmpty ≡ false
```

```
bool listFull(l);
```

Usage:

```
// l ≡ [1,2,3]
bool isFull = listFull(l);
// isFull ≡ false
```



Example Data Structure: Lists - Defining the List Operations *continued*

- **reporters**

```
int listSize(list);
```

Usage:

```
// l ≡ [1,2,3]
int n = listSize(l);
n ≡ 3
```



Example Data Structure: Lists - Defining the List Operations *continued*

- **reporters**

```
int listSize(list);
```

Usage:

```
// l ≡ [1,2,3]
int n = listSize(l);
n ≡ 3
```

```
int maxSize(l);
```

Usage:

```
// l ≡ [1,2,3]
int n = maxSize(l);
// n ≡ 100 (assuming definitions on previous page)
```



Example Data Structure: Lists - Using the List

- Note that if we separately compiled the implementation of our operations in another file we could actually use our list without knowing how it is implemented



Example Data Structure: Lists - Using the List

- Note that if we separately compiled the implementation of our operations in another file we could actually use our list without knowing how it is implemented

```
// ...
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};

list     createList(void);
void     destroyList(list);
void     clearList(list);
listItem getItem(list,int);
int      addItem(list,listItem);
int      insertItem(list,int,listItem);
bool     removeItem(list,int);
bool     setItem(list,int,listItem);
int      maxSize(list);
int      listSize(list);
bool     listEmpty(list);
```



Example Data Structure: Lists - Using the List

- Note that if we separately compiled the implementation of our operations in another file we could actually use our list without knowing how it is implemented

```
// ...
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};

list     createList(void);
void     destroyList(list);
void     clearList(list);
listItem getItem(list,int);
int      addItem(list,listItem);
int      insertItem(list,int,listItem);
bool     removeItem(list,int);
bool     setItem(list,int,listItem);
int      maxSize(list);
int      listSize(list);
bool     listEmpty(list);
```

```
int main(void){
  int i;
  list l = createList();
  for (i=1; i <=10; i++) addItem(l,i);
  for (i=0; i <10; i++) printf("%d ",getItem(l,i));
  printf("\n");
  destroyList(l);
  return 0;
}
```



Example Data Structure: Lists - Using the List

- Note that if we separately compiled the implementation of our operations in another file we could actually use our list without knowing how it is implemented

```
// ...
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};

list     createList(void);
void     destroyList(list);
void     clearList(list);
listItem getItem(list,int);
int      addItem(list,listItem);
int      insertItem(list,int,listItem);
bool     removeItem(list,int);
bool     setItem(list,int,listItem);
int      maxSize(list);
int      listSize(list);
bool     listEmpty(list);
```

```
int main(void){
  int i;
  list l = createList();
  for (i=1; i <=10; i++) addItem(l,i);
  for (i=0; i <10; i++) printf("%d ",getItem(l,i));
  printf("\n");
  destroyList(l);
  return 0;
}
```

Again – we can use an abstraction without knowing how it is implemented *if* we cleanly separate **what** it does from **how** it does it!



Example Data Structure: Lists - Implementing the List

```
list createList(void) {
  list l = malloc(sizeof(struct listType));
  l->count = 0; l->max = LIST_SIZE;
  return l;
}
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Example Data Structure: Lists - Implementing the List

```
list createList(void) {
  list l = malloc(sizeof(struct listType));
  l->count = 0; l->max = LIST_SIZE;
  return l;
}

void destroyList(list l) { free(l); }
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Example Data Structure: Lists - Implementing the List

```
list createList(void) {
  list l = malloc(sizeof(struct listType));
  l->count = 0; l->max = LIST_SIZE;
  return l;
}

void destroyList(list l) { free(l); }

void clearList(list l) { l->count = 0; }
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Example Data Structure: Lists - Implementing the List

```
list createList(void) {
  list l = malloc(sizeof(struct listType));
  l->count = 0; l->max = LIST_SIZE;
  return l;
}

void destroyList(list l) { free(l); }

void clearList(list l) { l->count = 0; }

int maxSize(list l) { return l->max; }
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Example Data Structure: Lists - Implementing the List

```
list createList(void) {
  list l = malloc(sizeof(struct listType));
  l->count = 0; l->max = LIST_SIZE;
  return l;
}

void destroyList(list l) { free(l); }

void clearList(list l) { l->count = 0; }

int maxSize(list l) { return l->max; }

int listSize(list l) { return l->count; }
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Example Data Structure: Lists - Implementing the List

```
list createList(void) {
  list l = malloc(sizeof(struct listType));
  l->count = 0; l->max = LIST_SIZE;
  return l;
}

void destroyList(list l) { free(l); }

void clearList(list l) { l->count = 0; }

int maxSize(list l) { return l->max; }

int listSize(list l) { return l->count; }

bool listEmpty(list l) { return (l->count==0); }
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN

typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Example Data Structure: Lists - Implementing the List *continued*

```
int addItem(list l,listItem item) {
  if(l->count < maxSize(l)) {
    l->data[l->count++] = item;
    return l->count-1;
  } else return -1;
}
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN
typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Example Data Structure: Lists - Implementing the List *continued*

```
typedef int listItem;
#define NULL_ITEM INT_MIN
typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```

```
int addItem(list l,listItem item) {
  if(l->count < maxSize(l)) {
    l->data[l->count++] = item;
    return l->count-1;
  } else return -1;
}
int insertItem(list l,int pos,listItem item){
  if(l->count < maxSize(l) && pos >= 0 && pos <= listSize(l)) {
    for(int i=l->count-1;i>=pos;i-) l->data[i+1] = l->data[i];
    l->data[pos]=item; l->count++; return pos;
  } else return -1;
}
```



Example Data Structure: Lists - Implementing the List *continued*

```
typedef int listItem;
#define NULL_ITEM INT_MIN
typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```

```
int addItem(list l,listItem item) {
  if(l->count < maxSize(l)) {
    l->data[l->count++] = item;
    return l->count-1;
  } else return -1;
}
int insertItem(list l,int pos,listItem item){
  if(l->count < maxSize(l) && pos >= 0 && pos <= listSize(l)) {
    for(int i=l->count-1;i>=pos;i-) l->data[i+1] = l->data[i];
    l->data[pos]=item; l->count++; return pos;
  } else return -1;
}
bool removeItem(list l,int pos){
  if (pos >= 0 && pos < l->count) {
    for(int i=pos+1;i<l->count;i++) l->data[i-1] = l->data[i];
    l->count-; return true;
  } else return false;
}
```



Example Data Structure: Lists - Implementing the List *continued*

```
listItem getItem(list l, int pos) {
  if (pos >= 0 && pos < l->count) {
    return l->data[pos];
  } else return NULL_ITEM;
}
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN
typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Example Data Structure: Lists - Implementing the List *continued*

```
listItem getItem(list l, int pos) {
  if (pos >= 0 && pos < l->count) {
    return l->data[pos];
  } else return NULL_ITEM;
}

bool setItem(list l, int pos,listItem item) {
  if(listSize(l) > 0 && pos >= 0 && pos < listSize(l)) {
    l->data[pos] = item;
    return true;
  } return false;
}
```

```
typedef int listItem;
#define NULL_ITEM INT_MIN
typedef struct listType *list;

#define LIST_SIZE 10 // arbitrary
struct listType {
  listItem data[LIST_SIZE]; int count, max;
};
```



Creating a List ADT

- Remember that a data structure that hides its implementation is called an ADT
- In C, unlike some other languages, it is impossible to completely hide a data structure's implementation details
- However, there are some common "best practices" or customs and practices on how to write an ADT in C



Creating a List ADT

- Remember that a data structure that hides its implementation is called an ADT
- In C, unlike some other languages, it is impossible to completely hide a data structure's implementation details
- However, there are some common "best practices" or customs and practices on how to write an ADT in C



Creating a List ADT

- Remember that a data structure that hides its implementation is called an ADT
- In C, unlike some other languages, it is impossible to completely hide a data structure's implementation details
- However, there are some common "best practices" or customs and practices on how to write an ADT in C



Creating a List ADT *continued*

- We wish to write a list ADT that allows us to write and execute the following code – file `listADT.c`:

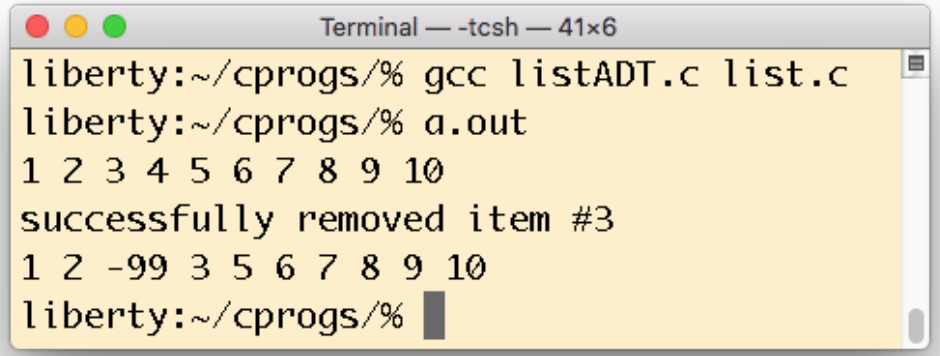
```
#include <stdio.h>
#include "list.h"

int main(void){
  int i,n;
  list l = createList();
  for (i=1; i <=10; i++) addItem(l,i);
  for (i=0; i <10; i++) printf("%d ",getItem(l,i));
  printf("\n");
  if(removeItem(l,3)) printf("successfully removed item #3\n");
  insertItem(l,2,-99);
  i=0;
  while((n=getItem(l,i++))!=NULL_ITEM) printf("%d ",n);
  printf("\n");
  destroyList(l);
  return 0;
}
```



Creating a List ADT *continued*

- We wish to write a list ADT that allows us to write and execute the following code – file `listADT.c`:

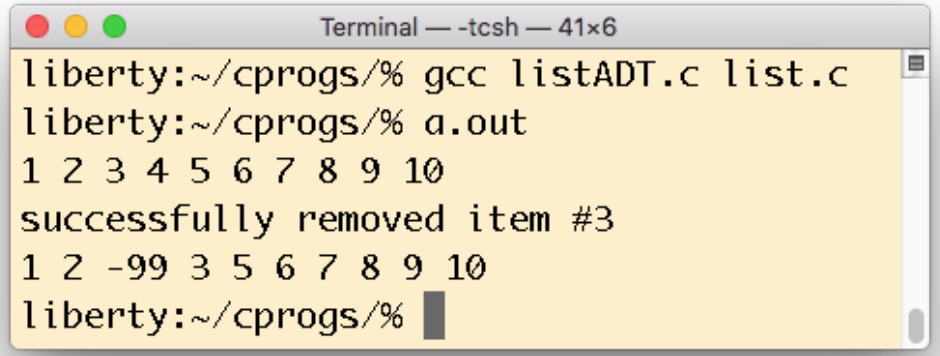



```
#include <stdio.h>
#include "list.h"

int main(void){
  int i,n;
  list l = createList();
  for (i=1; i <=10; i++) addItem(l,i);
  for (i=0; i <10; i++) printf("%d ",getItem(l,i));
  printf("\n");
  if(removeItem(l,3)) printf("successfully removed item #3\n");
  insertItem(l,2,-99);
  i=0;
  while((n=getItem(l,i++))!=NULL_ITEM) printf("%d ",n);
  printf("\n");
  destroyList(l);
  return 0;
}
```

Creating a List ADT *continued*

- We wish to write a list ADT that allows us to write and execute the following code – file `listADT.c`:




```
#include <stdio.h>
#include "list.h"

int main(void){
  int i,n;
  list l = createList();
  for (i=1; i <=10; i++) addItem(l,i);
  for (i=0; i <10; i++) printf("%d ",getItem(l,i));
  printf("\n");
  if(removeItem(l,3)) printf("successfully removed item #3\n");
  insertItem(l,2,-99);
  i=0;
  while((n=getItem(l,i++))!=NULL_ITEM) printf("%d ",n);
  printf("\n");
  destroyList(l);
  return 0;
}
```

Notice how the implementation details of list are hidden

Creating a List ADT *continued*

- How do we modify our list data structure to turn it into this list ADT?
  1. Create an abstraction for our list item type in a separate file named `itemType.h`:

```
#ifndef ITEMTYPE_H
#define ITEMTYPE_H
#include <limits.h>

typedef int listItem;
#define NULL_ITEM INT_MIN

#endif
```



Creating a List ADT *continued*

- How do we modify our list data structure to turn it into this list ADT?
  1. Create an abstraction for our list item type in a separate file named `itemType.h`:

```
#ifndef ITEMTYPE_H
#define ITEMTYPE_H
#include <limits.h>

typedef int listItem;
#define NULL_ITEM INT_MIN

#endif
```



Creating a List ADT *continued*

- How do we modify our list data structure to turn it into this list ADT?
  1. Create an abstraction for our list item type in a separate file named `itemType.h`:

```
#ifndef ITEMTYPE_H
#define ITEMTYPE_H
#include <limits.h>

typedef int listItem;
#define NULL_ITEM INT_MIN

#endif
```

Why is this a good idea?



Creating a List ADT *continued*

2. Create a separate header file for our ADT specification. This will be the “public” file available to the programmer so it often includes documentation. File `list.h`:

```
#include <stdbool.h>
#include "itemType.h"
#ifndef LIST_H
#define LIST_H

typedef struct listType *list;

list createList(void);
 // returns a newly created empty list; must be called before
 // using list routines
void destroyList(list);
 // deallocates list; operation undefined if list hasn't
 // been created with createList

void clearList(list);
 // deallocates list items (if applicable) and sets list to empty

listItem getItem(list,int);
 // returns item at location if it exists, otherwise
 // returns NULL_ITEM
```

(continued on next page)



Creating a List ADT *continued*

2. Create a separate header file for our ADT specification. This will be the "public" file available to the programmer so it often includes documentation. File `list.h`:

```
#include <stdbool.h>
#include "itemType.h"
#ifndef LIST_H
#define LIST_H

typedef struct listType *list;

list createList(void);
 // returns a newly created empty list; must be called before
 // using list routines
void destroyList(list);
 // deallocates list; operation undefined if list hasn't
 // been created with createList

void clearList(list);
 // deallocates list items (if applicable) and sets list to empty

listItem getItem(list,int);
 // returns item at location if it exists, otherwise
 // returns NULL_ITEM
```

declared as pointer so structure can be hidden

(continued on next page)



Creating a List ADT *continued*

File `list.h` continued:

```
int addItem(list,listItem);
 // adds item to end of list; returns index
 // of new item or -1 if add was unsuccesful
int insertItem(list,int,listItem);
 // inserts item into list at position; returns
 // index of new item or -1 if unsuccessful

bool removeItem(list,int);
 // removes item at position, returns true if operation
 // successful, false otherwise
bool setItem(list,int,listItem);
 // replaces item at location, returns true if operation
 // is succesful, false otherwise

int maxSize(list);
 // returns maximum number of items list can contain
int listSize(list);
 // returns current number of items in list
bool listEmpty(list);
 // returns true if list is empty, false otherwise
bool listFull(list);
 // returns true if list is full, false otherwise
#endif
```



Creating a List ADT *continued*

3. Create your implementation file `list.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>
#include "itemType.h"
#include "list.h"

#define LIST_SIZE 10

struct listType {
  listItem data[LIST_SIZE];
  int count;
  int max;
};

// all the functions go here exactly as they given before
```



Creating a List ADT *continued*

3. Create your implementation file `list.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>
#include "itemType.h"
#include "list.h"

#define LIST_SIZE 10

struct listType {
  listItem data[LIST_SIZE];
  int count;
  int max;
};

// all the functions go here exactly as they given before
```

note!

note!



Creating a List ADT *continued*

3. Create your implementation file `list.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>
#include "itemType.h"
#include "list.h"

#define LIST_SIZE 10

struct listType {
  listItem data[LIST_SIZE];
  int count;
  int max;
};

// all the functions go here exactly as they given before
```

note!

note!

Remember that this file can be separately compiled and distributed as object code



Creating a List ADT *continued*

3. Create your implementation file `list.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>
#include "itemType.h"
#include "list.h"

#define LIST_SIZE 10

struct listType {
  listItem data[LIST_SIZE];
  int count;
  int max;
};

// all the functions go here exactly as they given before
```

note!

note!

Remember that this file can be separately compiled and distributed as object code

List structure can be completely hidden when compiled



Creating a List ADT *continued*

4. To use the list ADT:
   - to compile ADT and main program that uses it:

     ```
     gcc main.c list.c
     ```
   - to separately compile ADT from main program:

     ```
     gcc -c list.c
     gcc main.c list.o
     ```



Creating a List ADT *continued*

4. To use the list ADT:
   - to compile ADT and main program that uses it:

     ```
     gcc main.c list.c
     ```
   - to separately compile ADT from main program:

     ```
     gcc -c list.c
     gcc main.c list.o
     ```



Creating a List ADT *continued*

4. To use the list ADT:
   - to compile ADT and main program that uses it:

     ```
     gcc main.c list.c
     ```
   - to separately compile ADT from main program:

     ```
     gcc -c list.c
     gcc main.c list.o
     ```



Dynamic Arrays (Revisted)

- Notice that the size of the `data` array in our list is fixed at compile time to `LIST_SIZE` number of items
- Could this be an issue? How, or why?
- One solution is to not allocate the array until runtime and then expand it (again at runtime) "automatically" when extra memory is needed
- As mentioned earlier, an array that can expand (and/or contract) at runtime as needed is often called a **dynamic array**
- What would we have to modify in our list ADT to use dynamic arrays?



Dynamic Arrays (Revisted)

- Notice that the size of the `data` array in our list is fixed at compile time to `LIST_SIZE` number of items
- Could this be an issue? How, or why?
- One solution is to not allocate the array until runtime and then expand it (again at runtime) "automatically" when extra memory is needed
- As mentioned earlier, an array that can expand (and/or contract) at runtime as needed is often called a **dynamic array**
- What would we have to modify in our list ADT to use dynamic arrays?



Dynamic Arrays (Revisted)

- Notice that the size of the `data` array in our list is fixed at compile time to `LIST_SIZE` number of items
- Could this be an issue? How, or why?
- One solution is to not allocate the array until runtime and then expand it (again at runtime) “automatically” when extra memory is needed
- As mentioned earlier, an array that can expand (and/or contract) at runtime as needed is often called a **dynamic array**
- What would we have to modify in our list ADT to use dynamic arrays?



Dynamic Arrays (Revisted)

- Notice that the size of the `data` array in our list is fixed at compile time to `LIST_SIZE` number of items
- Could this be an issue? How, or why?
- One solution is to not allocate the array until runtime and then expand it (again at runtime) “automatically” when extra memory is needed
- As mentioned earlier, an array that can expand (and/or contract) at runtime as needed is often called a **dynamic array**
- What would we have to modify in our list ADT to use dynamic arrays?



Dynamic Arrays (Revisted)

- Notice that the size of the `data` array in our list is fixed at compile time to `LIST_SIZE` number of items
- Could this be an issue? How, or why?
- One solution is to not allocate the array until runtime and then expand it (again at runtime) “automatically” when extra memory is needed
- As mentioned earlier, an array that can expand (and/or contract) at runtime as needed is often called a **dynamic array**
- What would we have to modify in our list ADT to use dynamic arrays?



Dynamic Arrays (Revisted) *continued*

- Changes to `list.c`:

```
// ...
#define SIZE_INCREMENT 4
#define DEBUG

struct listType {
  listItem *data;
  int count;
  int max;
};
```

- Now we'll add a **static function** (which can only be used in the file it is defined in) that expands the array as needed

```
static void expandArray(list l) {
#ifdef DEBUG
  printf("[DEBUG] expanding memory by %d to %d\n",SIZE_INCREMENT,l->max+SIZE_INCREMENT);
#endif
  listItem *tptr = malloc(sizeof(listItem)*(l->max+SIZE_INCREMENT));
  for (int i=0; i < l->count; i++) tptr[i] = l->data[i];
  free(l->data);
  l->data = tptr; l->max += SIZE_INCREMENT;
}
```



Dynamic Arrays (Revisted) *continued*

- Changes to `list.c`:

```
// ...
#define SIZE_INCREMENT 4
#define DEBUG

struct listType {
  listItem *data;
  int count;
  int max;
};
```

- Now we'll add a **static function** (which can only be used in the file it is defined in) that expands the array as needed

```
static void expandArray(list l) {
#ifdef DEBUG
  printf("[DEBUG] expanding memory by %d to %d\n",SIZE_INCREMENT,l->max+SIZE_INCREMENT);
#endif
  listItem *tptr = malloc(sizeof(listItem)*(l->max+SIZE_INCREMENT));
  for (int i=0; i < l->count; i++) tptr[i] = l->data[i];
  free(l->data);
  l->data = tptr; l->max += SIZE_INCREMENT;
}
```



Dynamic Arrays (Revisted) *continued*

- The only other changes that are needed are to any routines that create/destroy lists and add items to the list:

```
list createList(void) {
  list l = malloc(sizeof(struct listType));
  l->data = NULL; l->count = l->max = 0;
  return l;
}
void destroyList(list l) { free(l->data); free(l); }

int addItem(list l,listItem item) {
  if(l->count >= maxSize(l)) expandArray(l);
  l->data[l->count++] = item;
  return l->count-1;
}

int insertItem(list l,int pos,listItem item) {
  if(pos >= 0 && pos <= listSize(l)) {
    if(l->count >= maxSize(l)) expandArray(l);
    for(int i=l->count-1;i>=pos;i-) l->data[i+1] = l->data[i];
    l->data[pos]=item; l->count++;
    return pos;
  } else return -1;
}
```

- Note if desired we could add a routine that *shrank* lists as well



Dynamic Arrays (Revisted) *continued*

- The only other changes that are needed are to any routines that create/destroy lists and add items to the list:

```
list createList(void) {
  list l = malloc(sizeof(struct listType));
  l->data = NULL; l->count = l->max = 0;
  return l;
}
void destroyList(list l) { free(l->data); free(l); }

int addItem(list l,listItem item) {
  if(l->count >= maxSize(l)) expandArray(l);
  l->data[l->count++] = item;
  return l->count-1;
}

int insertItem(list l,int pos,listItem item) {
  if(pos >= 0 && pos <= listSize(l)) {
    if(l->count >= maxSize(l)) expandArray(l);
    for(int i=l->count-1;i>=pos;i-) l->data[i+1] = l->data[i];
    l->data[pos]=item; l->count++;
    return pos;
  } else return -1;
}
```

- Note if desired we could add a routine that *shrank* lists as well



Introduction

- A **linked list** is a container data structure used to implement a list of items arranged in some order
- Linked lists use dynamic memory that shrinks and grows as needed (but in a different manner than dynmaic arrays)
- They can perform similar functions as an array but are created, accessed, and manipulated much differently
- Data in a linked list isn't necessarily stored in continugous memory like arrays
- Instead, each item is stored in a **node** that is connected to other items through a **link**:



Introduction

- A **linked list** is a container data structure used to implement a list of items arranged in some order
- Linked lists use dynamic memory that shrinks and grows as needed (but in a different manner than dynmaic arrays)
- They can perform similar functions as an array but are created, accessed, and manipulated much differently
- Data in a linked list isn't necessarily stored in continugous memory like arrays
- Instead, each item is stored in a **node** that is connected to other items through a **link**:



Introduction

- A **linked list** is a container data structure used to implement a list of items arranged in some order
- Linked lists use dynamic memory that shrinks and grows as needed (but in a different manner than dynmaic arrays)
- They can perform similar functions as an array but are created, accessed, and manipulated much differently
- Data in a linked list isn't necessarily stored in continugous memory like arrays
- Instead, each item is stored in a **node** that is connected to other items through a **link**:



Introduction

- A **linked list** is a container data structure used to implement a list of items arranged in some order
- Linked lists use dynamic memory that shrinks and grows as needed (but in a different manner than dynmaic arrays)
- They can perform similar functions as an array but are created, accessed, and manipulated much differently
- Data in a linked list isn't necessarily stored in continugous memory like arrays
- Instead, each item is stored in a **node** that is connected to other items through a **link**:

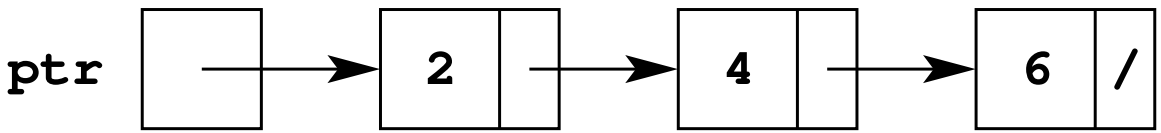
Introduction

- A **linked list** is a container data structure used to implement a list of items arranged in some order
- Linked lists use dynamic memory that shrinks and grows as needed (but in a different manner than dynmaic arrays)
- They can perform similar functions as an array but are created, accessed, and manipulated much differently
- Data in a linked list isn't necessarily stored in continugous memory like arrays
- Instead, each item is stored in a **node** that is connected to other items through a **link**:

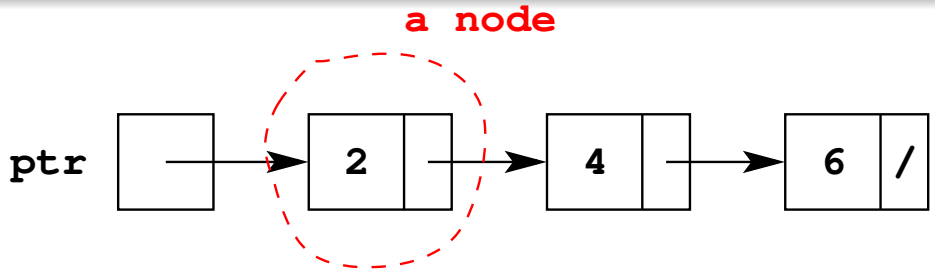


Introduction

- A **linked list** is a container data structure used to implement a list of items arranged in some order
- Linked lists use dynamic memory that shrinks and grows as needed (but in a different manner than dynmaic arrays)
- They can perform similar functions as an array but are created, accessed, and manipulated much differently
- Data in a linked list isn't necessarily stored in continugous memory like arrays
- Instead, each item is stored in a **node** that is connected to other items through a **link**:



Nodes

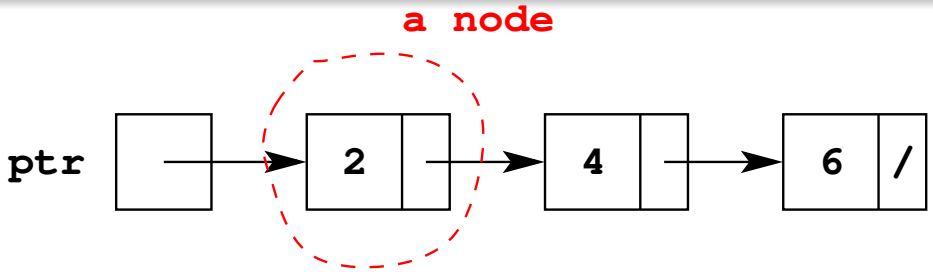


- Note above that a node has two parts: a section for the node's data and a link to the next node (if there is one)
- And, as the diagram suggests, this link to the next node is a pointer!
- Lets define a node as a C structure with two fields (we will redefine a node differently later):

```
struct node {
  int data;
  struct node *next;
}
```

- Consider this – in essence a node **is** a linked list!

Nodes

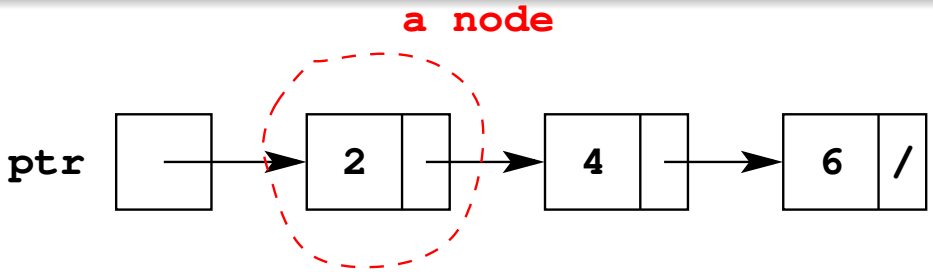


- Note above that a node has two parts: a section for the node's data and a link to the next node (if there is one)
- And, as the diagram suggests, this link to the next node is a pointer!
- Lets define a node as a C structure with two fields (we will redefine a node differently later):

```
struct node {
  int data;
  struct node *next;
}
```

- Consider this – in essence a node **is** a linked list!

Nodes

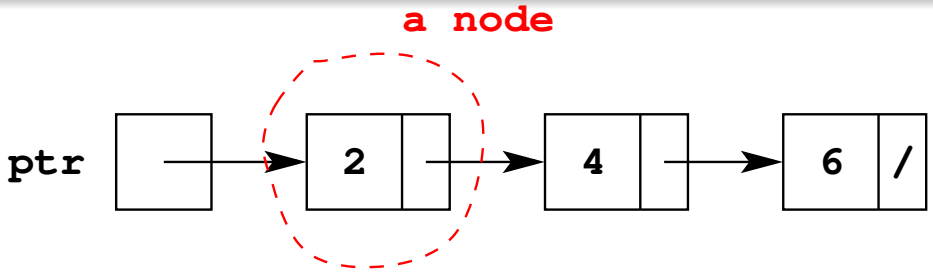


- Note above that a node has two parts: a section for the node's data and a link to the next node (if there is one)
- And, as the diagram suggests, this link to the next node is a pointer!
- Lets define a node as a C structure with two fields (we will redefine a node differently later):

```
struct node {
  int data;
  struct node *next;
}
```

- Consider this – in essence a node **is** a linked list!

Nodes

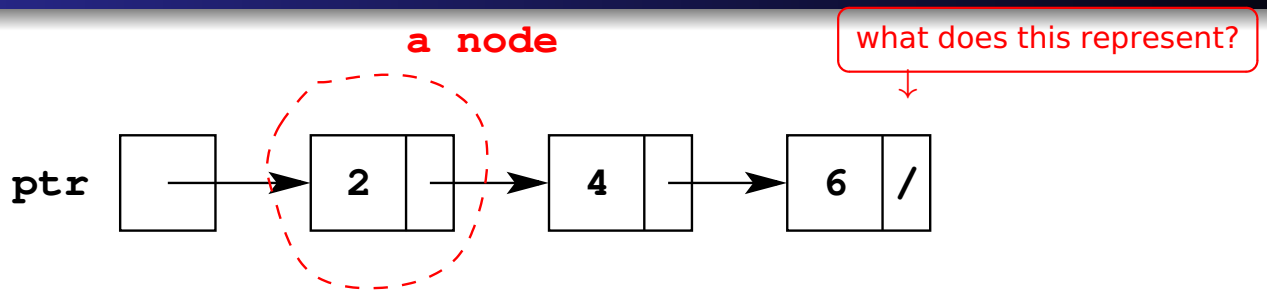


- Note above that a node has two parts: a section for the node's data and a link to the next node (if there is one)
- And, as the diagram suggests, this link to the next node is a pointer!
- Lets define a node as a C structure with two fields (we will redefine a node differently later):

```
struct node {
  int data;
  struct node *next;
}
```

- Consider this – in essence a node **is** a linked list!

Nodes

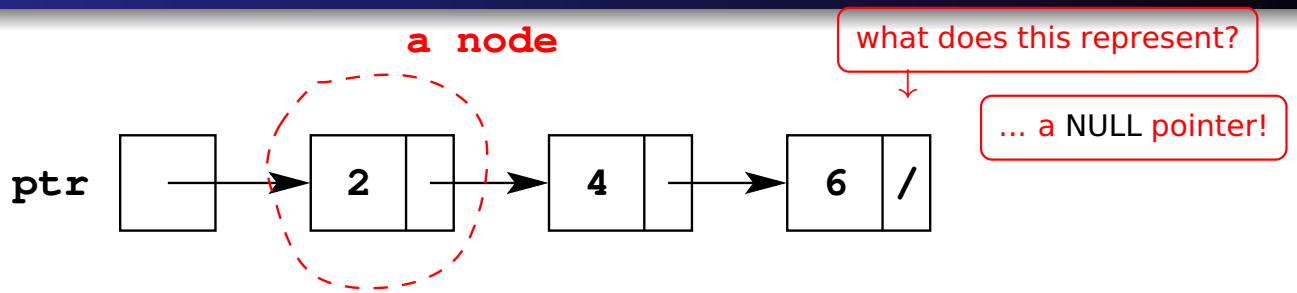


- Note above that a node has two parts: a section for the node's data and a link to the next node (if there is one)
- And, as the diagram suggests, this link to the next node is a pointer!
- Lets define a node as a C structure with two fields (we will redefine a node differently later):

```
struct node {
  int data;
  struct node *next;
}
```

- Consider this – in essence a node **is** a linked list!

Nodes



- Note above that a node has two parts: a section for the node's data and a link to the next node (if there is one)
- And, as the diagram suggests, this link to the next node is a pointer!
- Lets define a node as a C structure with two fields (we will redefine a node differently later):

```
struct node {
  int data;
  struct node *next;
}
```

- Consider this – in essence a node **is** a linked list!

NULL Pointers

- A special C constant that is used to represent a pointer variable that has no value, i.e. it points to nothing
- It can be assigned to any pointer variable:

```
int *ptr;
ptr = NULL;
```

- You can also test for it:

```
if(ptr != NULL) { /* do something */ }
```

- Dereferencing a NULL pointer will cause undefined behavior:

```
// ...
int main(void){
  int *ptr=NULL;
  printf("%d\n",*ptr); // WRONG!
}
```



NULL Pointers

- A special C constant that is used to represent a pointer variable that has no value, i.e. it points to nothing
- It can be assigned to any pointer variable:

```
int *ptr;
ptr = NULL;
```

- You can also test for it:

```
if(ptr != NULL) { /* do something */ }
```

- Dereferencing a NULL pointer will cause undefined behavior:

```
// ...
int main(void){
  int *ptr=NULL;
  printf("%d\n",*ptr); // WRONG!
}
```



NULL Pointers

- A special C constant that is used to represent a pointer variable that has no value, i.e. it points to nothing
- It can be assigned to any pointer variable:

```
int *ptr;
ptr = NULL;
```

- You can also test for it:

```
if(ptr != NULL) { /* do something */ }
```

- Dereferencing a NULL pointer will cause undefined behavior:

```
// ...
int main(void){
  int *ptr=NULL;
  printf("%d\n",*ptr); // WRONG!
}
```



NULL Pointers

- A special C constant that is used to represent a pointer variable that has no value, i.e. it points to nothing
- It can be assigned to any pointer variable:

```
int *ptr;
ptr = NULL;
```

- You can also test for it:

```
if(ptr != NULL) { /* do something */ }
```

- Dereferencing a NULL pointer will cause undefined behavior:

```
// ...
int main(void){
  int *ptr=NULL;
  printf("%d\n",*ptr); // WRONG!
}
```



Defining a Simple Linked List

- Following the example of the list defined in the last chapter, let's define our new linked list as follows:

Defining a Simple Linked List

- Following the example of the list defined in the last chapter, let's define our new linked list as follows:

```
typedef int listItem;
```



Defining a Simple Linked List

- Following the example of the list defined in the last chapter, let's define our new linked list as follows:

```
typedef int listItem;

struct node {
  listItem data;
  struct node *next;
};
```



Defining a Simple Linked List

- Following the example of the list defined in the last chapter, let's define our new linked list as follows:

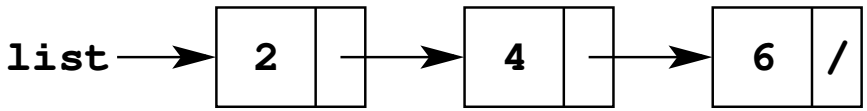
```
typedef int listItem;

struct node {
  listItem data;
  struct node *next;
};

typedef struct node *llist;
```

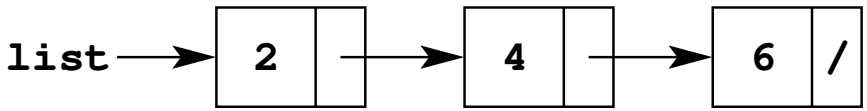


Basic Linked List Operations: Traversing a List




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

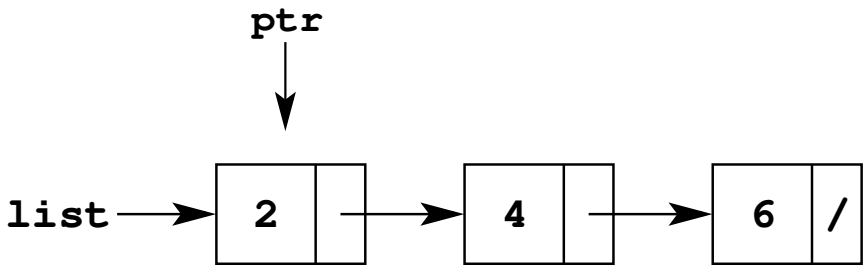
Basic Linked List Operations: Traversing a List




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

llist ptr = list;

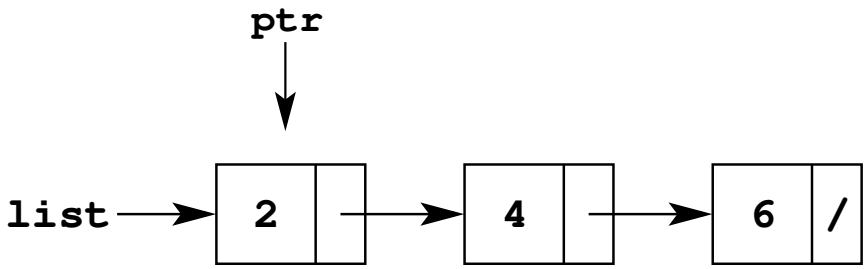
Basic Linked List Operations: Traversing a List




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

llist ptr = list;

Basic Linked List Operations: Traversing a List

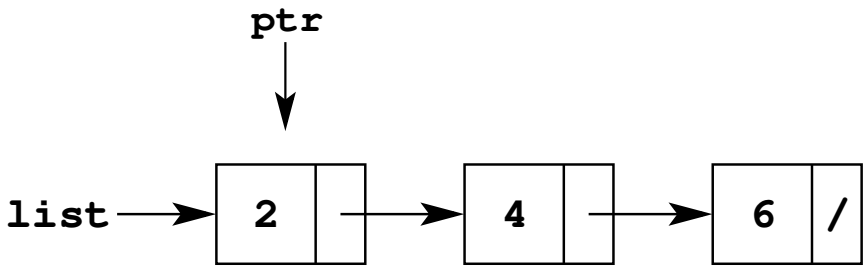



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {


}
```

Basic Linked List Operations: Traversing a List

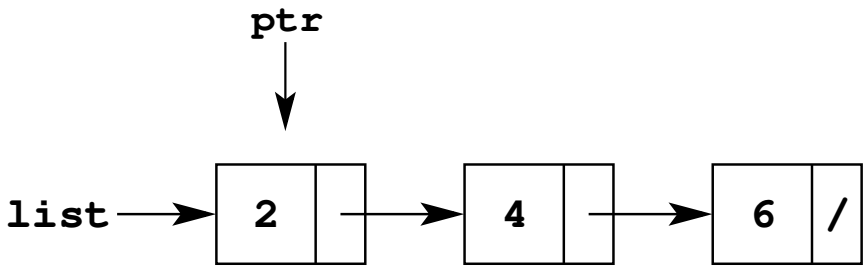



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);

}
```

Basic Linked List Operations: Traversing a List



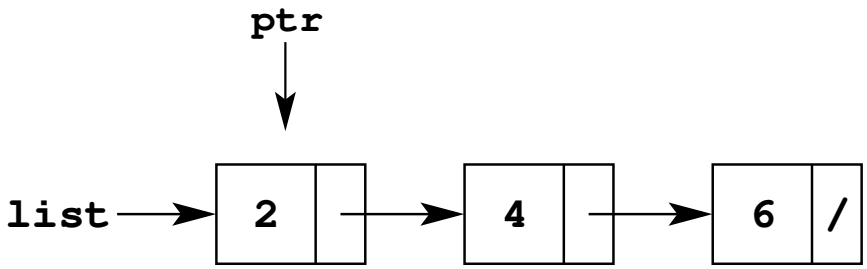

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);

}
```

```
2
```

Basic Linked List Operations: Traversing a List

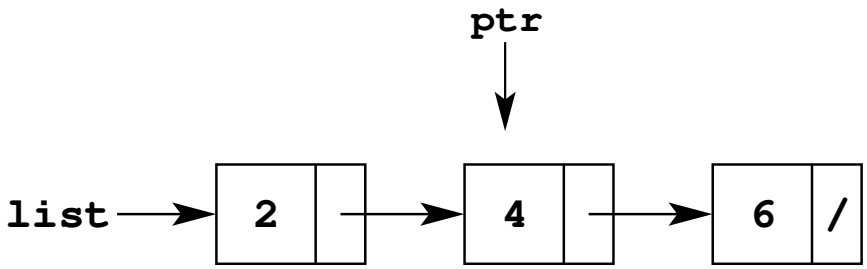



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
```

Basic Linked List Operations: Traversing a List

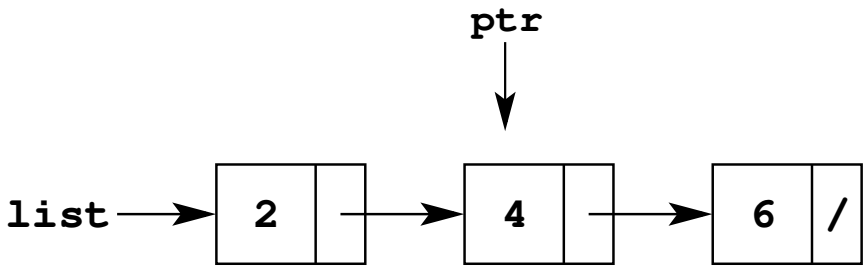



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
```

Basic Linked List Operations: Traversing a List

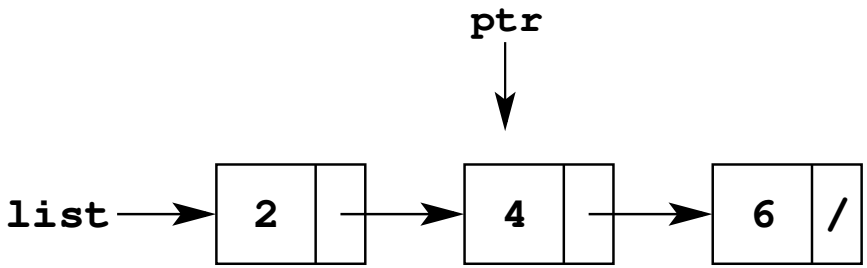



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
```

Basic Linked List Operations: Traversing a List

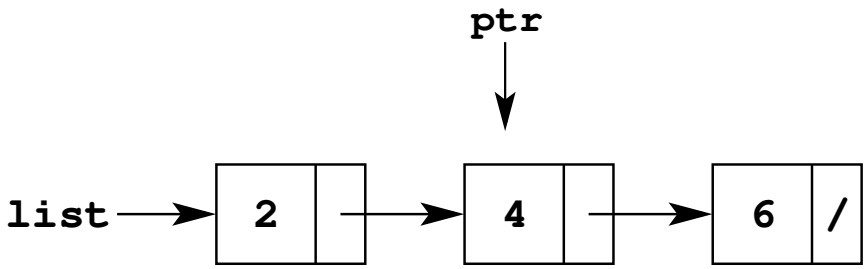



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
```

Basic Linked List Operations: Traversing a List

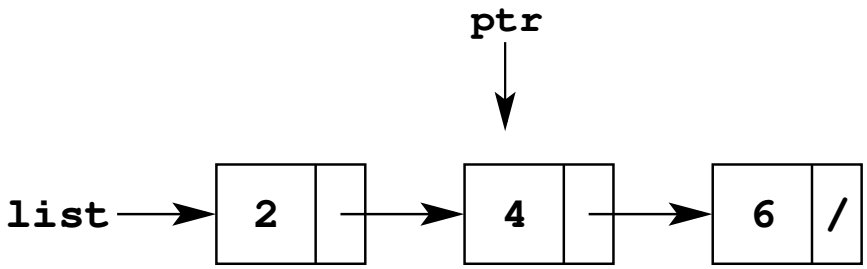



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
4
```

Basic Linked List Operations: Traversing a List

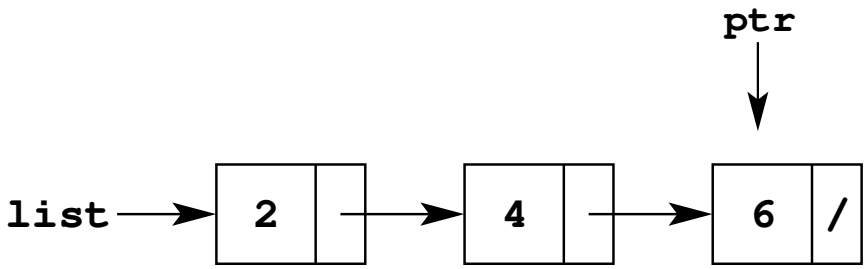



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
4
```

Basic Linked List Operations: Traversing a List

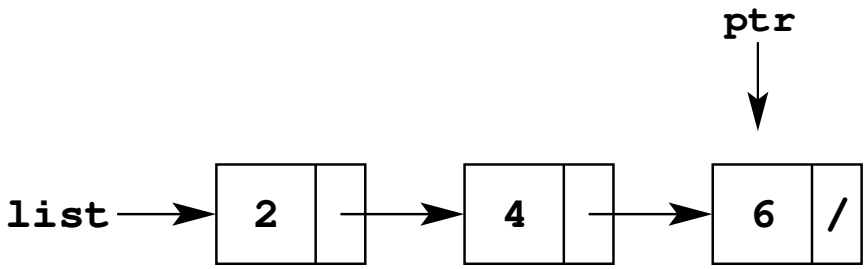



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
4
```

Basic Linked List Operations: Traversing a List

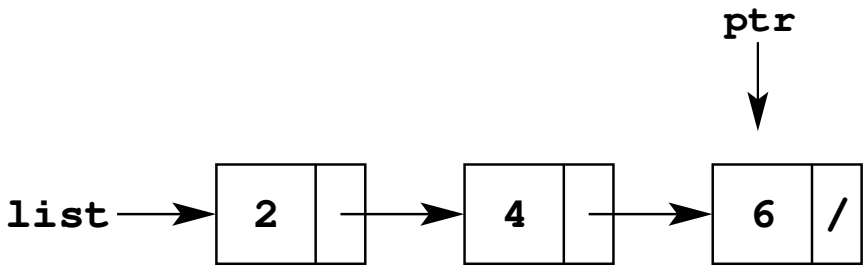



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
4
```

Basic Linked List Operations: Traversing a List

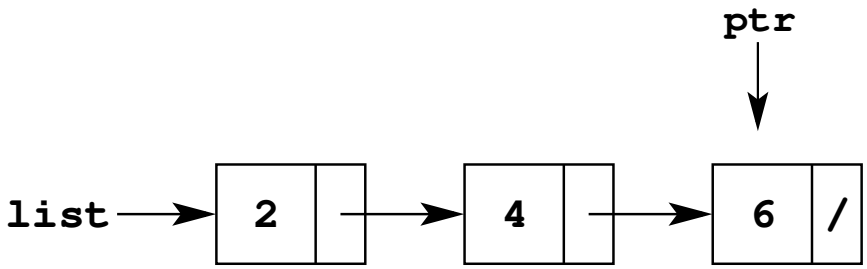



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
4
```

Basic Linked List Operations: Traversing a List

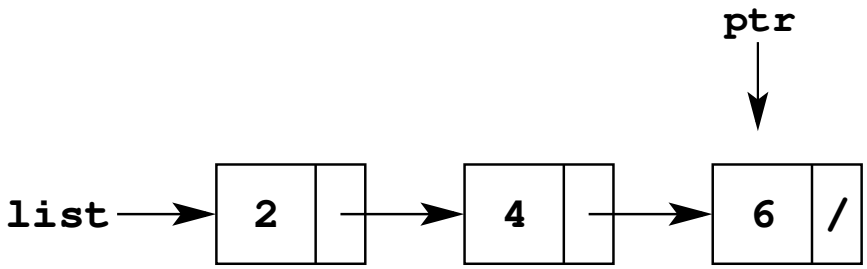



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
4
6
```

Basic Linked List Operations: Traversing a List

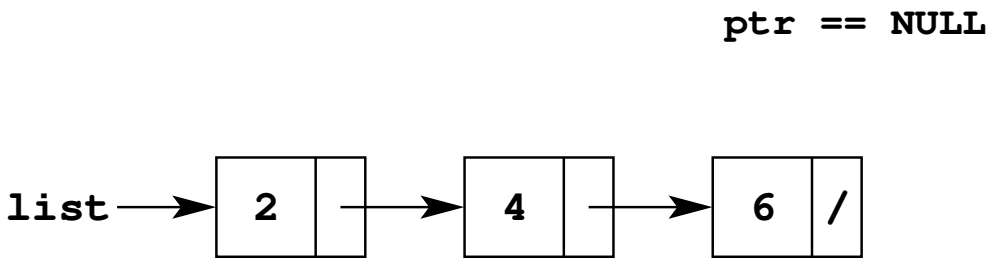



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
4
6
```

Basic Linked List Operations: Traversing a List

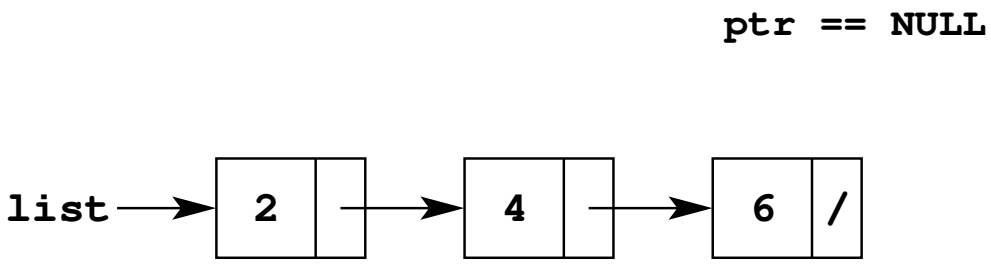



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

```
2
4
6
```

Basic Linked List Operations: Traversing a List



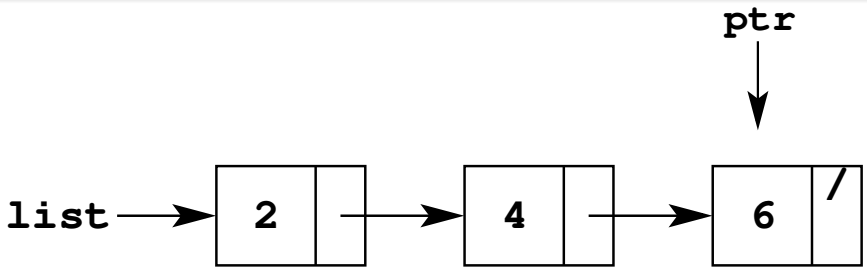

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist ptr = list;
while(ptr != NULL) {
  printf("%d\n",ptr->data);
  ptr = ptr->next;
}
```

done!

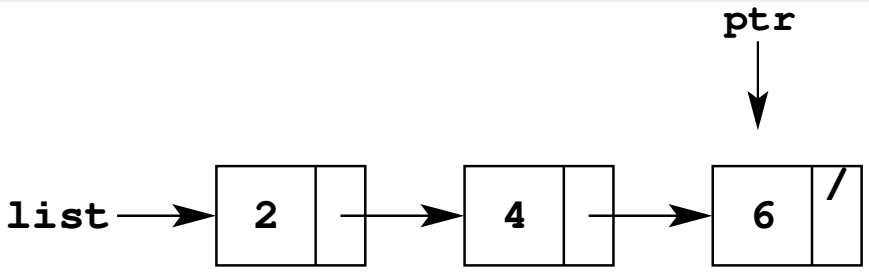
```
2
4
6
```

Basic Linked List Operations: Adding Item at the End




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

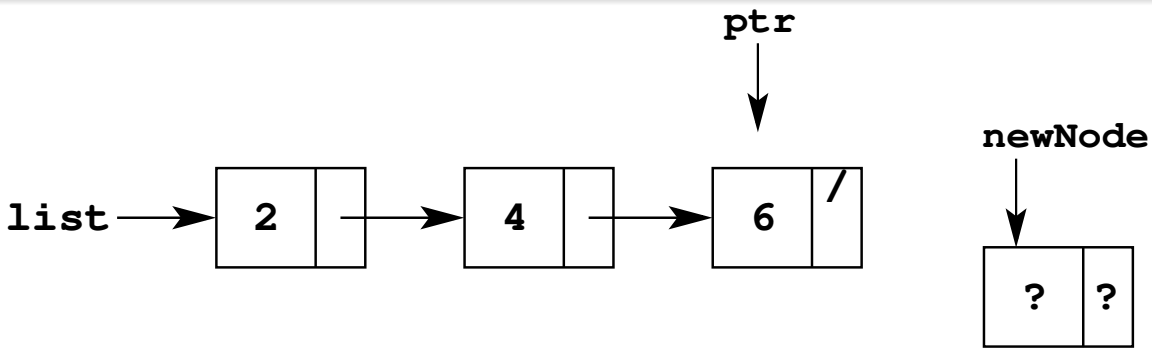
Basic Linked List Operations: Adding Item at the End




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

`llist newNode = malloc(sizeof (struct node));`

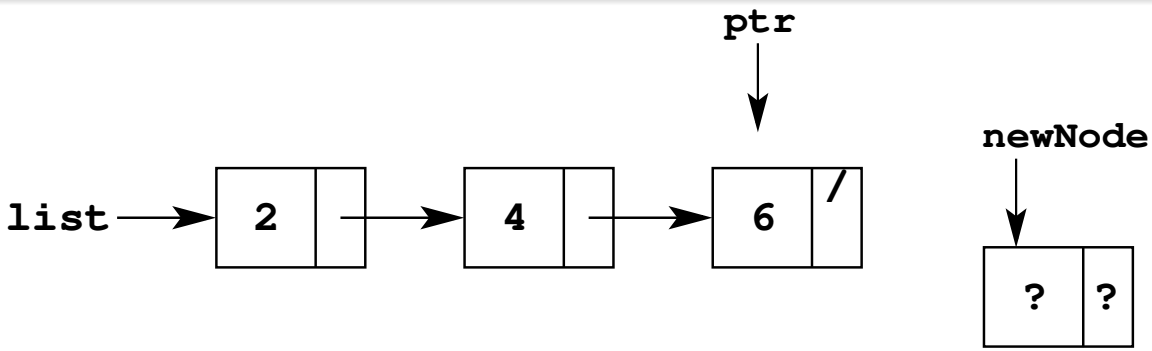
Basic Linked List Operations: Adding Item at the End




```
llist newNode = malloc(sizeof (struct node));
```

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

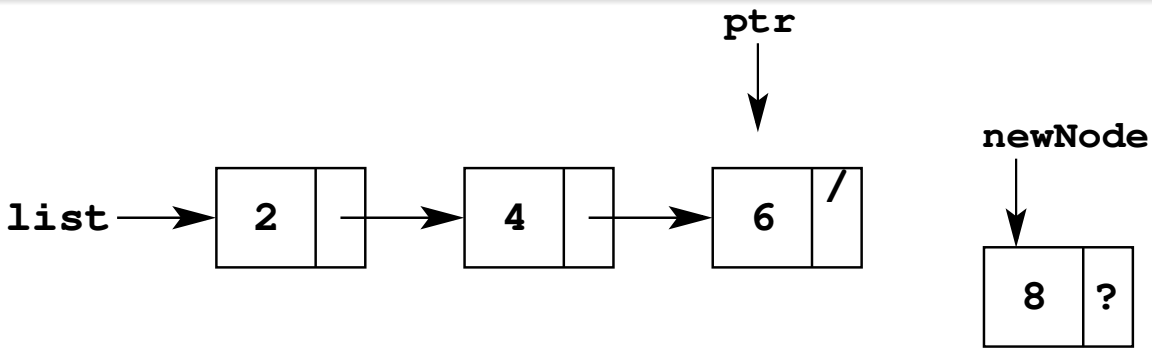
Basic Linked List Operations: Adding Item at the End




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 8;
```

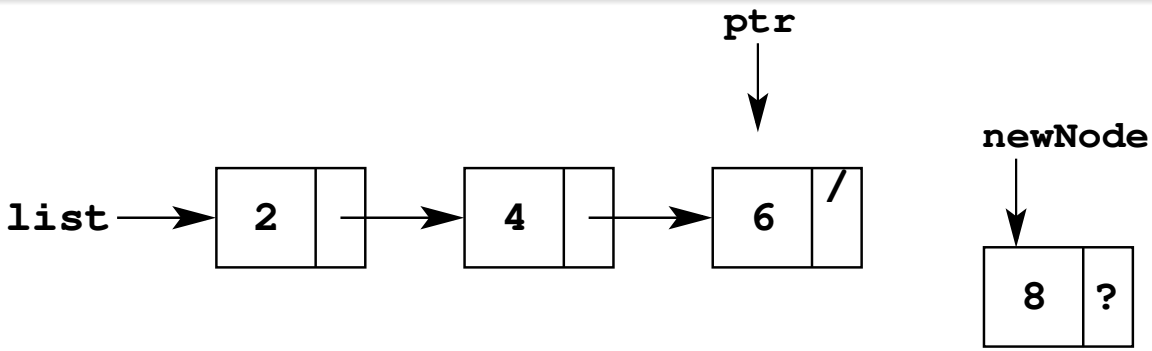
Basic Linked List Operations: Adding Item at the End




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 8;
```

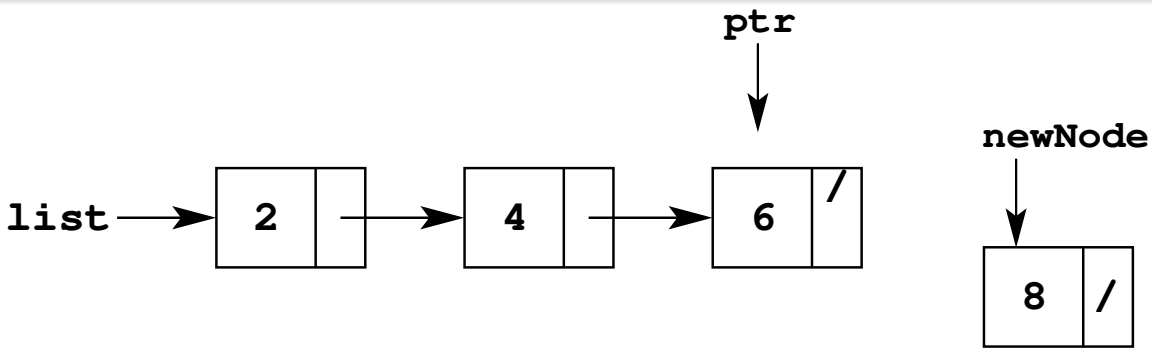
Basic Linked List Operations: Adding Item at the End




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 8;
newNode->next = NULL;
```

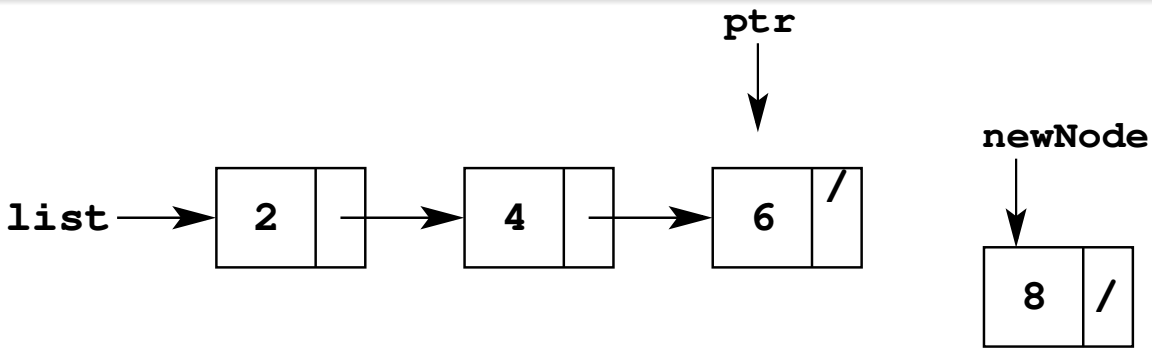
Basic Linked List Operations: Adding Item at the End




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 8;
newNode->next = NULL;
```

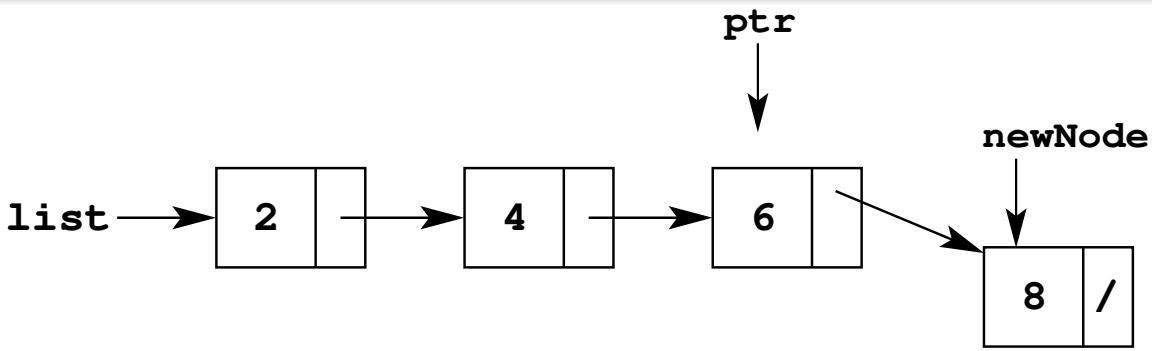
Basic Linked List Operations: Adding Item at the End




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 8;
newNode->next = NULL;
ptr->next = newNode;
```

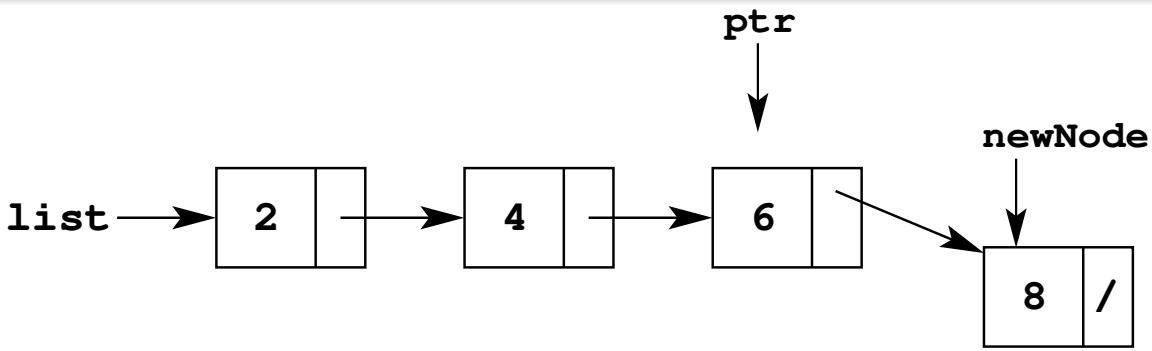
Basic Linked List Operations: Adding Item at the End




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 8;
newNode->next = NULL;
ptr->next = newNode;
```

Basic Linked List Operations: Adding Item at the End

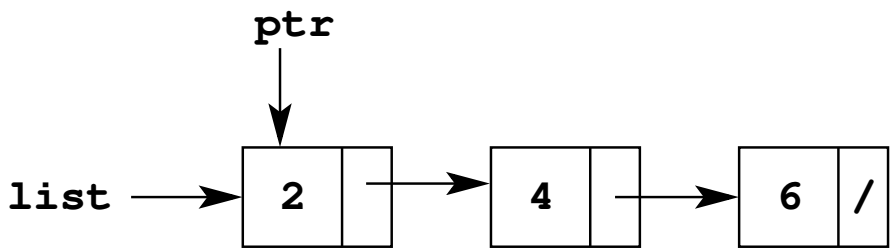



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 8;
newNode->next = NULL;
ptr->next = newNode;
```

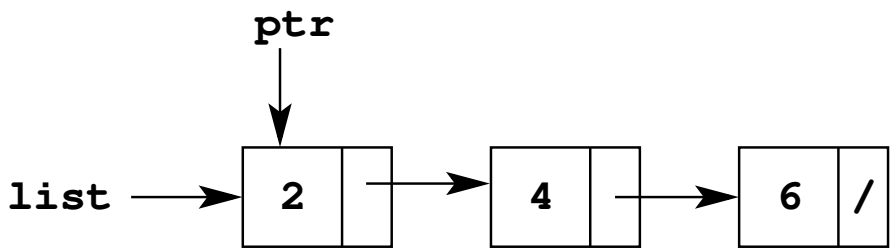
done!

Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

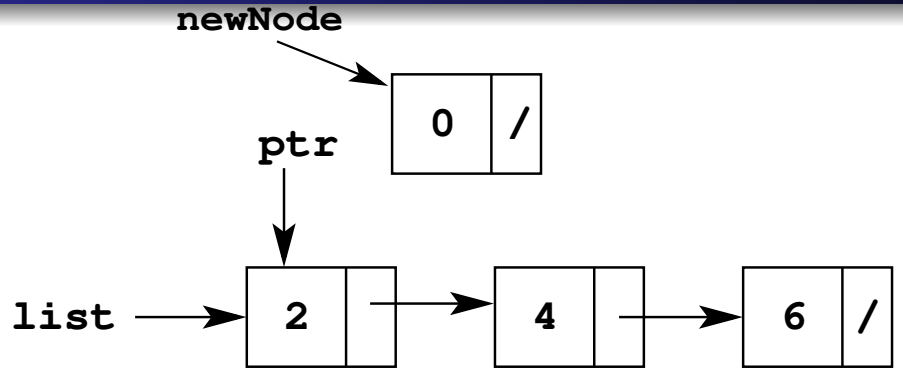
Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
```

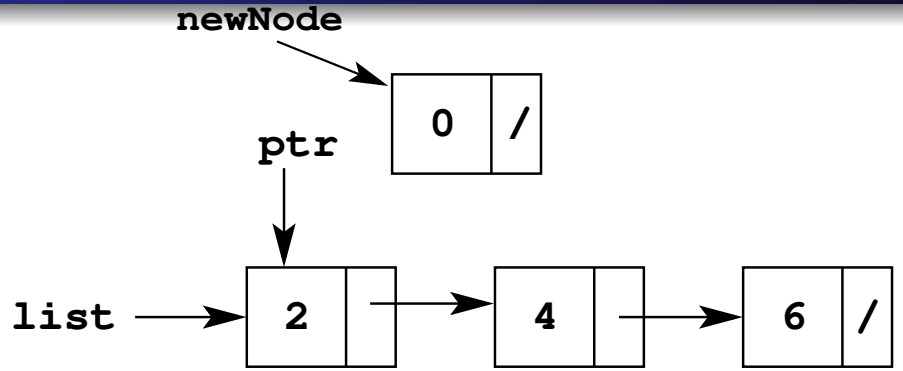
Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
```

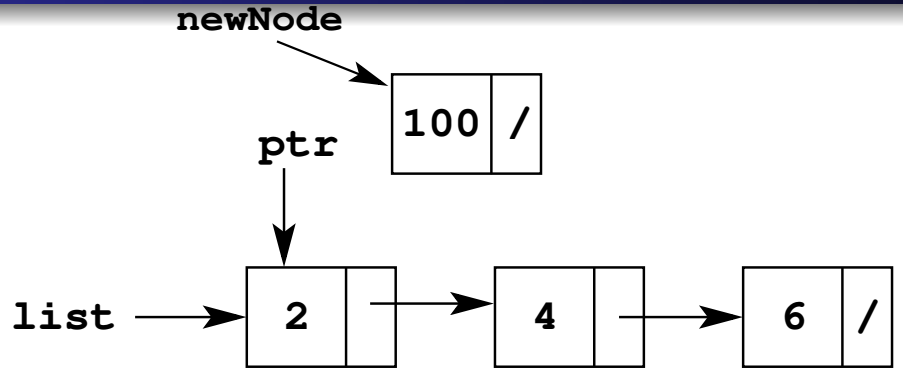
Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 100;
```

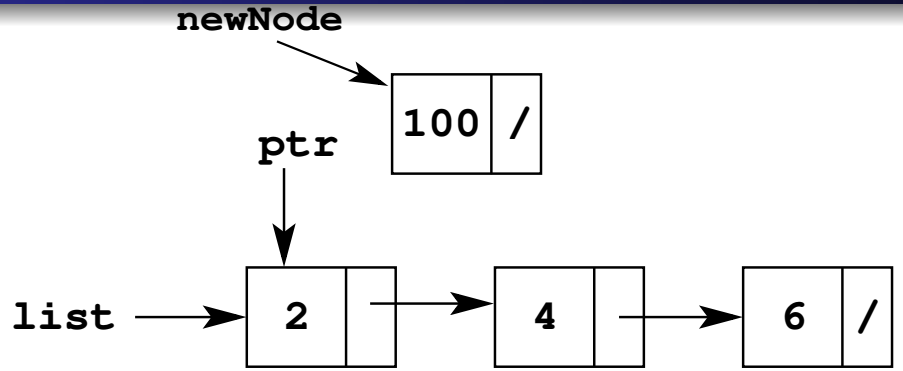
Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 100;
```

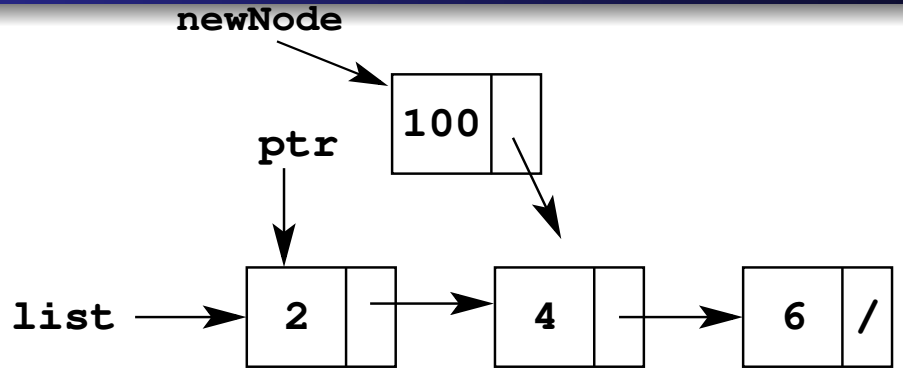
Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 100;
newNode->next = ptr->next;
```

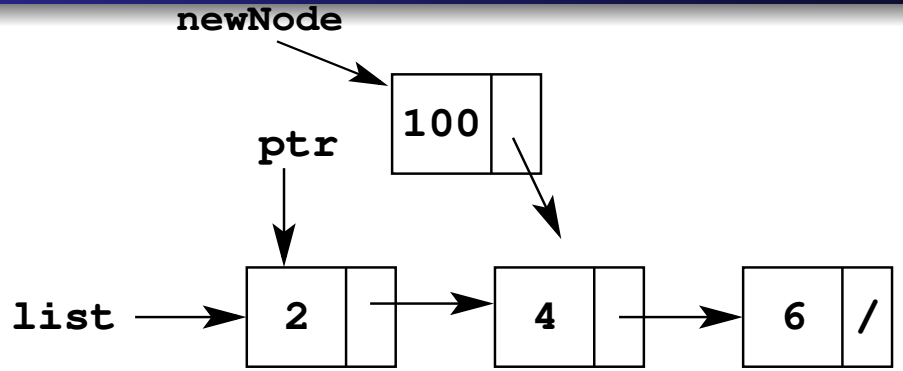
Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 100;
newNode->next = ptr->next;
```

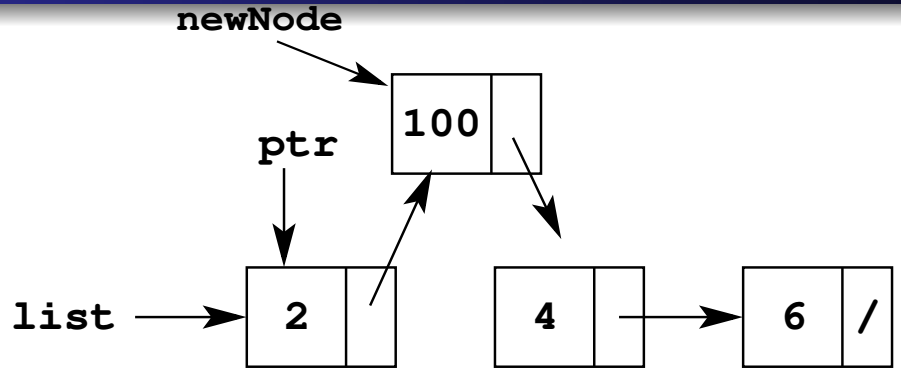
Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 100;
newNode->next = ptr->next;
ptr->next = newNode;
```

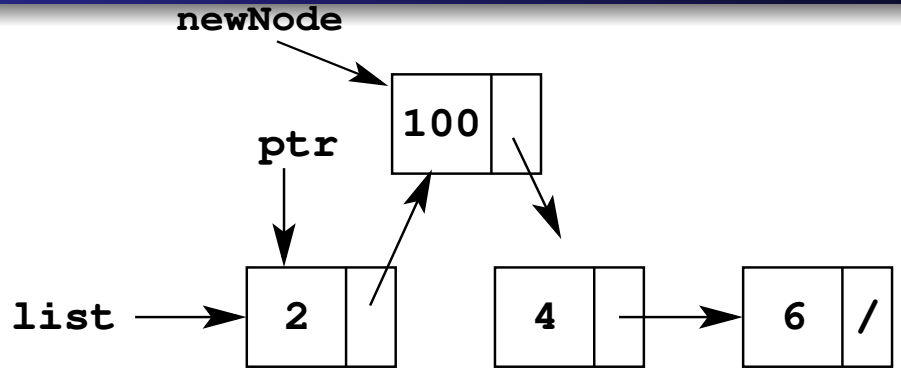
Basic Linked List Operations: Adding Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 100;
newNode->next = ptr->next;
ptr->next = newNode;
```

Basic Linked List Operations: Adding Item in the Middle

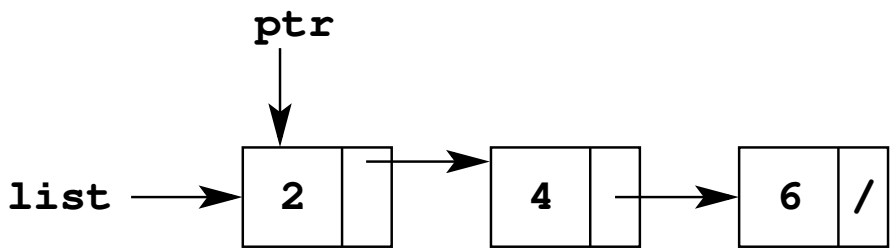



```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist newNode = malloc(sizeof (struct node));
newNode->data = 100;
newNode->next = ptr->next;
ptr->next = newNode;
```

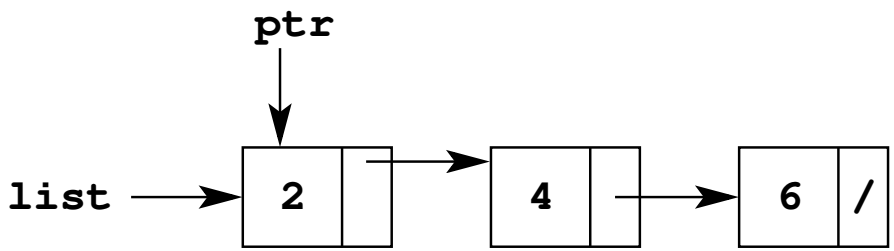
done!

Basic Linked List Operations: Removing Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

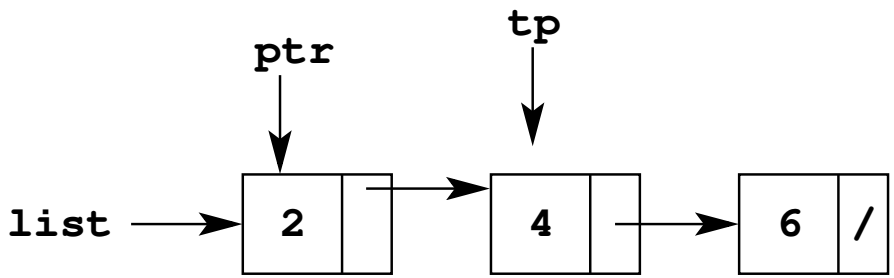
Basic Linked List Operations: Removing Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

`llist tp = ptr->next;`

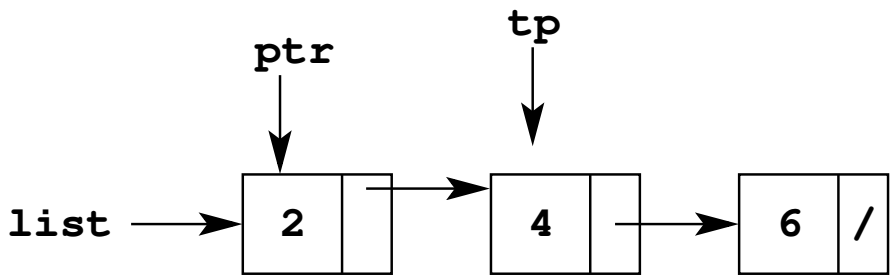
Basic Linked List Operations: Removing Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist tp = ptr->next;
```

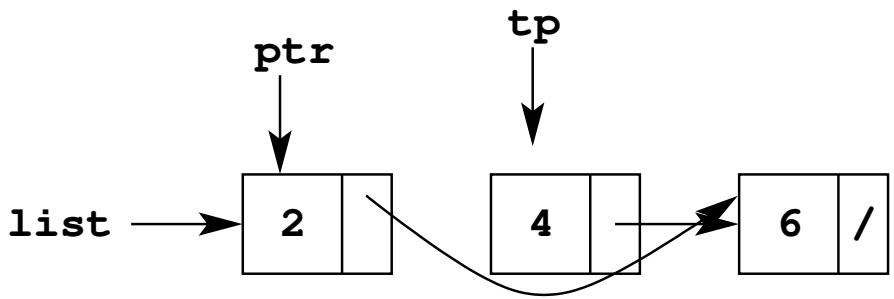
Basic Linked List Operations: Removing Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist tp = ptr->next;
ptr->next = tp->next;
```

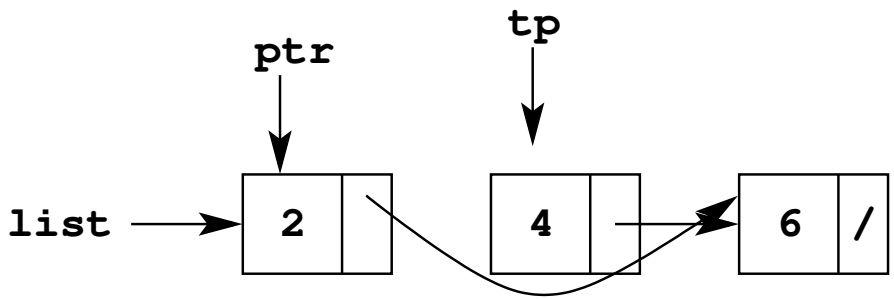
Basic Linked List Operations: Removing Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist tp = ptr->next;
ptr->next = tp->next;
```

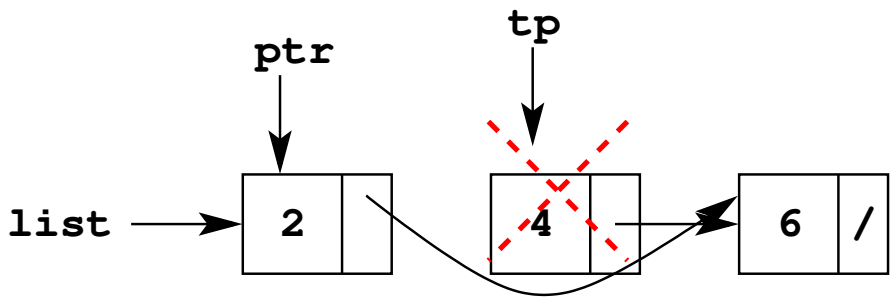
Basic Linked List Operations: Removing Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist tp = ptr->next;
ptr->next = tp->next;
free(tp);
```

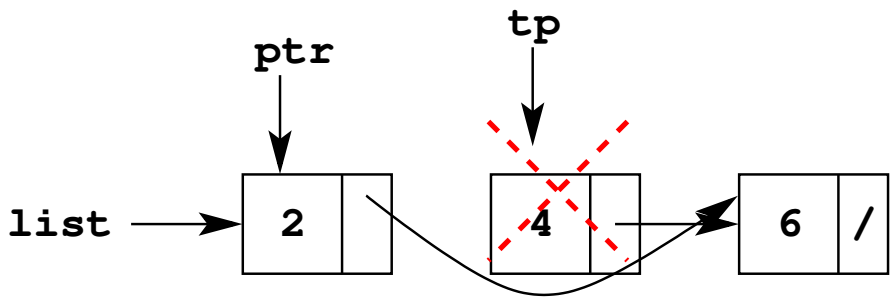
Basic Linked List Operations: Removing Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist tp = ptr->next;
ptr->next = tp->next;
free(tp);
```

Basic Linked List Operations: Removing Item in the Middle




```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

```
llist tp = ptr->next;
ptr->next = tp->next;
free(tp);
```

done!

Basic Linked List Operations: Other Operations

- adding item at the front?
- removing item at front?
- removing item at rear?
- deleting a whole list?
- making an identical copy of a list?
- reversing a list?
- creating a list of lists?

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

Basic Linked List Operations: Other Operations

- adding item at the front?
- removing item at front?
- removing item at rear?
- deleting a whole list?
- making an identical copy of a list?
- reversing a list?
- creating a list of lists?

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

Basic Linked List Operations: Other Operations

- adding item at the front?
- removing item at front?
- removing item at rear?
- deleting a whole list?
- making an identical copy of a list?
- reversing a list?
- creating a list of lists?

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

Basic Linked List Operations: Other Operations

- adding item at the front?
- removing item at front?
- removing item at rear?
- deleting a whole list?
- making an identical copy of a list?
- reversing a list?
- creating a list of lists?

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

Basic Linked List Operations: Other Operations

- adding item at the front?
- removing item at front?
- removing item at rear?
- deleting a whole list?
- making an identical copy of a list?
- reversing a list?
- creating a list of lists?

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

Basic Linked List Operations: Other Operations

- adding item at the front?
- removing item at front?
- removing item at rear?
- deleting a whole list?
- making an identical copy of a list?
- reversing a list?
- creating a list of lists?

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```

Basic Linked List Operations: Other Operations

- adding item at the front?
- removing item at front?
- removing item at rear?
- deleting a whole list?
- making an identical copy of a list?
- reversing a list?
- creating a list of lists?

```
typedef int listItem;
struct node {
  listItem data;
  struct node *next;
};
typedef struct node *llist;
```



Doubly Linked Lists

- One disadvantage of singly-linked lists: unidirectional movement through list
- If a lot of movement back and forth through the list is required one can use a **doubly linked list**
- A doubly linked list maintains a link to the previous node in the list:



Doubly Linked Lists

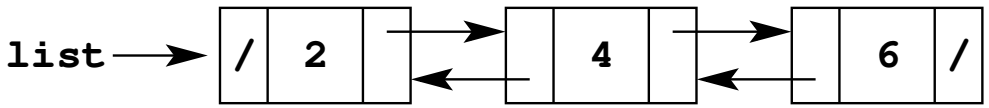
- One disadvantage of singly-linked lists: unidirectional movement through list
- If a lot of movement back and forth through the list is required one can use a **doubly linked list**
- A doubly linked list maintains a link to the previous node in the list:

Doubly Linked Lists

- One disadvantage of singly-linked lists: unidirectional movement through list
- If a lot of movement back and forth through the list is required one can use a **doubly linked list**
- A doubly linked list maintains a link to the previous node in the list:

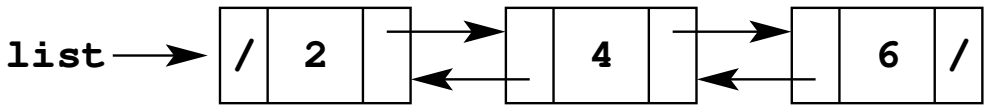
Doubly Linked Lists

- One disadvantage of singly-linked lists: unidirectional movement through list
- If a lot of movement back and forth through the list is required one can use a **doubly linked list**
- A doubly linked list maintains a link to the previous node in the list:



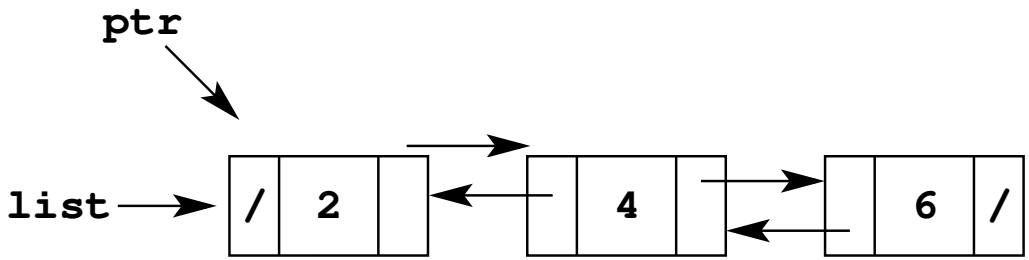
Doubly Linked Lists

- One disadvantage of singly-linked lists: unidirectional movement through list
- If a lot of movement back and forth through the list is required one can use a **doubly linked list**
- A doubly linked list maintains a link to the previous node in the list:



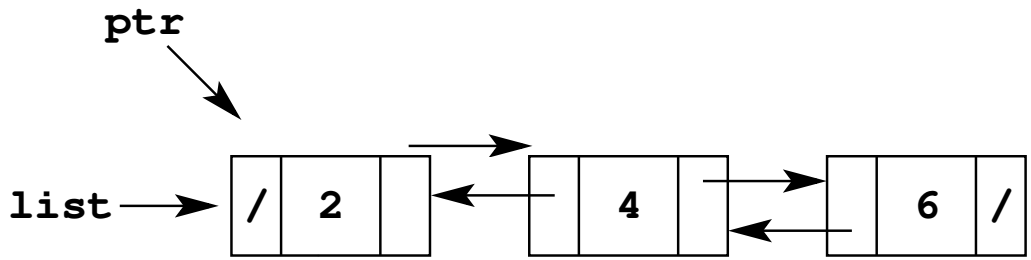

```
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
```

Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

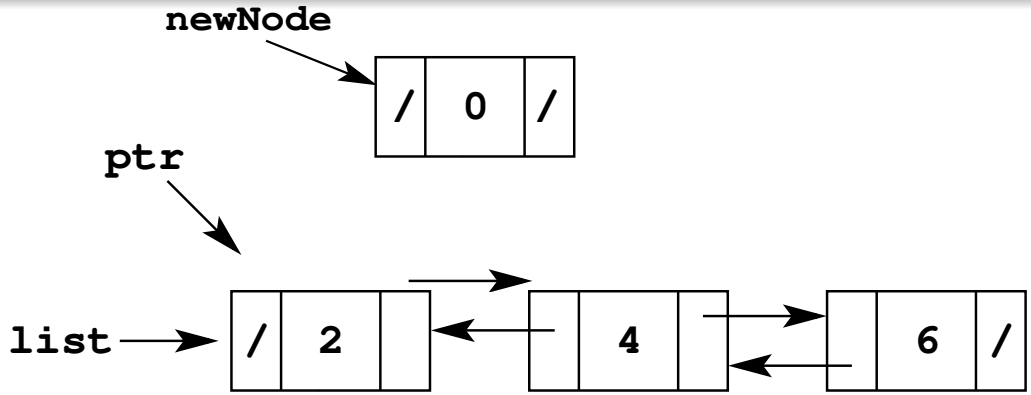
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
```

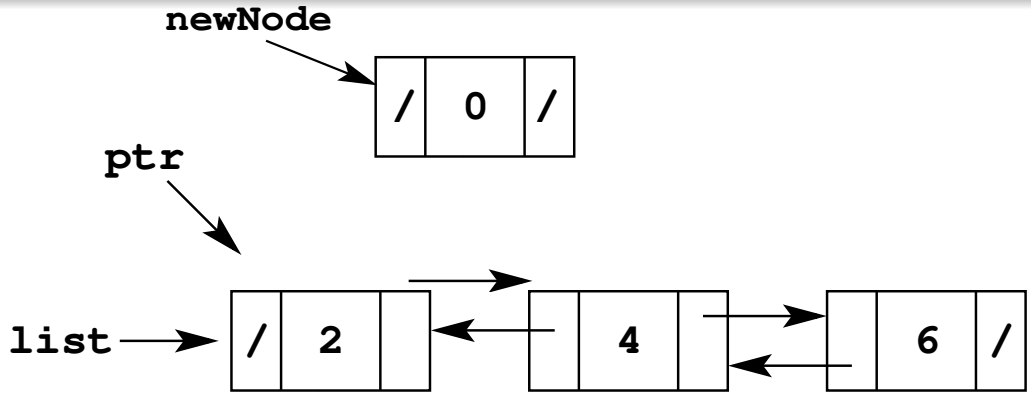
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
```

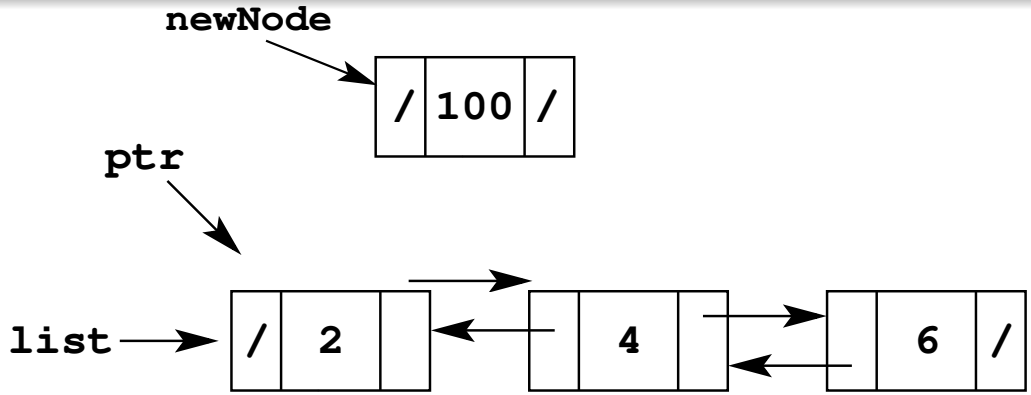
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
```

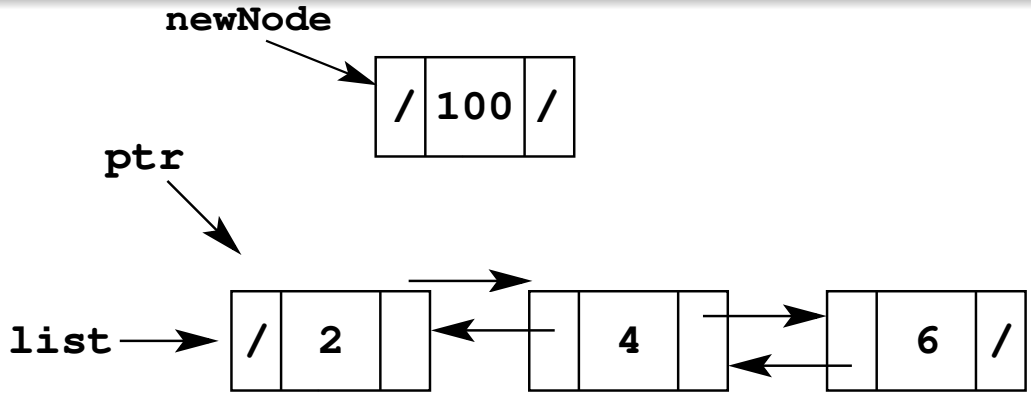
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
```

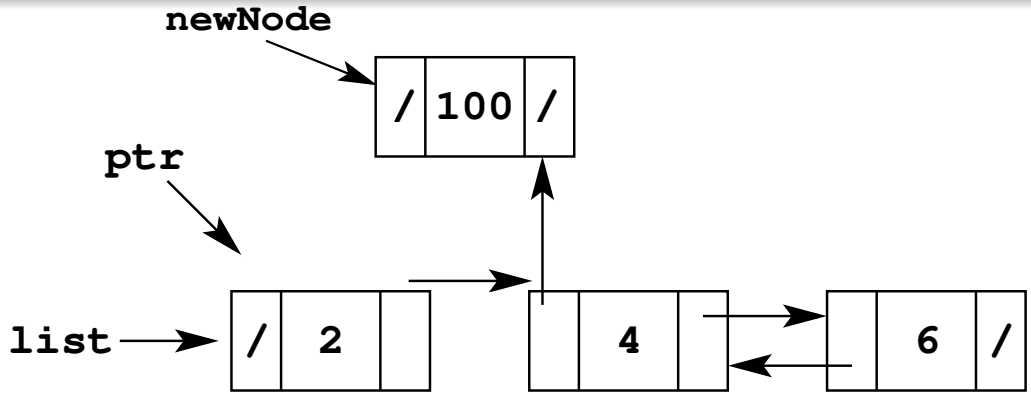
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
```

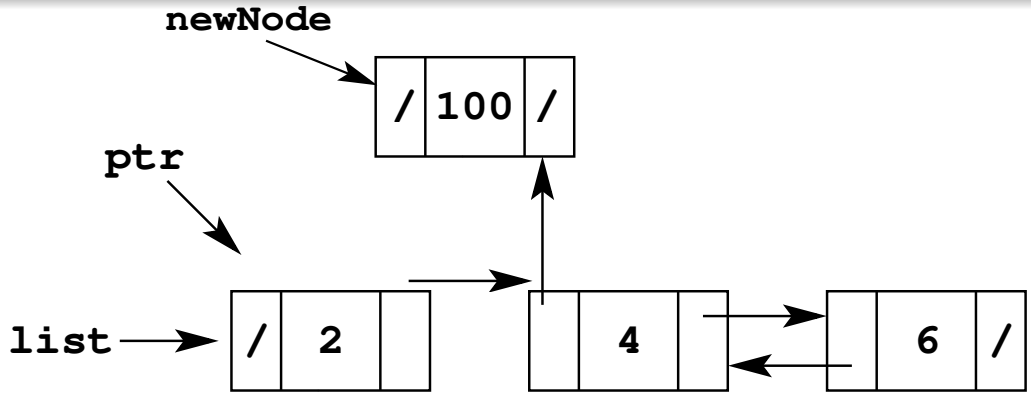
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
```

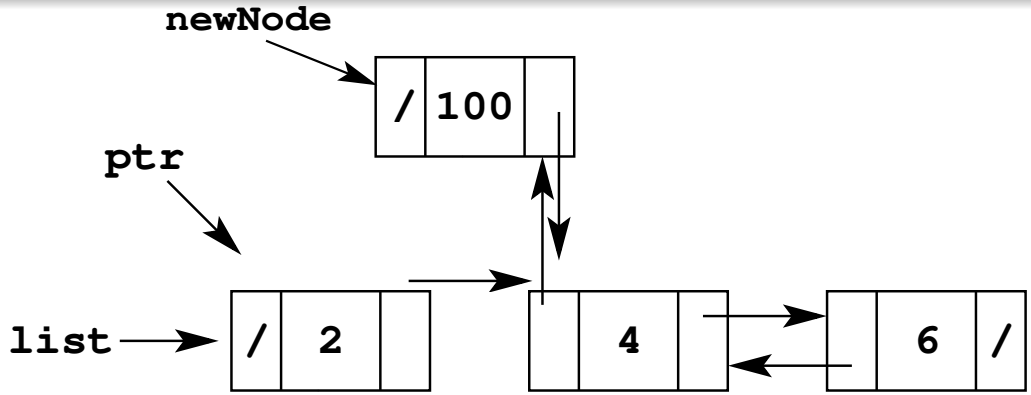
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
newNode->next = ptr->next;
```

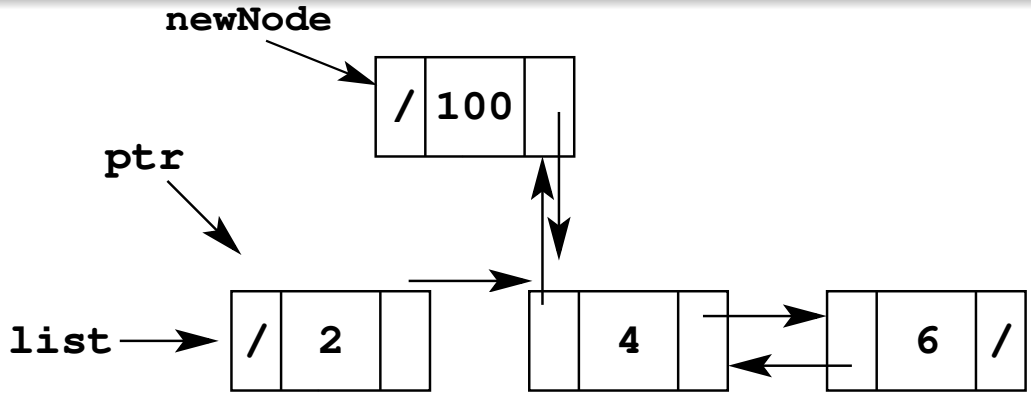
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
newNode->next = ptr->next;
```

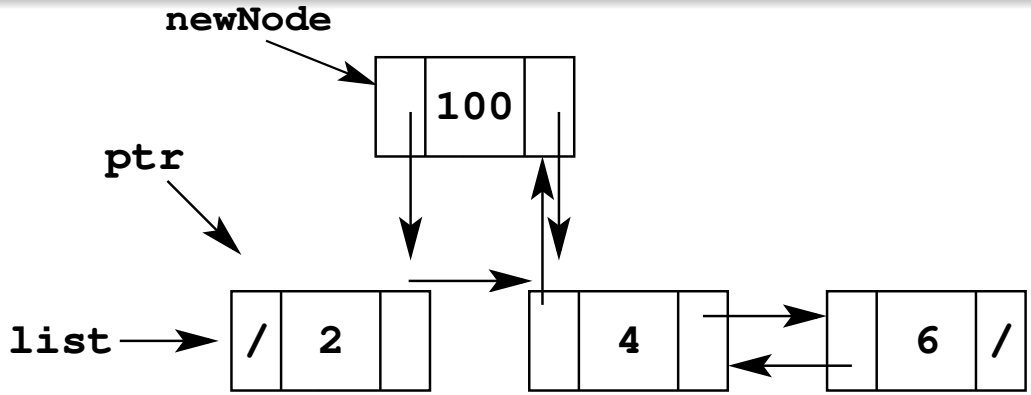
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
newNode->next = ptr->next;
newNode->prev = ptr;
```

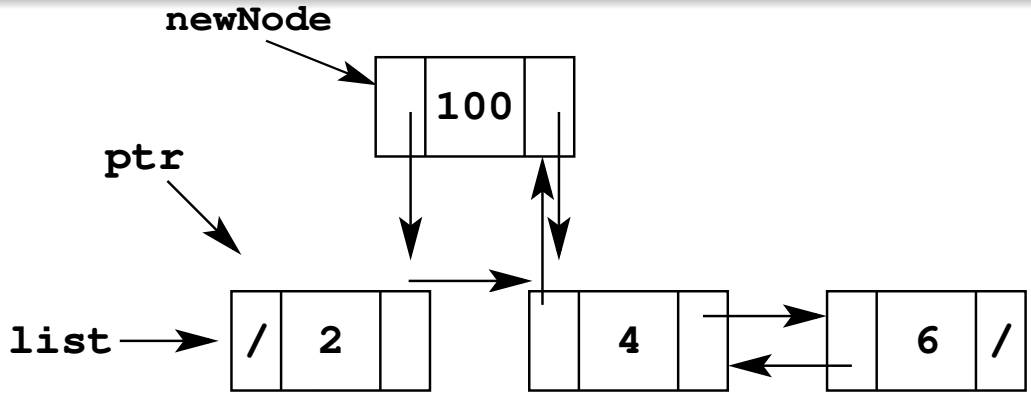
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
newNode->next = ptr->next;
newNode->prev = ptr;
```

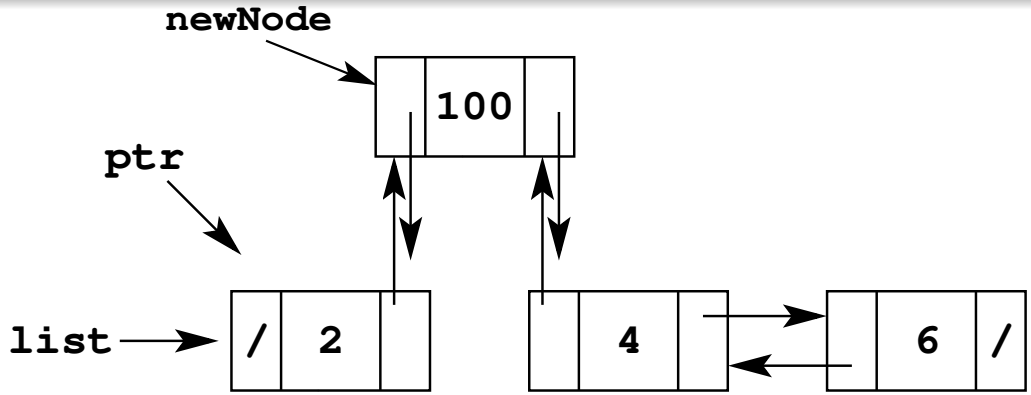
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
newNode->next = ptr->next;
newNode->prev = ptr;
ptr->next = newNode;
```

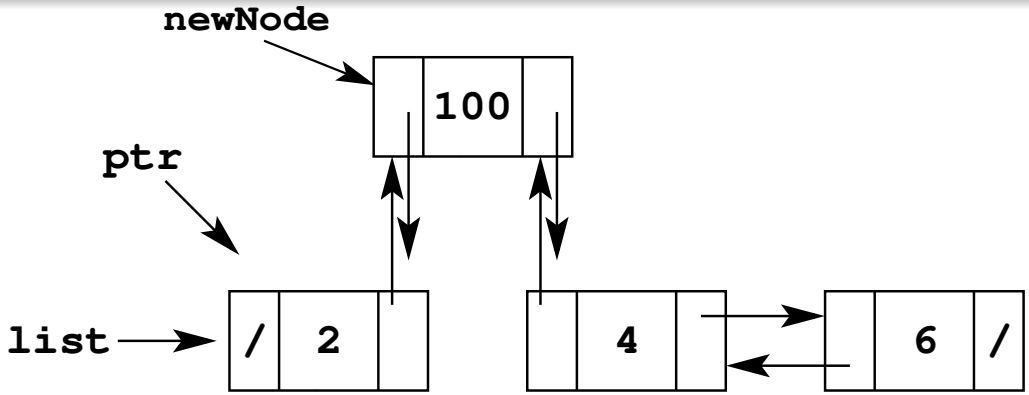
Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
newNode->next = ptr->next;
newNode->prev = ptr;
ptr->next = newNode;
```

Doubly Linked Lists: Adding Item in the Middle




```
typedef int listItem;
struct dlnode {
  int data;
  struct dlnode *prev;
  struct dlnode *next;
}
typedef struct dlnode *dllist;
```

```
dllist newNode = malloc(sizeof (struct dlnode));
newNode->data = 100;
ptr->next->prev = newNode;
newNode->next = ptr->next;
newNode->prev = ptr;
ptr->next = newNode;
```

done!

Choosing a Data Structure for Lists

If you ...	Then use ...

Choosing a Data Structure for Lists

If you ...	**Then use ...**
have frequent random access operations	

Choosing a Data Structure for Lists

If you ...	**Then use ...**
have frequent random access operations	dynamic array

Choosing a Data Structure for Lists

If you ...	**Then use ...**
have frequent random access operations	dynamic array
have many operations at a cursor	

Choosing a Data Structure for Lists

If you ...	**Then use ...**
have frequent random access operations	dynamic array
have many operations at a cursor	linked list

Choosing a Data Structure for Lists

If you ...	**Then use ...**
have frequent random access operations	dynamic array
have many operations at a cursor	linked list
have many operations at a two-way cursor	

Choosing a Data Structure for Lists

If you ...	**Then use ...**
have frequent random access operations	dynamic array
have many operations at a cursor	linked list
have many operations at a two-way cursor	doubly linked list

Choosing a Data Structure for Lists

If you ...	**Then use ...**
have frequent random access operations	dynamic array
have many operations at a cursor	linked list
have many operations at a two-way cursor	doubly linked list
have frequent resizing of the list	

Choosing a Data Structure for Lists

If you ...	**Then use ...**
have frequent random access operations	dynamic array
have many operations at a cursor	linked list
have many operations at a two-way cursor	doubly linked list
have frequent resizing of the list	linked list

Program Example: A List of Lists

- We need to write a program that maintains a list of medications (pills) available at a pharmacy
- Since each medication comes in various sizes and types, each drug will consist of a list of all its forms, e.g. drug CUREZIT might come in pill sizes 5mg, 10mg, 25mg, 50mg, 50mg Extended Release, and 100mg Extended Release
- Lists of drugs are stored in alphabetic order
- The different types (e.g. sizes) available of a specific drug do not change often
- New drugs come on the market and older drugs are discontinued frequently
- How would we design our data structures for this application?



Program Example: A List of Lists

- We need to write a program that maintains a list of medications (pills) available at a pharmacy
- Since each medication comes in various sizes and types, each drug will consist of a list of all its forms, e.g. drug CUREZIT might come in pill sizes 5mg, 10mg, 25mg, 50mg, 50mg Extended Release, and 100mg Extended Release
- Lists of drugs are stored in alphabetic order
- The different types (e.g. sizes) available of a specific drug do not change often
- New drugs come on the market and older drugs are discontinued frequently
- How would we design our data structures for this application?



Program Example: A List of Lists

- We need to write a program that maintains a list of medications (pills) available at a pharmacy
- Since each medication comes in various sizes and types, each drug will consist of a list of all its forms, e.g. drug CUREZIT might come in pill sizes 5mg, 10mg, 25mg, 50mg, 50mg Extended Release, and 100mg Extended Release
- Lists of drugs are stored in alphabetic order
- The different types (e.g. sizes) available of a specific drug do not change often
- New drugs come on the market and older drugs are discontinued frequently
- How would we design our data structures for this application?



Program Example: A List of Lists

- We need to write a program that maintains a list of medications (pills) available at a pharmacy
- Since each medication comes in various sizes and types, each drug will consist of a list of all its forms, e.g. drug CUREZIT might come in pill sizes 5mg, 10mg, 25mg, 50mg, 50mg Extended Release, and 100mg Extended Release
- Lists of drugs are stored in alphabetic order
- The different types (e.g. sizes) available of a specific drug do not change often
- New drugs come on the market and older drugs are discontinued frequently
- How would we design our data structures for this application?



Program Example: A List of Lists

- We need to write a program that maintains a list of medications (pills) available at a pharmacy
- Since each medication comes in various sizes and types, each drug will consist of a list of all its forms, e.g. drug CUREZIT might come in pill sizes 5mg, 10mg, 25mg, 50mg, 50mg Extended Release, and 100mg Extended Release
- Lists of drugs are stored in alphabetic order
- The different types (e.g. sizes) available of a specific drug do not change often
- New drugs come on the market and older drugs are discontinued frequently
- How would we design our data structures for this application?



Program Example: A List of Lists

- We need to write a program that maintains a list of medications (pills) available at a pharmacy
- Since each medication comes in various sizes and types, each drug will consist of a list of all its forms, e.g. drug CUREZIT might come in pill sizes 5mg, 10mg, 25mg, 50mg, 50mg Extended Release, and 100mg Extended Release
- Lists of drugs are stored in alphabetic order
- The different types (e.g. sizes) available of a specific drug do not change often
- New drugs come on the market and older drugs are discontinued frequently
- How would we design our data structures for this application?



Overview – Stacks

- A **stack** is a homogeneous data structure of ordered entries such that entries can be inserted and removed only at one end (called the **top**)
- Entries may be of any type
- By *ordered* all we mean is there is an item that can be accessed first (the one on top), one that is accessed second (the one below the top item), one that is accessed third, etc.
- A stack is a **LIFO** data structure – **L**ast-**I**n/**F**irst **O**ut (i.e. items are removed from the stack in the reverse order of their insertion)
- An attempt to add an item to a stack at its capacity creates **stack overflow**; trying to remove an item from an empty stack creates **stack underflow**



Overview – Stacks

- A **stack** is a homogeneous data structure of ordered entries such that entries can be inserted and removed only at one end (called the **top**)
- Entries may be of any type
- By *ordered* all we mean is there is an item that can be accessed first (the one on top), one that is accessed second (the one below the top item), one that is accessed third, etc.
- A stack is a **LIFO** data structure – **L**ast-**I**n/**F**irst **O**ut (i.e. items are removed from the stack in the reverse order of their insertion)
- An attempt to add an item to a stack at its capacity creates **stack overflow**; trying to remove an item from an empty stack creates **stack underflow**



Overview – Stacks

- A **stack** is a homogeneous data structure of ordered entries such that entries can be inserted and removed only at one end (called the **top**)
- Entries may be of any type
- By *ordered* all we mean is there is an item that can be accessed first (the one on top), one that is accessed second (the one below the top item), one that is accessed third, etc.
- A stack is a **LIFO** data structure – **L**ast-**I**n/**F**irst **O**ut (i.e. items are removed from the stack in the reverse order of their insertion)
- An attempt to add an item to a stack at its capacity creates **stack overflow**; trying to remove an item from an empty stack creates **stack underflow**



Overview – Stacks

- A **stack** is a homogeneous data structure of ordered entries such that entries can be inserted and removed only at one end (called the **top**)
- Entries may be of any type
- By *ordered* all we mean is there is an item that can be accessed first (the one on top), one that is accessed second (the one below the top item), one that is accessed third, etc.
- A stack is a **LIFO** data structure – **L**ast-**I**n/**F**irst **O**ut (i.e. items are removed from the stack in the reverse order of their insertion)
- An attempt to add an item to a stack at its capacity creates **stack overflow**; trying to remove an item from an empty stack creates **stack underflow**

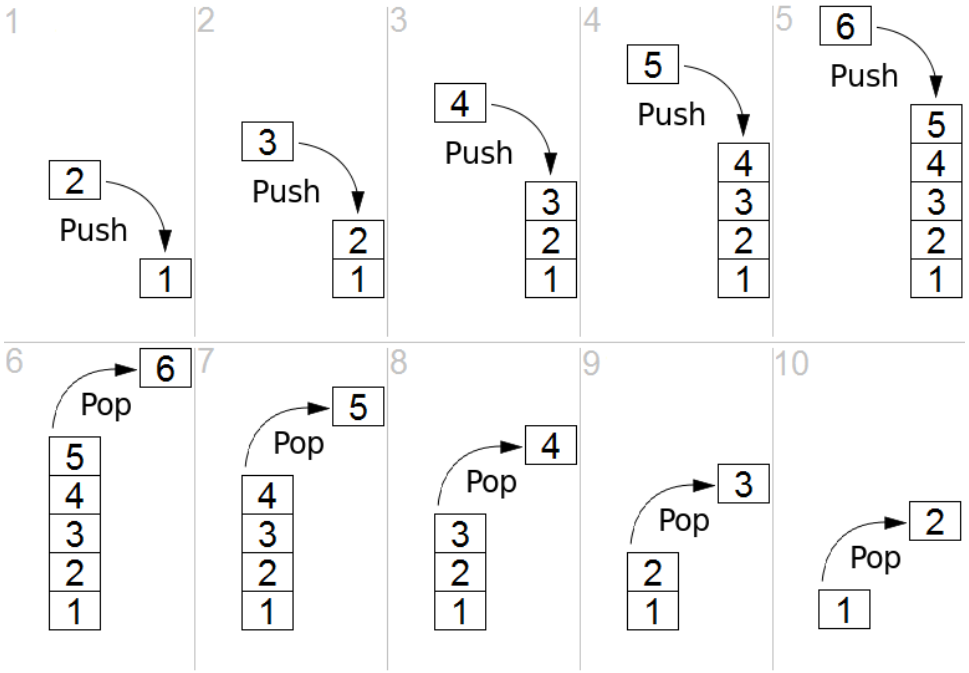


Overview – Stacks

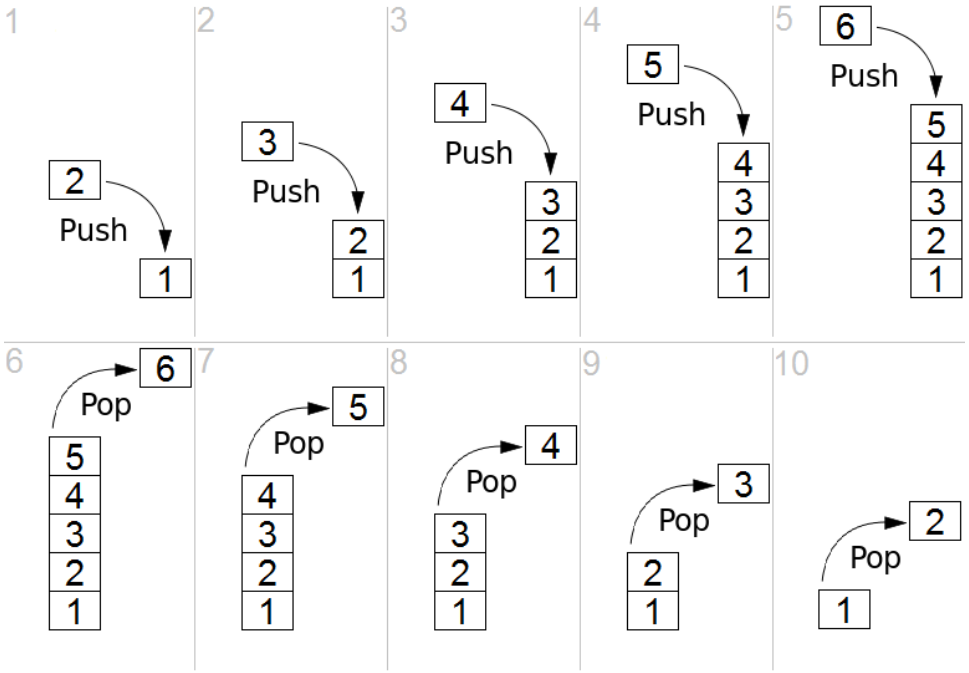
- A **stack** is a homogeneous data structure of ordered entries such that entries can be inserted and removed only at one end (called the **top**)
- Entries may be of any type
- By *ordered* all we mean is there is an item that can be accessed first (the one on top), one that is accessed second (the one below the top item), one that is accessed third, etc.
- A stack is a **LIFO** data structure – **L**ast-**I**n/**F**irst **O**ut (i.e. items are removed from the stack in the reverse order of their insertion)
- An attempt to add an item to a stack at its capacity creates **stack overflow**; trying to remove an item from an empty stack creates **stack underflow**



Stack Operations

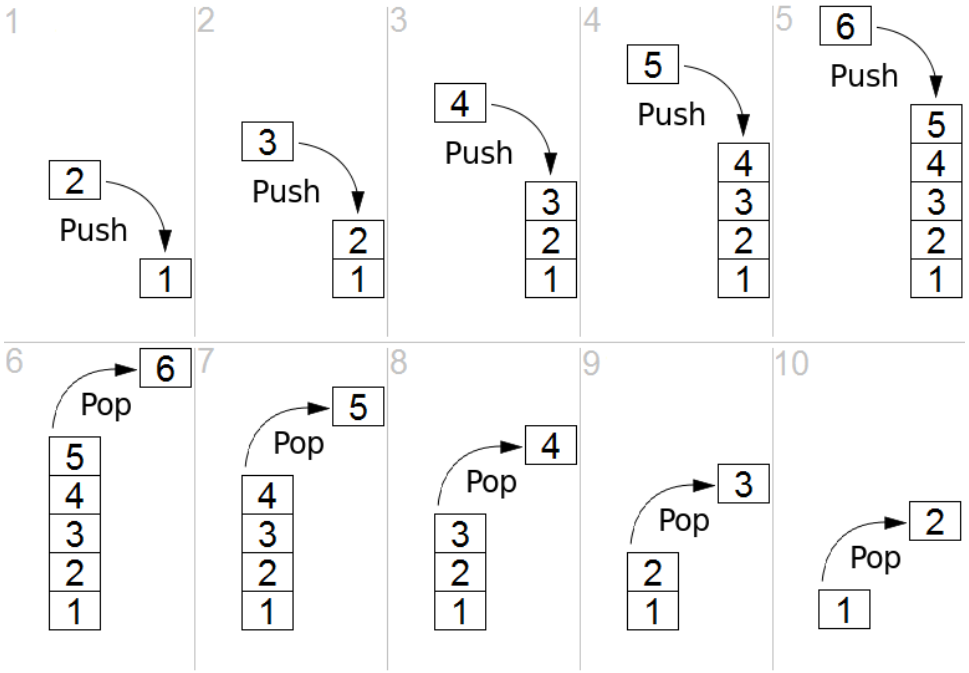


Stack Operations



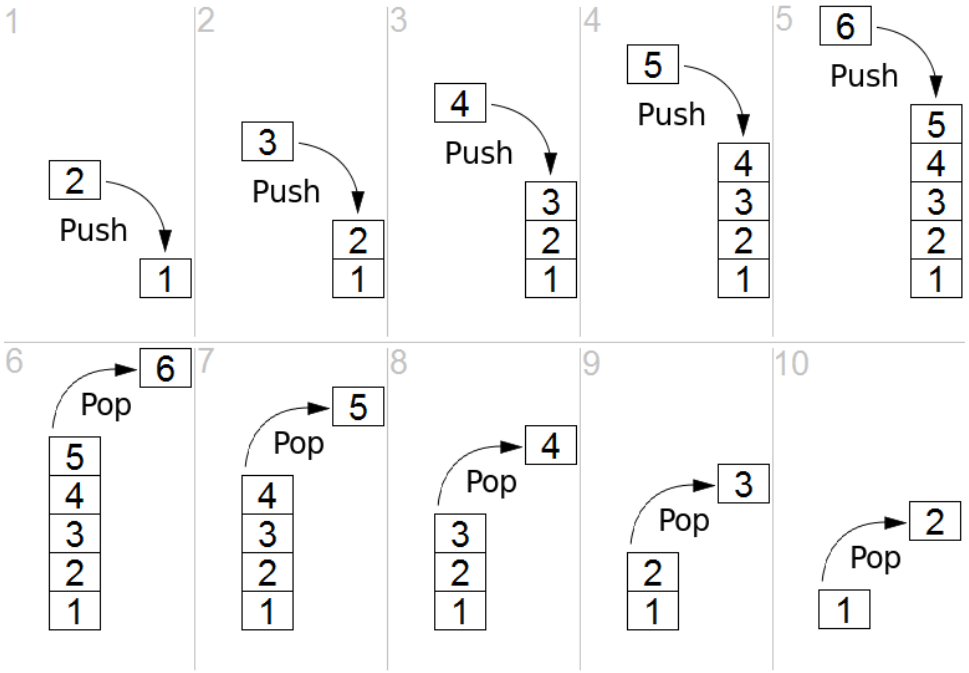
- Common operations on data type `stack`:
  - `void push(stack,itemType const);`
  - `void pop(stack);`
  - `itemType top(stack);`
  - `bool isEmpty(stack);`

Stack Operations



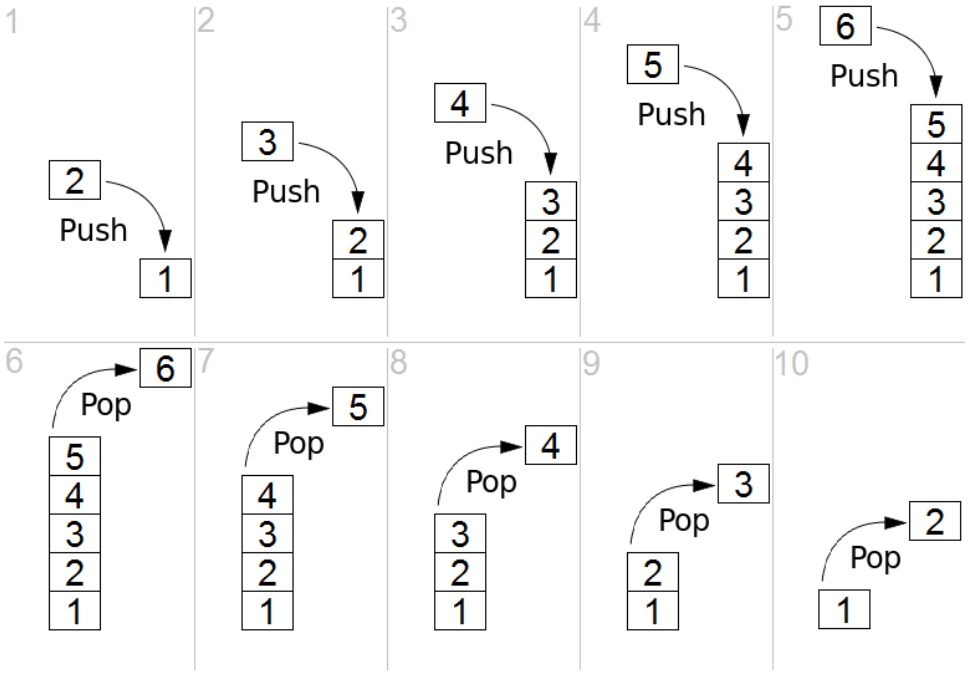
- Common operations on data type `stack`:
  - `void push(stack,itemType const);`
  - `void pop(stack);`
  - `itemType top(stack);`
  - `bool isEmpty(stack);`

Stack Operations



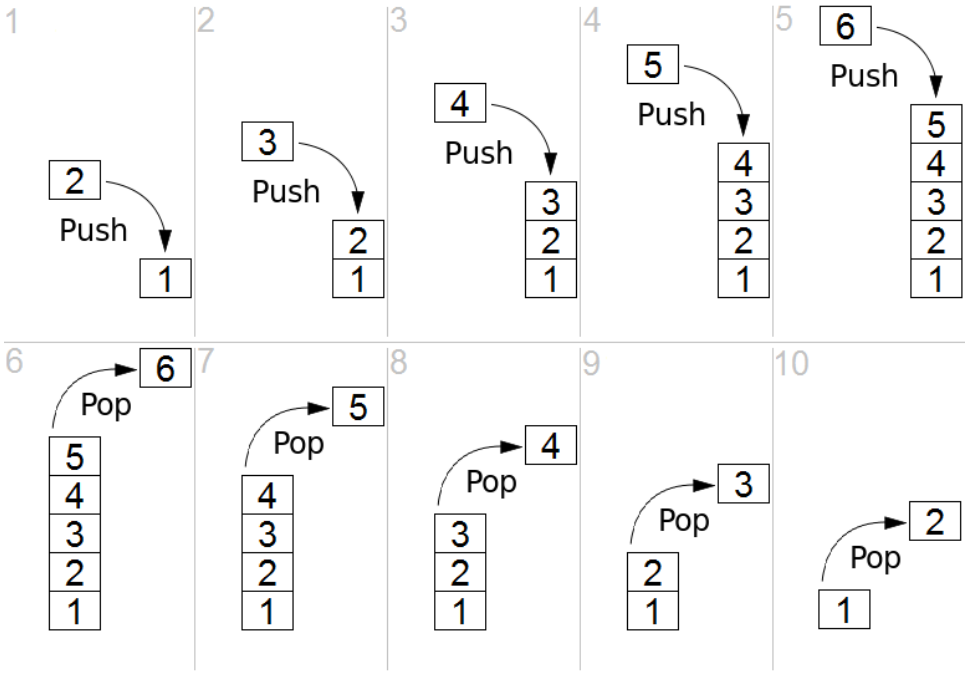
- Common operations on data type `stack`:
  - `void push(stack,itemType const);`
  - `void pop(stack);`
  - `itemType top(stack);`
  - `bool isEmpty(stack);`

Stack Operations



- Common operations on data type `stack`:
  - `void push(stack,itemType const);`
  - `void pop(stack);`
  - `itemType top(stack);`
  - `bool isEmpty(stack);`

Stack Operations

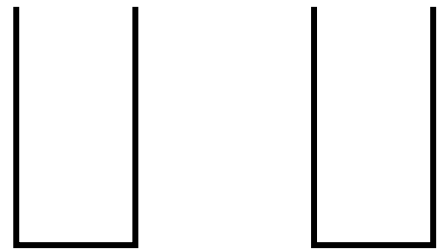


- Common operations on data type `stack`:
  - `void push(stack,itemType const);`
  - `void pop(stack);`
  - `itemType top(stack);`
  - `bool isEmpty(stack);`

An Application Using Stacks: Expression Evaluation

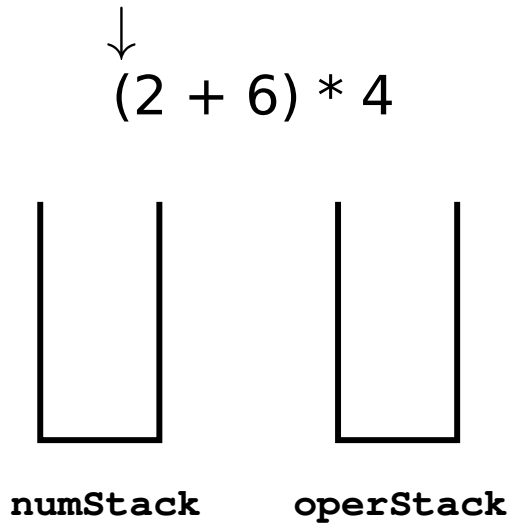
```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```

(2 + 6) * 4



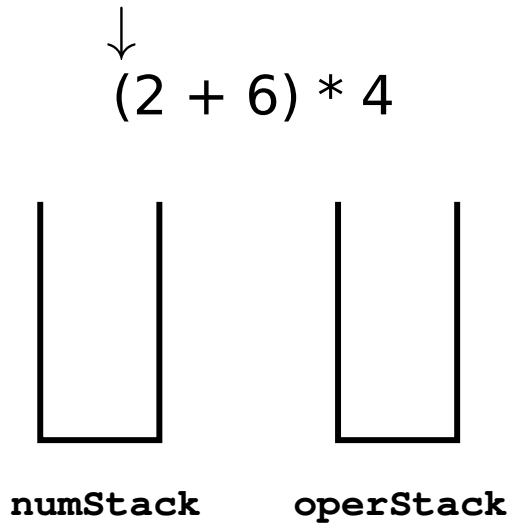
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



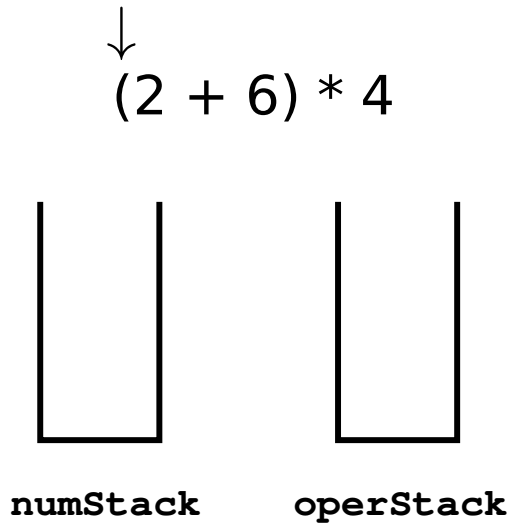
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



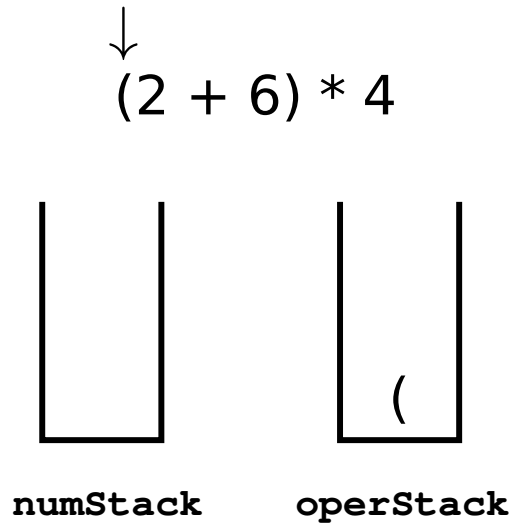
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



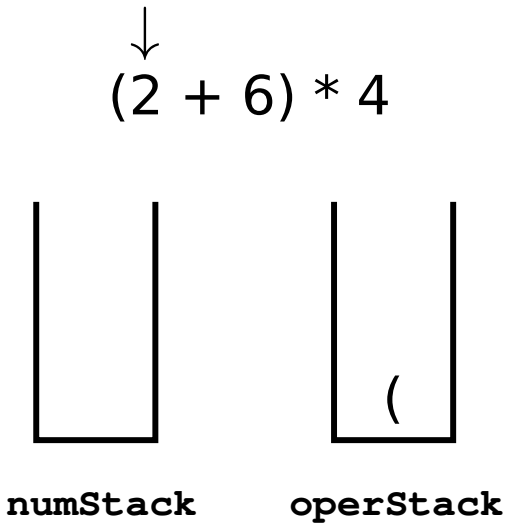
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



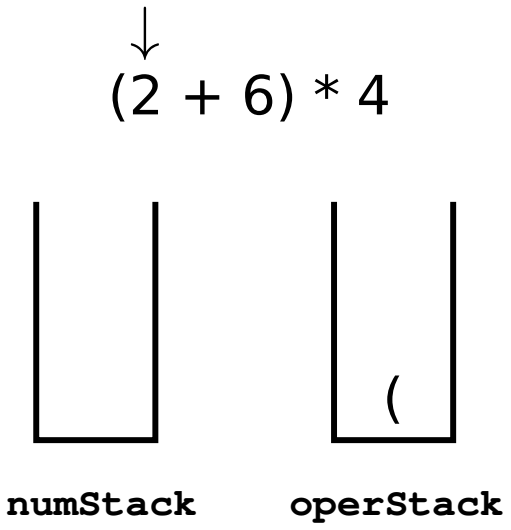
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



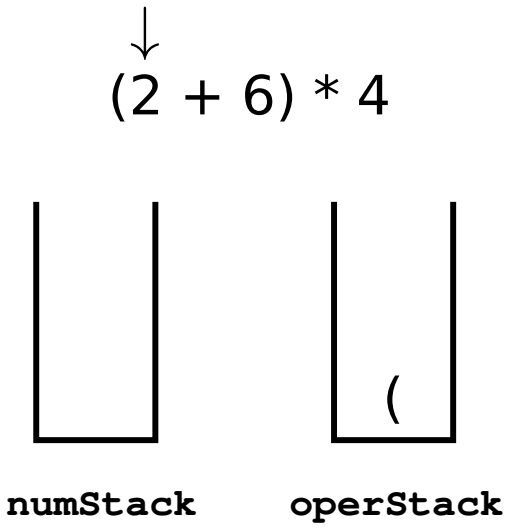
An Application Using Stacks: Expression Evaluation

if nextChar is number
then push onto numStack;
else if nextChar is operator
then if it has lower or equal precedence with operator on operStack
then eval operator on operStack
push current operator on operStack
else if nextChar is left paren
then push it onto operStack
else if nextChar is right paren
then while true do
if top of operStack equals left paren
then pop operStack
exit loop
perform operation at top of operStack
else if nextChar is the end of the line
then while the operStack is not empty
perform operation at top of operStack
print the top of numStack as the result
else skip character



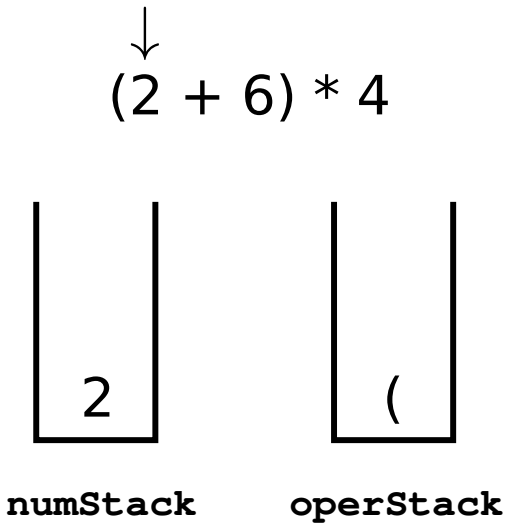
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



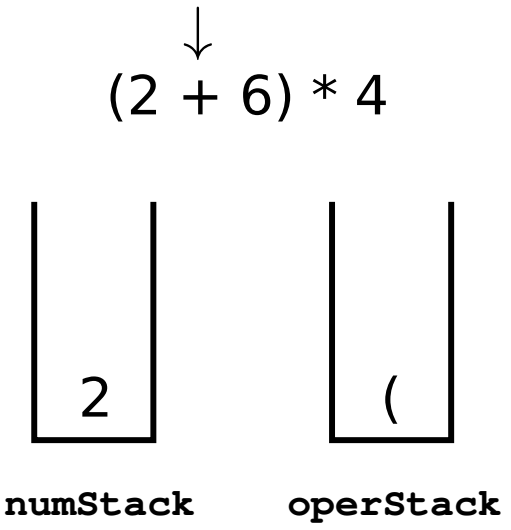
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



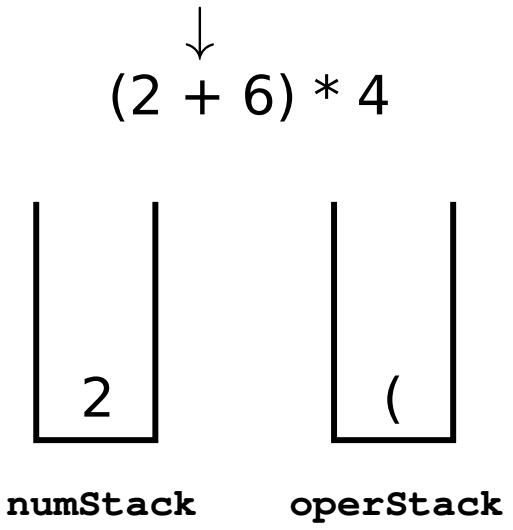
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



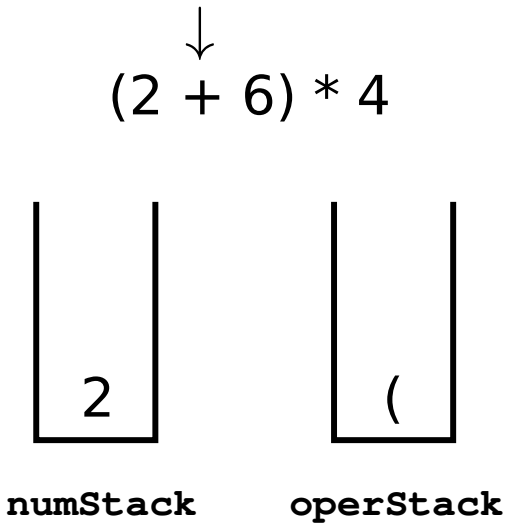
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



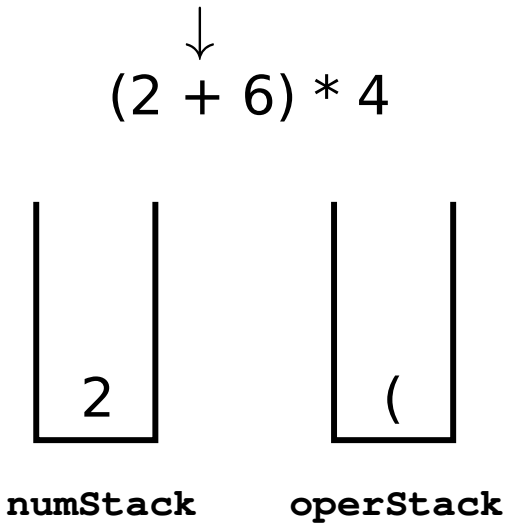
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



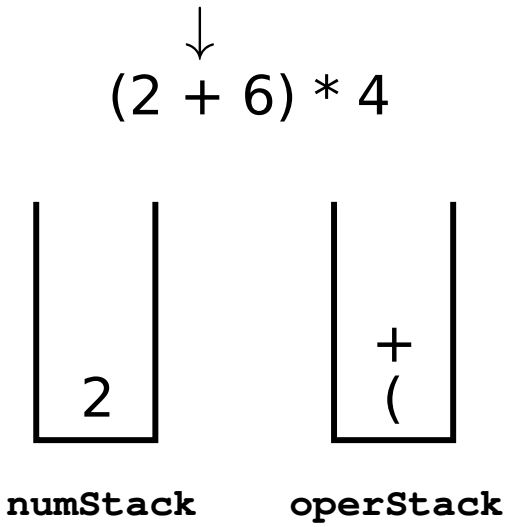
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



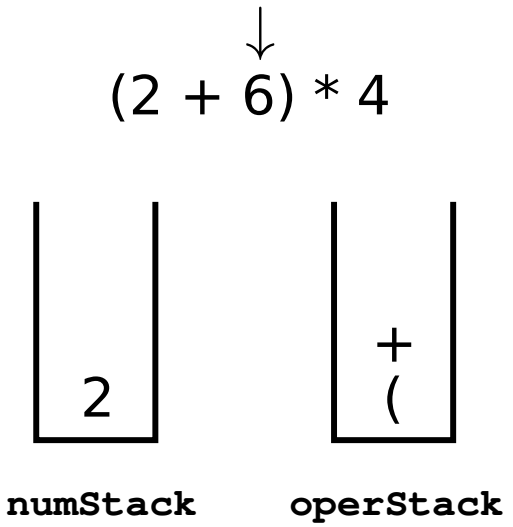
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



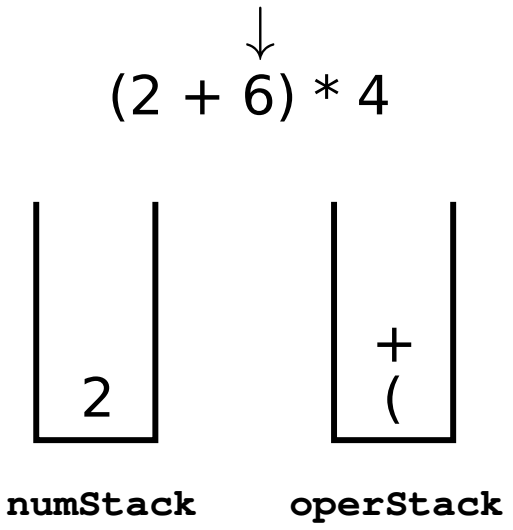
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



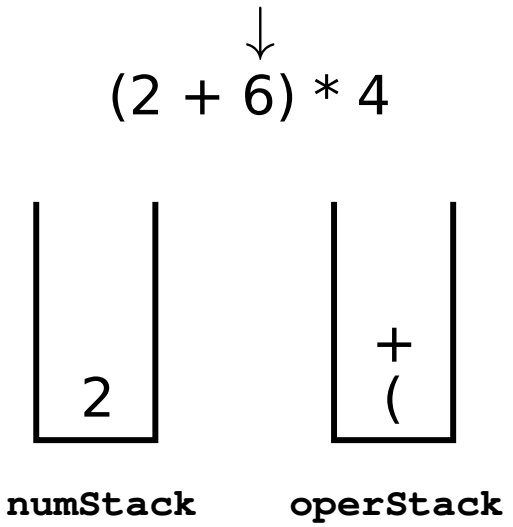
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



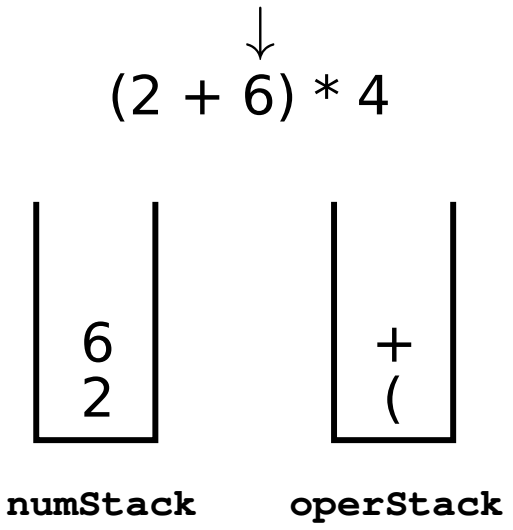
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



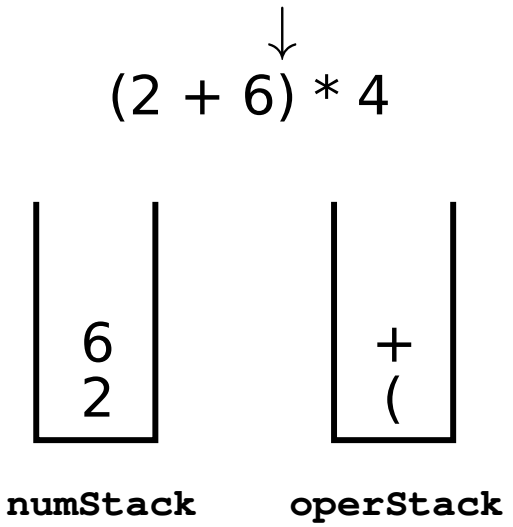
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



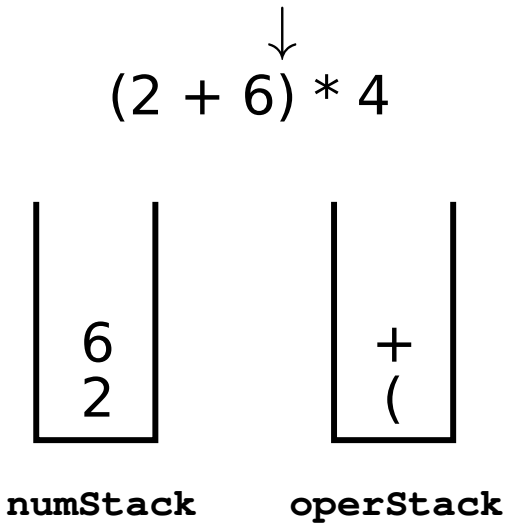
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



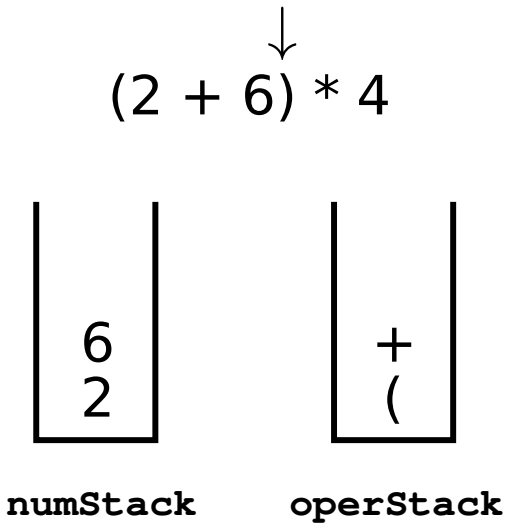
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



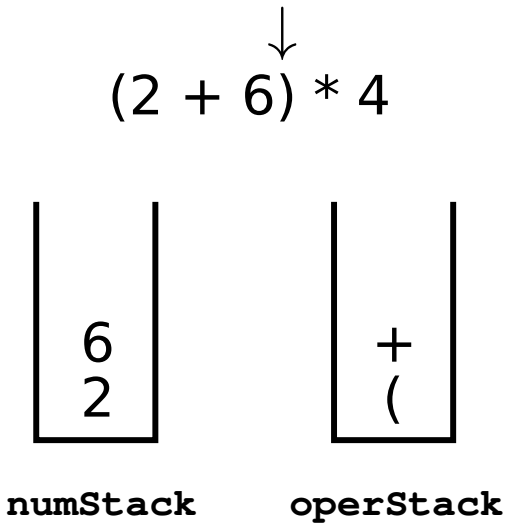
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



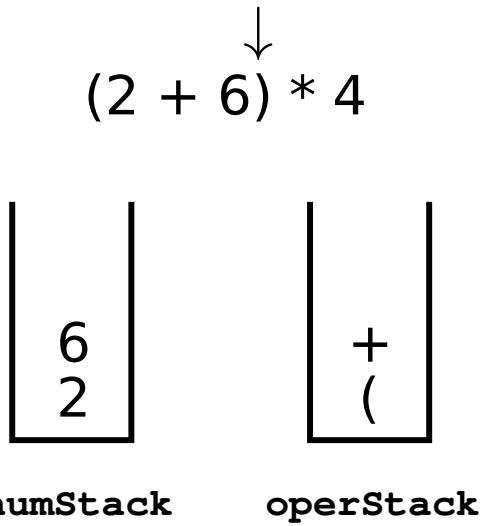
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                  if top of operStack equals left paren
                    then pop operStack
                         exit loop
                  perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                  perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



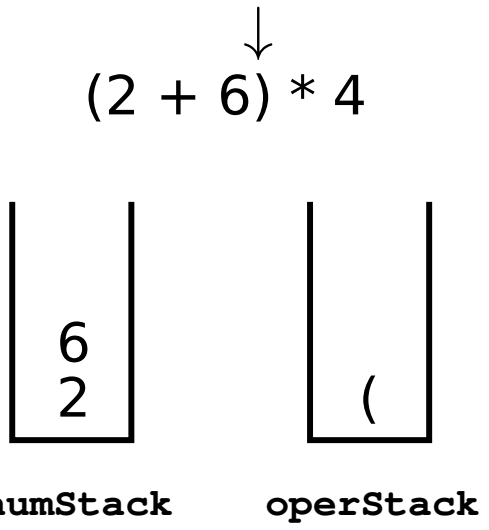
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



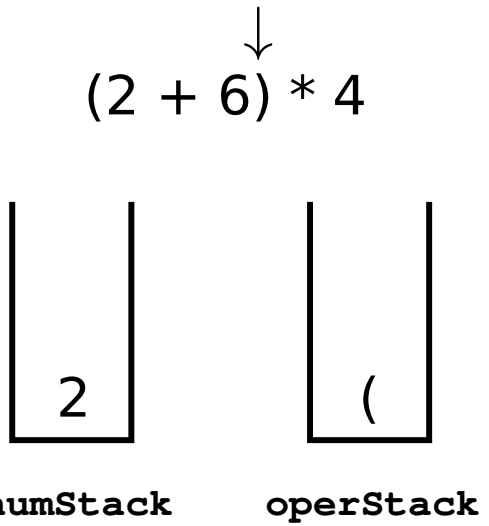
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



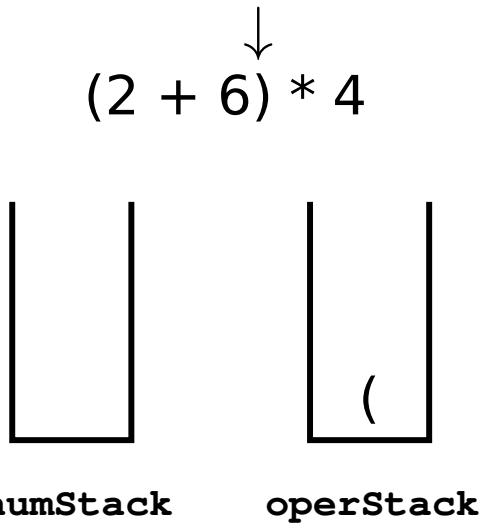
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



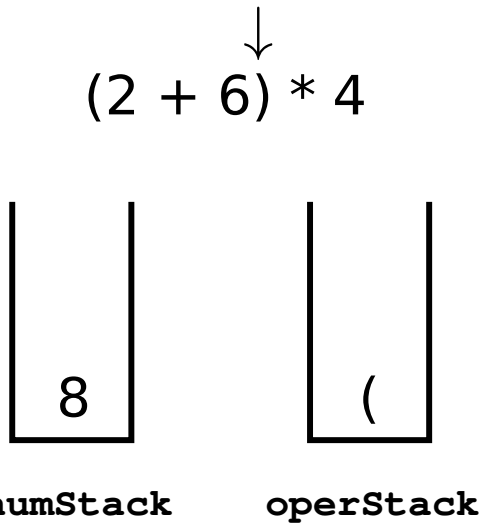
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



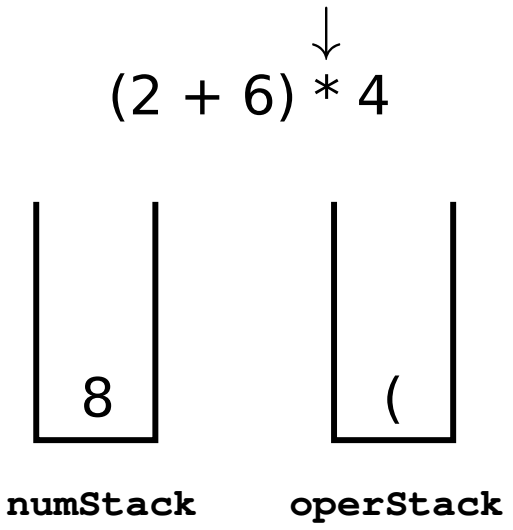
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



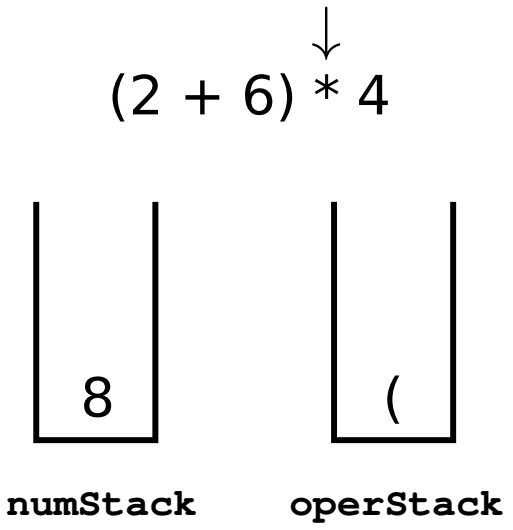
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



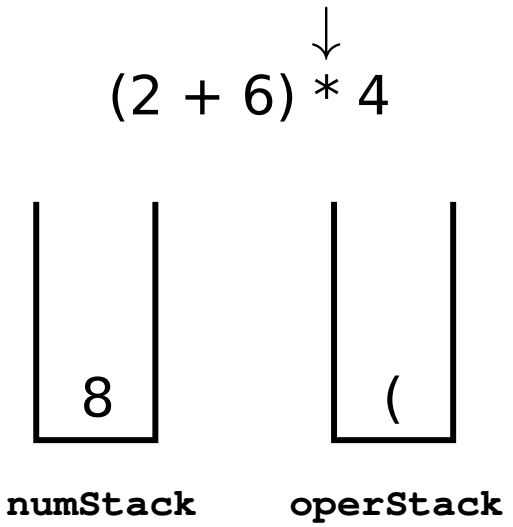
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



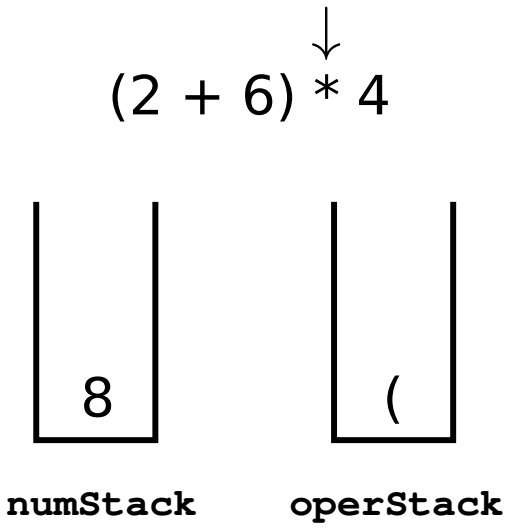
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



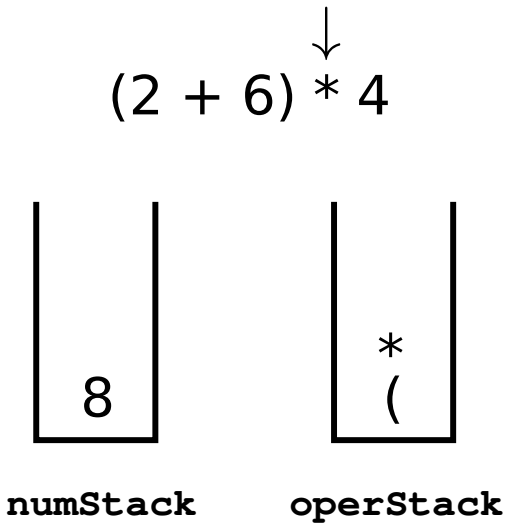
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



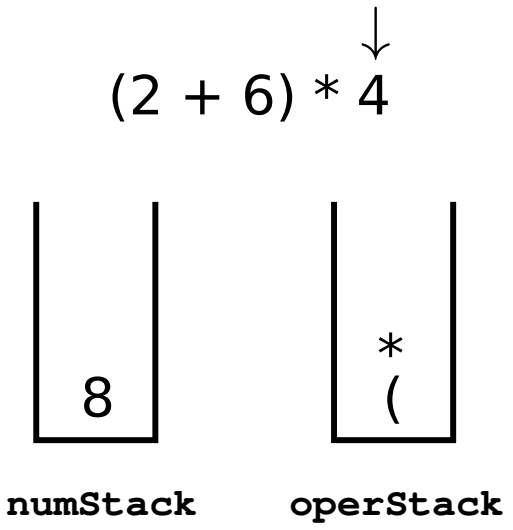
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



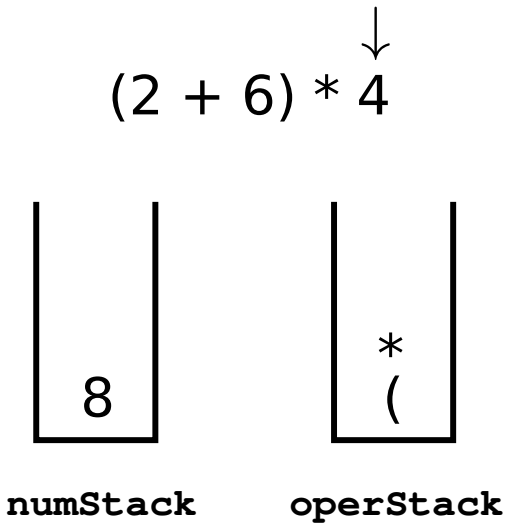
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



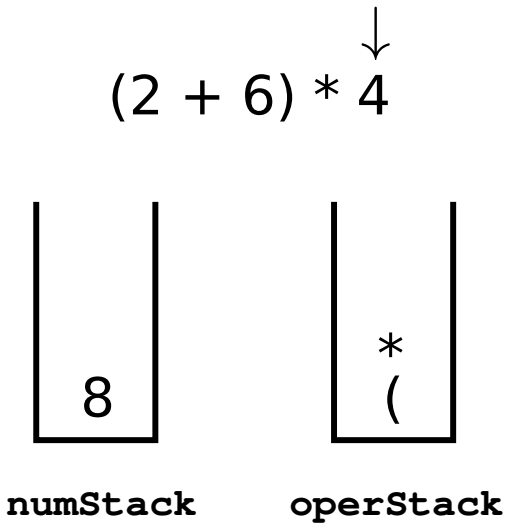
An Application Using Stacks: Expression Evaluation

if nextChar is number
then push onto numStack;
else if nextChar is operator
then if it has lower or equal precedence with operator on operStack
then eval operator on operStack
push current operator on operStack
else if nextChar is left paren
then push it onto operStack
else if nextChar is right paren
then while true do
if top of operStack equals left paren
then pop operStack
exit loop
perform operation at top of operStack
else if nextChar is the end of the line
then while the operStack is not empty
perform operation at top of operStack
print the top of numStack as the result
else skip character



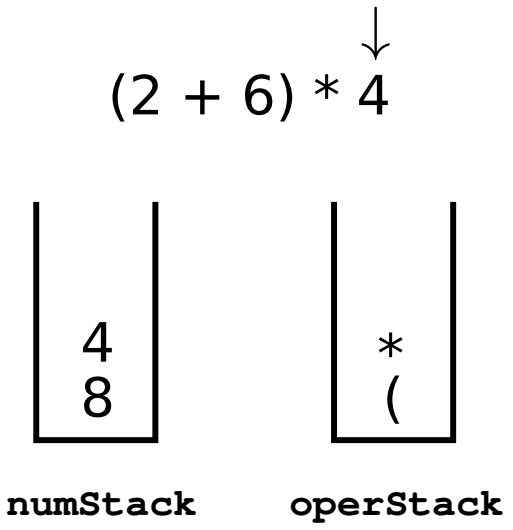
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



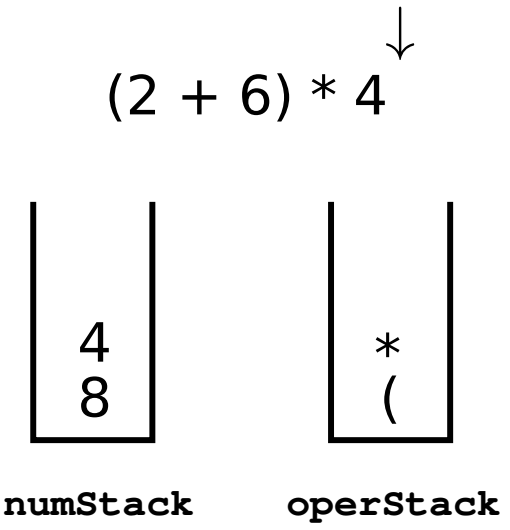
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



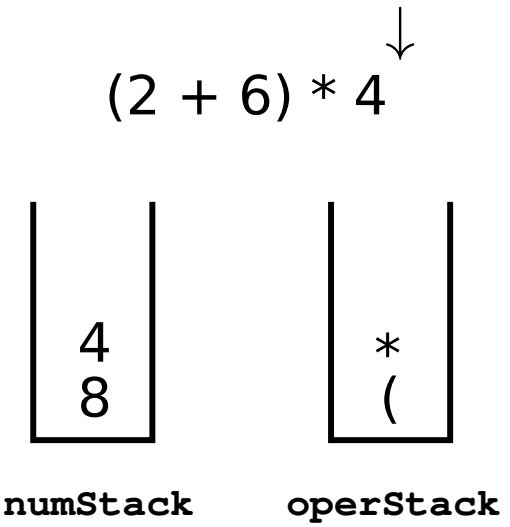
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



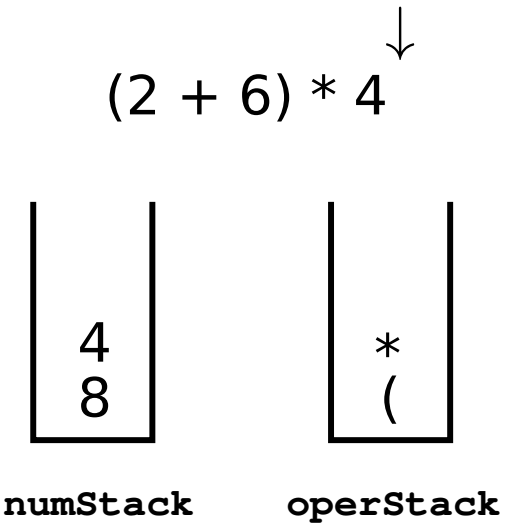
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
             perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



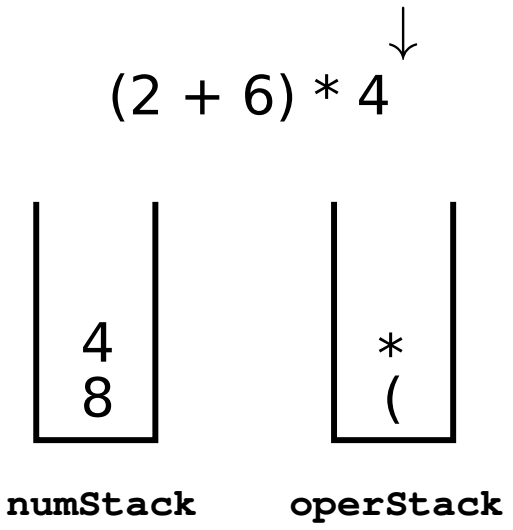
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



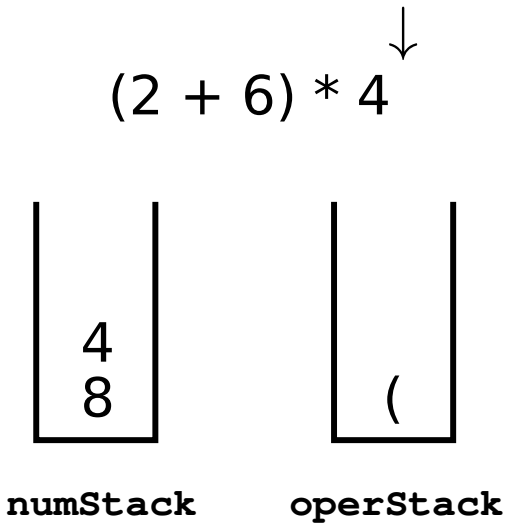
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



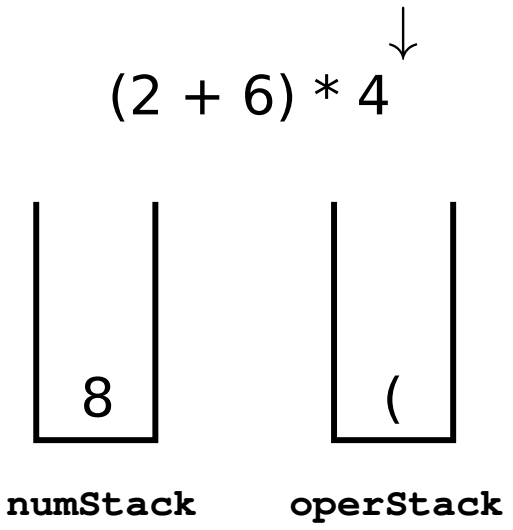
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



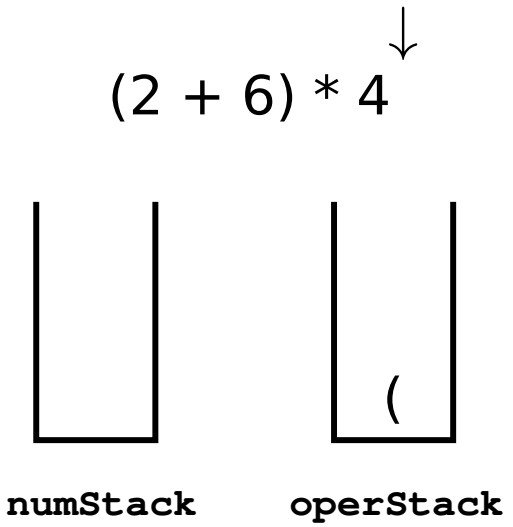
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



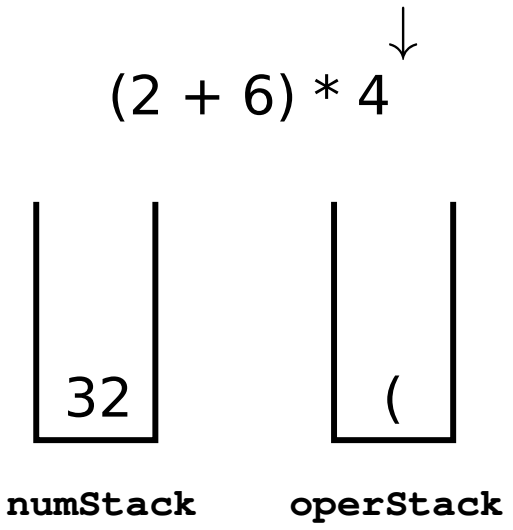
An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



An Application Using Stacks: Expression Evaluation

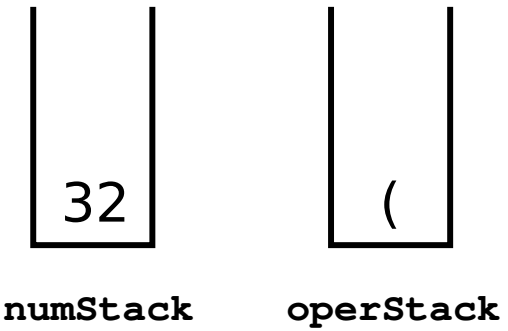
```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```



An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```

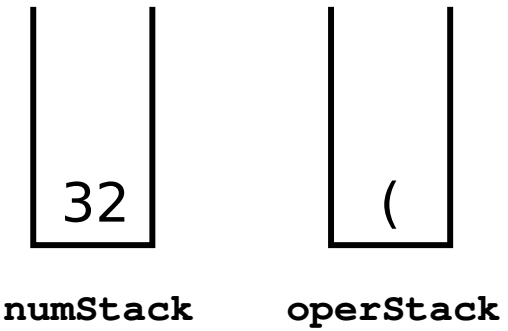
(2 + 6) * 4



An Application Using Stacks: Expression Evaluation

```
if nextChar is number
  then push onto numStack;
else if nextChar is operator
        then if it has lower or equal precedence with operator on operStack
                then eval operator on operStack
             push current operator on operStack
else if nextChar is left paren
        then push it onto operStack
else if nextChar is right paren
        then while true do
                if top of operStack equals left paren
                  then pop operStack
                       exit loop
                perform operation at top of operStack
else if nextChar is the end of the line
        then while the operStack is not empty
                perform operation at top of operStack
             print the top of numStack as the result
else skip character
```

(2 + 6) * 4 =32



Implementing a Stack Using Arrays

- File `stack.h`:

```
#include <stdbool.h>
#include "itemType.h"
#ifndef STACK_H
#define STACK_H

typedef struct stackType *stack;

stack createStack();
void freeStack(stack);

void push(stack,itemType);
void pop(stack);
itemType top(stack);
bool isEmpty(stack);
bool isFull(stack);

#endif
```



Implementing a Stack Using Arrays *continued*

- File `stack.c`:

```
// ...
#include "stack.h"
#include "itemType.h"

#define STACKSIZE 100 // arbitrary

struct stackType {
  itemType data[STACKSIZE];
  int top;
};

stack createStack() {
  stack s = malloc(sizeof(struct stackType));
  s->top = -1;
  return s;
}
```



Implementing a Stack Using Arrays *continued*

```
void push(stack s,itemType p) {
  if(isFull(s)) {
    printf("Error: stack overflow\n");
    exit(EXIT_FAILURE);
  } else {
    s->data[++s->top] = p;
  }
}

void pop(stack s){
  if(isEmpty(s)){
    printf("Error: stack underflow\n");
    exit(EXIT_FAILURE);
  } else {
    s->top--;
  }
}
```



Implementing a Stack Using Arrays *continued*

```
void push(stack s,itemType p) {
  if(isFull(s)) {
    printf("Error: stack overflow\n");
    exit(EXIT_FAILURE);
  } else {
    s->data[++s->top] = p;
  }
}

void pop(stack s){
  if(isEmpty(s)){
    printf("Error: stack underflow\n");
    exit(EXIT_FAILURE);
  } else {
    s->top--;
  }
}
```

Instead of exiting if stack is full you can allocate a larger array as needed



Implementing a Stack Using Arrays *continued*

```
itemType top(stack s) {
  if(isEmpty(s)){
    printf("Error: stack underflow\n");
    exit(EXIT_FAILURE);
  } else {
    return s->data[s->top];
  }
}

bool isEmpty(stack s) { return s->top == -1; }
bool isFull(stack s) { return s->top == STACKSIZE-1; }
```



Implementing a Stack as a Linked List

- Essentially treat the front (or rear) of the list as the top of the stack
- Algorithms become easy as you only have a single case (insert/remove from top) to consider



Overview – Queues

- A **queue** is a homogeneous data structure of ordered entries such that entries can be inserted at the rear and removed from the front
- Entries may be of any type
- As with stacks, by *ordered* all we mean is there is an item that can be accessed first (the one at the front), one that is accessed second (the one after the front item), one that is accessed third, etc.
- A stack is a **FIFO** data structue – **F**irst-**I**n/**F**irst **O**ut (i.e. items are removed from the queue in the same order of their insertion)
- An attempt to add an item to a queue at its capacity creates **queue overflow**; trying to remove an item from an empty queue creates **queue underflow**



Overview – Queues

- A **queue** is a homogeneous data structure of ordered entries such that entries can be inserted at the rear and removed from the front
- Entries may be of any type
- As with stacks, by *ordered* all we mean is there is an item that can be accessed first (the one at the front), one that is accessed second (the one after the front item), one that is accessed third, etc.
- A stack is a **FIFO** data structue – **F**irst-**I**n/**F**irst **O**ut (i.e. items are removed from the queue in the same order of their insertion)
- An attempt to add an item to a queue at its capacity creates **queue overflow**; trying to remove an item from an empty queue creates **queue underflow**



Overview – Queues

- A **queue** is a homogeneous data structure of ordered entries such that entries can be inserted at the rear and removed from the front
- Entries may be of any type
- As with stacks, by *ordered* all we mean is there is an item that can be accessed first (the one at the front), one that is accessed second (the one after the front item), one that is accessed third, etc.
- A stack is a **FIFO** data structue – **F**irst-**I**n/**F**irst **O**ut (i.e. items are removed from the queue in the same order of their insertion)
- An attempt to add an item to a queue at its capacity creates **queue overflow**; trying to remove an item from an empty queue creates **queue underflow**



Overview – Queues

- A **queue** is a homogeneous data structure of ordered entries such that entries can be inserted at the rear and removed from the front
- Entries may be of any type
- As with stacks, by *ordered* all we mean is there is an item that can be accessed first (the one at the front), one that is accessed second (the one after the front item), one that is accessed third, etc.
- A stack is a **FIFO** data structue – **F**irst-**I**n/**F**irst **O**ut (i.e. items are removed from the queue in the same order of their insertion)
- An attempt to add an item to a queue at its capacity creates **queue overflow**; trying to remove an item from an empty queue creates **queue underflow**

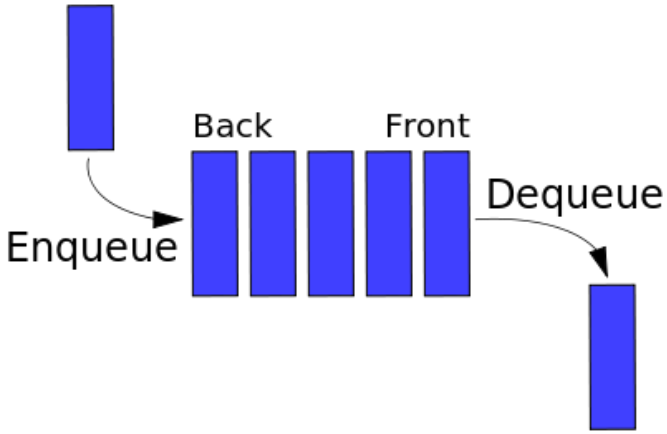


Overview – Queues

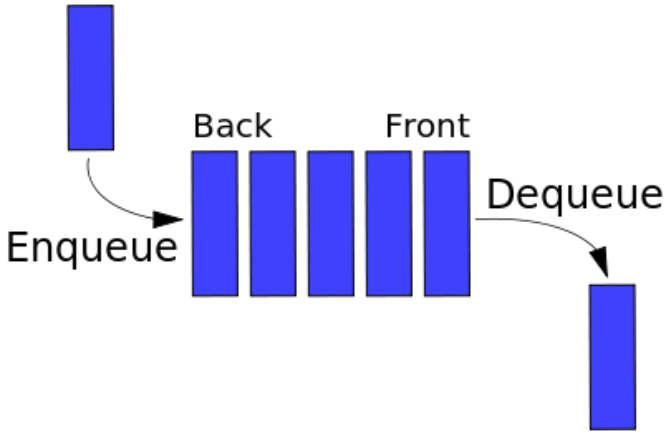
- A **queue** is a homogeneous data structure of ordered entries such that entries can be inserted at the rear and removed from the front
- Entries may be of any type
- As with stacks, by *ordered* all we mean is there is an item that can be accessed first (the one at the front), one that is accessed second (the one after the front item), one that is accessed third, etc.
- A stack is a **FIFO** data structue – **F**irst-**I**n/**F**irst **O**ut (i.e. items are removed from the queue in the same order of their insertion)
- An attempt to add an item to a queue at its capacity creates **queue overflow**; trying to remove an item from an empty queue creates **queue underflow**



Queue Operations

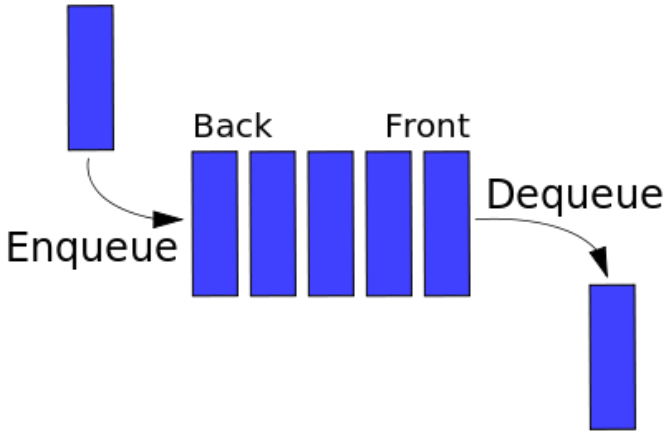


Queue Operations



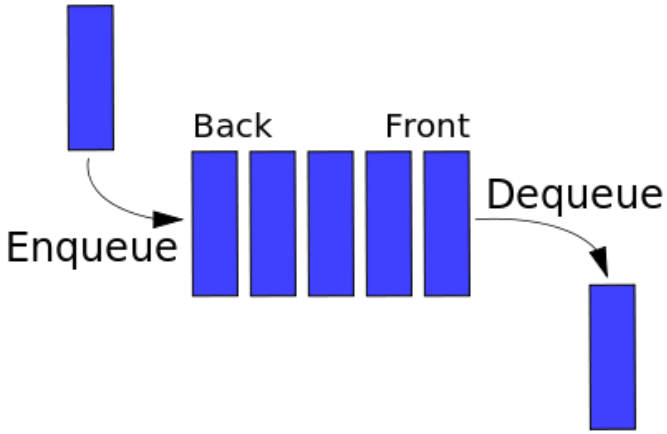
- Common operations on data type queue:
  - `void enqueue(queue,itemType);`
  - `void dequeue(queue);`
  - `itemType front(queue);`
  - `bool isEmpty(queue);`

Queue Operations



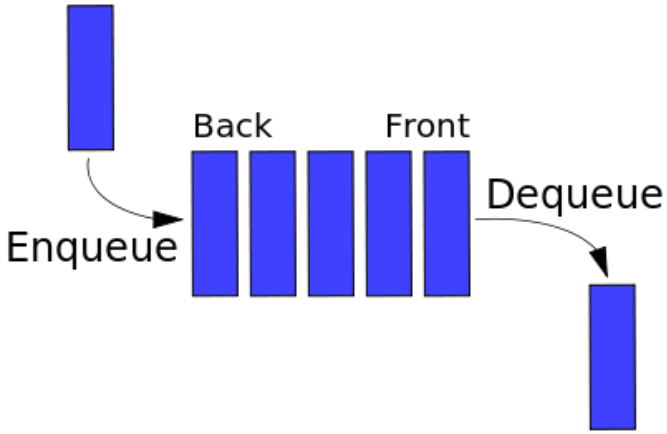
- Common operations on data type queue:
  - `void enqueue(queue,itemType);`
  - `void dequeue(queue);`
  - `itemType front(queue);`
  - `bool isEmpty(queue);`

Queue Operations



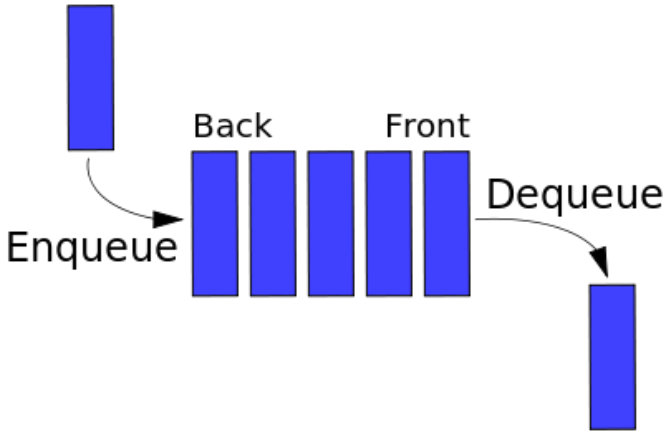
- Common operations on data type queue:
  - `void enqueue(queue,itemType);`
  - `void dequeue(queue);`
  - `itemType front(queue);`
  - `bool isEmpty(queue);`

Queue Operations



- Common operations on data type queue:
  - `void enqueue(queue,itemType);`
  - `void dequeue(queue);`
  - `itemType front(queue);`
  - `bool isEmpty(queue);`

Queue Operations



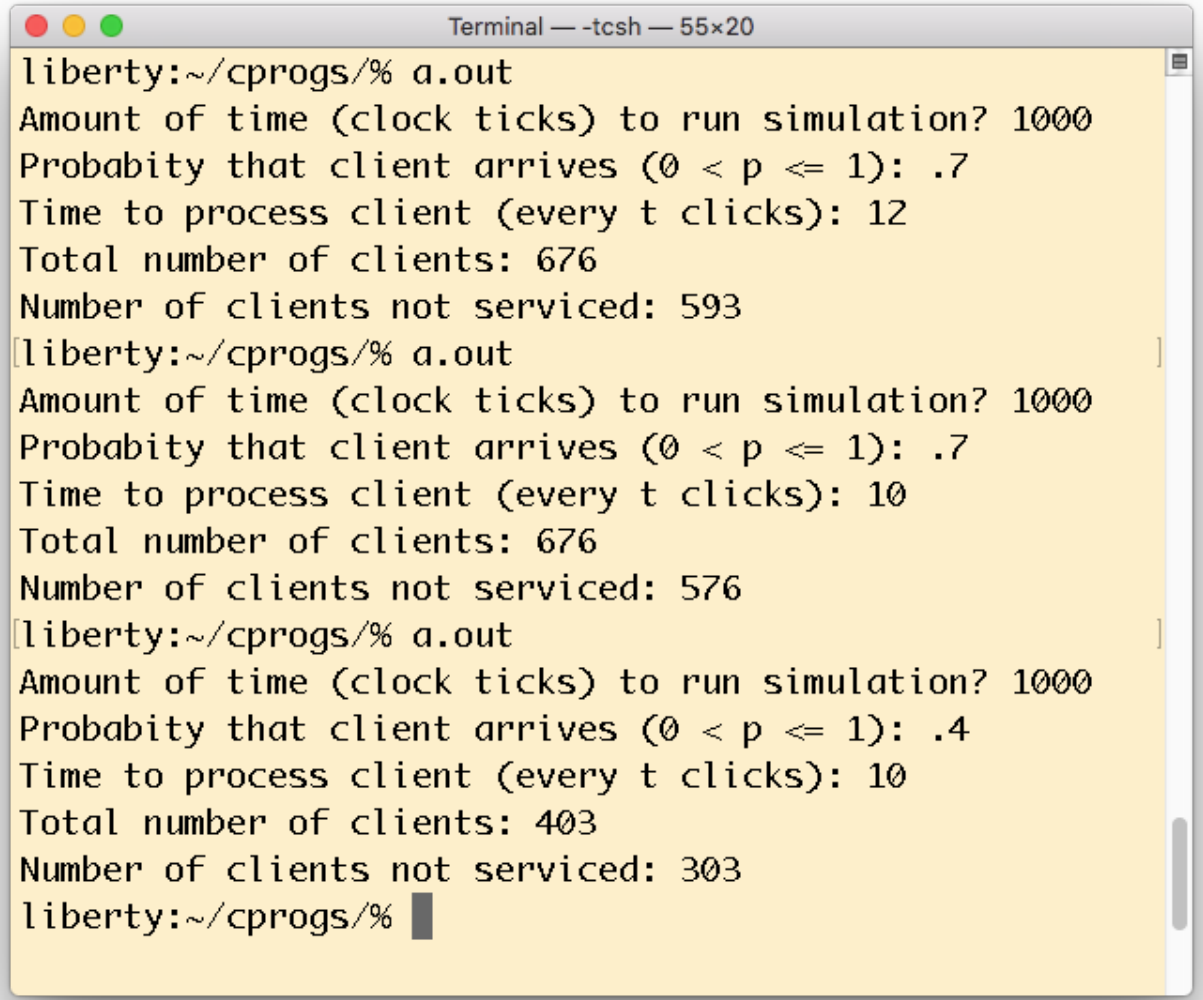
- Common operations on data type queue:
  - `void enqueue(queue,itemType);`
  - `void dequeue(queue);`
  - `itemType front(queue);`
  - `bool isEmpty(queue);`

Applications of Queues

- Queues are used whenever you have a situation for something must “wait it’s turn”
- Usually this is some sort of client-server application where items are being served at a slower rate than they arrive wanting to be served
- Common uses include print queues, CPU job queues, keyboard buffer queues, etc.



An Application Using Queues: Client–Server Simulation



An Application Using Queues: Client–Server Simulation *continued*

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "itemType.h"
#include "queue.h"

#define CLIENT 0 // arbitrary int

int main(void){
  queue q = createQueue();
  float prob;
  int totalClicks,seed,processTime,totalClients = 0;
  //seed = time(NULL);
  seed = 9999;
  srand(seed);
```



An Application Using Queues: Client–Server Simulation *continued*

```
  printf("Amount of time (clock ticks) to run simulation? ");
  scanf("%d",&totalClicks);
  printf("Probabity that client arrives (0 < p <= 1): ");
  scanf("%f",&prob);
  printf("Time to process client (every t clicks): ");
  scanf("%d",&processTime);
  for (int clock=1; clock <= totalClicks; clock++) {
    if(clock % processTime == 0) // time to move job out of queue
      if(!isEmpty(q)) dequeue(q);
    float randNum = rand()/(float)RAND_MAX;
    if (randNum <= prob) { enqueue(q,CLIENT); totalClients++; }
  }
  printf("Total number of clients: %d\n",totalClients);
  printf("Number of clients not serviced: %d\n",size(q));
}
```



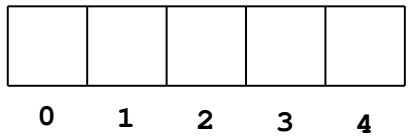
Implementing a Queue Using Arrays

- Design attempt #1 - keep front anchored at 0:

Implementing a Queue Using Arrays

- Design attempt #1 - keep front anchored at 0:

```
q = createQueue()
```

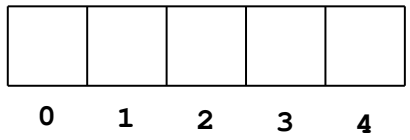



```
front: 0
back: -1
```

Implementing a Queue Using Arrays

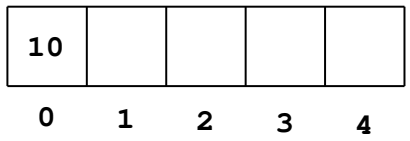
- Design attempt #1 - keep front anchored at 0:

```
q = createQueue()
```




```
front: 0
back: -1
```

```
enqueue(q,10)
```

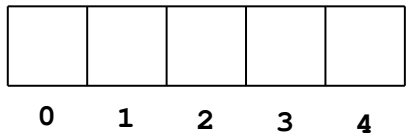



```
front: 0
back: 0
```

Implementing a Queue Using Arrays

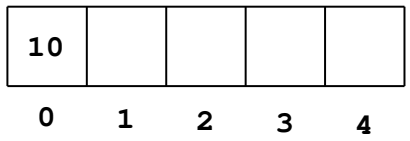
- Design attempt #1 - keep front anchored at 0:

```
q = createQueue()
```



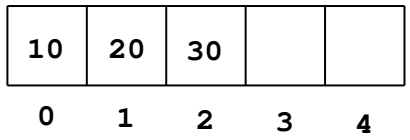

```
front: 0
back: -1
```

```
enqueue(q,10)
```




```
front: 0
back: 0
```

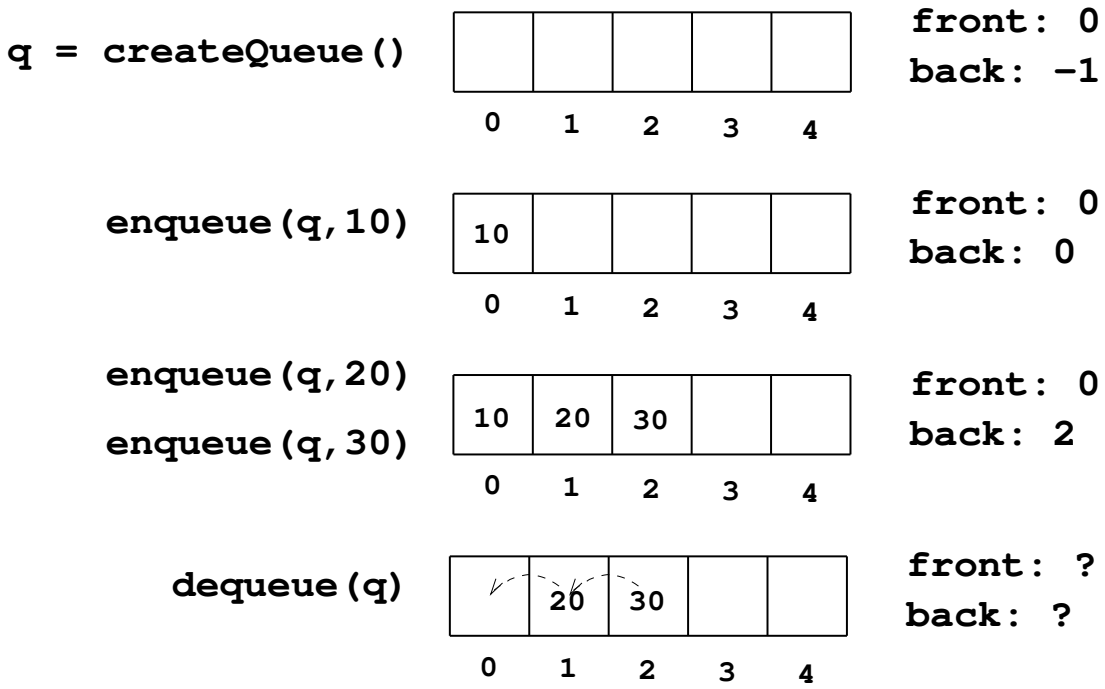
```
enqueue(q,20)
enqueue(q,30)
```




```
front: 0
back: 2
```

Implementing a Queue Using Arrays

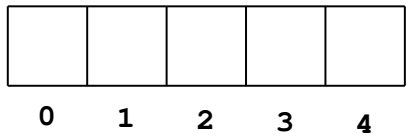
- Design attempt #1 - keep front anchored at 0:



Implementing a Queue Using Arrays

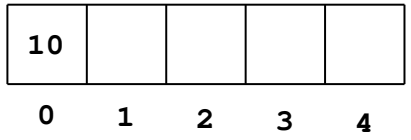
- Design attempt #1 - keep front anchored at 0:

```
q = createQueue()
```



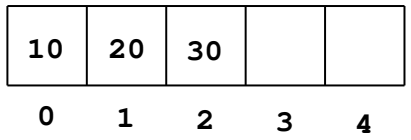

```
front: 0
back: -1
```

```
enqueue(q,10)
```



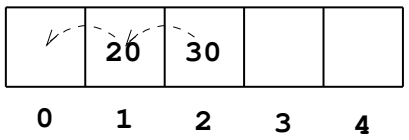

```
front: 0
back: 0
```

```
enqueue(q,20)
enqueue(q,30)
```

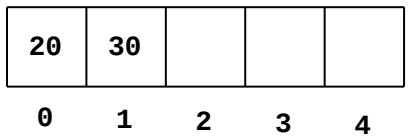



```
front: 0
back: 2
```

```
dequeue(q)
```



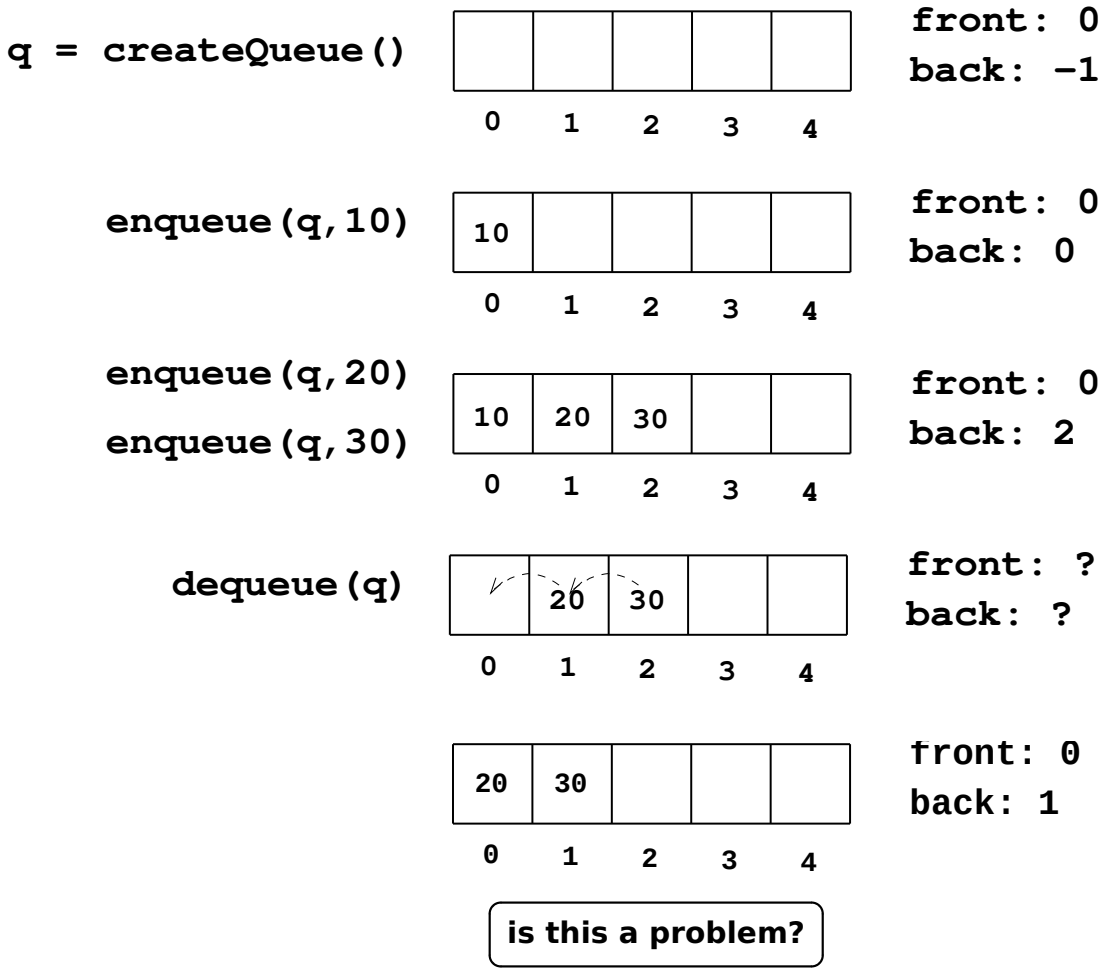

```
front: ?
back: ?
```




```
front: 0
back: 1
```

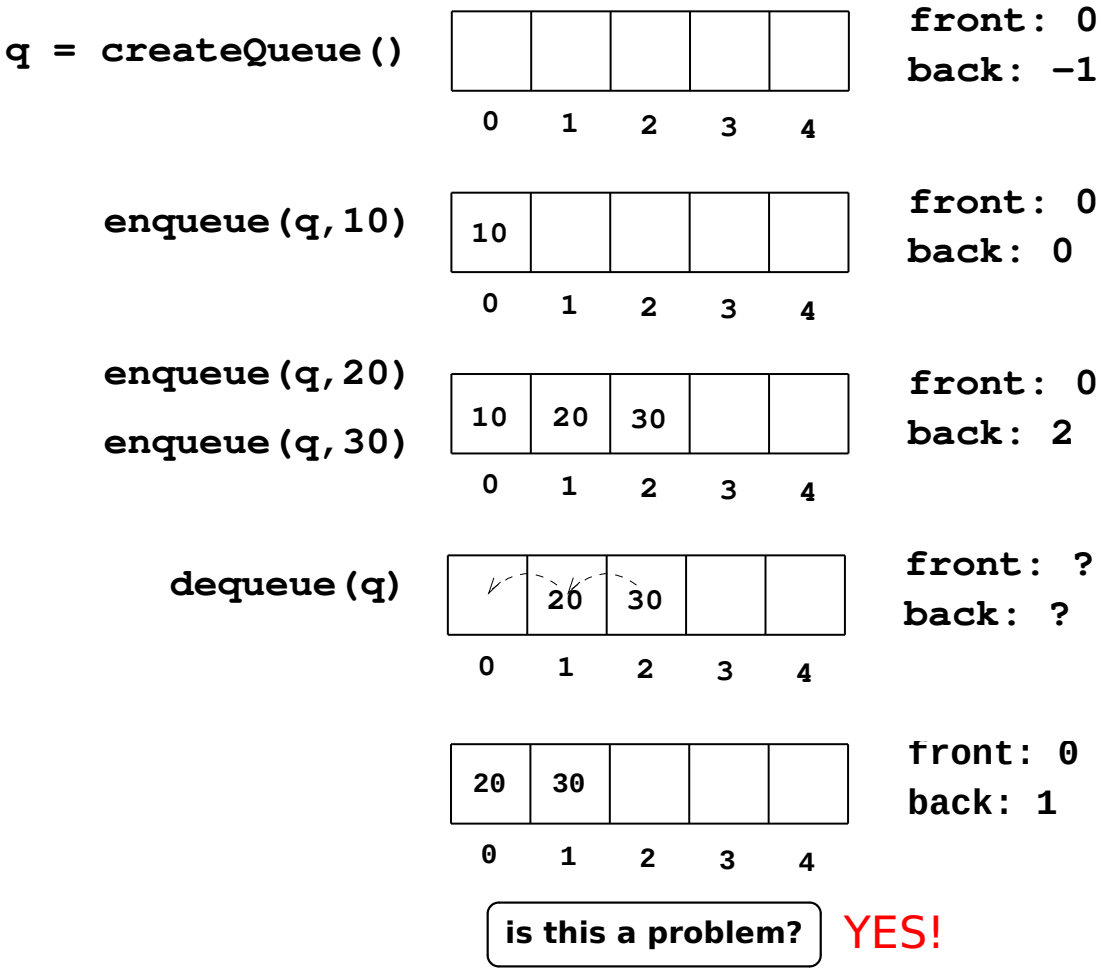
Implementing a Queue Using Arrays

- Design attempt #1 - keep front anchored at 0:



Implementing a Queue Using Arrays

- Design attempt #1 - keep front anchored at 0:

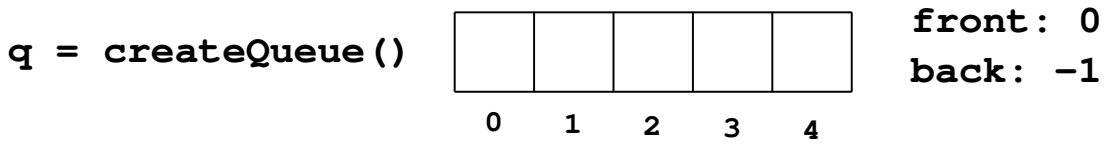


Implementing a Queue Using Arrays *continued*

- Design attempt #2 - do not anchor the front:

Implementing a Queue Using Arrays *continued*

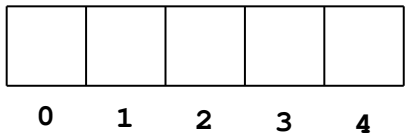
- Design attempt #2 - do not anchor the front:



Implementing a Queue Using Arrays *continued*

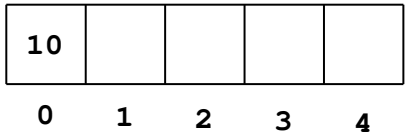
- Design attempt #2 - do not anchor the front:

```
q = createQueue()
```




```
front: 0
back: -1
```

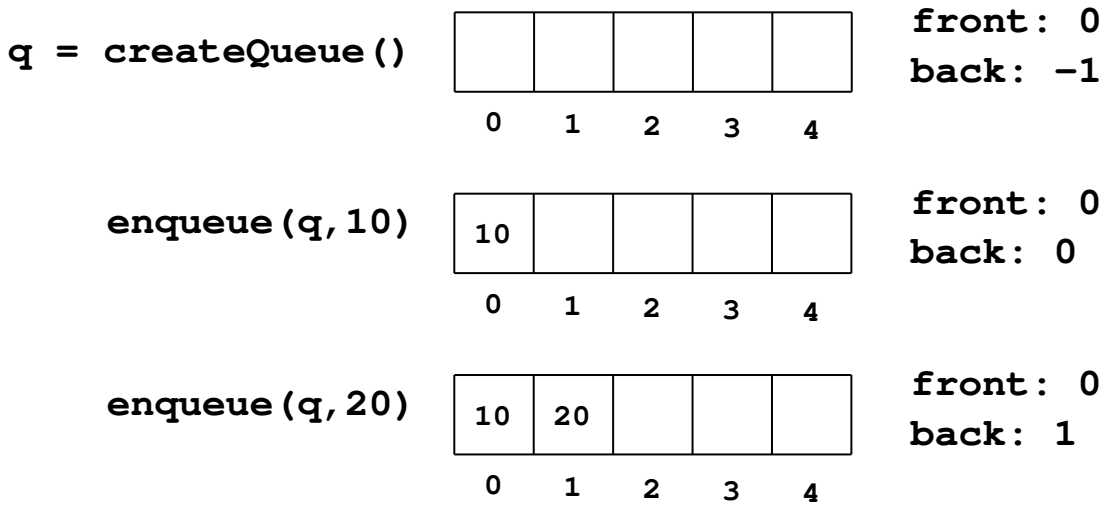
```
enqueue(q,10)
```




```
front: 0
back: 0
```

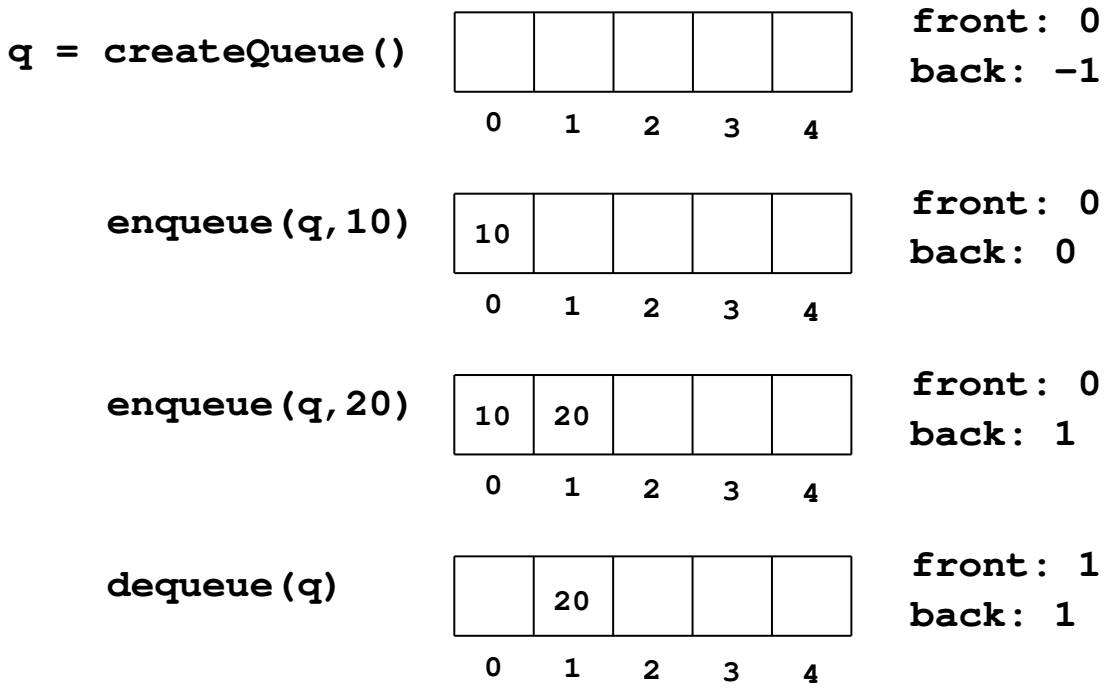
Implementing a Queue Using Arrays *continued*

- Design attempt #2 - do not anchor the front:



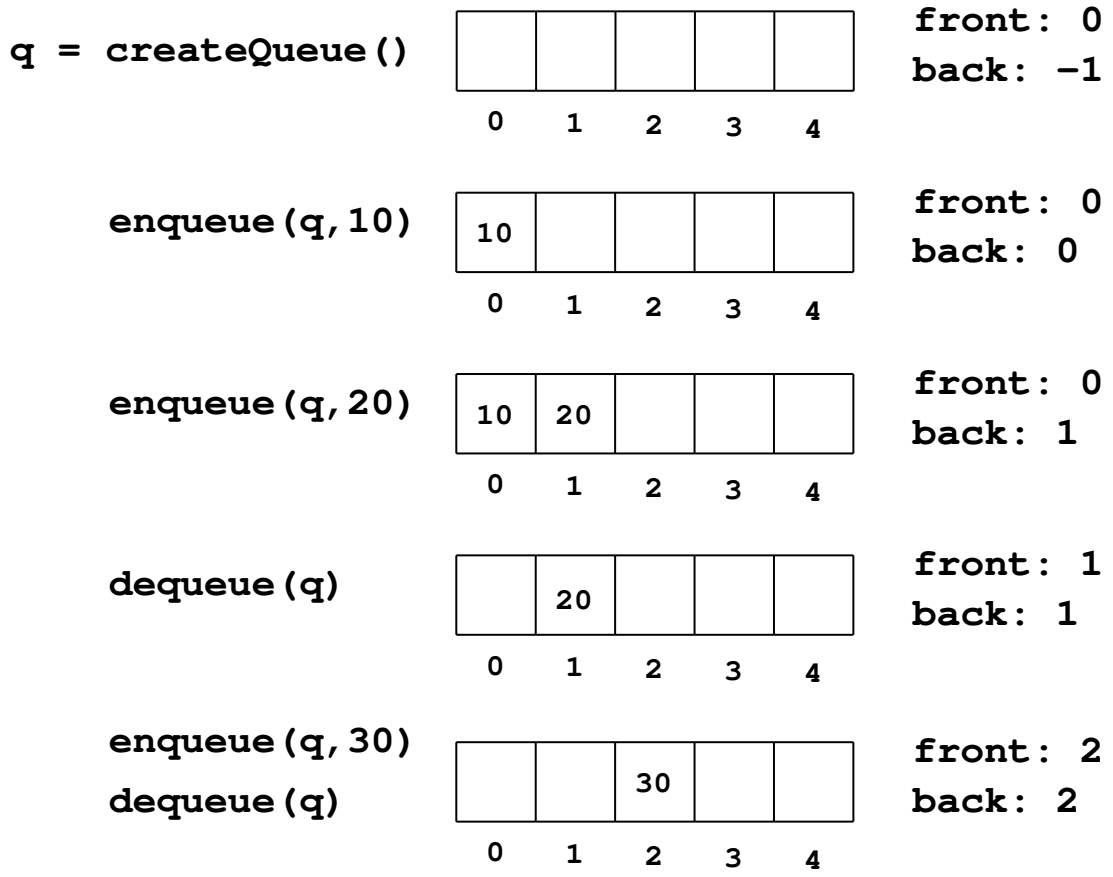
Implementing a Queue Using Arrays *continued*

- Design attempt #2 - do not anchor the front:



Implementing a Queue Using Arrays *continued*

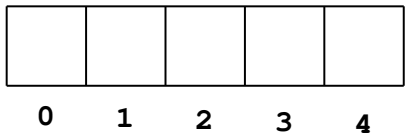
- Design attempt #2 - do not anchor the front:



Implementing a Queue Using Arrays *continued*

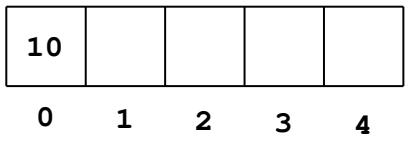
- Design attempt #2 - do not anchor the front:

```
q = createQueue()
```



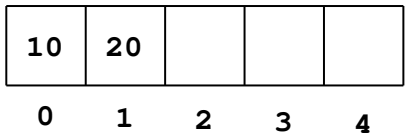

```
front: 0
back: -1
```

```
enqueue(q,10)
```



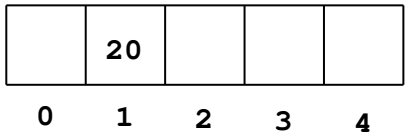

```
front: 0
back: 0
```

```
enqueue(q,20)
```



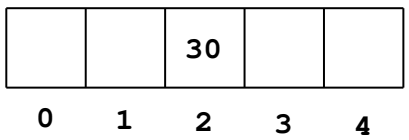

```
front: 0
back: 1
```

```
dequeue(q)
```




```
front: 1
back: 1
```

```
enqueue(q,30)
dequeue(q)
```




```
front: 2
back: 2
```

still a problem!

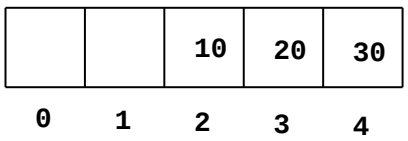
Implementing a Queue Using Arrays *continued*

- Design attempt #3 - wrap the queue around in the array:

- Referred to as a circular array implementation

Implementing a Queue Using Arrays *continued*

- Design attempt #3 - wrap the queue around in the array:

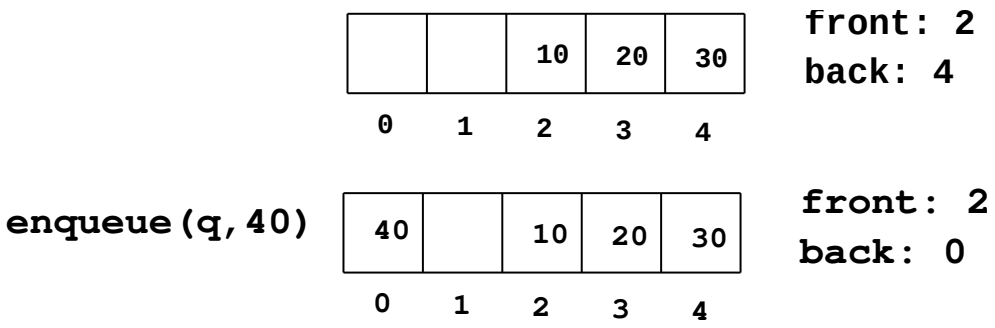


front: 2
back: 4

- Referred to as a circular array implementation

Implementing a Queue Using Arrays *continued*

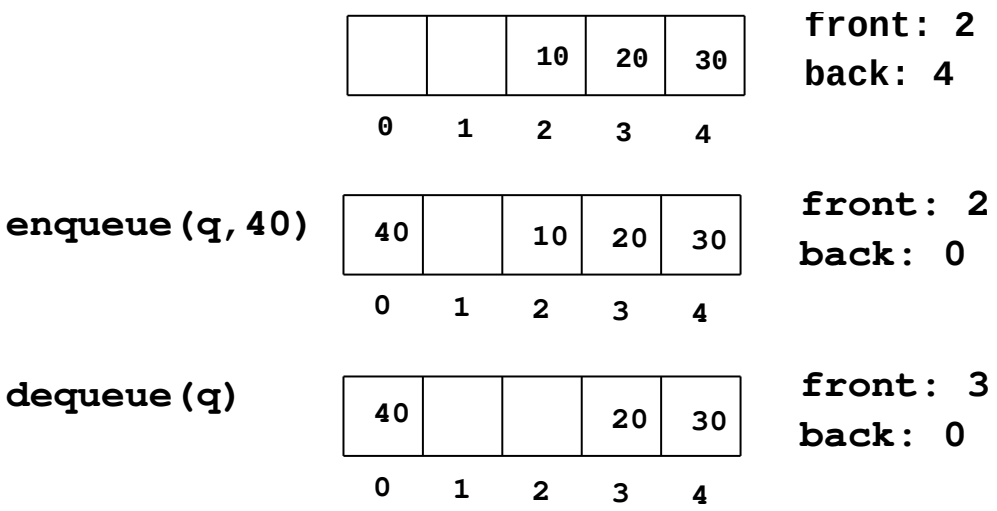
- Design attempt #3 - wrap the queue around in the array:



- Referred to as a circular array implementation

Implementing a Queue Using Arrays *continued*

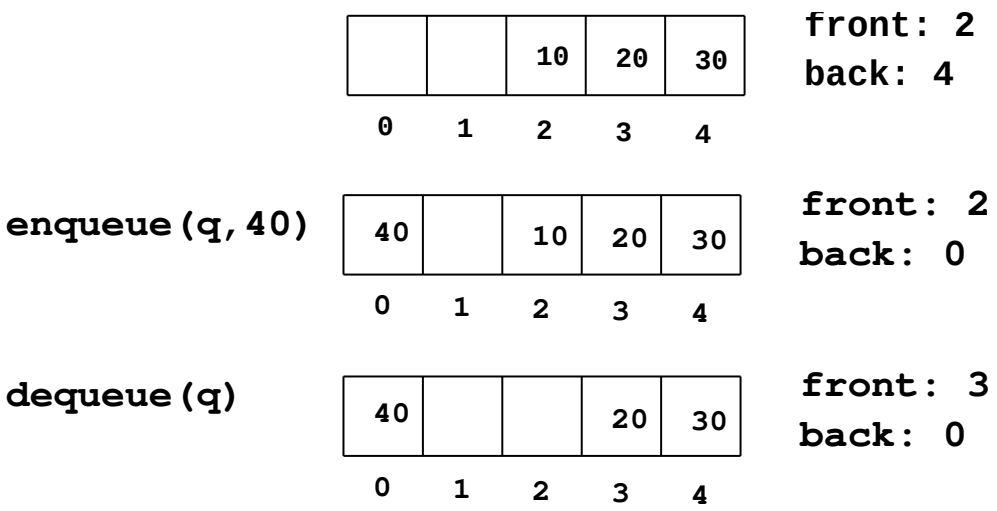
- Design attempt #3 - wrap the queue around in the array:



- Referred to as a circular array implementation

Implementing a Queue Using Arrays *continued*

- Design attempt #3 - wrap the queue around in the array:



- Referred to as a circular array implementation

Implementing a Queue Using Arrays *continued*

- File `queue.h`:

```
#include <stdbool.h>
#include "itemType.h"
#ifndef QUEUE_H
#define QUEUE_H

typedef struct queueType *queue;

queue createQueue();
void freeQueue(queue);
void enqueue(queue,itemType);
void dequeue(queue);
itemType front(queue);
bool isEmpty(queue);
bool isFull(queue);
int size(queue);
#endif
```



Implementing a Queue Using Arrays *continued*

- File `queue.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include "queue.h"
#include "itemType.h"

#define CHUNKSIZE 20

struct queueType {
  itemType *data;
  int front, back,
      count, // # of items in queue
      size;  // size of the data array
};

static int inc(int n,queue q) { return ++n % q->size; }
```



Implementing a Queue Using Arrays *continued*

```
static void expandQ(queue q) {
  itemType *t = malloc(sizeof(itemType)*(q->size+CHUNKSIZE));
  for (int i = 0; i < q->size; i++) t[i] = q->data[i];
  q->size += CHUNKSIZE;
  q->data = t;
}

queue createQueue() {
  queue q = malloc(sizeof(struct queueType));
  q->data = malloc(sizeof(itemType)*CHUNKSIZE);
  q->front = 0; q->back = -1;
  q->size = CHUNKSIZE; q->count = 0;
  return q;
}

void freeQueue(queue q) { free(q->data); free(q); }
```



Implementing a Queue Using Arrays *continued*

```
void enqueue(queue q,itemType p) {
  if(isFull(q)) expandQ(q);
  q->data[q->back=inc(q->back,q)] = p;
  q->count++;
}

void dequeue(queue q){
  if(isEmpty(q)){
    printf("Error: queue underflow\n");
    exit(EXIT_FAILURE);
  } else {
    q->front = inc(q->front,q);
    q->count--;
  }
}
```



Implementing a Queue Using Arrays *continued*

```
itemType front(queue q) {
  if(isEmpty(q)){
    printf("Error: queue underflow\n");
    exit(EXIT_FAILURE);
  } else {
    return q->data[q->front];
  }
}

bool isEmpty(queue q) { return q->count == 0; }
bool isFull(queue q) { return q->count == q->size; }
int size(queue q) { return q->count; }
```



Implementing a Queue as a Linked List

- File queueLL.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include "queue.h"
#include "itemType.h"

struct node {
  itemType data;
  struct node *next;
};

struct queueType {
  struct node *front, *back;
  int count;
};
```



Implementing a Queue as a Linked List *continued*

```
queue createQueue() {
  queue q = malloc(sizeof(struct queueType));
  q->front = q->back = NULL; q->count = 0;
  return q;
}

static void freeNodes(struct node *p) {
  if(p==NULL)
    return;
  else {
    freeNodes(p->next);
    free(p);
  }
}

void freeQueue(queue q) { freeNodes(q->front); free(q); }
```



Implementing a Queue as a Linked List *continued*

```
void enqueue(queue q,itemType itm) {
  if(isFull(q)) {
    printf("Error: queue overflow\n");
    exit(EXIT_FAILURE);
  } else {
    struct node *tp = malloc(sizeof(struct node));
    tp->data = itm; tp->next = NULL;
    if(isEmpty(q)) {
     q->front = q->back = tp;
    } else {
      q->back = q->back->next = tp;
    }
    q->count++;
  }
}
```



Implementing a Queue as a Linked List *continued*

```
void dequeue(queue q){
  if(isEmpty(q)){
    printf("Error: queue underflow\n");
    exit(EXIT_FAILURE);
  } else {
    struct node *tp = q->front;
    q->front = (q->count==1) ? (q->back = NULL) : q->front->next;
    free(tp);
    q->count--;
  }
}

itemType front(queue q) {
  if(isEmpty(q)){
    printf("Error: queue underflow\n");
    exit(EXIT_FAILURE);
  } else return q->front->data;
}
```



Implementing a Queue as a Linked List *continued*

```
bool isEmpty(queue q) { return q->front==NULL; }
bool isFull(queue q) { return false; }

int size(queue q) { return q->count; }
```



Generic Programming

- Note that the behavior of most container modules is exactly the same no matter what type of values they hold
- It seems like it would be convenient if there was a way to write only a single version of a container for all types!
- **Generic programming** is a style of programming where a program or module's data type(s) are set or determined when the program runs instead of at compile time
- Most modern languages have the capability of doing this; however, in C you must simulate it by using void pointers
- A **void pointer** is a pointer that has no associated data type with it
- A void pointer can both the hold address of and can be typecasted to to a pointer of any type



Generic Programming

- Note that the behavior of most container modules is exactly the same no matter what type of values they hold
- It seems like it would be convenient if there was a way to write only a single version of a container for all types!
- **Generic programming** is a style of programming where a program or module's data type(s) are set or determined when the program runs instead of at compile time
- Most modern languages have the capability of doing this; however, in C you must simulate it by using void pointers
- A **void pointer** is a pointer that has no associated data type with it
- A void pointer can both the hold address of and can be typecasted to to a pointer of any type



Generic Programming

- Note that the behavior of most container modules is exactly the same no matter what type of values they hold
- It seems like it would be convenient if there was a way to write only a single version of a container for all types!
- **Generic programming** is a style of programming where a program or module's data type(s) are set or determined when the program runs instead of at compile time
- Most modern languages have the capability of doing this; however, in C you must simulate it by using void pointers
- A **void pointer** is a pointer that has no associated data type with it
- A void pointer can both the hold address of and can be typecasted to to a pointer of any type



Generic Programming

- Note that the behavior of most container modules is exactly the same no matter what type of values they hold
- It seems like it would be convenient if there was a way to write only a single version of a container for all types!
- **Generic programming** is a style of programming where a program or module's data type(s) are set or determined when the program runs instead of at compile time
- Most modern languages have the capability of doing this; however, in C you must simulate it by using void pointers
- A **void pointer** is a pointer that has no associated data type with it
- A void pointer can both the hold address of and can be typecasted to to a pointer of any type



Generic Programming

- Note that the behavior of most container modules is exactly the same no matter what type of values they hold
- It seems like it would be convenient if there was a way to write only a single version of a container for all types!
- **Generic programming** is a style of programming where a program or module's data type(s) are set or determined when the program runs instead of at compile time
- Most modern languages have the capability of doing this; however, in C you must simulate it by using void pointers
- A **void pointer** is a pointer that has no associated data type with it
- A void pointer can both the hold address of and can be typecasted to to a pointer of any type



Generic Programming

- Note that the behavior of most container modules is exactly the same no matter what type of values they hold
- It seems like it would be convenient if there was a way to write only a single version of a container for all types!
- **Generic programming** is a style of programming where a program or module's data type(s) are set or determined when the program runs instead of at compile time
- Most modern languages have the capability of doing this; however, in C you must simulate it by using void pointers
- A **void pointer** is a pointer that has no associated data type with it
- A void pointer can both the hold address of and can be typecasted to to a pointer of any type



Generic Programming *continued*

```
#include <stdbool.h>

#ifndef STACK_H
#define STACK_H

typedef struct stackType *stack;

stack createStack();
void freeStack(stack);
void push(stack,void *);
void pop(stack);
void *top(stack);
bool isEmpty(stack);
bool isFull(stack);

#endif
```



Generic Programming *continued*

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include "stackGENERIC.h"

#define STACKSIZE 10

struct stackType {
  void *data[STACKSIZE];
  int top;
};

stack createStack() {
  stack s = malloc(sizeof(struct stackType));
  s->top = -1;
  return s;
}

void freeStack(stack s) { free(s); }

void push(stack s,void *p) {
  if(isFull(s)) {
    printf("Error:  stack overflow\n");
    exit(EXIT_FAILURE);
  } else s->data[++s->top] = p;
}
```

```
void pop(stack s){
  if(isEmpty(s)){
    printf("Error:  stack underflow\n");
    exit(EXIT_FAILURE);
  } else s->top--;
}

void *top(stack s) {
  if(isEmpty(s)){
    printf("Error:  stack underflow\n");
    exit(EXIT_FAILURE);
  } else {
    return s->data[s->top];
  }
}

bool isEmpty(stack s) { return s->top == -1; }
bool isFull(stack s) {
  return s->top == STACKSIZE-1;
}
```



Generic Programming *continued*

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include "stackGENERIC.h"

#define STACKSIZE 10

struct stackType {
  void *data[STACKSIZE];
  int top;
};

stack createStack() {
  stack s = malloc(sizeof(struct stackType));
  s->top = -1;
  return s;
}

void freeStack(stack s) { free(s); }

void push(stack s,void *p) {
  if(isFull(s)) {
    printf("Error:  stack overflow\n");
    exit(EXIT_FAILURE);
  } else s->data[++s->top] = p;
}
```

```
void pop(stack s){
  if(isEmpty(s)){
    printf("Error:  stack underflow\n");
    exit(EXIT_FAILURE);
  } else s->top--;
}

void *top(stack s) {
  if(isEmpty(s)){
    printf("Error:  stack underflow\n");
    exit(EXIT_FAILURE);
  } else {
    return s->data[s->top];
  }
}

bool isEmpty(stack s) { return s->top == -1; }
bool isFull(stack s) {
  return s->top == STACKSIZE-1;
}
```

How would we use this?



Generic Programming *continued*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "stackGENERIC.h"

int main(void) {
  int l1[] = {1,2,3};
  float l2[] = {10.0,20.0,30.0};
  char *str = "how now brown cow";

  stack s = createStack();
  for(int i=0; i < 3; i++) {
    push(s,&l1[i]); push(s,&l2[i]);
  }
  push(s,str);

  printf("%s\n",(char*)top(s)); pop(s);
  while(!isEmpty(s)) {
    printf("%.2f \n",*(float*)top(s)); pop(s);
    printf("%d \n",*(int*)top(s));     pop(s);
  }
}
```



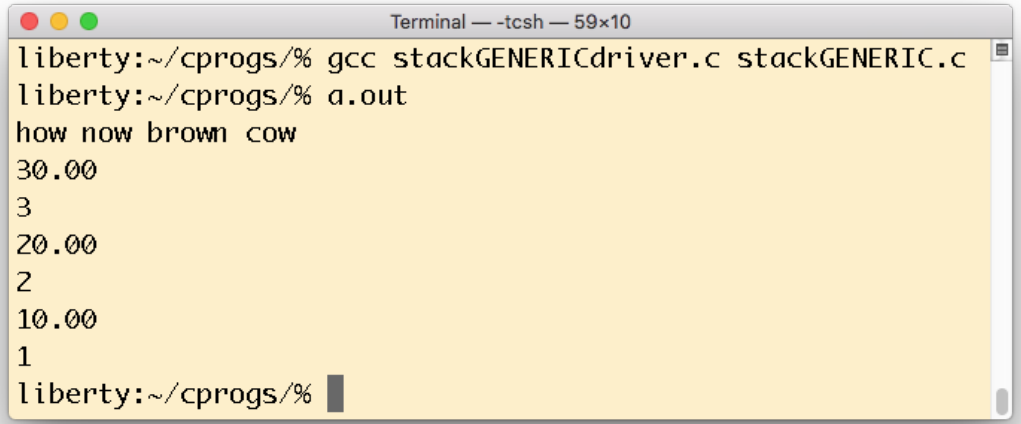
Generic Programming *continued*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "stackGENERIC.h"

int main(void) {
  int l1[] = {1,2,3};
  float l2[] = {10.0,20.0,30.0};
  char *str = "how now brown cow";

  stack s = createStack();
  for(int i=0; i < 3; i++) {
    push(s,&l1[i]); push(s,&l2[i]);
  }
  push(s,str);

  printf("%s\n",(char*)top(s)); pop(s);
  while(!isEmpty(s)) {
    printf("%.2f \n",*(float*)top(s)); pop(s);
    printf("%d \n",*(int*)top(s));     pop(s);
  }
}
```



Rationale for Trees

- Searching for an item in a list is one of the most common activities performed by computers around the world
- Therefore it is important to do it in an efficient manner
- Let's consider two methods: **sequential search** and **binary search**



Rationale for Trees

- Searching for an item in a list is one of the most common activities performed by computers around the world
- Therefore it is important to do it in an efficient manner
- Let's consider two methods: **sequential search** and **binary search**

Rationale for Trees

- Searching for an item in a list is one of the most common activities performed by computers around the world
- Therefore it is important to do it in an efficient manner
- Let's consider two methods: **sequential search** and **binary search**



Rationale for Trees *continued*

- How can we determine if one algorithm is more efficient (faster) than another?
- Running each while timing their performance is an inaccurate method; why?
- Computer scientists have determined that there is a correlation between the run-time efficiency of an algorithm and the number of compares it performs to accomplish a task
- Therefore let's consider the number of compares required to search for an item in a list for two different algorithms



Rationale for Trees *continued*

- How can we determine if one algorithm is more efficient (faster) than another?
- Running each while timing their performance is an inaccurate method; why?
- Computer scientists have determined that there is a correlation between the run-time efficiency of an algorithm and the number of compares it performs to accomplish a task
- Therefore let's consider the number of compares required to search for an item in a list for two different algorithms



Rationale for Trees *continued*

- How can we determine if one algorithm is more efficient (faster) than another?
- Running each while timing their performance is an inaccurate method; why?
- Computer scientists have determined that there is a correlation between the run-time efficiency of an algorithm and the number of compares it performs to accomplish a task
- Therefore let's consider the number of compares required to search for an item in a list for two different algorithms



Rationale for Trees *continued*

- How can we determine if one algorithm is more efficient (faster) than another?
- Running each while timing their performance is an inaccurate method; why?
- Computer scientists have determined that there is a correlation between the run-time efficiency of an algorithm and the number of compares it performs to accomplish a task
- Therefore let's consider the number of compares required to search for an item in a list for two different algorithms



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:

Case	Best Case	Worst Case	Average Case



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:

Case	Best Case	Worst Case	Average Case
item is present			



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:

Case	Best Case	Worst Case	Average Case
item is present	1		



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:

Case	Best Case	Worst Case	Average Case
item is present	1	n	



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:

Case	**Best Case**	**Worst Case**	**Average Case**
item is present	1	n	**???**



Rationale for Trees *continued*

- Consider a list of n integers
- Searching for a **key** in a list with sequential search involves looking at each item in turn comparing it to the key until we find the value or run out of list to search
- It will take 1 compare if the key is located at the head of the list, 5 compares to find it if it's in the list at position 5, and n compares to determine it's not in the list
- Therefore sequential search requires the following number of compares:

Case	**Best Case**	**Worst Case**	**Average Case**
item is present	1	n	**???**
item is not present	n	n	n



Rationale for Trees *continued*

- One possible method to find the average case is to determine how many comparisons needed to search for *all* entries in the list and then divide this amount by n

- Therefore for sequential search:

Case	Best Case	Worst Case	Average Case
item is present	1	n	$\frac{1}{2}(n+1)$
item is not present	n	n	n



Rationale for Trees *continued*

- One possible method to find the average case is to determine how many comparisons needed to search for *all* entries in the list and then divide this amount by n, i.e.

$$\frac{1 + 2 + 3 + \cdots + n}{n}$$

- Therefore for sequential search:

Case	Best Case	Worst Case	Average Case
item is present	1	n	$\frac{1}{2}(n+1)$
item is not present	n	n	n



Rationale for Trees *continued*

- One possible method to find the average case is to determine how many comparisons needed to search for *all* entries in the list and then divide this amount by n, i.e.

$$\frac{1+2+3+\cdots+n}{n}$$

And since $1+2+3+\cdots+n = \frac{1}{2}n(n+1)$, we can say that the average number of key comparisons in a successful sequential search is

$$\frac{1}{n}\left(\frac{n(n+1)}{2}\right) = \frac{1}{2}(n+1)$$

- Therefore for sequential search:

Case	Best Case	Worst Case	Average Case
item is present	1	n	$\frac{1}{2}(n+1)$
item is not present	n	n	n

Rationale for Trees *continued*

- One possible method to find the average case is to determine how many comparisons needed to search for *all* entries in the list and then divide this amount by n, i.e.

$$\frac{1+2+3+\cdots+n}{n}$$

And since $1+2+3+\cdots+n = \frac{1}{2}n(n+1)$, we can say that the average number of key comparisons in a successful sequential search is

$$\frac{1}{n}\left(\frac{n(n+1)}{2}\right) = \frac{1}{2}(n+1)$$

- Therefore for sequential search:

Case	**Best Case**	**Worst Case**	**Average Case**
item is present	1	n	$\frac{1}{2}(n+1)$
item is not present	n	n	n



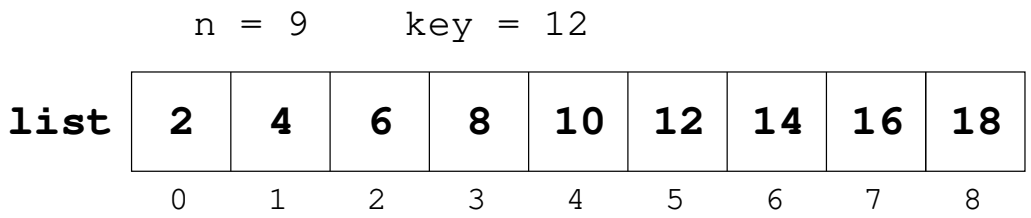
Rationale for Trees *continued*

- What is **binary search**?



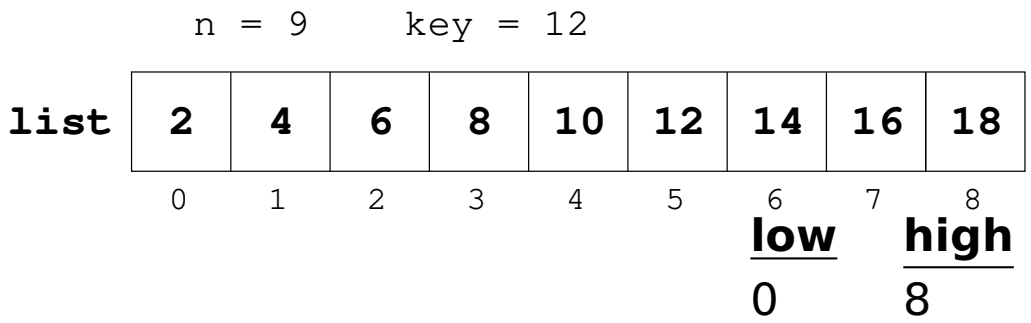
Rationale for Trees *continued*

- What is **binary search**?



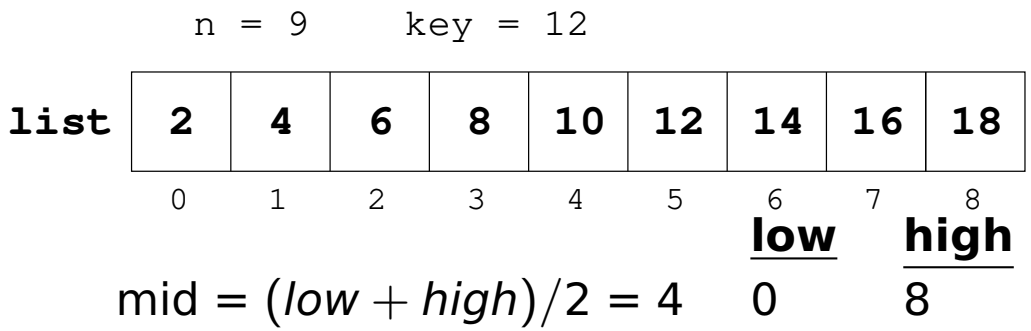
Rationale for Trees *continued*

- What is **binary search**?



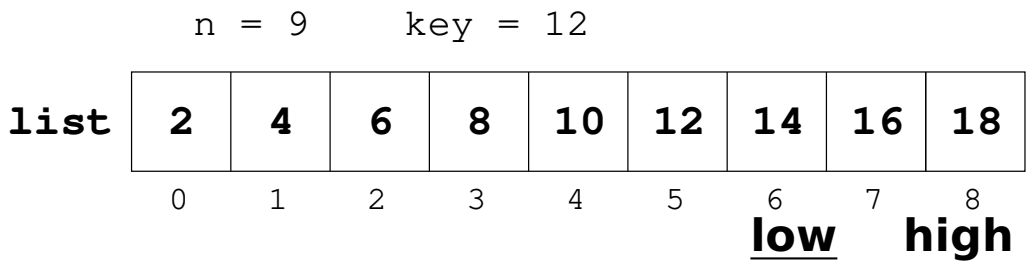
Rationale for Trees *continued*

- What is **binary search**?



Rationale for Trees *continued*

- What is **binary search**?



Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2 = 4$	0	8

is list[mid] == key? **no**



Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2 = 4$	0	8

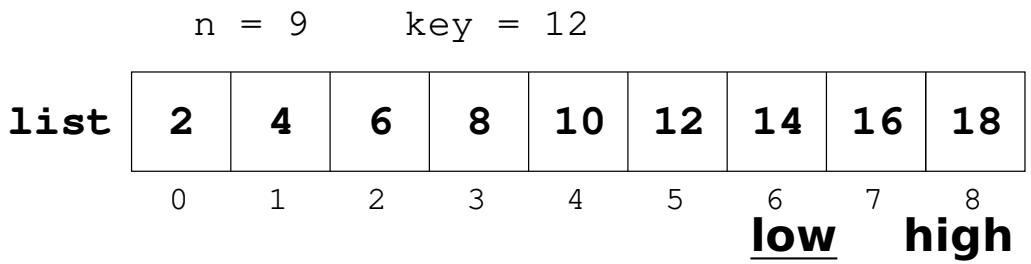
is list[mid] == key? **no**

is list[mid] < key?



Rationale for Trees *continued*

- What is **binary search**?



	low	**high**
mid = $(low + high)/2 = 4$	0	8

is list[mid] == key? **no**

is list[mid] < key? **yes**

Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2 = 4$	0	8

is list[mid] == key? **no**

is list[mid] < key? **yes**

low = mid + 1



Rationale for Trees *continued*

- What is **binary search**?

`n = 9` `key = 12`

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2$ = 4	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8

Rationale for Trees *continued*

- What is **binary search**?

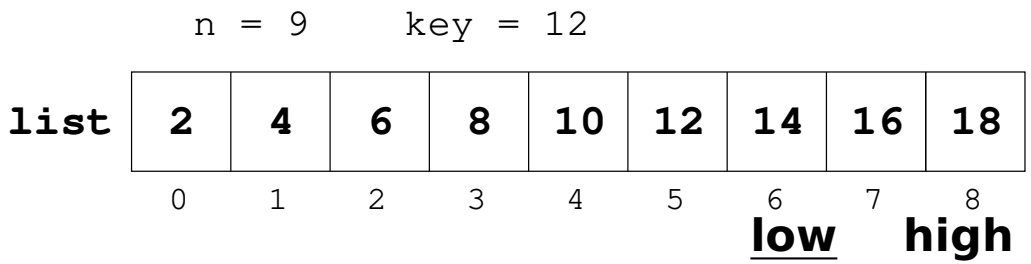
`n = 9` `key = 12`

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2 = 4$	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2 = 6$		

Rationale for Trees *continued*

- What is **binary search**?



	low	high
mid = $(low + high)/2 = 4$	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2 = 6$		
is list[mid] == key?		

Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2 = 4$	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2 = 6$		
is list[mid] == key? **no**		

Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2$ = 4	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2$ = 6		
is list[mid] == key? **no**		
is list[mid] < key?		

Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**
	0	1	2	3	4	5	6	7	8

	low	**high**
mid = $(low + high)/2$ = 4	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2$ = 6		
is list[mid] == key? **no**		
is list[mid] < key? **no**		

Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2$ = 4	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2$ = 6		
is list[mid] == key? **no**		
is list[mid] < key? **no**		
high = mid - 1		

Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2$ = 4	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2$ = 6		
is list[mid] == key? **no**		
is list[mid] < key? **no**		
high = mid - 1	5	5

Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**
	0	1	2	3	4	5	6	7	8

	low	**high**
mid = $(low + high)/2$ = 4	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2$ = 6		
is list[mid] == key? **no**		
is list[mid] < key? **no**		
high = mid - 1	5	5
mid = $(low + high)/2$ = 5		



Rationale for Trees *continued*

- What is **binary search**?

n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2$ = 4	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2$ = 6		
is list[mid] == key? **no**		
is list[mid] < key? **no**		
high = mid - 1	5	5
mid = $(low + high)/2$ = 5		
is list[mid] == key?		

Rationale for Trees *continued*

- What is **binary search**?

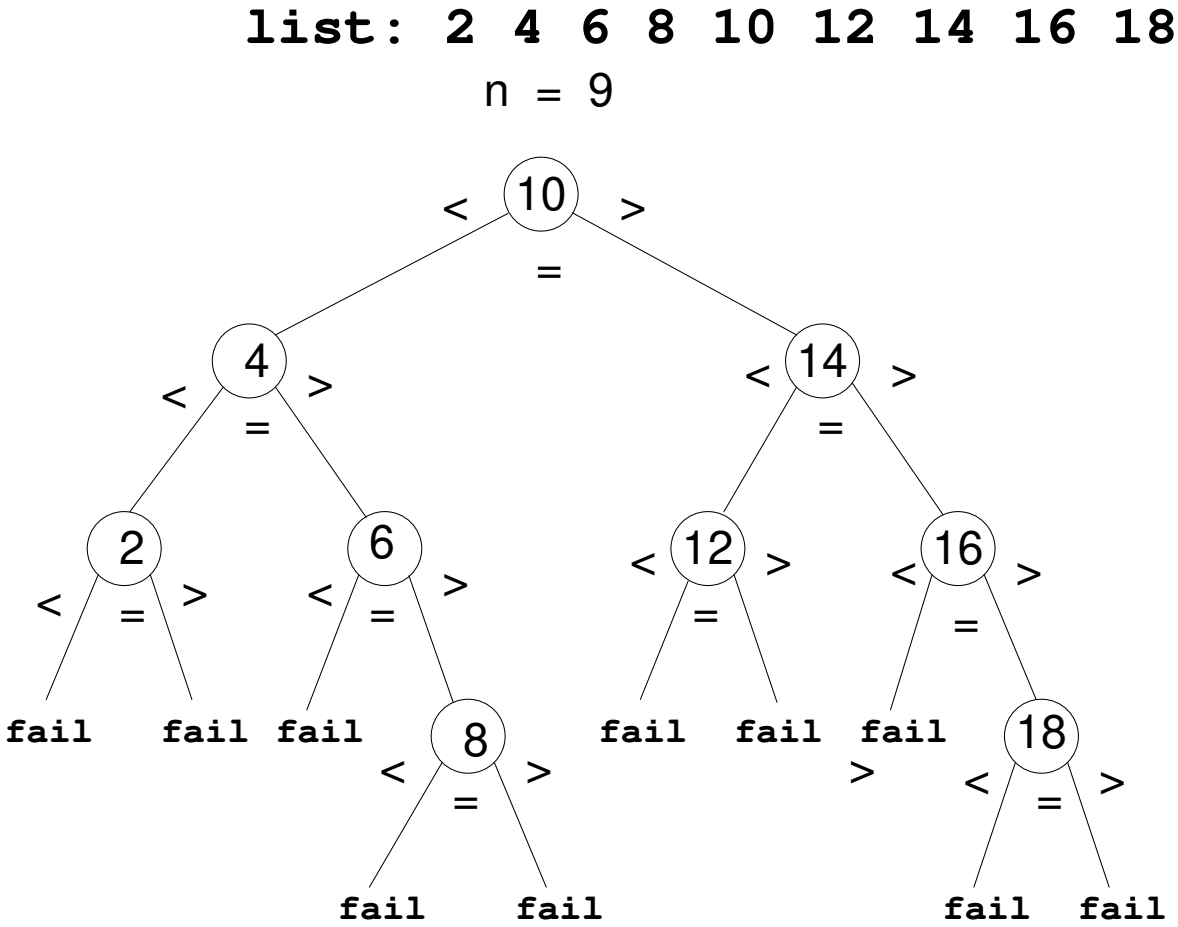
n = 9 key = 12

	0	1	2	3	4	5	6	7	8
list	**2**	**4**	**6**	**8**	**10**	**12**	**14**	**16**	**18**

	low	**high**
mid = $(low + high)/2$ = 4	0	8
is list[mid] == key? **no**		
is list[mid] < key? **yes**		
low = mid + 1	5	8
mid = $(low + high)/2$ = 6		
is list[mid] == key? **no**		
is list[mid] < key? **no**		
high = mid - 1	5	5
mid = $(low + high)/2$ = 5		
is list[mid] == key? **yes!**		

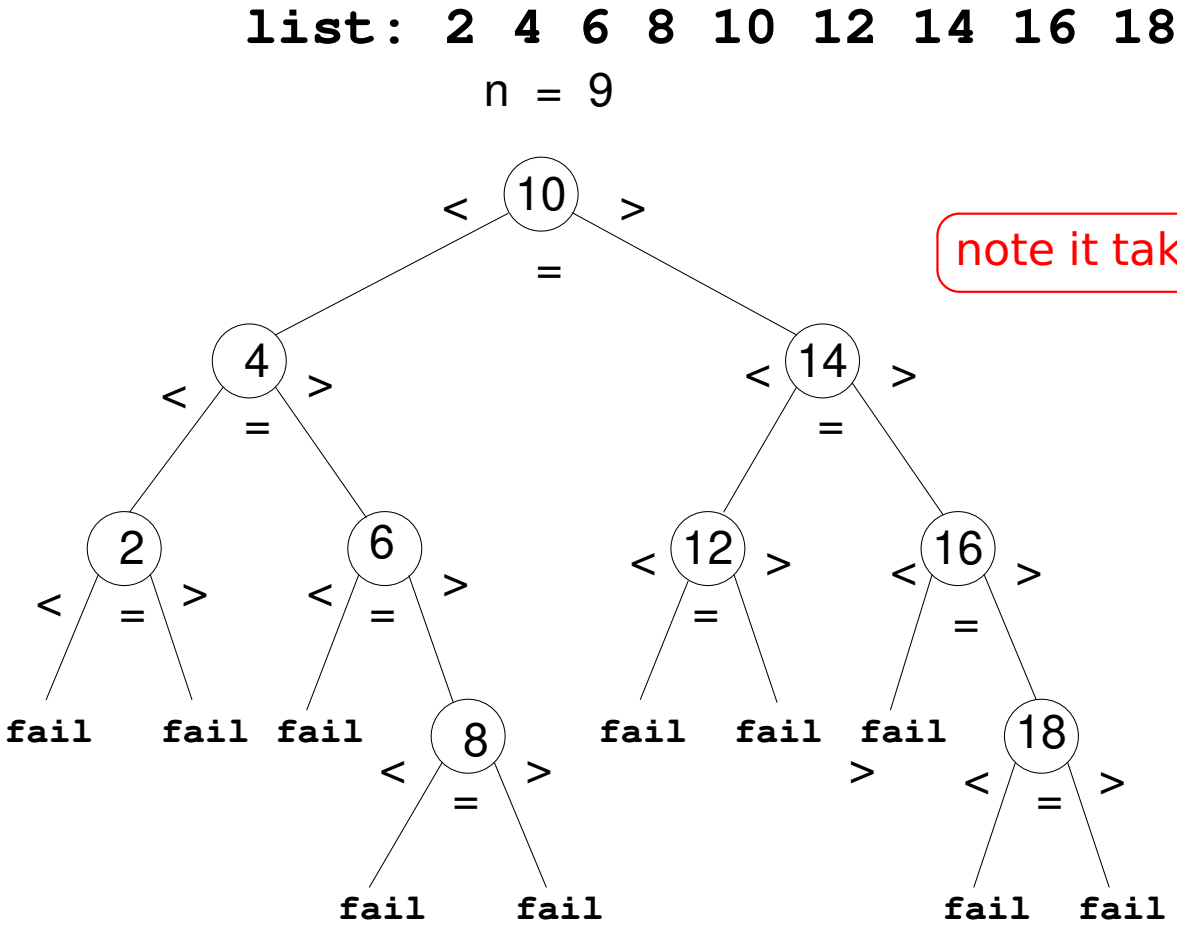
Rationale for Trees *continued*

- Analysis of binary search using a **comparison tree**:



Rationale for Trees *continued*

- Analysis of binary search using a **comparison tree**:

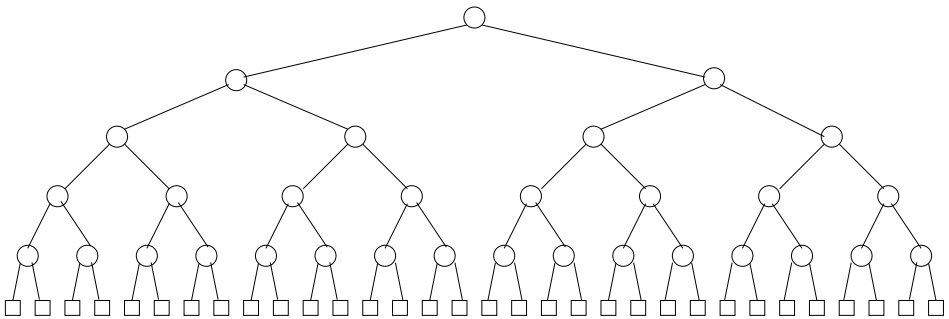


note it takes 2 compares per level

Rationale for Trees *continued*

Binary search analysis continued ...

- Turns out the average and worst case is similar for binary search



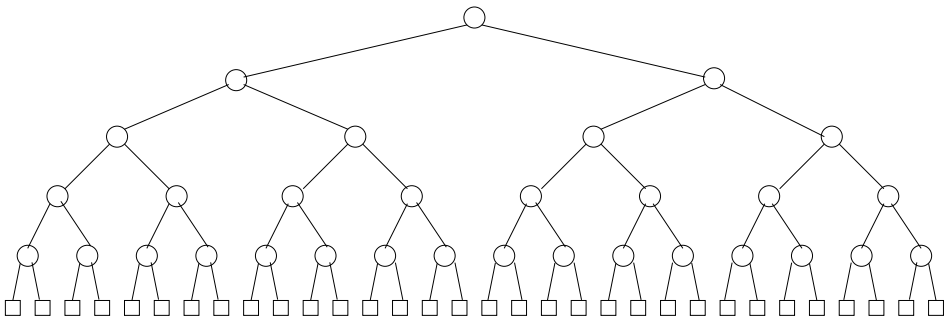
n = 31

- **Worst case**, with 2 comparisons per level: 10 comparisons
- **Average case:** about the same since the number of vertices near the top of the tree is too small in comparison to the number at the bottom to make a significant difference

Rationale for Trees *continued*

Binary search analysis continued ...

- Turns out the average and worst case is similar for binary search



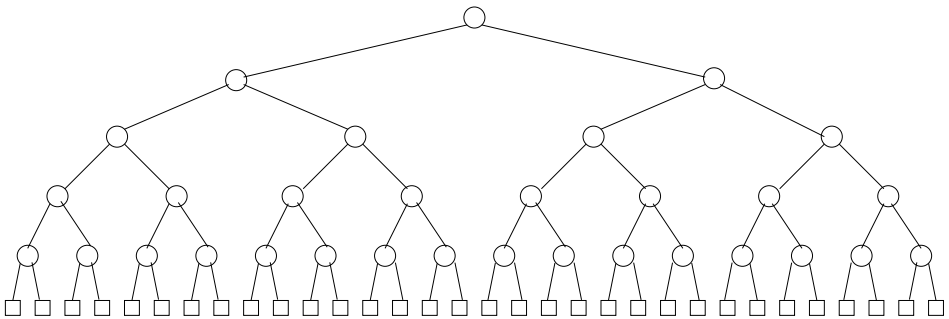
n = 31

- **Worst case**, with 2 comparisons per level: 10 comparisons
- **Average case:** about the same since the number of vertices near the top of the tree is too small in comparison to the number at the bottom to make a significant difference

Rationale for Trees *continued*

Binary search analysis continued ...

- Turns out the average and worst case is similar for binary search



n = 31

- **Worst case**, with 2 comparisons per level: 10 comparisons
- **Average case:** about the same since the number of vertices near the top of the tree is too small in comparison to the number at the bottom to make a significant difference

Rationale for Trees *continued*

Binary search analysis continued ...

- What about the general case? Behavior is similar, at least for large values of n
- Analysis: how many compares will it take for the worst case for a given n?



Rationale for Trees *continued*

Binary search analysis continued ...

- What about the general case? Behavior is similar, at least for large values of n
- Analysis: how many compares will it take for the worst case for a given n? First ask – what is the height of the tree for a given n?



Rationale for Trees *continued*

Binary search analysis continued ...

- What about the general case? Behavior is similar, at least for large values of n
- Analysis: how many compares will it take for the worst case for a given n? First ask – what is the height of the tree for a given n?

height	# of vertices (n)
1	1
2	3
3	7
4	15
5	31
⋮	⋮
k	$2^k - 1$



Rationale for Trees *continued*

Binary search analysis continued ...

- What about the general case? Behavior is similar, at least for large values of n
- Analysis: how many compares will it take for the worst case for a given n? First ask – what is the height of the tree for a given n?

height	# of vertices (n)
1	1
2	3
3	7
4	15
5	31
⋮	⋮
k	$2^k - 1$



Rationale for Trees *continued*

Binary search analysis continued ...

- What about the general case? Behavior is similar, at least for large values of n
- Analysis: how many compares will it take for the worst case for a given n? First ask – what is the height of the tree for a given n?

height	**# of vertices (n)**
1	1
2	3
3	7
4	15
5	31
⋮	⋮
k	$2^k - 1$



Rationale for Trees *continued*

Binary search analysis continued ...

- Now solving for k:

$$\begin{aligned} n &= 2^k - 1 \\ n+1 &= 2^k \end{aligned}$$

take the log of both sides:

$$\mathbf{k = \lg(n+1)}$$

- For large n, the difference between $\lg(n+1)$ and $\lg n$ is insignificant
- Recall that binary search does 2 comparisons per vertex, therefore binary search makes $2 \lg n$ comparisons of keys in searching a list of n entries
- There is little difference between the average and worst cases, or between unsuccessful and successful searches



Rationale for Trees *continued*

Binary search analysis continued ...

- Now solving for k:

$$n = 2^k - 1$$
$$n + 1 = 2^k$$

take the log of both sides:

$$\mathbf{k = \lg(n + 1)}$$

- For large n, the difference between $\lg(n+1)$ and $\lg n$ is insignificant
- Recall that binary search does 2 comparisons per vertex, therefore binary search makes $2 \lg n$ comparisons of keys in searching a list of n entries
- There is little difference between the average and worst cases, or between unsuccessful and successful searches



Rationale for Trees *continued*

Binary search analysis continued ...

- Now solving for k:

$$\begin{aligned} n &= 2^k - 1 \\ n+1 &= 2^k \end{aligned}$$

take the log of both sides:

$$\mathbf{k = \lg(n+1)}$$

- For large n, the difference between $\lg(n+1)$ and $\lg n$ is insignificant
- Recall that binary search does 2 comparisons per vertex, therefore binary search makes $2 \lg n$ comparisons of keys in searching a list of n entries
- There is little difference between the average and worst cases, or between unsuccessful and successful searches



Rationale for Trees *continued*

Binary search analysis continued ...

- Now solving for k:

$$\begin{aligned} n &= 2^k - 1 \\ n+1 &= 2^k \end{aligned}$$

take the log of both sides:

$$\mathbf{k = \lg(n+1)}$$

- For large n, the difference between $\lg(n+1)$ and $\lg n$ is insignificant
- Recall that binary search does 2 comparisons per vertex, therefore binary search makes $2 \lg n$ comparisons of keys in searching a list of n entries
- There is little difference between the average and worst cases, or between unsuccessful and successful searches



Rationale for Trees *continued*

Binary search analysis continued ...

- Now solving for k:

$$\begin{aligned} n &= 2^k - 1 \\ n+1 &= 2^k \end{aligned}$$

take the log of both sides:

$$\mathbf{k = \lg(n+1)}$$

- For large n, the difference between $\lg(n+1)$ and $\lg n$ is insignificant
- Recall that binary search does 2 comparisons per vertex, therefore binary search makes $2 \lg n$ comparisons of keys in searching a list of n entries
- There is little difference between the average and worst cases, or between unsuccessful and successful searches



Rationale for Trees *continued*

Binary search analysis continued ...

- Now solving for k:

$$\begin{aligned} n &= 2^k - 1 \\ n+1 &= 2^k \end{aligned}$$

take the log of both sides:

$$\mathbf{k = \lg(n+1)}$$

- For large n, the difference between $\lg(n+1)$ and $\lg n$ is insignificant
- Recall that binary search does 2 comparisons per vertex, therefore binary search makes $2 \lg n$ comparisons of keys in searching a list of n entries
- There is little difference between the average and worst cases, or between unsuccessful and successful searches



Rationale for Trees *continued*

Binary search analysis continued ...

- Now solving for k:

$$\begin{aligned} n &= 2^k - 1 \\ n+1 &= 2^k \end{aligned}$$

take the log of both sides:

$$\mathbf{k = \lg(n+1)}$$

- For large n, the difference between $\lg(n+1)$ and $\lg n$ is insignificant
- Recall that binary search does 2 comparisons per vertex, therefore binary search makes $2 \lg n$ comparisons of keys in searching a list of n entries
- There is little difference between the average and worst cases, or between unsuccessful and successful searches



Rationale for Trees *continued*

Binary search analysis continued ...

- Now solving for k:

$$\begin{aligned} n &= 2^k - 1 \\ n+1 &= 2^k \end{aligned}$$

take the log of both sides:

$$\mathbf{k} = \mathbf{lg}(\mathbf{n}+\mathbf{1})$$

- For large n, the difference between $\lg(n+1)$ and $\lg n$ is insignificant
- Recall that binary search does 2 comparisons per vertex, therefore binary search makes $2 \lg n$ comparisons of keys in searching a list of n entries
- There is little difference between the average and worst cases, or between unsuccessful and successful searches



Rationale for Trees *continued*

- Therefore for binary search:

Case	Best Case	Worst Case	Average Case
item is present	1	2 lg n	2 lg n
item is not present	2 lg n	2 lg n	2 lg n

- Is this difference between the efficiency of the two search algorithms significant?
- Let's do a test with code



Rationale for Trees *continued*

- Therefore for binary search:

Case	Best Case	Worst Case	Average Case
item is present	1	2 lg n	2 lg n
item is not present	2 lg n	2 lg n	2 lg n

- Is this difference between the efficiency of the two search algorithms significant?
- Let's do a test with code



Rationale for Trees *continued*

- Therefore for binary search:

Case	Best Case	Worst Case	Average Case
item is present	1	2 lg n	2 lg n
item is not present	2 lg n	2 lg n	2 lg n

- Is this difference between the efficiency of the two search algorithms significant?
- Let's do a test with code



Rationale for Trees *continued*

```
#include <stdio.h>

#define SIZE 2000000

int seqSearch(int[],int,int);
int binSearch(int[],int,int);

int main(void){
  int list[SIZE], key = (int) SIZE*.93172;

  printf("Number of items: %d\n",SIZE);
  for(int i = 0; i < SIZE; i++) list[i] = i;
  printf("key = %d\n",key);
  printf("seq. search compares to find key = %d\n",
         seqSearch(list,SIZE,key));
  printf("bin. search compares to find key = %d\n",
         binSearch(list,SIZE,key));
}

int seqSearch(int a[], int n, int key) {
  int compares = 0;
  for (int i = 0; i < n; i++) {
    compares++;
    if (a[i] == key) return compares;
  }
  return -1;
}
```

```
int binSearch(int a[], int n, int key) {
  int compares = 0, mid=0;
  int low=0, high=n-1;
  while (low <= high) {
    mid = (high+low)/2;
    ++compares;
    if (a[mid] == key) return compares;
    else {
      ++compares;
      if (a[mid] < key) {
        low = mid+1;
      } else {
        high = mid-1;
      }
    }
  }
  return -1;
}
```



Rationale for Trees *continued*

```
#include <stdio.h>

#define SIZE 2000000

int seqSearch(int[],int,int);
int binSearch(int[],int,int);

int main(void){
  int list[SIZE], key = (int) SIZE*.93172;

  printf("Number of items: %d\n",SIZE);
  for(int i = 0; i < SIZE; i++) list[i] = i;
  printf("key = %d\n",key);
  printf("seq. search compares to find key = %d\n",
         seqSearch(list,SIZE,key));
  printf("bin. search compares to find key = %d\n",
         binSearch(list,SIZE,key));
}

int seqSearch(int a[], int n, int key) {
  int compares = 0;
  for (int i = 0; i < n; i++) {
    compares++;
    if (a[i] == key) return compares;
  }
  return -1;
}
```

```
int binSearch(int a[], int n, int key) {
  int compares = 0, mid=0;
  int low=0, high=n-1;
  while (low <= high) {
    mid = (high+low)/2;
    ++compares;
    if (a[mid] == key) return compares;
    else {
      ++compares;
      if (a[mid] < key) {
        low = mid+1;
      } else {
        high = mid-1;
      }
    }
  }
  return -1;
}
```

```
Terminal — -tcsh — 43×7
homebox:~/cprogs/% gcc binSeqComparison.c
homebox:~/cprogs/% a.out
Number of items: 2000000
key = 1863440
seq. search compares to find key = 1863441
bin. search compares to find key = 41
homebox:~/cprogs/%
```

Rationale for Trees *continued*

- There are two issues with using binary searches on a list stored in an array:
  1. it only works on sorted lists
  2. suffers the drawbacks of costly insertions and deletions
- Is there a way we can get the efficiency of binary search along with the more efficient insertions/deletions we get with linked structures?
- Yes – with trees!



Rationale for Trees *continued*

- There are two issues with using binary searches on a list stored in an array:
  1. it only works on sorted lists
  2. suffers the drawbacks of costly insertions and deletions
- Is there a way we can get the efficiency of binary search along with the more efficient insertions/deletions we get with linked structures?
- Yes – with trees!



Rationale for Trees *continued*

- There are two issues with using binary searches on a list stored in an array:
  1. it only works on sorted lists is this really a problem?
  2. suffers the drawbacks of costly insertions and deletions
- Is there a way we can get the efficiency of binary search along with the more efficient insertions/deletions we get with linked structures?
- Yes – with trees!



Rationale for Trees *continued*

- There are two issues with using binary searches on a list stored in an array:
  1. it only works on sorted lists is this really a problem?
  2. suffers the drawbacks of costly insertions and deletions
- Is there a way we can get the efficiency of binary search along with the more efficient insertions/deletions we get with linked structures?
- Yes – with trees!



Rationale for Trees *continued*

- There are two issues with using binary searches on a list stored in an array:
  1. it only works on sorted lists is this really a problem?
  2. suffers the drawbacks of costly insertions and deletions
- Is there a way we can get the efficiency of binary search along with the more efficient insertions/deletions we get with linked structures?
- Yes – with trees!



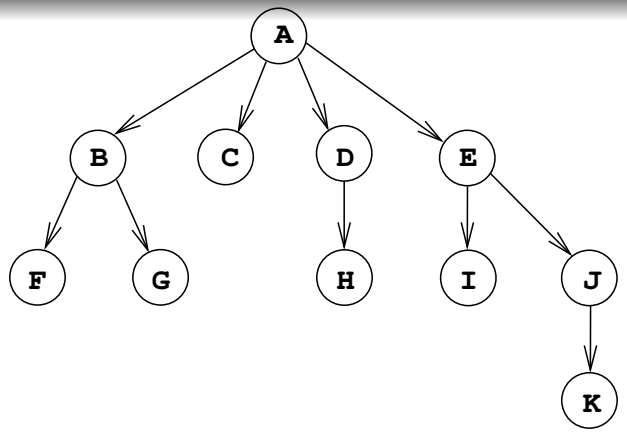
Rationale for Trees *continued*

- There are two issues with using binary searches on a list stored in an array:
  1. it only works on sorted lists is this really a problem?
  2. suffers the drawbacks of costly insertions and deletions
- Is there a way we can get the efficiency of binary search along with the more efficient insertions/deletions we get with linked structures?
- Yes – with trees!



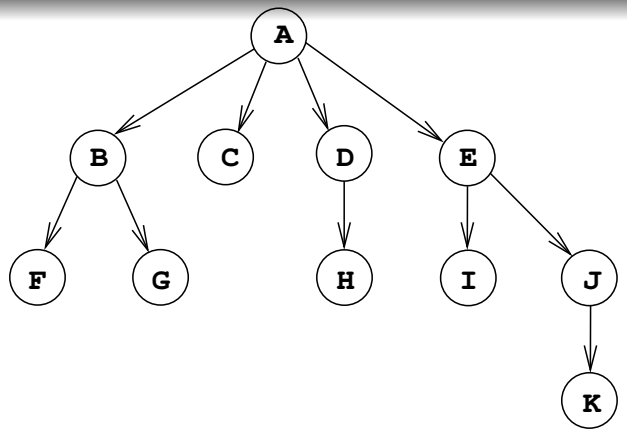
Introduction to Trees

- A **tree** is a set of nodes and directed edges that connect them



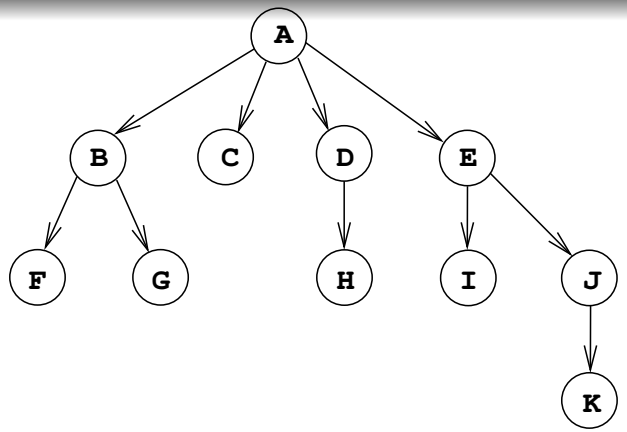
- One node is distinguished as the **root**
- Every node *c*, except the root, is connected by an edge from exactly one other node *p*; *p* is *c*'s **parent**, and *c* is one of *p*'s **children**
- Nodes with no children are often called **leaves**
- Each node in a tree can be thought of as the root of a **subtree**
- There is always a unique path from the root to any node in the tree
- We will mostly be interested in **binary trees**

Introduction to Trees



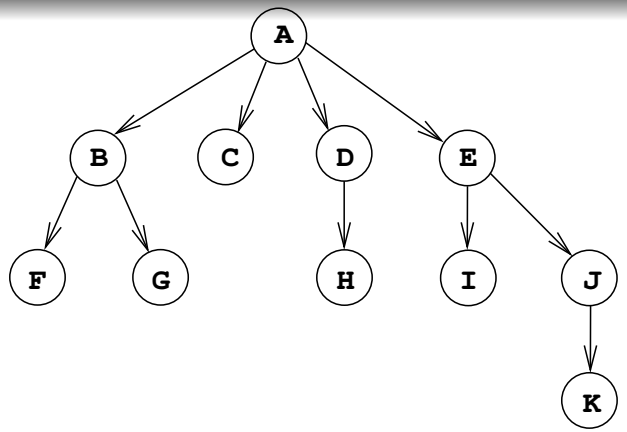
- A **tree** is a set of nodes and directed edges that connect them
- One node is distinguished as the **root**
- Every node *c*, except the root, is connected by an edge from exactly one other node *p*; *p* is *c*'s **parent**, and *c* is one of *p*'s **children**
- Nodes with no children are often called **leaves**
- Each node in a tree can be thought of as the root of a **subtree**
- There is always a unique path from the root to any node in the tree
- We will mostly be interested in **binary trees**

Introduction to Trees



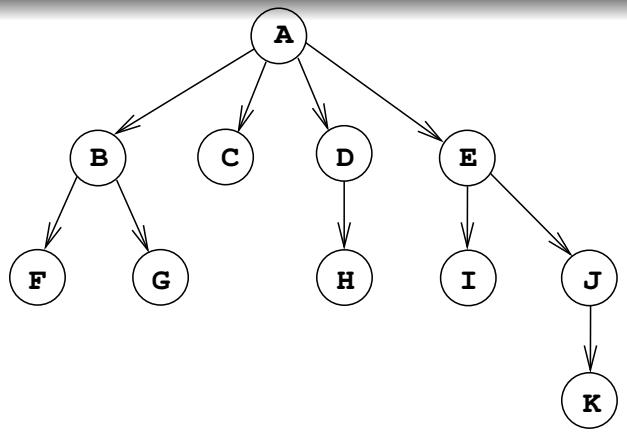
- A **tree** is a set of nodes and directed edges that connect them
- One node is distinguished as the **root**
- Every node *c*, except the root, is connected by an edge from exactly one other node *p*; *p* is *c*'s **parent**, and *c* is one of *p*'s **children**
- Nodes with no children are often called **leaves**
- Each node in a tree can be thought of as the root of a **subtree**
- There is always a unique path from the root to any node in the tree
- We will mostly be interested in **binary trees**

Introduction to Trees



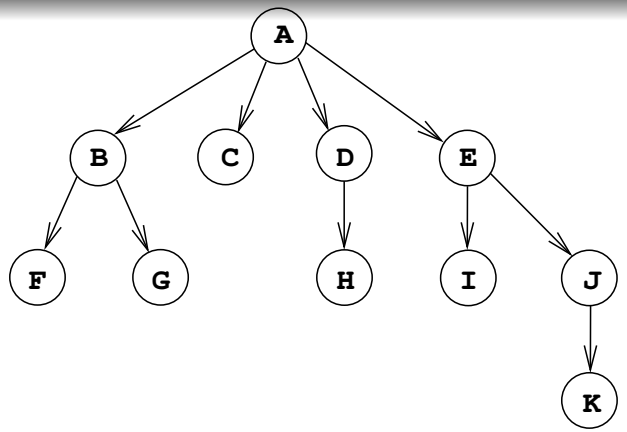
- A **tree** is a set of nodes and directed edges that connect them
- One node is distinguished as the **root**
- Every node *c*, except the root, is connected by an edge from exactly one other node *p*; *p* is *c*'s **parent**, and *c* is one of *p*'s **children**
- Nodes with no children are often called **leaves**
- Each node in a tree can be thought of as the root of a **subtree**
- There is always a unique path from the root to any node in the tree
- We will mostly be interested in **binary trees**

Introduction to Trees



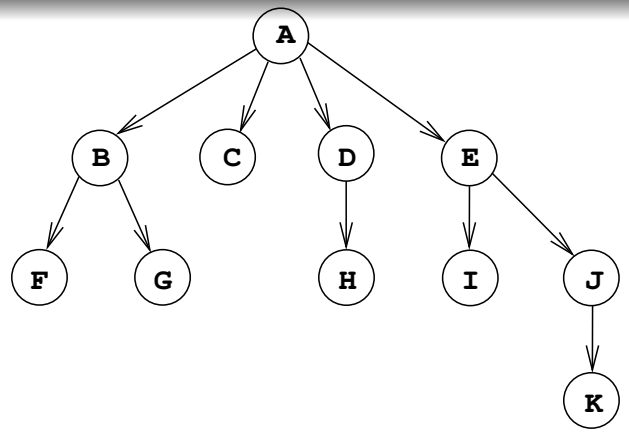
- A **tree** is a set of nodes and directed edges that connect them
- One node is distinguished as the **root**
- Every node *c*, except the root, is connected by an edge from exactly one other node *p*; *p* is *c*'s **parent**, and *c* is one of *p*'s **children**
- Nodes with no children are often called **leaves**
- Each node in a tree can be thought of as the root of a **subtree**
- There is always a unique path from the root to any node in the tree
- We will mostly be interested in **binary trees**

Introduction to Trees



- A **tree** is a set of nodes and directed edges that connect them
- One node is distinguished as the **root**
- Every node *c*, except the root, is connected by an edge from exactly one other node *p*; *p* is *c*'s **parent**, and *c* is one of *p*'s **children**
- Nodes with no children are often called **leaves**
- Each node in a tree can be thought of as the root of a **subtree**
- There is always a unique path from the root to any node in the tree
- We will mostly be interested in **binary trees**

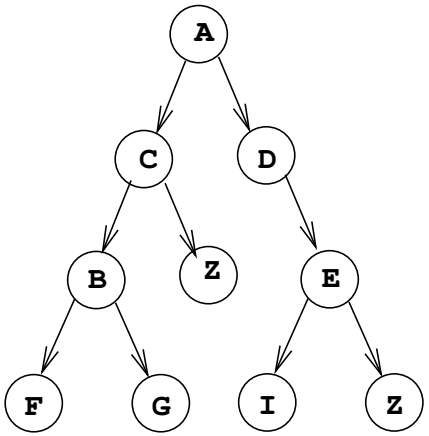
Introduction to Trees



- A **tree** is a set of nodes and directed edges that connect them
- One node is distinguished as the **root**
- Every node *c*, except the root, is connected by an edge from exactly one other node *p*; *p* is *c*'s **parent**, and *c* is one of *p*'s **children**
- Nodes with no children are often called **leaves**
- Each node in a tree can be thought of as the root of a **subtree**
- There is always a unique path from the root to any node in the tree
- We will mostly be interested in **binary trees**

Introduction to Binary Trees

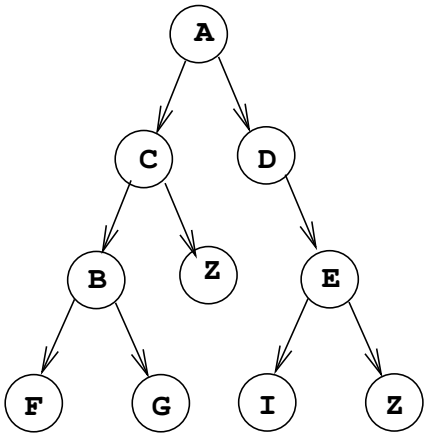
- A **binary tree** is a tree which no node can have more than two children (i.e. it can have 0, 1, or 2 children)



- The two children are named **left** and **right**
- A **full binary tree** is a binary tree where every non-leaf has two children
- Let's look at a special kind of binary tree called a **binary search tree** (often abbreviated BST)

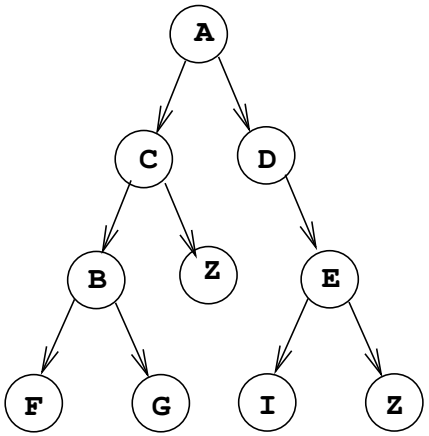
Introduction to Binary Trees

- A **binary tree** is a tree which no node can have more than two children (i.e. it can have 0, 1, or 2 children)



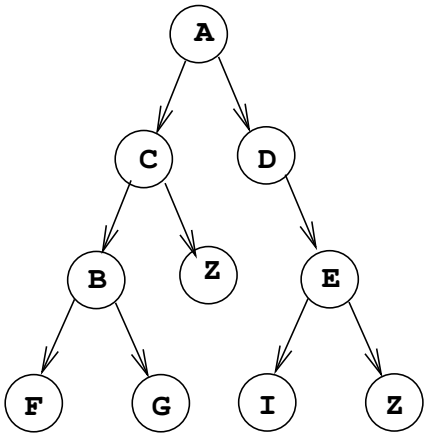
- The two children are named **left** and **right**
- A **full binary tree** is a binary tree where every non-leaf has two children
- Let's look at a special kind of binary tree called a **binary search tree** (often abbreviated BST)

Introduction to Binary Trees



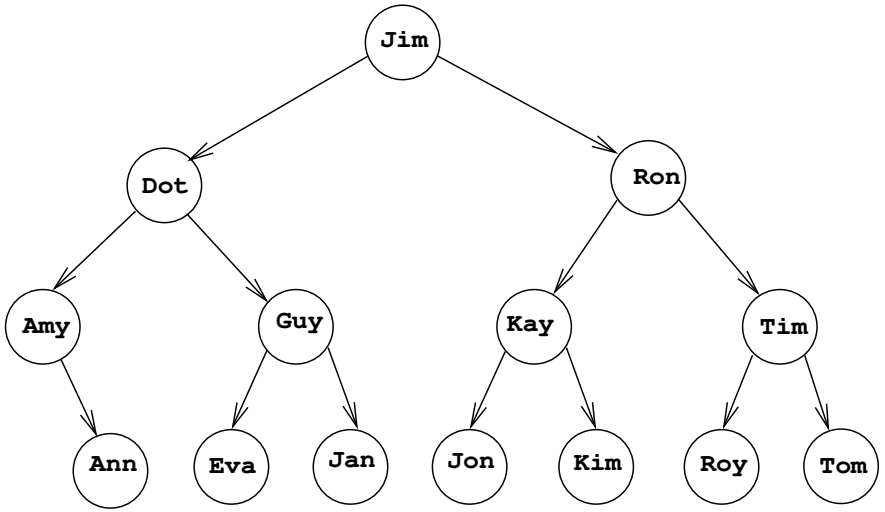
- A **binary tree** is a tree which no node can have more than two children (i.e. it can have 0, 1, or 2 children)
- The two children are named **left** and **right**
- A **full binary tree** is a binary tree where every non-leaf has two children
- Let's look at a special kind of binary tree called a **binary search tree** (often abbreviated BST)

Introduction to Binary Trees



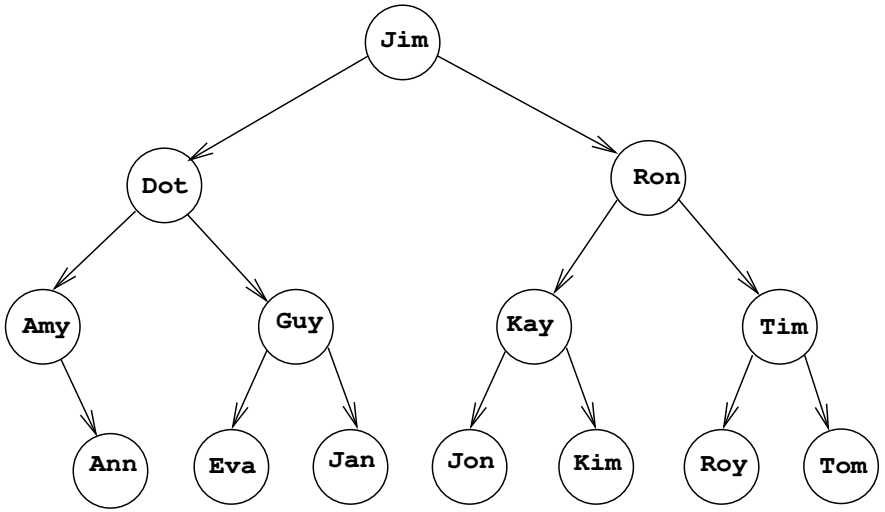
- A **binary tree** is a tree which no node can have more than two children (i.e. it can have 0, 1, or 2 children)
- The two children are named **left** and **right**
- A **full binary tree** is a binary tree where every non-leaf has two children
- Let's look at a special kind of binary tree called a **binary search tree** (often abbreviated BST)

Overview of Binary Search Trees



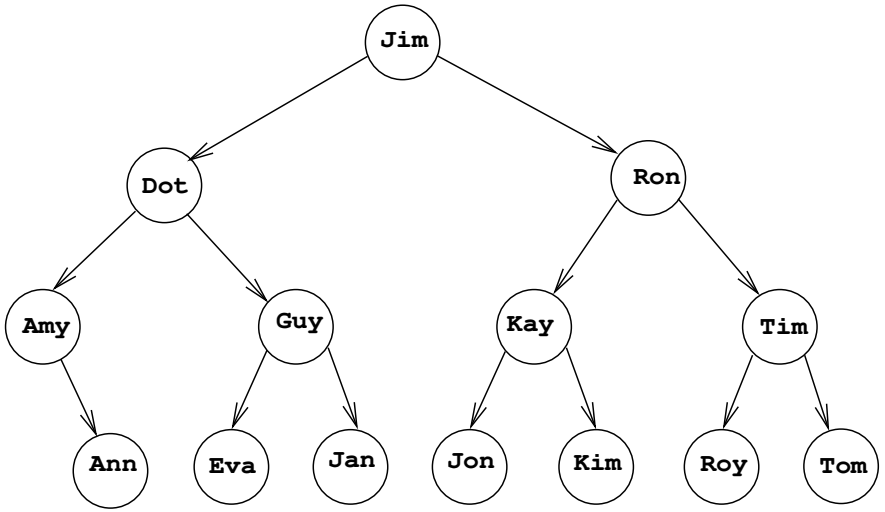
- A **binary search tree** (BST) is an ordered binary tree where
  - every left subtree contains values less than its root
  - every right subtree contains values greater than its root
  - duplicate items are not allowed
- Entries in a BST must be able to be compared to determine ordering
- Items are searched for by their **key** – a data object field that has a unique value for each instance of the object

Overview of Binary Search Trees



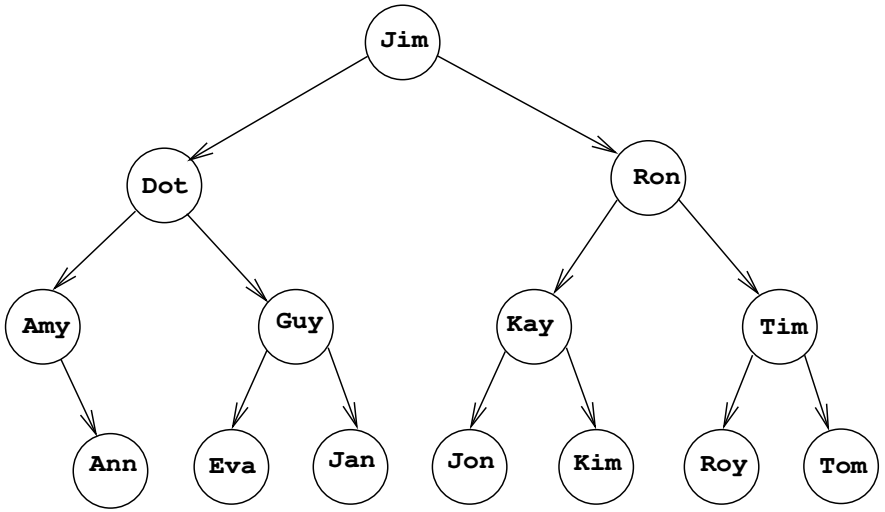
- A **binary search tree** (BST) is an ordered binary tree where
  - every left subtree contains values less than its root
  - every right subtree contains values greater than its root
  - duplicate items are not allowed
- Entries in a BST must be able to be compared to determine ordering
- Items are searched for by their **key** – a data object field that has a unique value for each instance of the object

Overview of Binary Search Trees



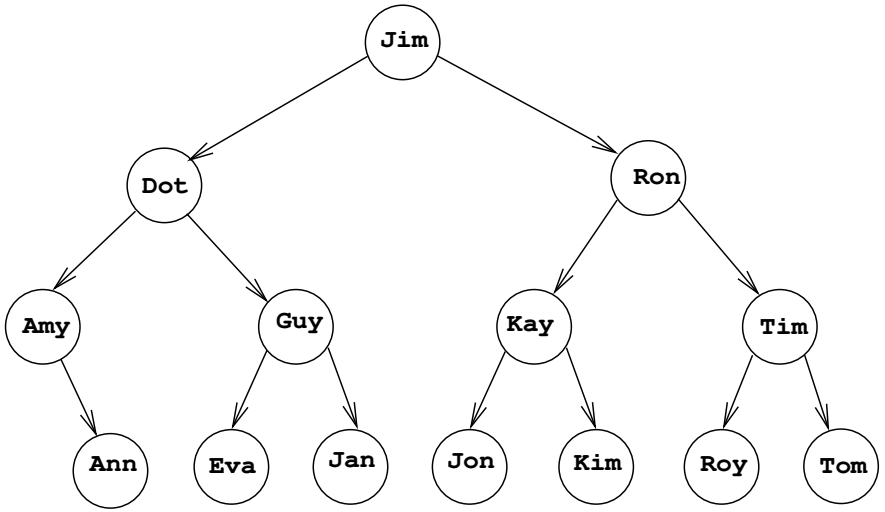
- A **binary search tree** (BST) is an ordered binary tree where
  - every left subtree contains values less than its root
  - every right subtree contains values greater than its root
  - duplicate items are not allowed
- Entries in a BST must be able to be compared to determine ordering
- Items are searched for by their **key** – a data object field that has a unique value for each instance of the object

Overview of Binary Search Trees



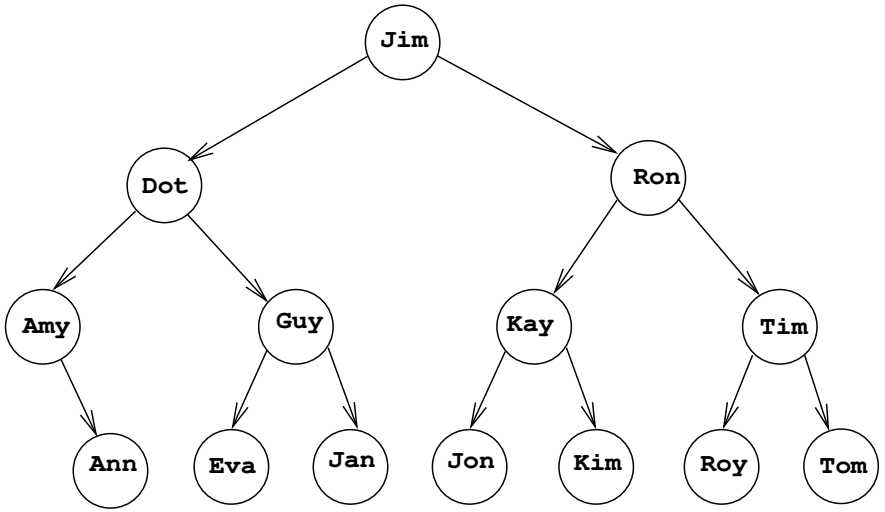
- A **binary search tree** (BST) is an ordered binary tree where
  - every left subtree contains values less than its root
  - every right subtree contains values greater than its root
  - duplicate items are not allowed
- Entries in a BST must be able to be compared to determine ordering
- Items are searched for by their **key** – a data object field that has a unique value for each instance of the object

Overview of Binary Search Trees



- A **binary search tree** (BST) is an ordered binary tree where
  - every left subtree contains values less than its root
  - every right subtree contains values greater than its root
  - duplicate items are not allowed
- Entries in a BST must be able to be compared to determine ordering
- Items are searched for by their **key** – a data object field that has a unique value for each instance of the object

Overview of Binary Search Trees



- A **binary search tree** (BST) is an ordered binary tree where
  - every left subtree contains values less than its root
  - every right subtree contains values greater than its root
  - duplicate items are not allowed
- Entries in a BST must be able to be compared to determine ordering
- Items are searched for by their **key** – a data object field that has a unique value for each instance of the object

Binary Search Tree Traversals

- The concept of traversing a linear data structure is easy to grasp; how does one traverse a tree?
- As we arrive at each node we will possibly have 0, 1, or 2 paths to take to continue the traversal
- As we traverse a tree, at each node there will be three tasks that we wish to do in some order: visit the node (V), traverse the left subtree (L), and traverse the right subtree (R)
- Rather than randomly choosing one of these activities each time we arrive at a node, let's look at 3 patterns of traversing a binary tree:

Preorder	VLR
Inorder	LVR
Postorder	LRV



Binary Search Tree Traversals

- The concept of traversing a linear data structure is easy to grasp; how does one traverse a tree?
- As we arrive at each node we will possibly have 0, 1, or 2 paths to take to continue the traversal
- As we traverse a tree, at each node there will be three tasks that we wish to do in some order: visit the node (V), traverse the left subtree (L), and traverse the right subtree (R)
- Rather than randomly choosing one of these activities each time we arrive at a node, let's look at 3 patterns of traversing a binary tree:

Preorder	VLR
Inorder	LVR
Postorder	LRV



Binary Search Tree Traversals

- The concept of traversing a linear data structure is easy to grasp; how does one traverse a tree?
- As we arrive at each node we will possibly have 0, 1, or 2 paths to take to continue the traversal
- As we traverse a tree, at each node there will be three tasks that we wish to do in some order: visit the node (V), traverse the left subtree (L), and traverse the right subtree (R)
- Rather than randomly choosing one of these activities each time we arrive at a node, let's look at 3 patterns of traversing a binary tree:

Preorder	VLR
Inorder	LVR
Postorder	LRV



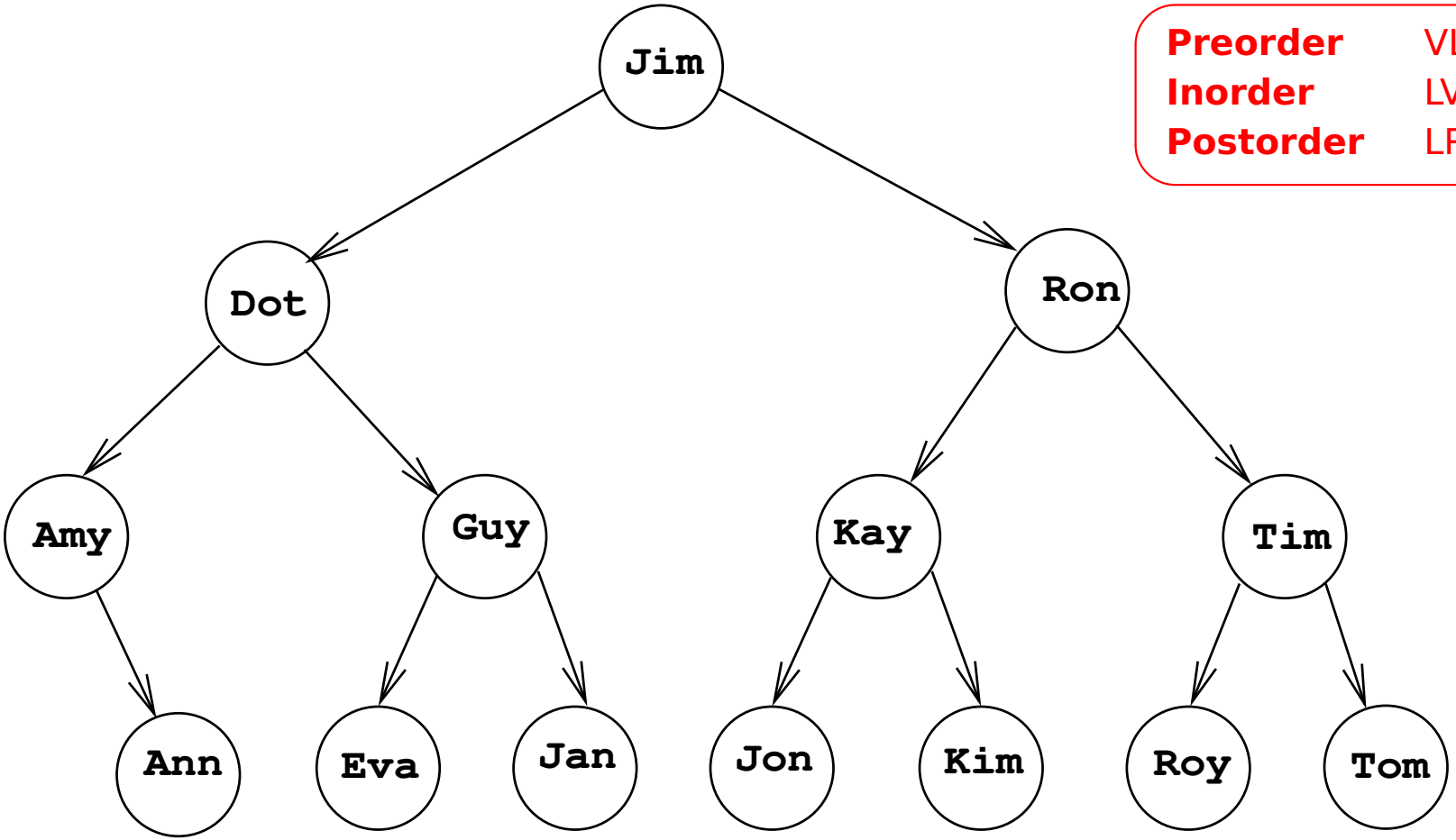
Binary Search Tree Traversals

- The concept of traversing a linear data structure is easy to grasp; how does one traverse a tree?
- As we arrive at each node we will possibly have 0, 1, or 2 paths to take to continue the traversal
- As we traverse a tree, at each node there will be three tasks that we wish to do in some order: visit the node (V), traverse the left subtree (L), and traverse the right subtree (R)
- Rather than randomly choosing one of these activities each time we arrive at a node, let's look at 3 patterns of traversing a binary tree:

Preorder	VLR
Inorder	LVR
Postorder	LRV



Binary Search Tree Traversals *continued*



Preorder	VLR
Inorder	LVR
Postorder	LRV

Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:
- Now consider functions instead of arrays:
- In C we can define and use pointers to functions
- Using a function name without the parentheses () evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:
- Now consider functions instead of arrays:
- In C we can define and use pointers to functions
- Using a function name without the parentheses () evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:
  p1 // p1
- Now consider functions instead of arrays:
- In C we can define and use pointers to functions
- Using a function name without the parentheses ( ) evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1          // p1
*p1         // is a pointer
```

- Now consider functions instead of arrays:
- In C we can define and use pointers to functions
- Using a function name without the parentheses ( ) evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1          // p1
*p1         // is a pointer
(*p1)[5]    // to an array of size 5
```

- Now consider functions instead of arrays:
- In C we can define and use pointers to functions
- Using a function name without the parentheses () evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

- Now consider functions instead of arrays:
- In C we can define and use pointers to functions
- Using a function name without the parentheses () evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:
- In C we can define and use pointers to functions
- Using a function name without the parentheses ( ) evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

- In C we can define and use pointers to functions
- Using a function name without the parentheses ( ) evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

```
p2                          // p2
```

- In C we can define and use pointers to functions
- Using a function name without the parentheses ( ) evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

```
p2                          // p2
*p2                         // is a pointer
```

- In C we can define and use pointers to functions
- Using a function name without the parentheses ( ) evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

```
p2                          // p2
*p2                         // is a pointer
(*p2)()                     // to a function
```

- In C we can define and use pointers to functions
- Using a function name without the parentheses () evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

```
p2                      // p2
*p2                     // is a pointer
(*p2)()                 // to a function
(*p2)(float n)          // passed a float
```

- In C we can define and use pointers to functions
- Using a function name without the parentheses () evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

```
p2                          // p2
*p2                         // is a pointer
(*p2)()                     // to a function
(*p2)(float n)              // passed a float
int (*p2)(float n)          // returning an integer
```

- In C we can define and use pointers to functions
- Using a function name without the parentheses () evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

```
p2                          // p2
*p2                         // is a pointer
(*p2)()                     // to a function
(*p2)(float n)              // passed a float
int (*p2)(float n)          // returning an integer
```

compare to `int *p2(float n)`

- In C we can define and use pointers to functions
- Using a function name without the parentheses ( ) evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

```
p2                          // p2
*p2                         // is a pointer
(*p2)()                     // to a function
(*p2)(float n)              // passed a float
int (*p2)(float n)          // returning an integer
```

compare to `int *p2(float n)`

- In C we can define and use pointers to functions
- Using a function name without the parentheses ( ) evaluates to the address of the function



Binary Search Tree Traversals *continued*

- A digression ... passing functions into functions
- In EGRE245 we learned the following about pointers and arrays:

```
p1                  // p1
*p1                 // is a pointer
(*p1)[5]            // to an array of size 5
int (*p1)[5]        // of integers
```

compare to `int *p1[5]`

- Now consider functions instead of arrays:

```
p2                          // p2
*p2                         // is a pointer
(*p2)()                     // to a function
(*p2)(float n)              // passed a float
int (*p2)(float n)          // returning an integer
```

compare to `int *p2(float n)`

- In C we can define and use pointers to functions
- Using a function name without the parentheses `()` evaluates to the address of the function



Binary Search Tree Traversals *continued*

```
#include <stdio.h>
void fun() { printf("this is fun!\n"); }
void twoUp(int *n) { *n += 2; }
void threeUp(int *n) { *n += 3; }
void apply(void (*f)(int*),int data[], int n) {
  for (int i=0; i < n; i++) f(&data[i]);
}
void printList(int l[],int n) {
  for (int i=0; i<n; i++) printf("%d ",l[i]);
}

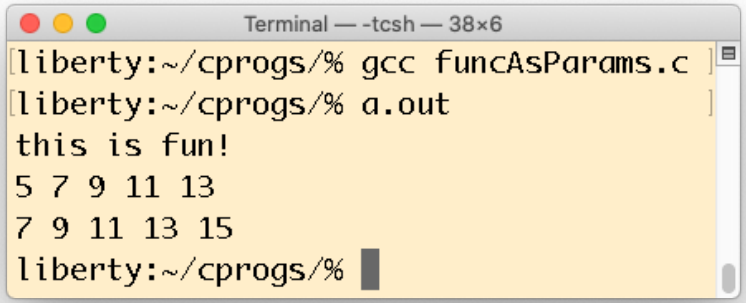
int main(void) {
  void (*fp)();
  fp = fun;
  (*fp)();
  int list1[5] = {2,4,6,8,10};
  for (int i=0; i<5; i++) threeUp(&list1[i]);
  printList(list1,5); printf("\n");
  apply(twoUp,list1,5);
  printList(list1,5); printf("\n");
}
```



Binary Search Tree Traversals *continued*

```
#include <stdio.h>
void fun() { printf("this is fun!\n"); }
void twoUp(int *n) { *n += 2; }
void threeUp(int *n) { *n += 3; }
void apply(void (*f)(int*),int data[], int n) {
  for (int i=0; i < n; i++) f(&data[i]);
}
void printList(int l[],int n) {
  for (int i=0; i<n; i++) printf("%d ",l[i]);
}

int main(void) {
  void (*fp)();
  fp = fun;
  (*fp)();
  int list1[5] = {2,4,6,8,10};
  for (int i=0; i<5; i++) threeUp(&list1[i]);
  printList(list1,5); printf("\n");
  apply(twoUp,list1,5);
  printList(list1,5); printf("\n");
}
```



Binary Search Tree Traversals *continued*

```
void preOrder(struct node *t,void (*f)()) {
  if(t!=NULL) {
    f(t->data);
    preOrder(t->left,f);
    preOrder(t->right,f);
  }
}
void inOrder(struct node *t,void (*f)()) {
  if(t!=NULL) {
    inOrder(t->left,f);
    f(t->data);
    inOrder(t->right,f);
  }
}
void postOrder(struct node *t,void (*f)()) {
  if(t!=NULL) {
    postOrder(t->left,f);
    postOrder(t->right,f);
    f(t->data);
  }
}
```

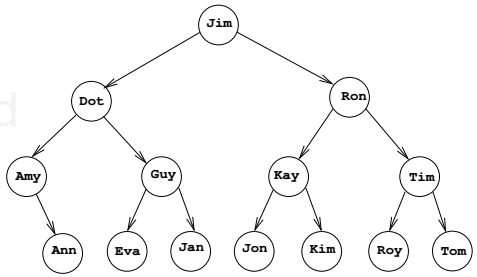
```
struct node {
  itemType data;
  struct node *left, *right;
};
```



Implementing Binary Search Trees

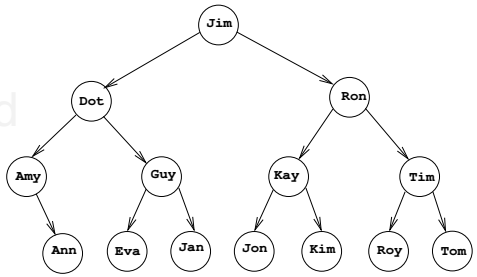
- Just like our linked structures, will first define a node
- Our node will define a “generic” binary search tree node
- Remember that with a doubly linked list structure each node had three components: data plus previous and next pointers
- Therefore we defined a doubly linked node class with data, prev, and next fields as private data
- A binary search tree node also has 3 fields – a data field plus two pointers, one to the left subtree and one to the right subtree
- Therefore our binary search tree node structure will have 3 private fields: `data`, `left`, and `right`

```
struct node {
  itemType data;
  struct node *left, *right;
};
```



Implementing Binary Search Trees

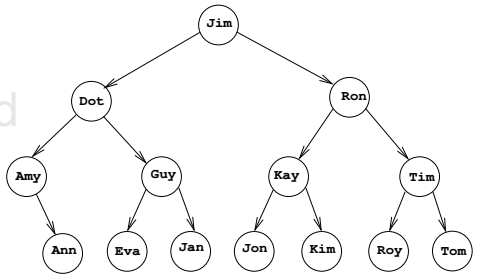
- Just like our linked structures, will first define a node
- Our node will define a “generic” binary search tree node
- Remember that with a doubly linked list structure each node had three components: data plus previous and next pointers
- Therefore we defined a doubly linked node class with data, prev, and next fields as private data
- A binary search tree node also has 3 fields – a data field plus two pointers, one to the left subtree and one to the right subtree
- Therefore our binary search tree node structure will have 3 private fields: `data`, `left`, and `right`




```
struct node {
  itemType data;
  struct node *left, *right;
};
```

Implementing Binary Search Trees

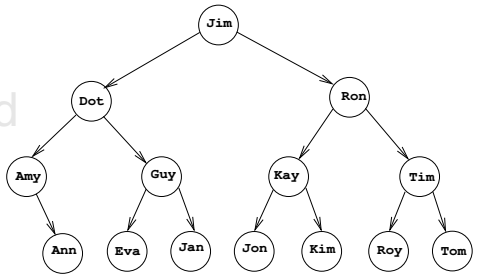
- Just like our linked structures, will first define a node
- Our node will define a “generic” binary search tree node
- Remember that with a doubly linked list structure each node had three components: data plus previous and next pointers
- Therefore we defined a doubly linked node class with data, prev, and next fields as private data
- A binary search tree node also has 3 fields – a data field plus two pointers, one to the left subtree and one to the right subtree
- Therefore our binary search tree node structure will have 3 private fields: `data`, `left`, and `right`




```
struct node {
  itemType data;
  struct node *left, *right;
};
```

Implementing Binary Search Trees

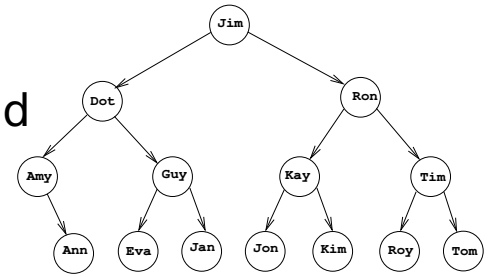
- Just like our linked structures, will first define a node
- Our node will define a “generic” binary search tree node
- Remember that with a doubly linked list structure each node had three components: data plus previous and next pointers
- Therefore we defined a doubly linked node class with data, prev, and next fields as private data
- A binary search tree node also has 3 fields – a data field plus two pointers, one to the left subtree and one to the right subtree
- Therefore our binary search tree node structure will have 3 private fields: `data`, `left`, and `right`




```
struct node {
  itemType data;
  struct node *left, *right;
};
```

Implementing Binary Search Trees

- Just like our linked structures, will first define a node
- Our node will define a “generic” binary search tree node
- Remember that with a doubly linked list structure each node had three components: data plus previous and next pointers
- Therefore we defined a doubly linked node class with data, prev, and next fields as private data
- A binary search tree node also has 3 fields – a data field plus two pointers, one to the left subtree and one to the right subtree

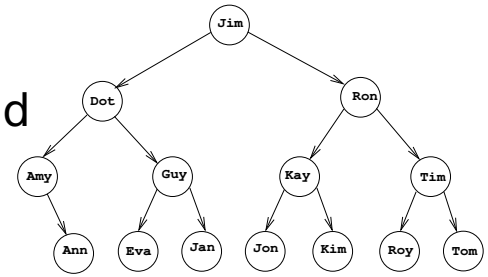


- Therefore our binary search tree node structure will have 3 private fields: `data`, `left`, and `right`

```
struct node {
  itemType data;
  struct node *left, *right;
};
```

Implementing Binary Search Trees

- Just like our linked structures, will first define a node
- Our node will define a “generic” binary search tree node
- Remember that with a doubly linked list structure each node had three components: data plus previous and next pointers
- Therefore we defined a doubly linked node class with data, prev, and next fields as private data
- A binary search tree node also has 3 fields – a data field plus two pointers, one to the left subtree and one to the right subtree



- Therefore our binary search tree node structure will have 3 private fields: `data`, `left`, and `right`

```
struct node {
  itemType data;
  struct node *left, *right;
};
```

Implementing Binary Search Trees *continued*

- File `bst.h`:

```
#include <stdbool.h>
#include "itemType.h"

#ifndef BST_H
#define BST_H

typedef struct bstType *bst;

bst createBST();
void freeBST(bst);

void insertItem(bst,itemType);
void removeItem(bst,itemType);

bool contains(bst,itemType);

void preorder(bst,void (*f)(itemType));
void inorder(bst,void (*f)(itemType));
void postorder(bst,void (*f)(itemType));

int size(bst);
bool isEmpty(bst);

#endif
```



Insertion into a Binary Search Tree

- How does one insert into a BST?
  - algorithm to insert item *n* into BST *t*:

```
if t is empty then
    return a one-node tree containing n
    else
        if n is less than the value in root of t then
            return a tree with n inserted in the left subtree of t
        else if n is greater than the value in the root of t then
            return a tree with n inserted in the right subtree of t
        else
            error – duplicates are not allowed
```



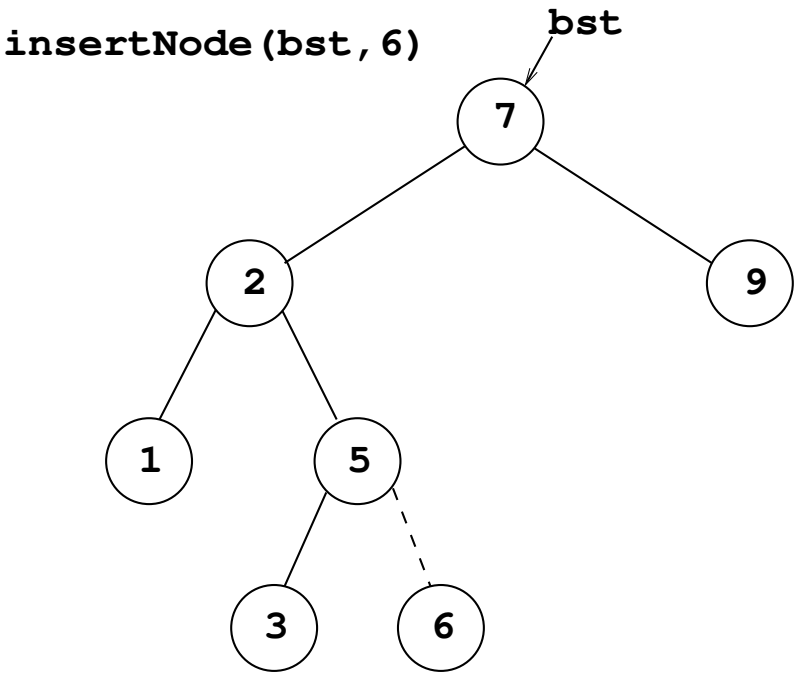
Insertion into a Binary Search Tree

- How does one insert into a BST?
  - algorithm to insert item *n* into BST *t*:

```
if t is empty then
    return a one-node tree containing n
    else
        if n is less than the value in root of t then
            return a tree with n inserted in the left subtree of t
        else if n is greater than the value in the root of t then
            return a tree with n inserted in the right subtree of t
        else
            error – duplicates are not allowed
```



Insertion into a Binary Search Tree *continued*




```
struct node *insertNode(struct node *t, itemType item) {
  if(t==NULL) {
    t = malloc(sizeof(struct node));
    t->data = item; t->left = t->right = NULL;
  } else if(itemcmp(item,t->data) < 0)
    t->left = insertNode(t->left,item);
  else if(itemcmp(item,t->data) > 0)
    t->right = insertNode(t->right,item);
  else
    printf("Warning:  duplicate item '%s' ignored\n",
           toItemString(item));
  return t;
}
```

```
struct node {
  itemType data;
  struct node *left, *right;
};
```

Removing an Item from a Binary Search Tree

- Removal from a BST?
  - remove is more difficult because nonleaf (internal) nodes hold the tree together
  - Also need to avoid making the tree deeper, if possible
  - Three cases to consider:



Removing an Item from a Binary Search Tree

- Removal from a BST?
  - remove is more difficult because nonleaf (internal) nodes hold the tree together
  - Also need to avoid making the tree deeper, if possible
  - Three cases to consider:



Removing an Item from a Binary Search Tree

- Removal from a BST?
  - remove is more difficult because nonleaf (internal) nodes hold the tree together
  - Also need to avoid making the tree deeper, if possible
  - Three cases to consider:
    1. removing a leaf node



Removing an Item from a Binary Search Tree

- Removal from a BST?
  - remove is more difficult because nonleaf (internal) nodes hold the tree together
  - Also need to avoid making the tree deeper, if possible
  - Three cases to consider:
    - removing a leaf node
    - removing a node with one child
    - removing a node with two children



Removing an Item from a Binary Search Tree

- Removal from a BST?
  - remove is more difficult because nonleaf (internal) nodes hold the tree together
  - Also need to avoid making the tree deeper, if possible
  - Three cases to consider:
    - ★ removing a leaf node
    - ★ removing a node with one child
    - ★ removing a node with two children



Removing an Item from a Binary Search Tree

- Removal from a BST?
  - remove is more difficult because nonleaf (internal) nodes hold the tree together
  - Also need to avoid making the tree deeper, if possible
  - Three cases to consider:
    - removing a leaf node
    - removing a node with one child
    - removing a node with two children

Removing an Item from a Binary Search Tree

- Removal from a BST?
  - remove is more difficult because nonleaf (internal) nodes hold the tree together
  - Also need to avoid making the tree deeper, if possible
  - Three cases to consider:
    - ★ removing a leaf node
    - ★ removing a node with one child
    - ★ removing a node with two children



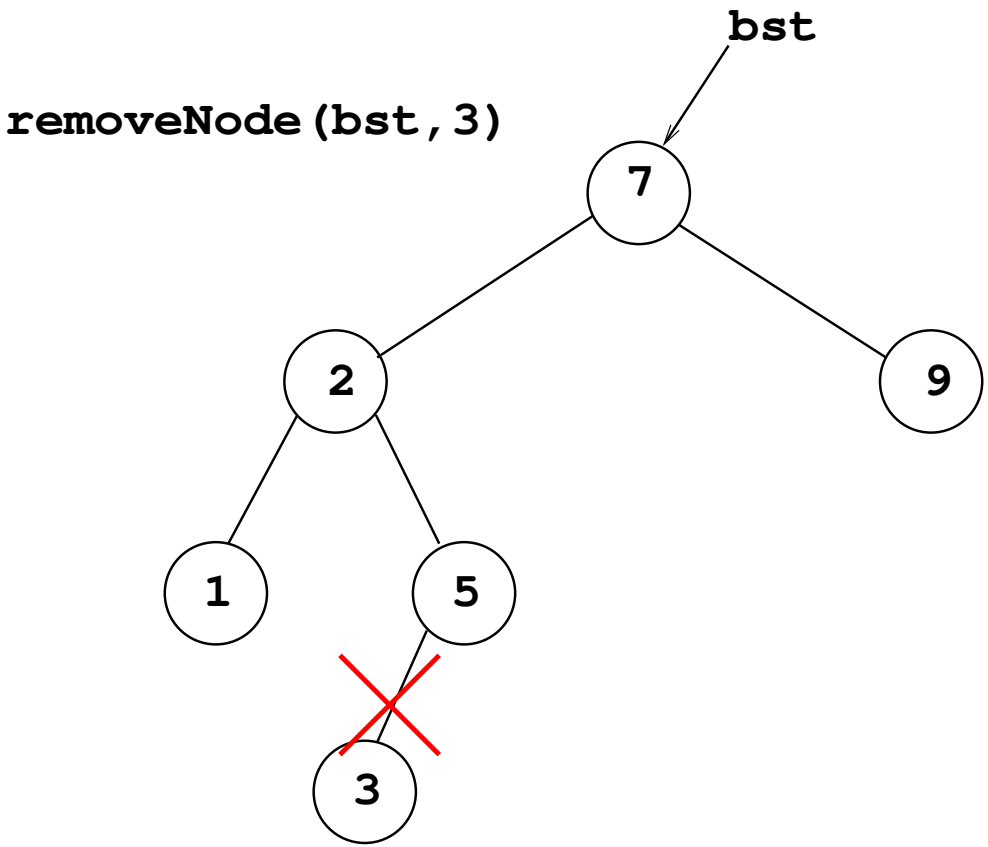
Removing an Item from a Binary Search Tree *continued*

- Case 1: node to remove is a leaf



Removing an Item from a Binary Search Tree *continued*

- Case 1: node to remove is a leaf



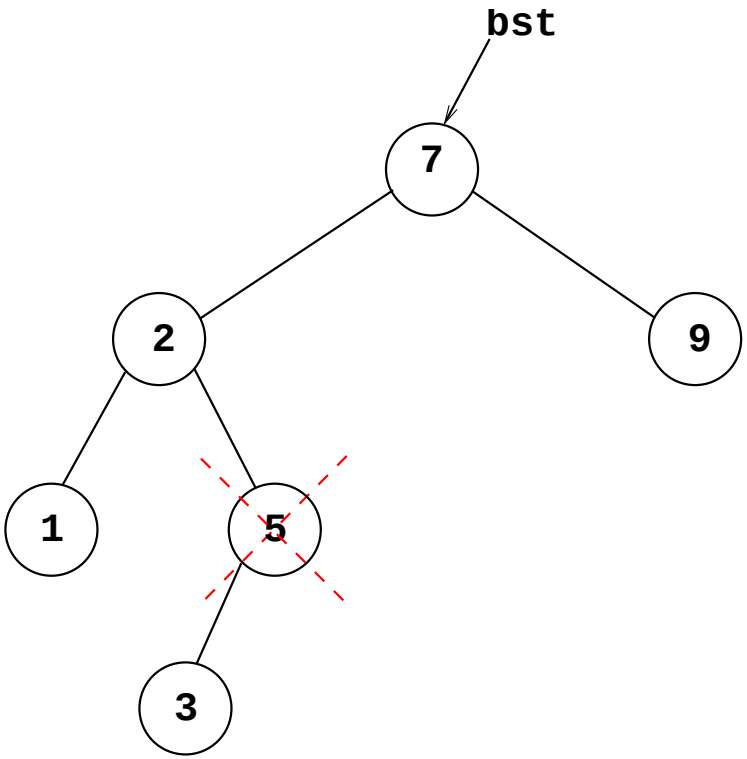
Removing an Item from a Binary Search Tree *continued*

- Case 2: node to remove has one child



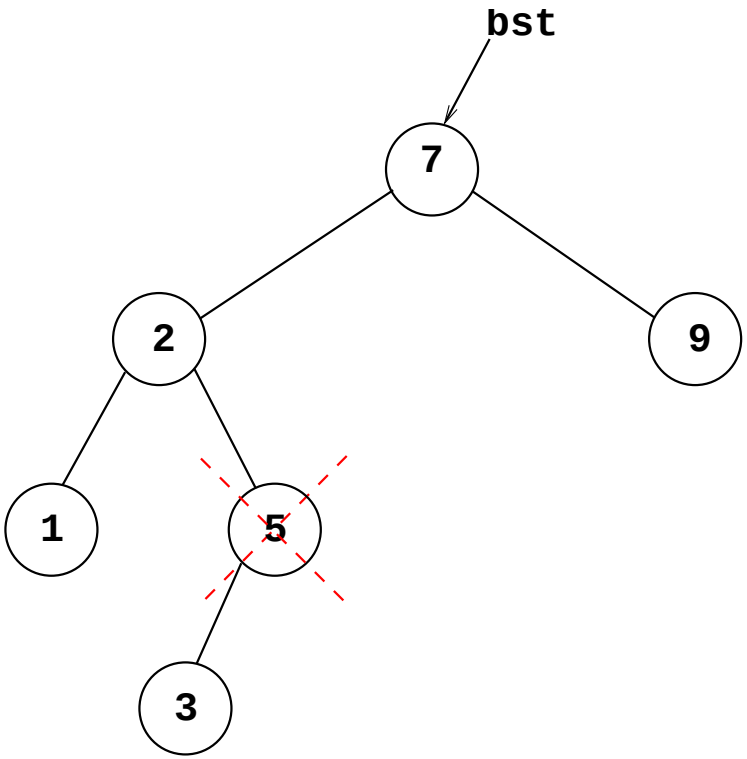
Removing an Item from a Binary Search Tree *continued*

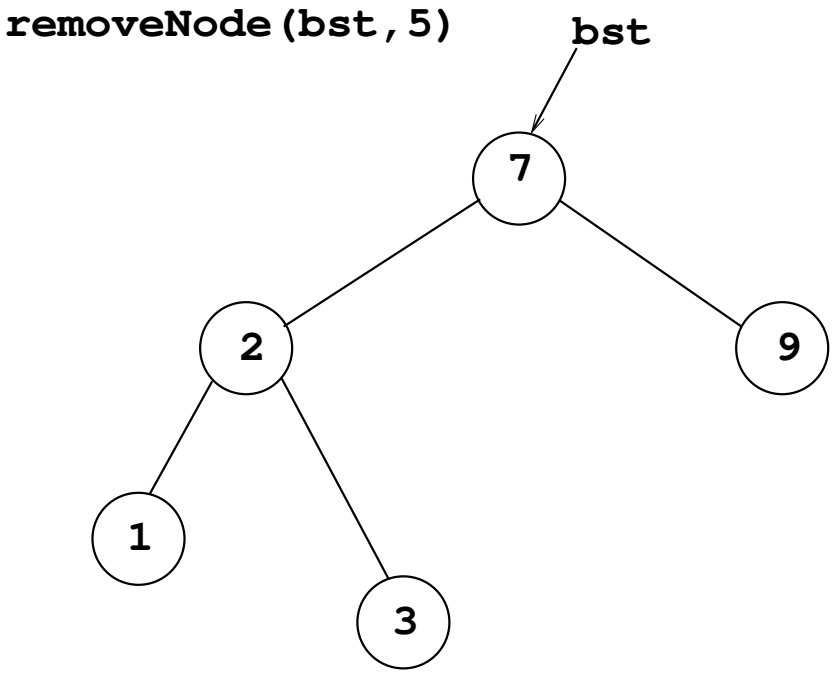
- Case 2: node to remove has one child



Removing an Item from a Binary Search Tree *continued*

- Case 2: node to remove has one child





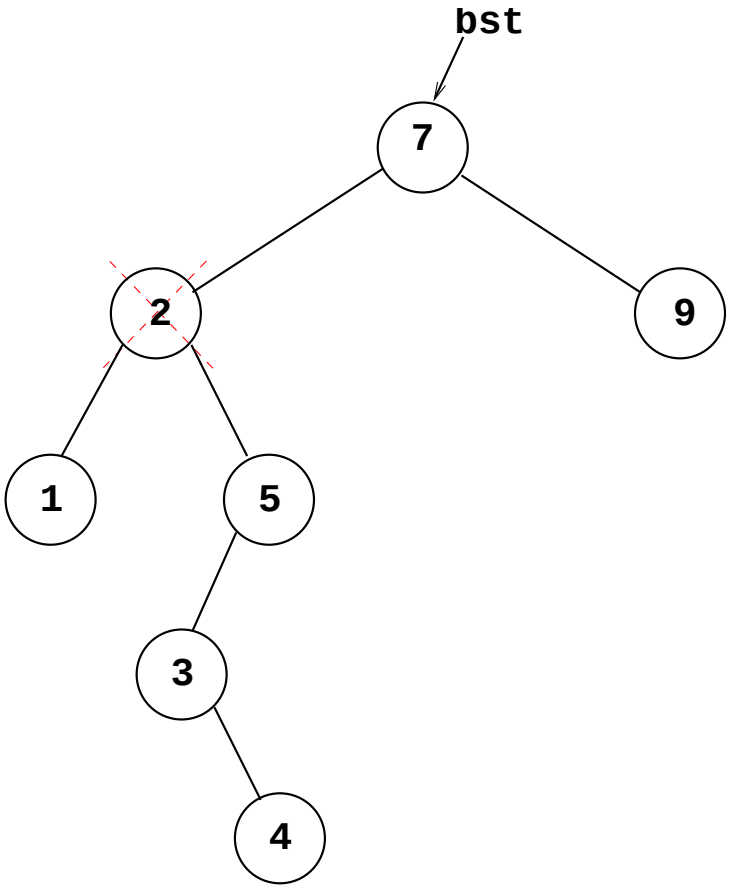
Removing an Item from a Binary Search Tree *continued*

- Case 3: node to remove has two children



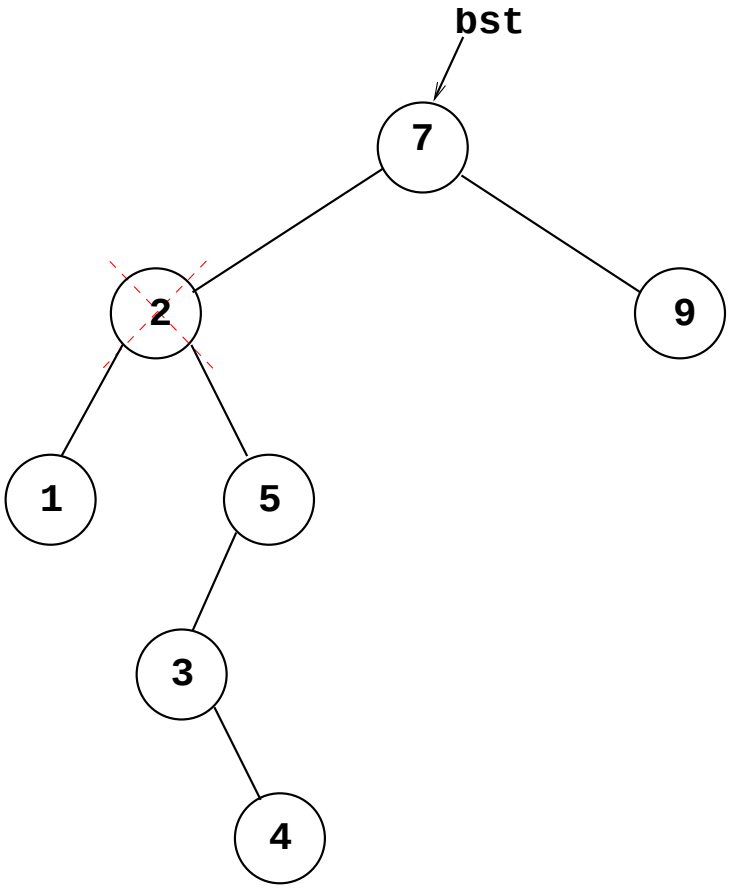
Removing an Item from a Binary Search Tree *continued*

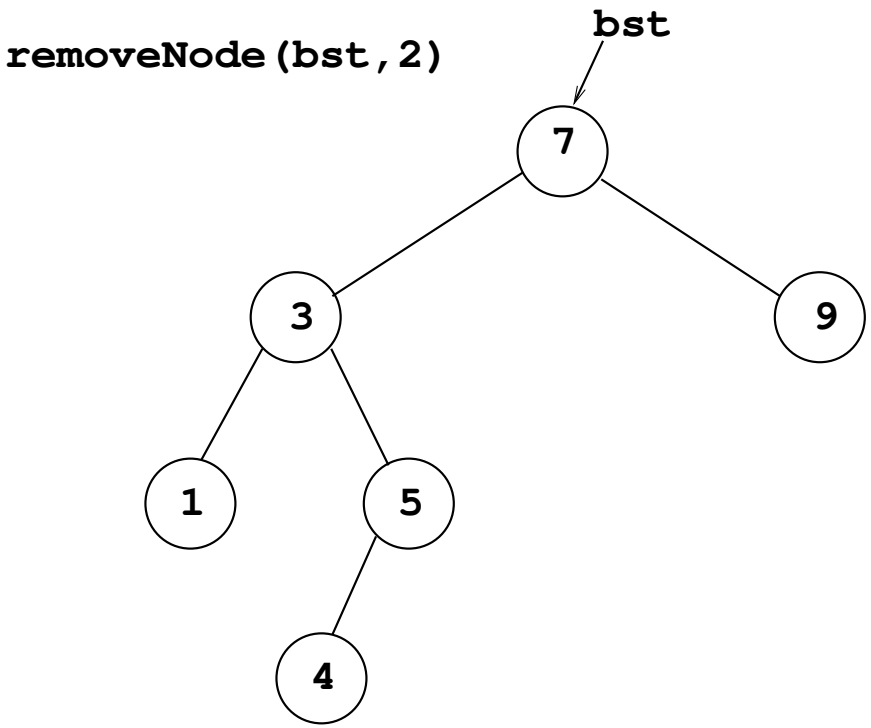
- Case 3: node to remove has two children



Removing an Item from a Binary Search Tree *continued*

- Case 3: node to remove has two children





Removing an Item from a Binary Search Tree *continued*

```
struct node *removeNode(struct node *t, itemType item) {
  if(t == NULL)
    printf("Warning: item '%s' not found\n",toItemString(item));
  else if(itemcmp(item,t->data) < 0)
    t->left = removeNode(t->left,item);
  else if(itemcmp(item,t->data) > 0)
    t->right = removeNode(t->right,item);
  else // found it!
    if(t->left!=NULL && t->right!=NULL) {
      t->data = findMin(t->right)->data;
      t->right = removeMin(t->right);
    } else
      t = ((t->left!=NULL)?t->left:t->right);
  return t;
}
```

```
struct node {
  itemType data;
  struct node *left, *right;
};
```



Removing an Item from a Binary Search Tree *continued*

```
static struct node *findMin(struct node *t) {
  return ((t->left==NULL)?t:findMin(t->left));
}
```

```
struct node {
  itemType data;
  struct node *left, *right;
};
```

```
static struct node *removeMin(struct node *t) {
  if(t!=NULL) {
    if(t->left!=NULL)
      t->left = removeMin(t->left);
    else {
      struct node *ptr = t;
      t = t->right;
#ifdef NEED_TO_FREE
      freeItemType(ptr->data);
#endif
      free(ptr);
    }
  }
  return t;
}
```

```
#ifndef ITEMTYPE_H
#define ITEMTYPE_H
#include "limits.h"

#define NULLITEM INT_MIN
//#define NEED_TO_FREE

typedef int itemType;

void freeItemType(itemType);

int itemcmp(itemType,itemType);
char *toItemString(itemType);

#endif
```



Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AVL trees
  - Red-black trees
  - Splay trees
- Time does not allow us to cover these algorithms in this class



Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AVL trees
  - Red-black trees
  - Splay trees
- Time does not allow us to cover these algorithms in this class

Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AA tree
- Time does not allow us to cover these algorithms in this class

Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AA tree
  - AVL trees
  - Red-black trees
  - Splay trees
- Time does not allow us to cover these algorithms in this class



Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AA tree
  - AVL trees
  - Red-black trees
  - Splay trees
- Time does not allow us to cover these algorithms in this class



Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AA tree
  - AVL trees
  - Red-black trees
  - Splay trees
- Time does not allow us to cover these algorithms in this class



Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AA tree
  - AVL trees
  - Red-black trees
  - Splay trees
- Time does not allow us to cover these algorithms in this class



Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AA tree
  - AVL trees
  - Red-black trees
  - Splay trees
- Time does not allow us to cover these algorithms in this class

Self-Balancing Binary Search Trees

- It is advantageous to keep the height of a BST as small as possible
- For many (even most) BSTs performing a large number of arbitrary insertions & deletions will eventually result in a degradation of performance
- Good solution: monitor and reorganize the tree periodically to maintain a certain level of efficiency
- Many different algorithms:
  - AA tree
  - AVL trees
  - Red-black trees
  - Splay trees
- Time does not allow us to cover these algorithms in this class



bst.c

```
#include <stdio.h>
#include <stdlib.h>
#include "bst.h"
#include "itemType.h"

struct node {
  itemType data;
  struct node *left, *right;
};

struct bstType {
  struct node *root;
  // int count;
};
```



bst.c *continued*

```
bst createBST() {
  bst t = malloc(sizeof(struct bstType));
  t->root = NULL;
  return t;
}

static void freeNodes(struct node *t) {
  if (t!=NULL) {
    freeNodes(t->left);
    freeNodes(t->right);
#ifdef NEED_TO_FREE
    freeItemType(t->data);
#endif
    printf("freeing %s\n",toItemString(t->data));
    free(t);
  }
}
void freeBST(bst t) { freeNodes(t->root); t->root = NULL;}
```



bst.c *continued*

```
static struct node *findMin(struct node *t) {
  return ((t->left==NULL)?t:findMin(t->left));
}
static struct node *removeMin(struct node *t) {
  if(t!=NULL) {
    if(t->left!=NULL)
      t->left = removeMin(t->left);
    else {
      struct node *ptr = t;
      t = t->right;
#ifdef NEED_TO_FREE
      freeItemType(ptr->data);
#endif
      free(ptr);
    }
  }
  return t;
}
```



bst.c *continued*

```
static struct node *insertNode(struct node *t, itemType item) {
  if(t==NULL) {
    t = malloc(sizeof(struct node));
    t->data = item; t->left = t->right = NULL;
  } else if(itemcmp(item,t->data) < 0)
    t->left = insertNode(t->left,item);
  else  if(itemcmp(item,t->data) > 0)
    t->right = insertNode(t->right,item);
  else
    printf("Warning: duplicate item '%s' ignored\n",
           toItemString(item));
  return t;
}

void insertItem(bst t,itemType item) {
  t->root = insertNode(t->root,item);
}
```



bst.c *continued*

```
static struct node *removeNode(struct node *t, itemType item) {
  if(t == NULL)
    printf("Warning: item '%s' not found\n",toItemString(item));
  else if(itemcmp(item,t->data) < 0)
    t->left = removeNode(t->left,item);
  else if(itemcmp(item,t->data) > 0)
    t->right = removeNode(t->right,item);
  else // found it!
    if(t->left!=NULL && t->right!=NULL) {
      t->data = findMin(t->right)->data;
      t->right = removeMin(t->right);
    } else
      t = ((t->left!=NULL)?t->left:t->right);
  return t;
}
void removeItem(bst t,itemType item) {
  t->root = removeNode(t->root,item);
}
```



bst.c *continued*

```
static void preOrder(struct node *t,void (*f)()) {
  if(t!=NULL) {
    f(t->data); preOrder(t->left,f); preOrder(t->right,f);
  }
}
static void inOrder(struct node *t,void (*f)()) {
  if(t!=NULL) {
    inOrder(t->left,f); f(t->data); inOrder(t->right,f);
  }
}
static void postOrder(struct node *t,void (*f)()) {
  if(t!=NULL) {
    postOrder(t->left,f); postOrder(t->right,f); f(t->data);
  }
}
void preorder(bst t,void (*f)()) { preOrder(t->root,f); }
void inorder(bst t,void (*f)()) { inOrder(t->root,f); }
void postorder(bst t,void (*f)()) { postOrder(t->root,f); }
```



bst.c *continued*

```
static int tcount(struct node *t) {
  return ((t==NULL)?0:1+tcount(t->left)+tcount(t->right));
}
int size(bst t) { return tcount(t->root); }

bool isEmpty(bst t) { return t->root==NULL; }
```

